CSE 309 (Compilers)

Relevant Information

Dr. Muhammad Masroor Ali

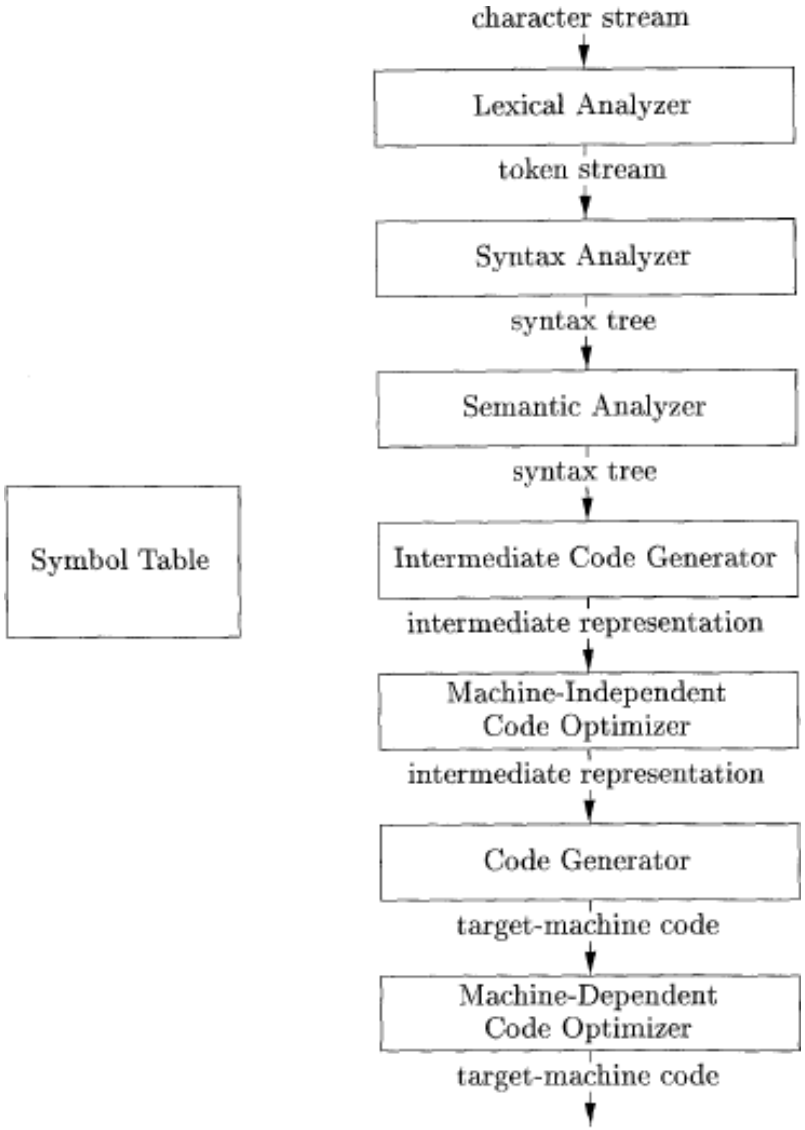
Professor
Department of Computer Science and Engineering
Bangladesh University of Engineering and Technology
Dhaka-1205, Bangladesh

January 2025

Adapted by
Ahmed Mahir Sultan Rumi
Lecturer, Department of CSE, BUET

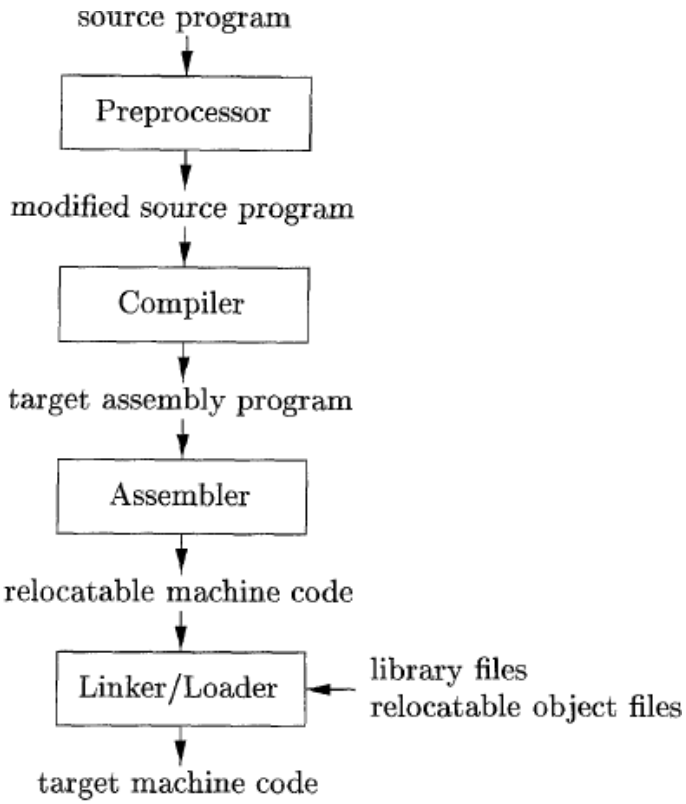
Version: 1.2, Last modified: June 18, 2026

Study of Compilers

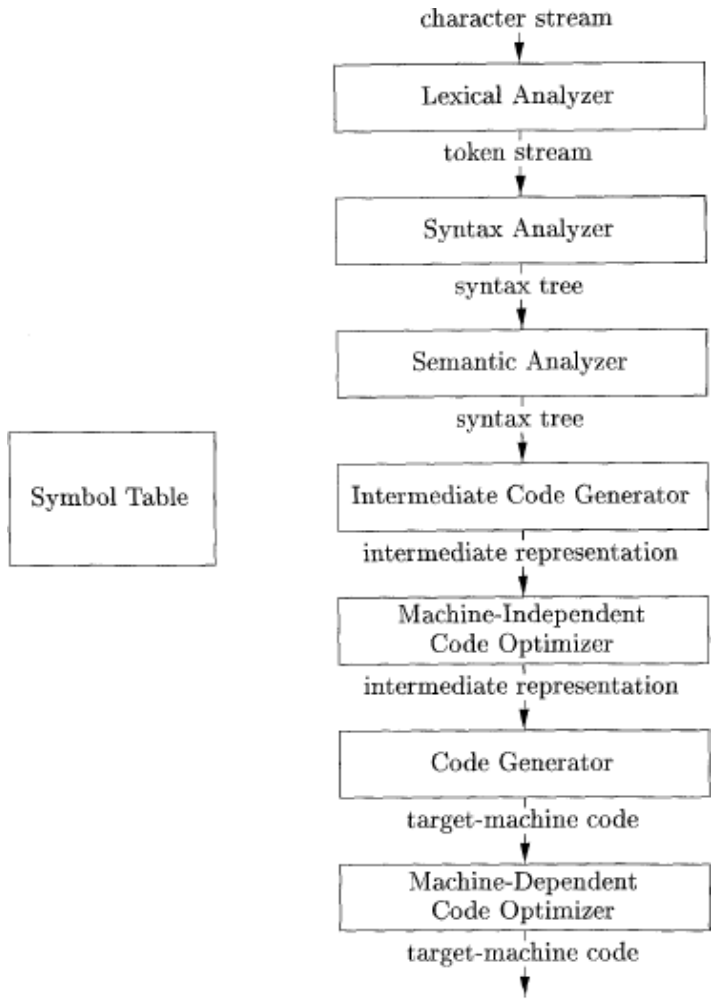


Phases of a compiler

Study of Compilers



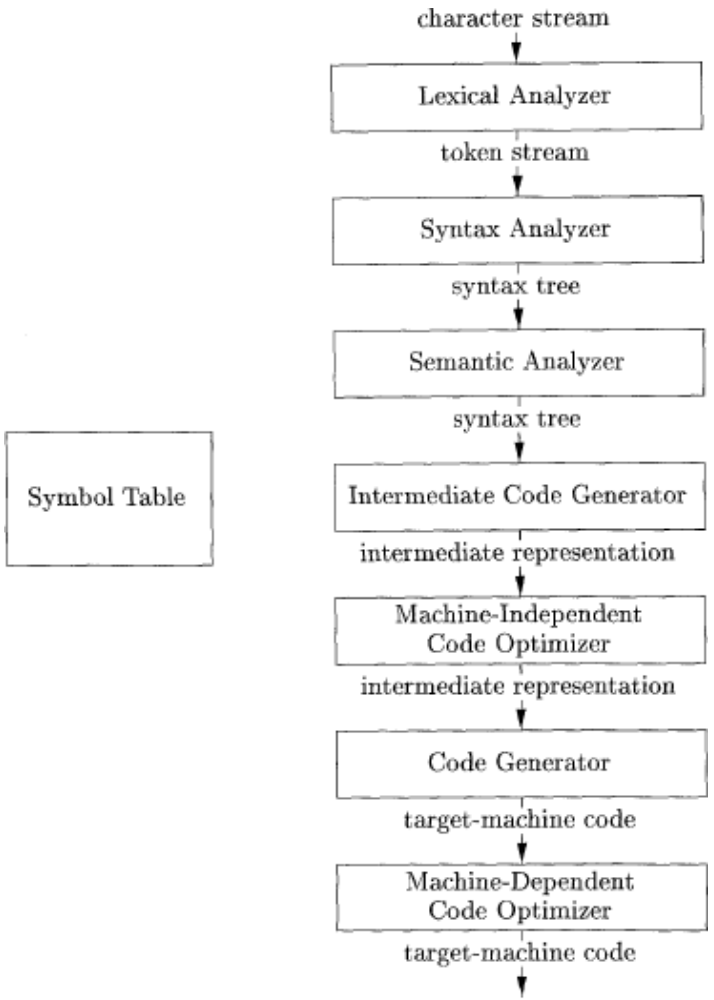
A language-processing system



Phases of a compiler

Study of Compilers

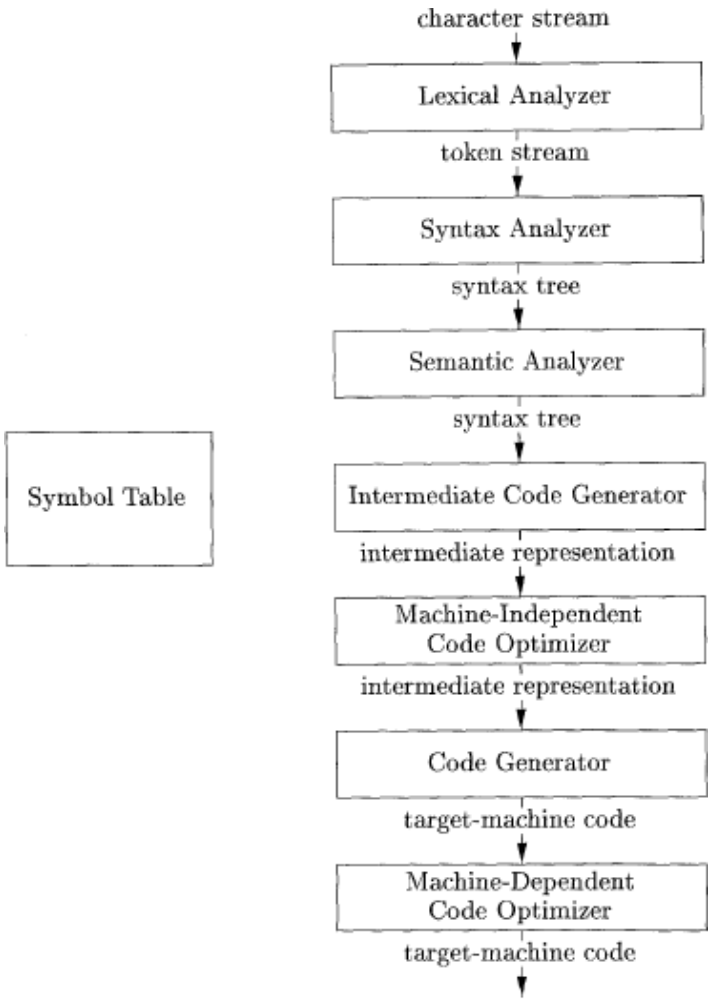
- Introduction and basic concepts
- Lexical analysis
- Syntax analysis
- Syntax-directed translation
- Intermediate-code generation
- Run-time environments
- Code generation



Phases of a compiler

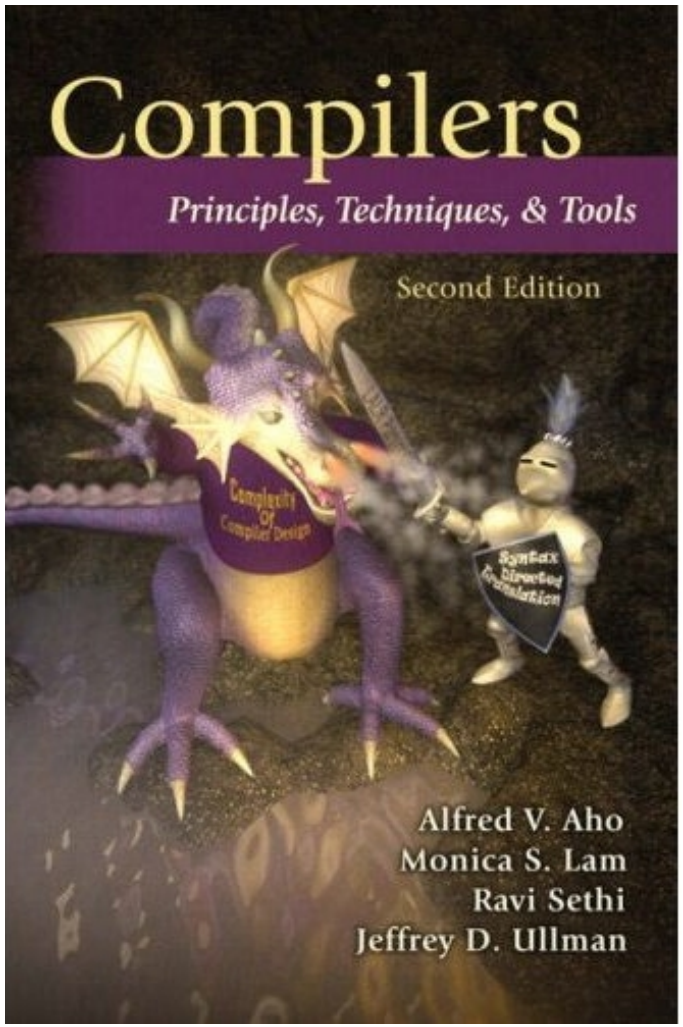
Study of Compilers - 1st part

- Introduction and basic concepts
- Lexical analysis
- Syntax analysis
- Syntax-directed translation



Phases of a compiler

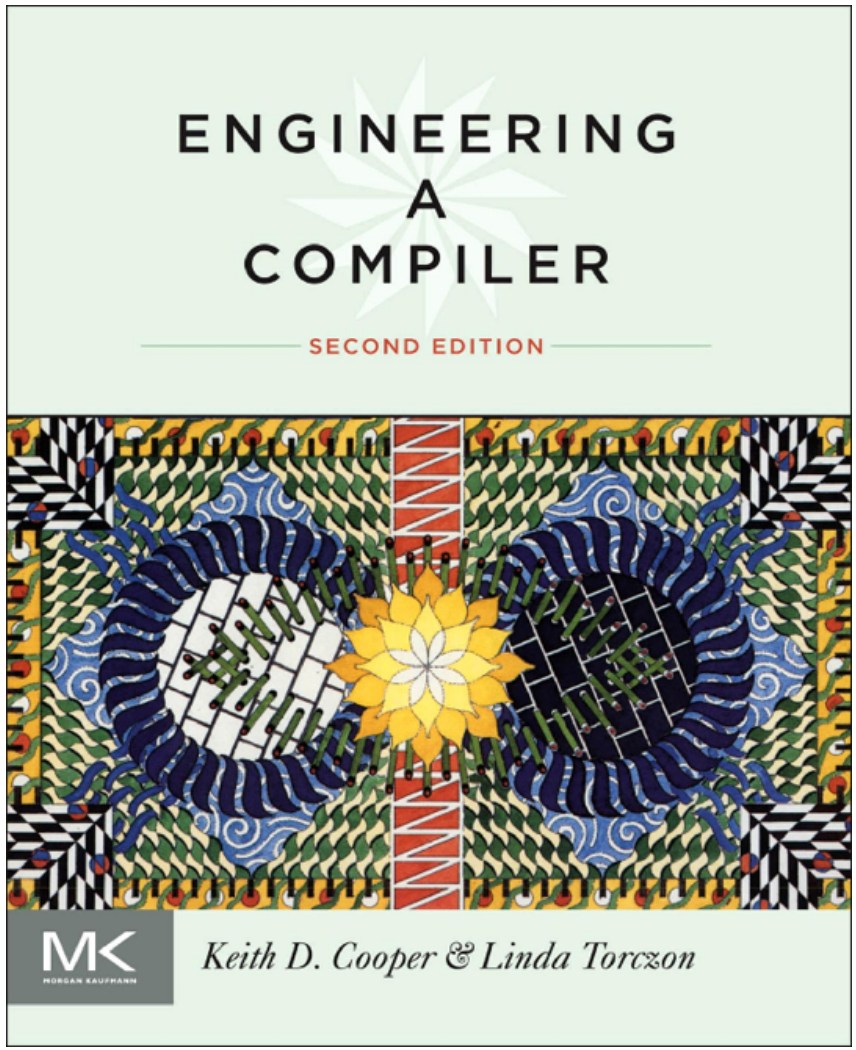
References



- ALFRED V. AHO, MONICA S. LAM, RAVI SETHI AND JEFFREY D. ULLMAN Compilers: Principles, Techniques & Tools, Second edition. Pearson/Addison Wesley, 2007.

References

- Linda Torczon and Keith Cooper Engineering A Compiler, Second edition. Morgan Kaufmann Publishers Inc., 2011.



Class Tests

- This is a 3.0 credit course.
- So, there will be $3 + 1 = 4$ class tests.
- Out of these, best three will be selected.
- Class tests will be held on the dates announced by the department.

Course Website

- Departmental Moodle site, `http://cse.buet.ac.bd/moodle/`, will be used for all course material dissemination, announcements, and communication.

CSE 309 (Compilers)

Basic Concepts

Dr. Muhammad Masroor Ali

Professor
Department of Computer Science and Engineering
Bangladesh University of Engineering and Technology
Dhaka-1205, Bangladesh

January 2025

Adapted by
Ahmed Mahir Sultan Rumi
Lecturer, Department of CSE, BUET

Version: 1.2, Last modified: June 20, 2026

Introduction

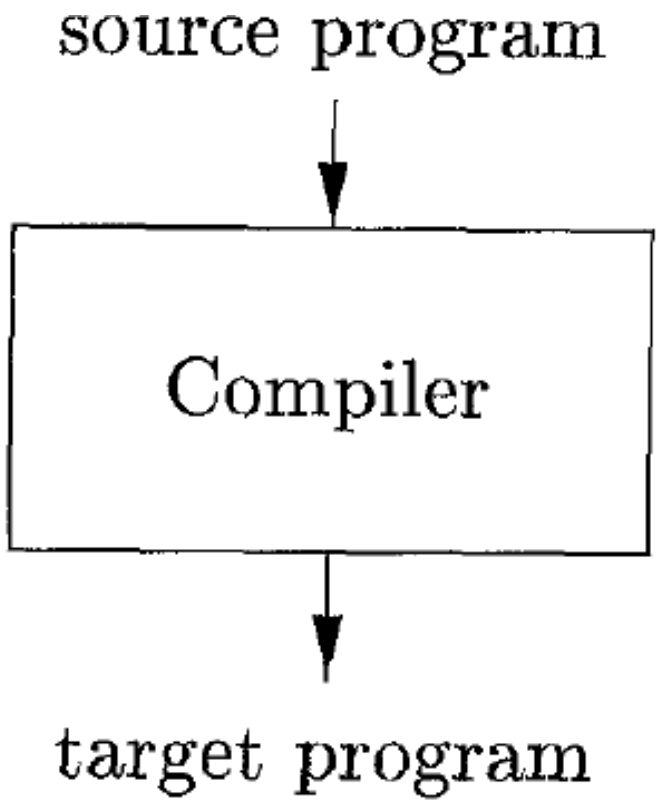
- Programming languages are notations for describing computations to people and to machines.
- The world as we know it depends on programming languages, because all the software running on all the computers was written in some programming language.
- But, before a program can be run, it first must be translated into a form in which it can be executed by a computer.
- The software systems that do this translation are called compilers.

Introduction

- Programming languages are notations for describing computations to people and to machines.
- The world as we know it depends on programming languages, because all the software running on all the computers was written in some programming language.
- But, before a program can be run, it first must be translated into a form in which it can be executed by a computer.
- The software systems that do this translation are called compilers.

Language Processors

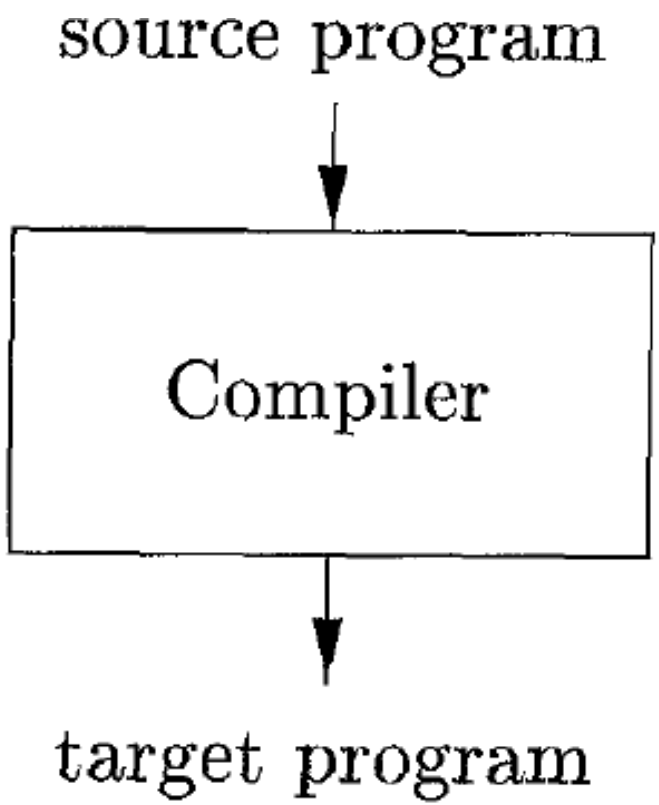
- Simply stated, a compiler is a program,
  - that can read a program in one language — the source language — and
  - translate it into an equivalent program in another language — the target language.



A compiler

Language Processors

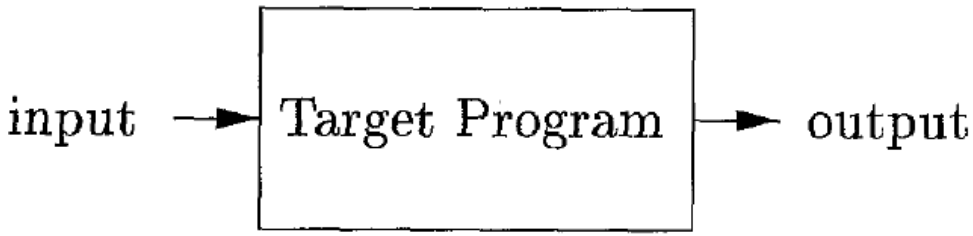
- An important role of the compiler is to report any errors in the source program that it detects during the translation process.



A compiler

Language Processors

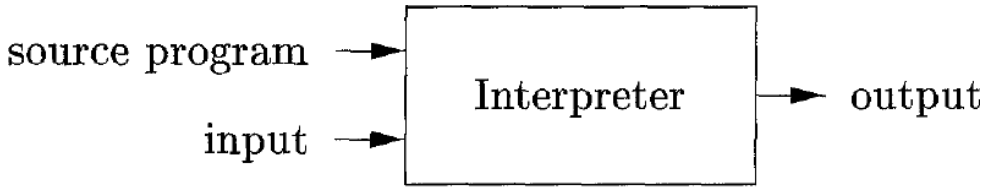
- If the target program is an executable machine-language program, it can then be called by the user to process inputs and produce outputs.



Running the target program

Language Processors — *continued*

- An interpreter is another common kind of language processor.
- Instead of producing a target program as a translation, an interpreter appears to directly execute the operations specified in the source program on inputs supplied by the user.



An interpreter

Language Processors — *continued*

- The machine-language target program produced by a compiler is usually much faster than an interpreter at mapping inputs to outputs.
- An interpreter, however, can usually give better error diagnostics than a compiler, because it executes the source program statement by statement.

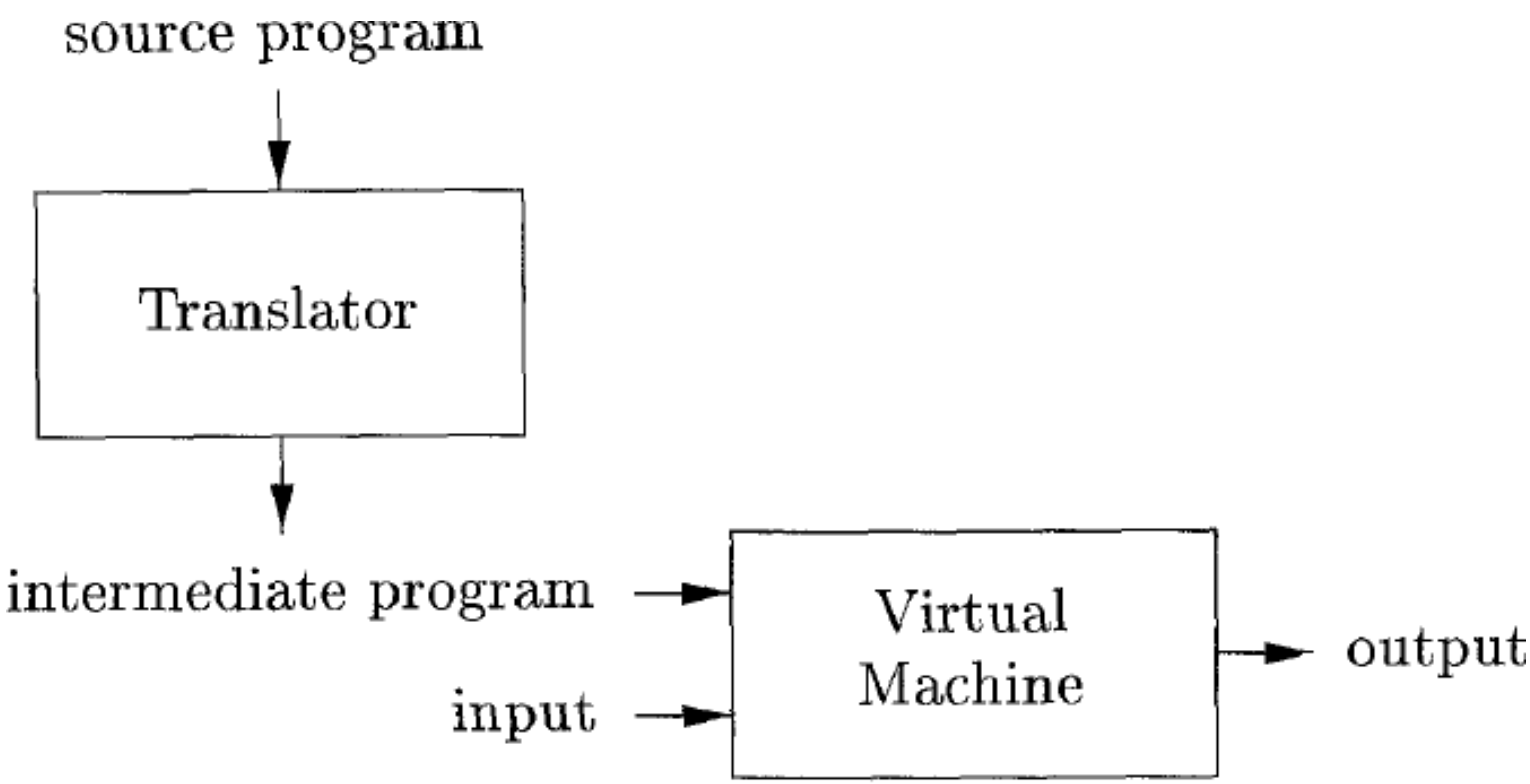
Language Processors — *continued*

- Then which path do the modern compilers take?
- Obvious answer. The middle way.
  - Hybrid compilers
  - Just-in-time compilers (JIT)

Language Processors — *continued*

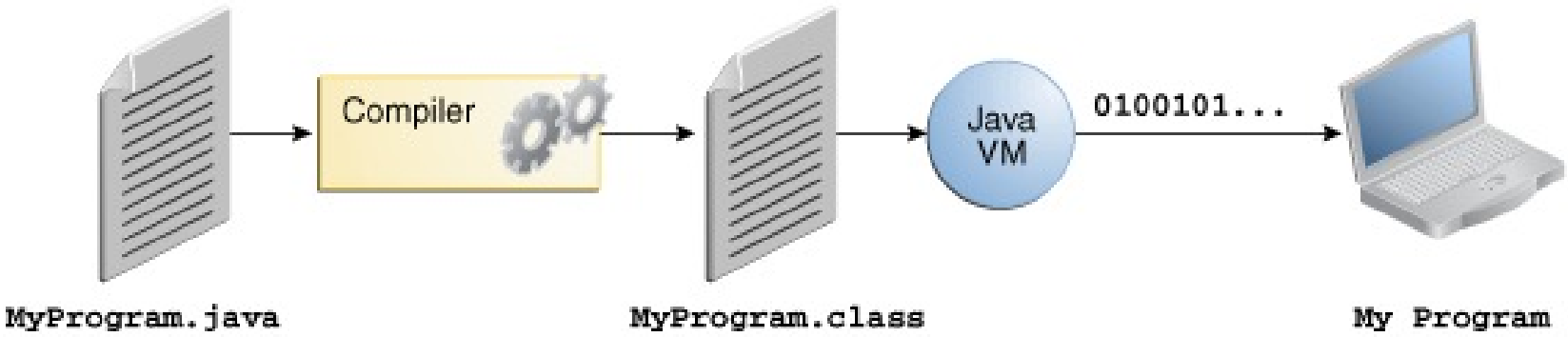
- Then which path do the modern compilers take?
- Obvious answer. The middle way.
  - Hybrid compilers
  - Just-in-time compilers (JIT)

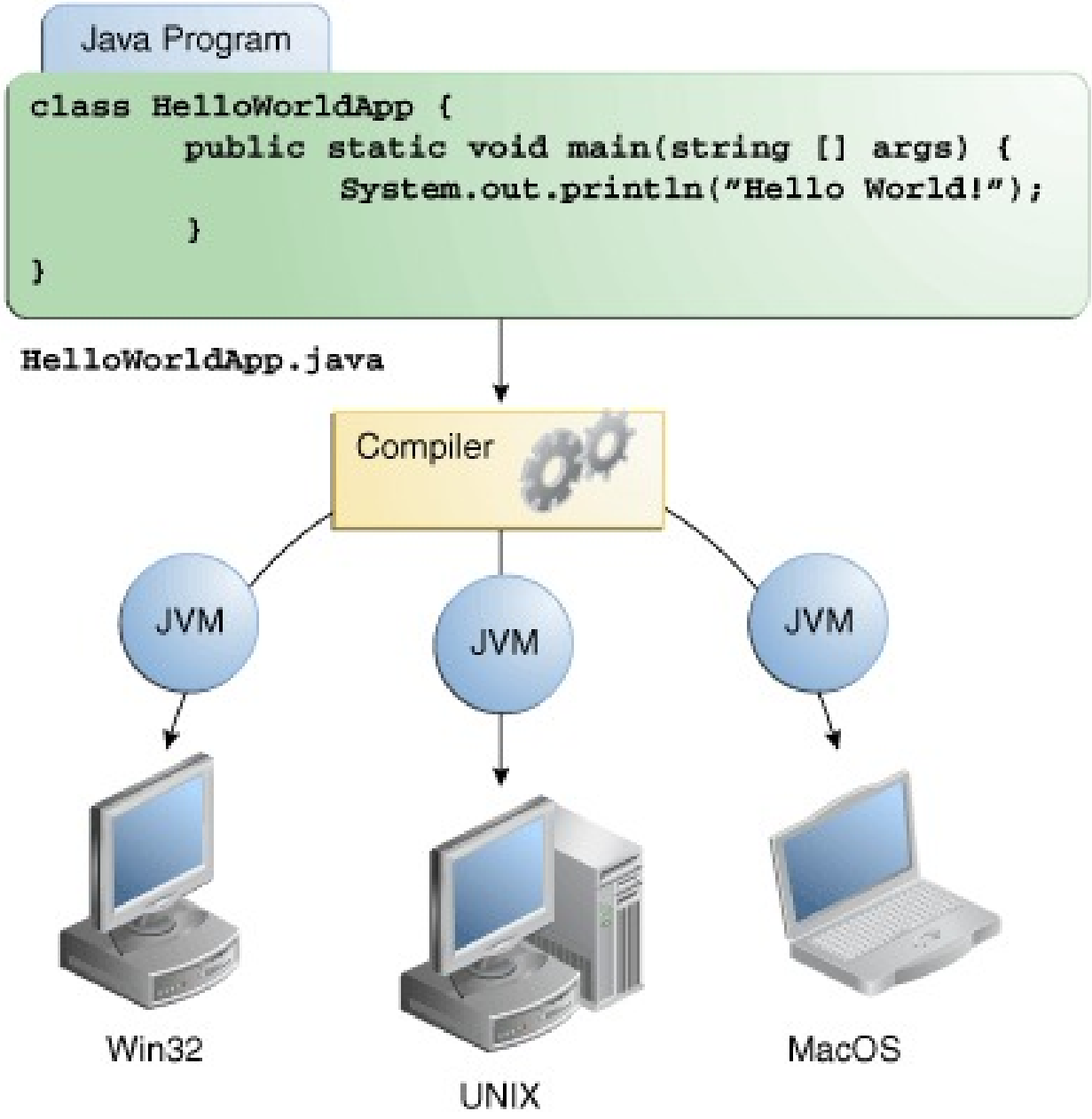
Example



A hybrid compiler

- Java language processors combine compilation and interpretation.


Compiler
Java
VM
0100101...
MyProgram.java
MyProgram.class
My Program


Java Program
class HelloWorldApp {
public static void main(string [] args) {
System.out.println("Hello World!");
}
}
HelloWorldApp.java
Compiler
JVM
JVM
JVM
Win32
UNIX
MacOS

Example — *continued*

- A Java source program may first be compiled into an intermediate form called bytecodes.
- The bytecodes are then interpreted by a virtual machine.
- ”Write once, run anywhere”

Example — *continued*

- A Java source program may first be compiled into an intermediate form called bytecodes.
- The bytecodes are then interpreted by a virtual machine.
- ”Write once, run anywhere” A benefit of this arrangement is that bytecodes compiled on one machine can be interpreted on another machine, perhaps across a network.

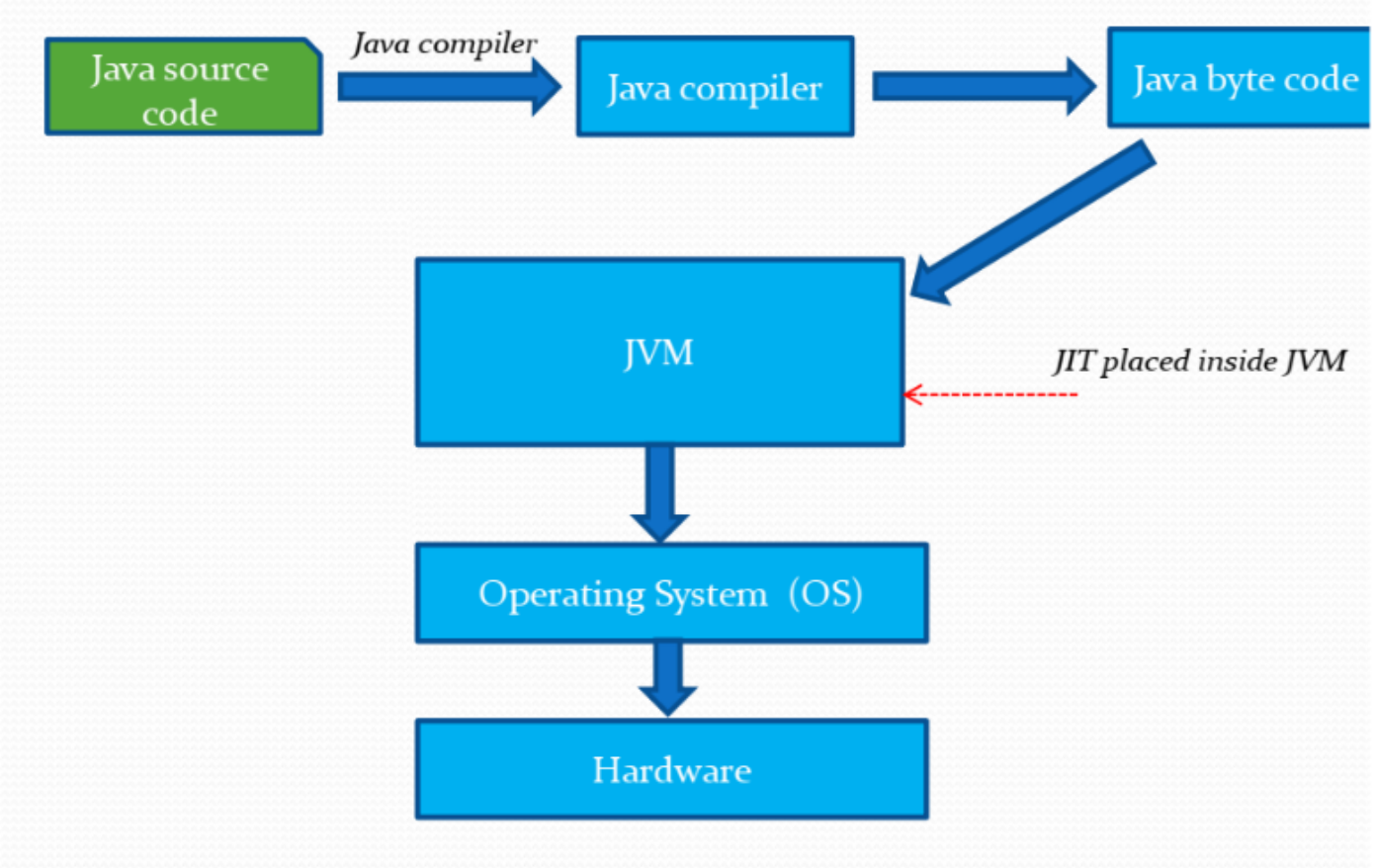
Example — *continued*

- A Java source program may first be compiled into an intermediate form called bytecodes.
- The bytecodes are then interpreted by a virtual machine.
- ”Write once, run anywhere” A benefit of this arrangement is that bytecodes compiled on one machine can be interpreted on another machine, perhaps across a network.
- Faster and better performance than pure interpretation

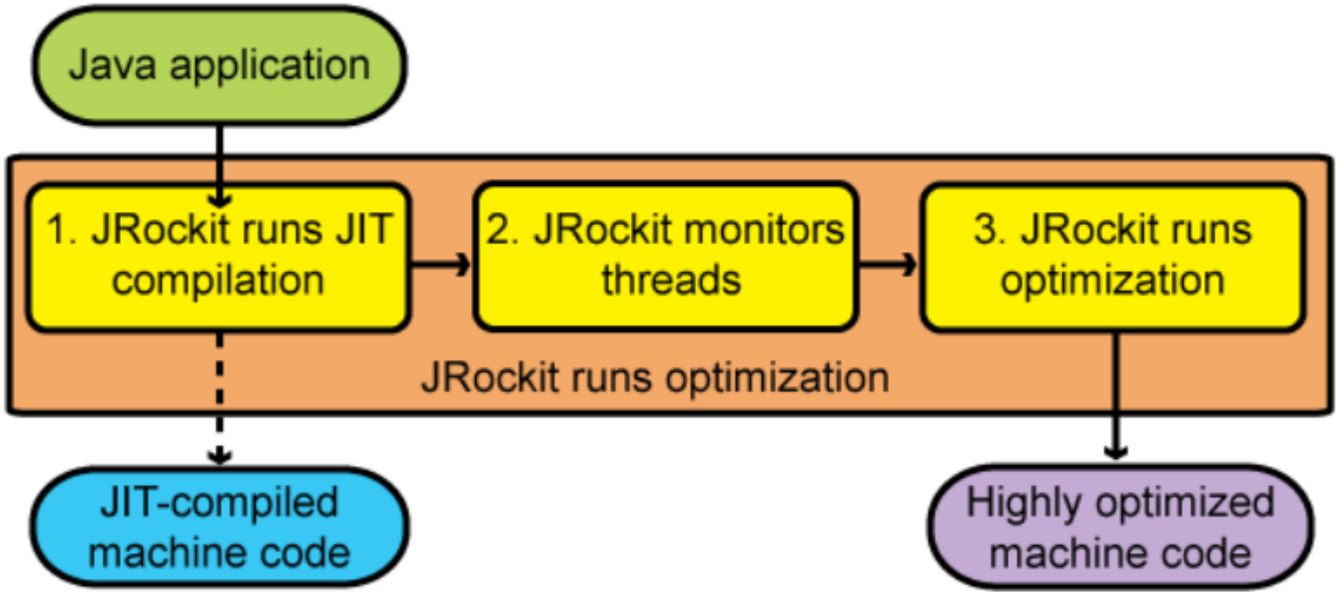
Example — *continued*

- A Java source program may first be compiled into an intermediate form called bytecodes.
- The bytecodes are then interpreted by a virtual machine.
- ”Write once, run anywhere” A benefit of this arrangement is that bytecodes compiled on one machine can be interpreted on another machine, perhaps across a network.
- Faster and better performance than pure interpretation
- One important point. You will need JVM in the machine to run bytecode. That is platform dependent.

Just-in-Time Compilation

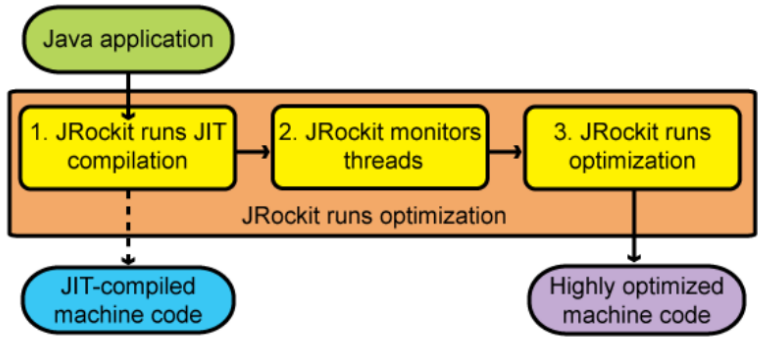


Just-in-Time Compilation - JRockit — *continued*



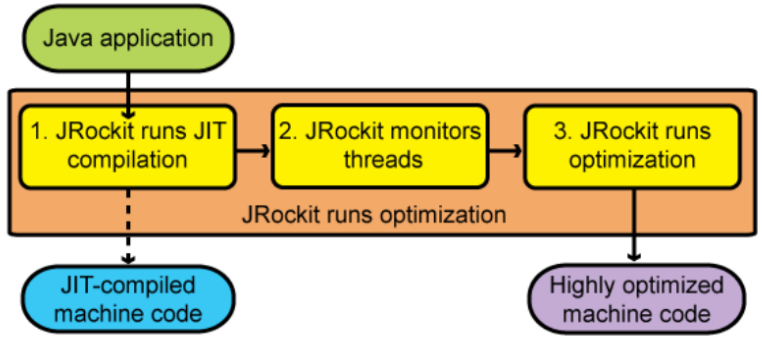
JRockit Just-in-Time Compilation

- When a section of instructions is called:
  - Compile bytecode into machine code just in time.
  - Run the compiled machine code.
  - The compiled code is not fully optimized.
  - It may be slower than bytecode interpretation.
  - JVM startup may be slower than execution.



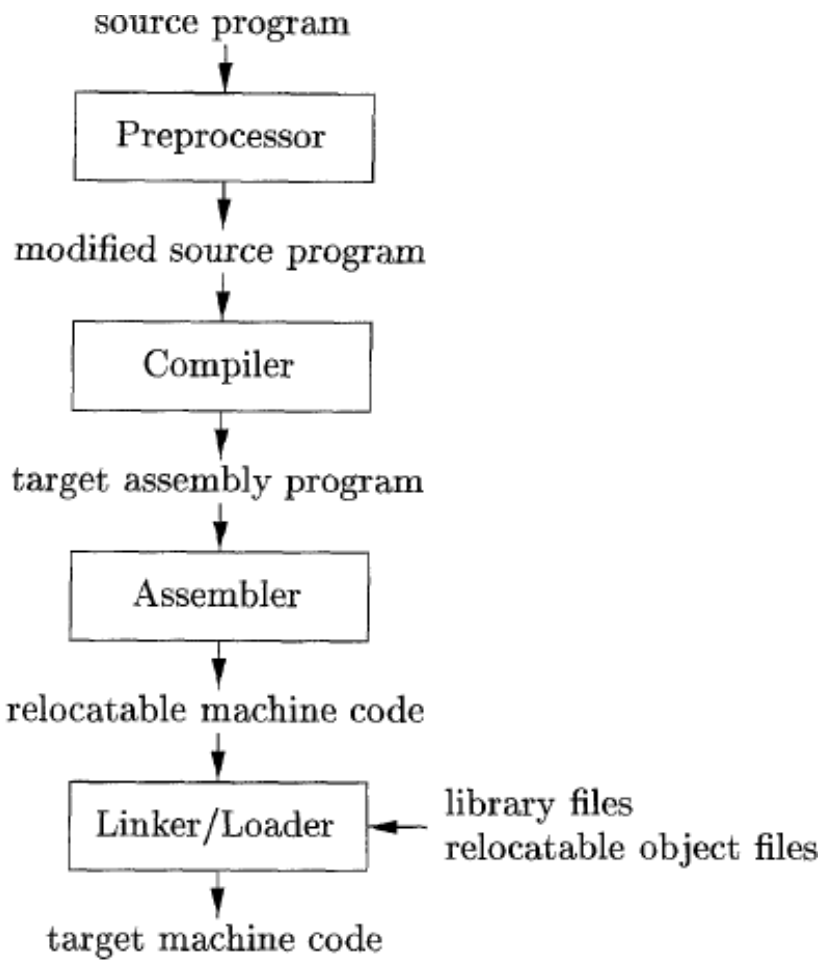
JRockit Optimization

- Identify which functions merit optimization:
  - A sampler thread checks the status of active threads.
  - Hot methods are earmarked for optimization.
  - Optimization opportunities occur early on.
- In the background, compile optimized hot methods into machine-readable instructions.



Language Processors — *continued*

- In addition to a compiler, several other programs may be required to create an executable target program.
- A source program may be divided into modules stored in separate files.
- The task of collecting the source program is sometimes entrusted to a separate program, called a preprocessor.
- The preprocessor may also expand shorthands, called macros, into source language statements.



A language-processing system

C Preprocessor

- The C preprocessor or `cpp` is the macro preprocessor for the C and C++ computer programming languages.
- The preprocessor provides the ability for:
  - the inclusion of header files,
  - macro expansions,
  - conditional compilation, and
  - line control.

C Preprocessor — *continued*

- In many C implementations, it is a separate program invoked by the compiler as the first part of translation.

C Preprocessor — Including Files

- One of the most common uses of the preprocessor is to include another file:

```
#include <stdio.h>
int main(void)
{
  printf("Hello, world!\n");
  return 0;
}
```

C Preprocessor — Including Files

```
#include <stdio.h>
int main(void)
{
  printf("Hello, world!\n");
  return 0;
}
```

- The preprocessor replaces the line `#include <stdio.h>` with the text of the file '`stdio.h`'.

C Preprocessor — Conditional Compilation

- The `if-else` directives `#if`, `#ifdef`, `#ifndef`, `#else`, `#elif` and `#endif` can be used for conditional compilation.
- What is the difference between if and `#if`?

Difference Between `#if` and `if`

- The main difference is when the condition is checked.
- `#if` is a preprocessor-time condition.
- `if` is a runtime condition.
- Therefore, `#if` decides whether code reaches the compiler.
- `if` decides whether compiled code executes while the program runs.

`#if`: Preprocessor-Time Condition

```
#define DEBUG 1

#if DEBUG
printf("Debug mode\n");
#endif
```

- The preprocessor handles `#if` before actual compilation.
- If `DEBUG` is `1`, the line is kept in the source code.
- If `DEBUG` is `0`, the line is removed before compilation.

`if`: Runtime Condition

```
int x = 5;

if (x > 0) {
    printf("Positive\n");
}
```

- `if` is a normal C/C++ statement.
- It is handled by the compiler and checked at runtime.
- Here, the condition `x > 0` is checked when the program runs.

C Preprocessor — Conditional Compilation — *continued*

- Most compilers targeting Microsoft Windows implicitly define `_WIN32`.
- This allows code, including preprocessor commands, to compile only when targeting Windows systems.

C Preprocessor — Conditional Compilation — *continued*

```
#ifdef _unix_
/* _unix_ is usually defined by compilers
   targeting Unix systems */
# include <unistd.h>
#elif defined _WIN32
/* _WIN32 is usually defined by compilers
   targeting 32 or 64 bit Windows systems */
# include <windows.h>
#endif
```

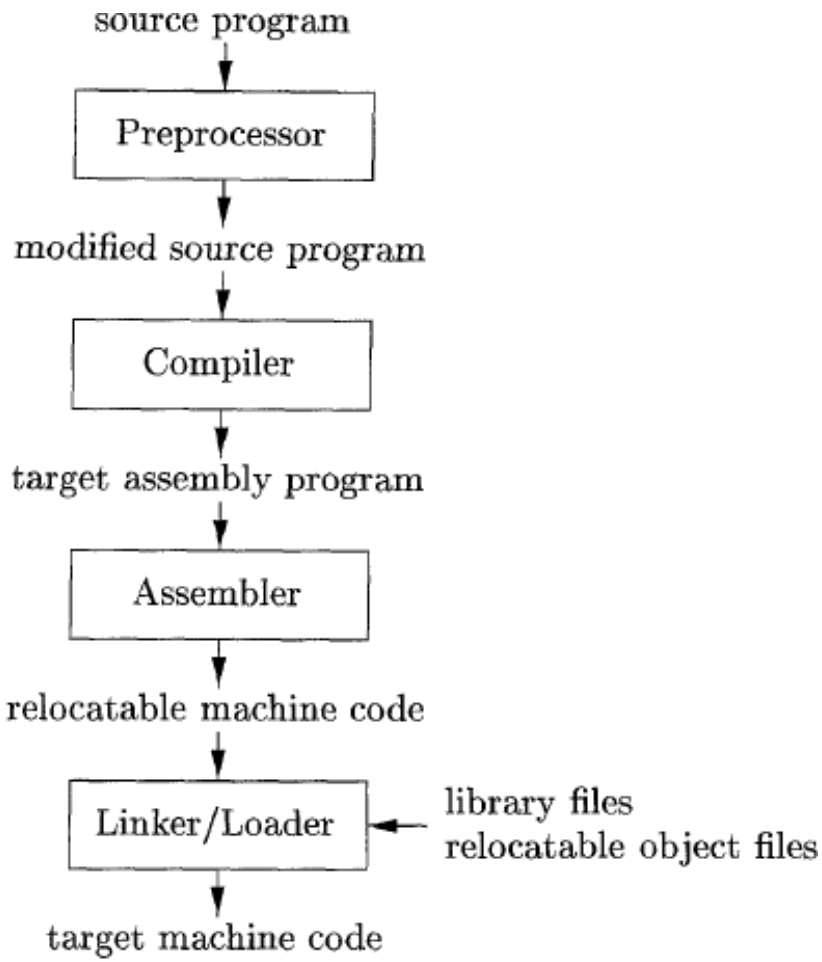
- The example code tests if a macro `_unix_` is defined.
- If it is, the file `<unistd.h>` is then included.
- Otherwise, it tests if a macro `_WIN32` is defined instead.
- If it is, the file `<windows.h>` is then included.

C Preprocessor — Macro Definition and Expansion

```
#define PI 3.14159
#define RADTODEG(x) ((x) * 57.29578)
```

Language Processors — *continued*

- The modified source program is then fed to a compiler.
- The compiler may produce an assembly-language program as its output, because assembly language is easier to produce as output and is easier to debug.
- The assembly language is then processed by a program called an assembler that produces relocatable machine code as its output.



A language-processing system

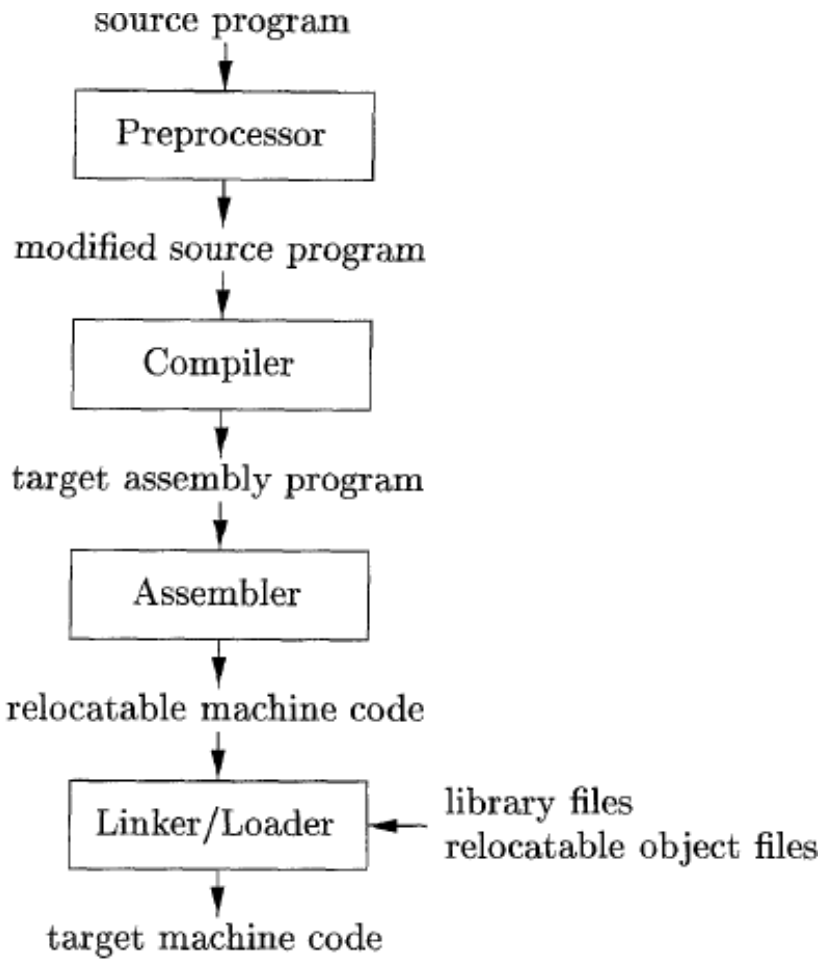
Assembler

https://cs.stackexchange.com/a/13905/7200

- An assembler translates assembly code to machine code.
- The translation is mechanical, and can be done in only one way.
- Assembly languages are usually designed to be nearly trivial to parse and type check and tend to involve a lot of table-driven generators.
- Why do we have this assembler step in the pipeline?

Language Processors — *continued*

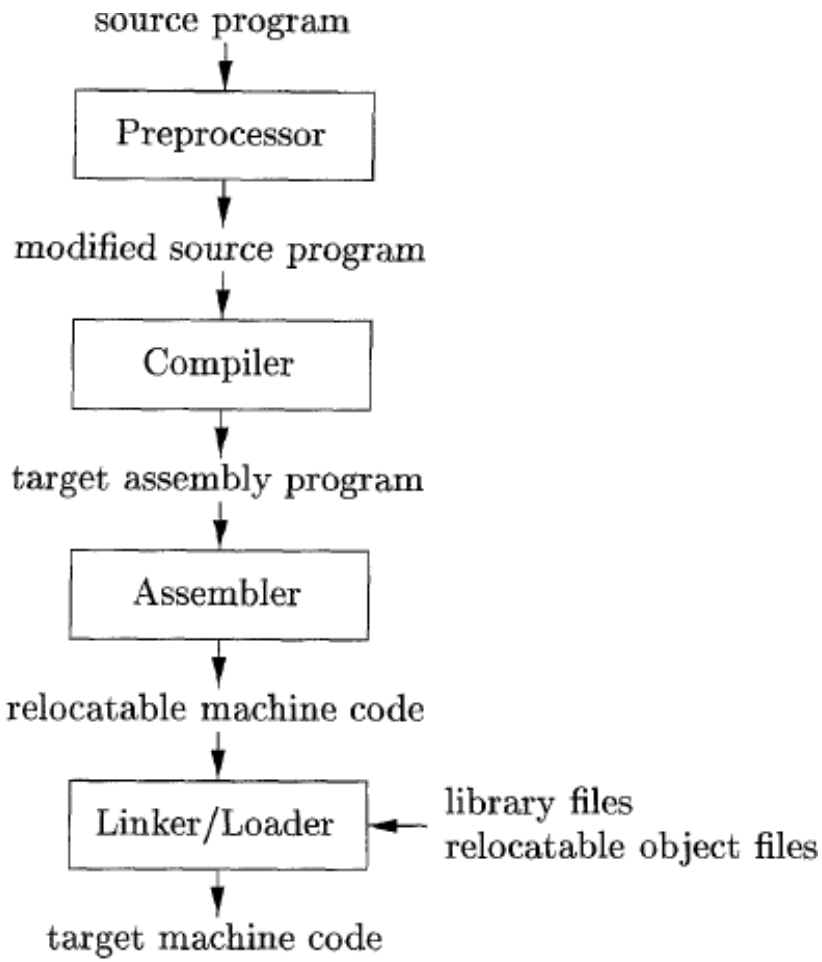
- Large programs are often compiled in pieces.
- So the relocatable machine code may have to be linked together with other relocatable object files and library files into the code that actually runs on the machine.



A language-processing system

Language Processors — *continued*

- The linker resolves external memory addresses, where the code in one file may refer to a location in another file.
- The loader then puts together all of the executable object files into memory for execution.



A language-processing system

Relocatable Machine Code

https://www.wikiwand.com/en/Relocation_(computing), https://stackoverflow.com/a/46101660/1337854 and https://gateoverflow.in/18466/difference-between-relocatable-machine-absolute-machine

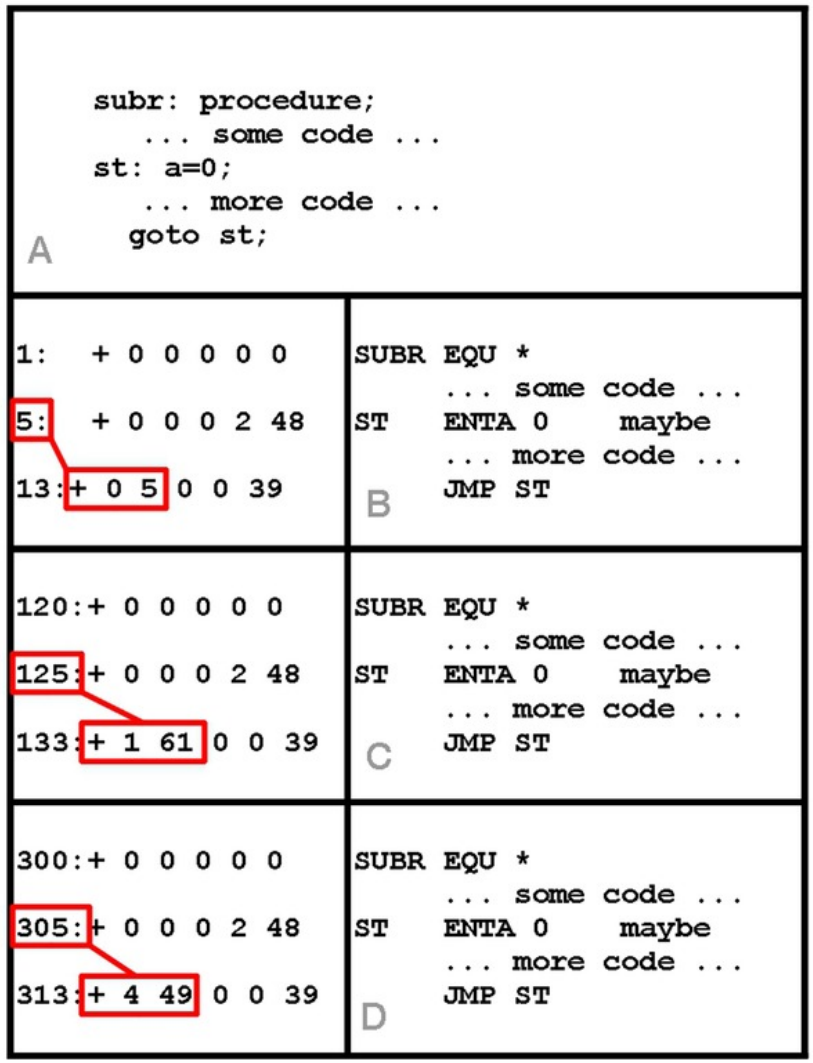
- Relocation is the process of assigning load addresses for position-dependent code and data of a program and adjusting the code and data to reflect the assigned addresses.
- Relocatable-Code, as the name suggests, is relocatable in that code can be loaded anywhere in memory.
- It usually has be relocated or fixed up before it is executable.

Relocatable Machine Code — *continued*

- When the machine code is generated then the computer does not know where the code will be placed in RAM till run time.
- So all the lines are numbered from 0-1-2. . . and so on.
- Now suppose your program is loaded at 230 address then this address on run time will be converted to $230 + 0$, $230 + 1$, $230 + 2$.
- This means every line will be adjusted acordingly.

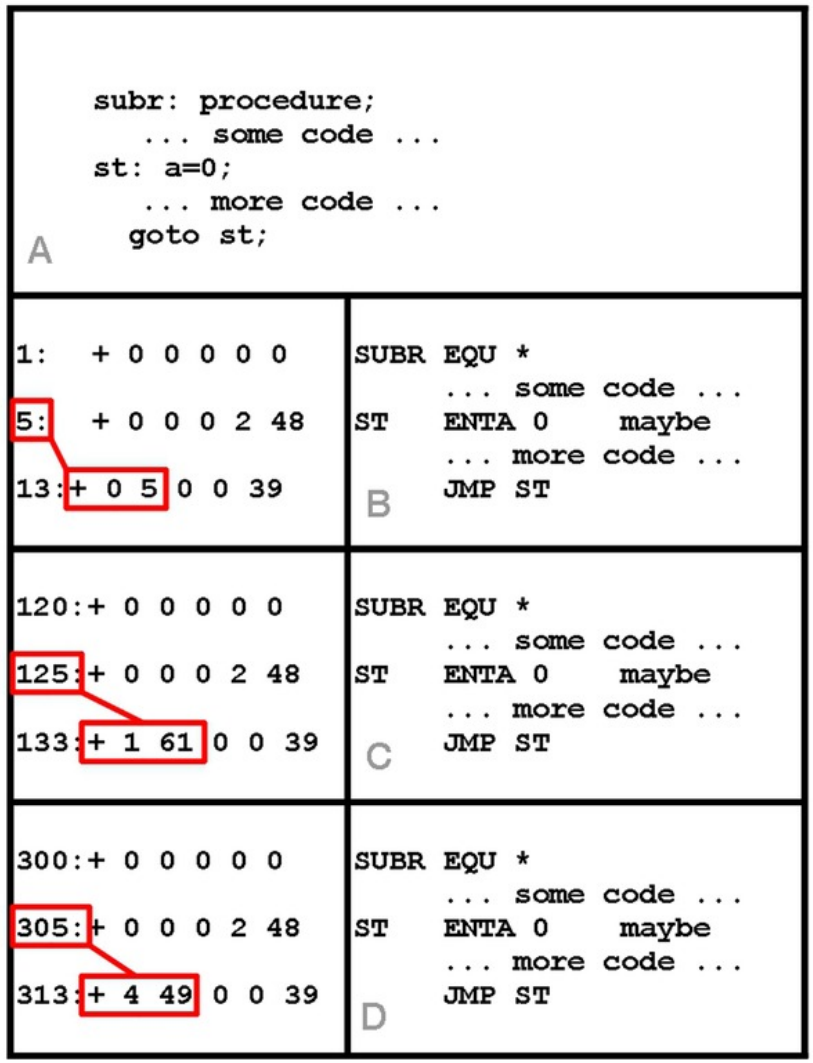
Relocatable Machine Code — *continued*

- (A) Program `SUBR` is compiled to produce object file (B), shown as both machine code and assembler.
- The compiler may start the compiled code at an arbitrary location, often location 1 as shown.
- Location 13 contains the machine code for the jump instruction to statement `ST` in location 5.



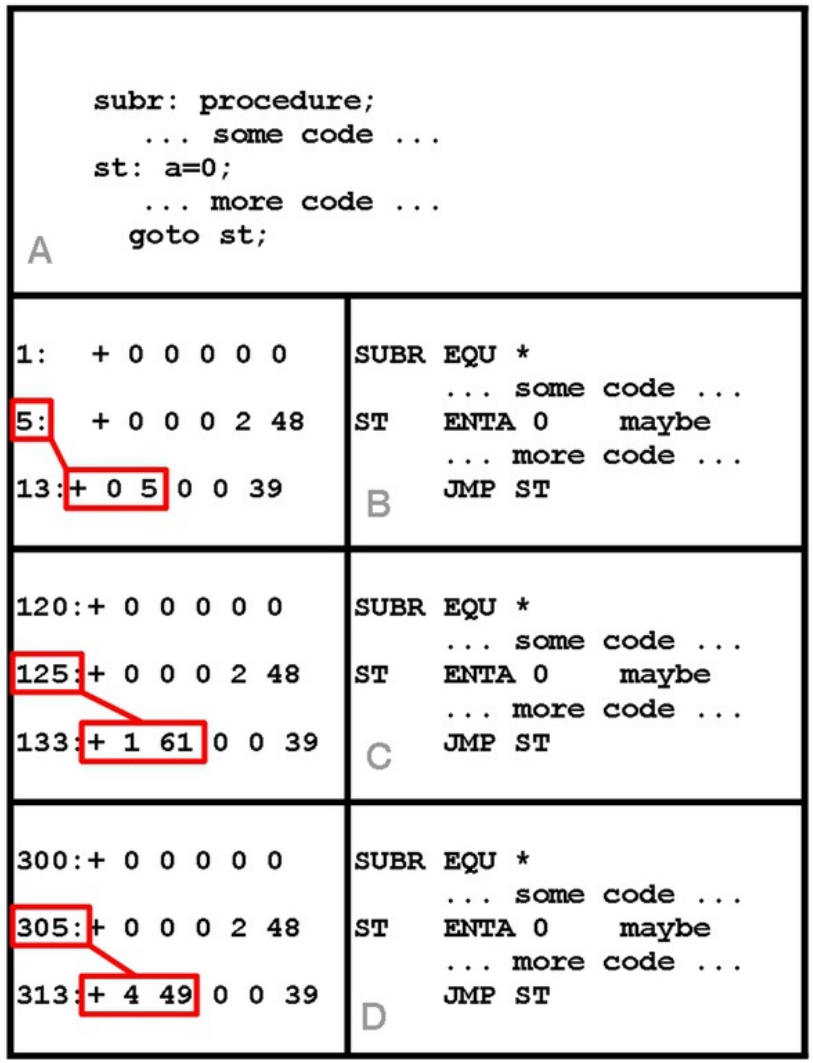
Relocatable Machine Code — *continued*

- (C) If `SUBR` is later linked with other code it may be stored at a location other than 1.
- In this example the linker places it at location 120.



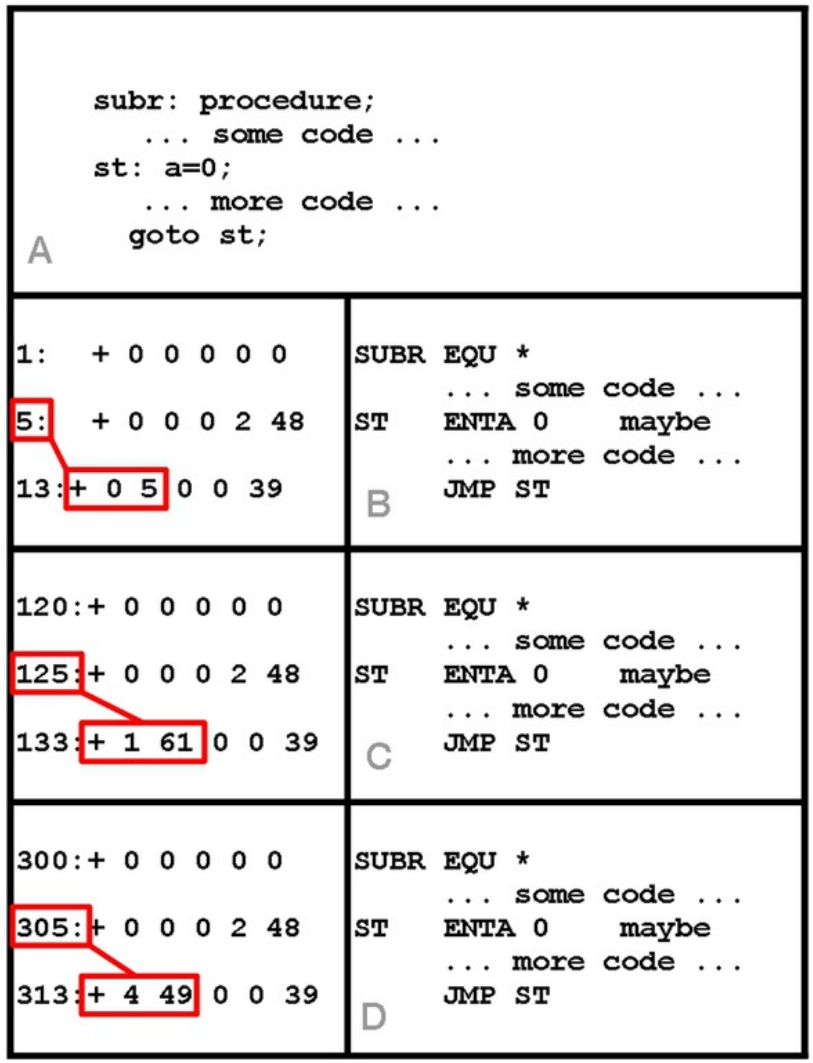
Relocatable Machine Code — *continued*

- The address in the jump instruction, which is now at location 133, must be relocated to point to the new location of the code for statement `ST`, now 125.
- `1 61` shown in the instruction is the MIX machine code representation of 125.



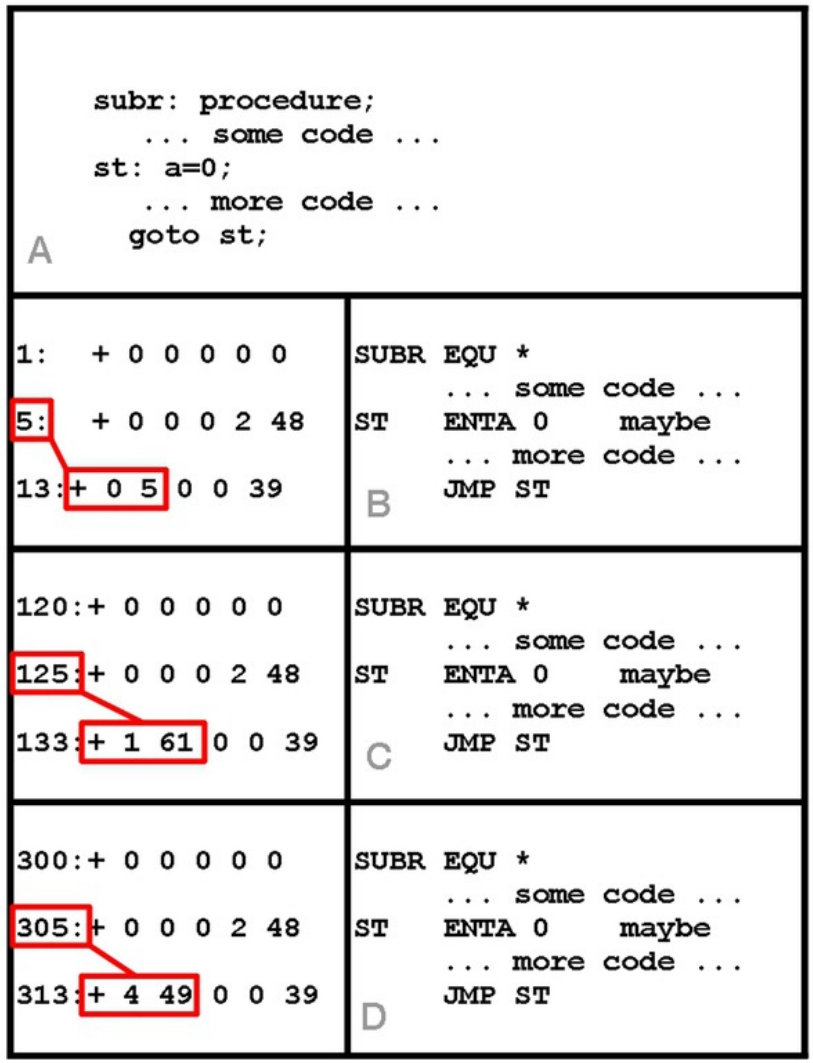
Relocatable Machine Code — *continued*

- (D) When the program is loaded into memory to run it may be loaded at some location other than the one assigned by the linker.
- This example shows `SUBR` now at location 300.



Relocatable Machine Code — *continued*

- The address in the jump instruction, now at 313, needs to be relocated again so that it points to the updated location of `ST`, 305.
- `4 49` is the MIX machine representation of 305.



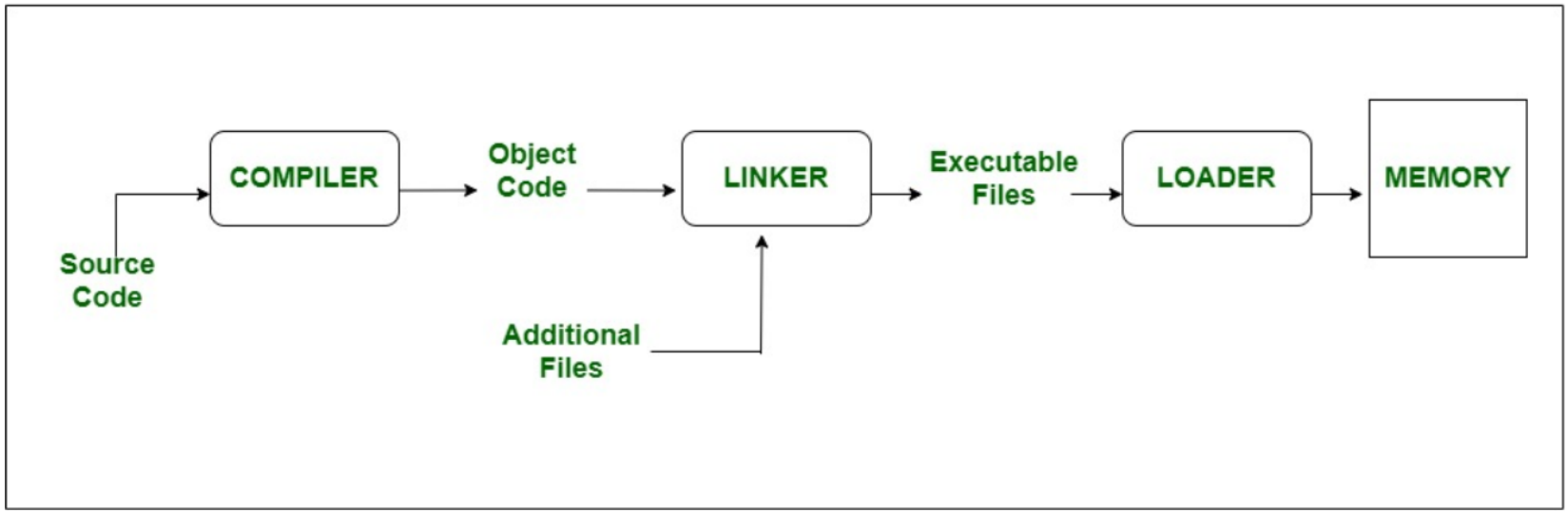
Difference between Linker and Loader

https://www.geeksforgeeks.org/difference-between-linker-and-loader/

Linker

- A linker is special program that combines the object files, generated by compiler/assembler, and other pieces of codes to originate an executable file.
- In the object file, linker searches and append all libraries needed for execution of file.
- It regulates memory space that code from each module will hold.
- It also merges two or more separate object programs and establishes link among them.

Difference between Linker and Loader — *continued*



Difference between Linker and Loader — *continued*

Loader

- The loader is special program that takes input of object code from linker, loads it to main memory, and prepares this code for execution by computer.
- Loader allocates memory space to program.
- Even it settles down symbolic reference between objects.
- It in charge of loading programs and libraries in operating system.

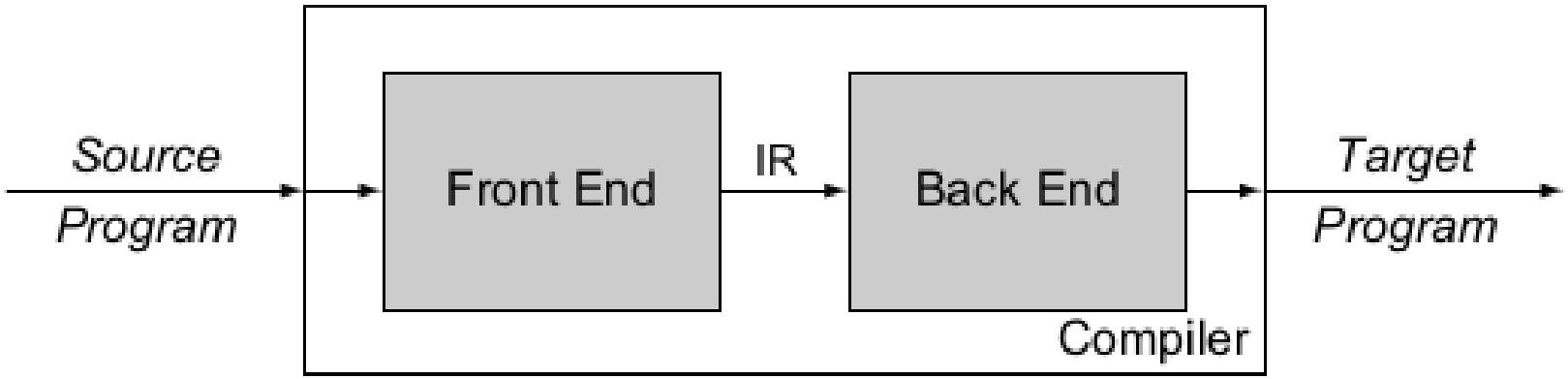
The Structure of a Compiler — *continued*

- To translate text from one language to another, the tool must understand both the form, or syntax, and content, or meaning, of the input language.
- It needs to understand the rules that govern syntax and meaning in the output language.
- Finally, it needs a scheme for mapping content from the source language to the target language.

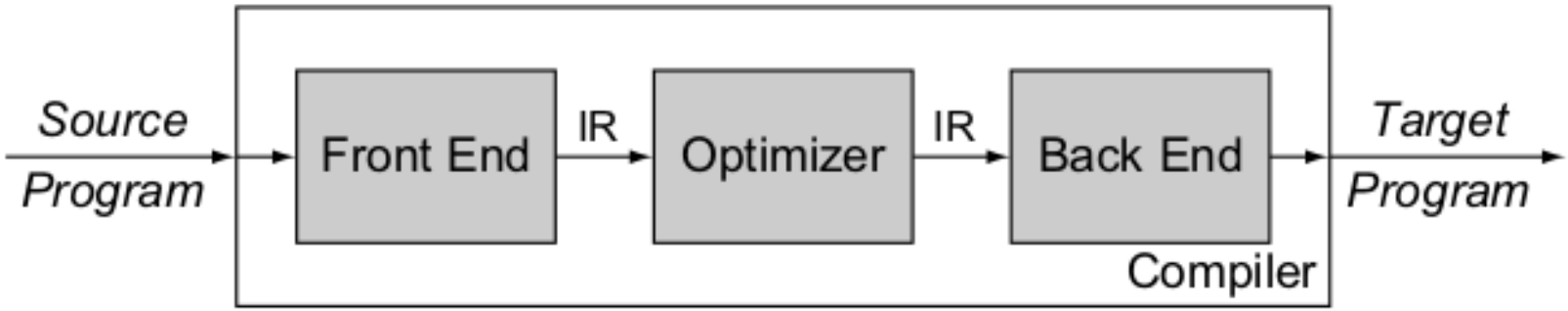
The Structure of a Compiler — *continued*

- The structure of a typical compiler derives from these simple observations.
- The compiler has a front end to deal with the source language.
- It has a back end to deal with the target language.
- Connecting the front end and the back end, it has a formal structure for representing the program in an intermediate form whose meaning is largely independent of either language.
- To improve the translation, a compiler often includes an optimizer that analyzes and rewrites that intermediate form.

The Structure of a Compiler — *continued*



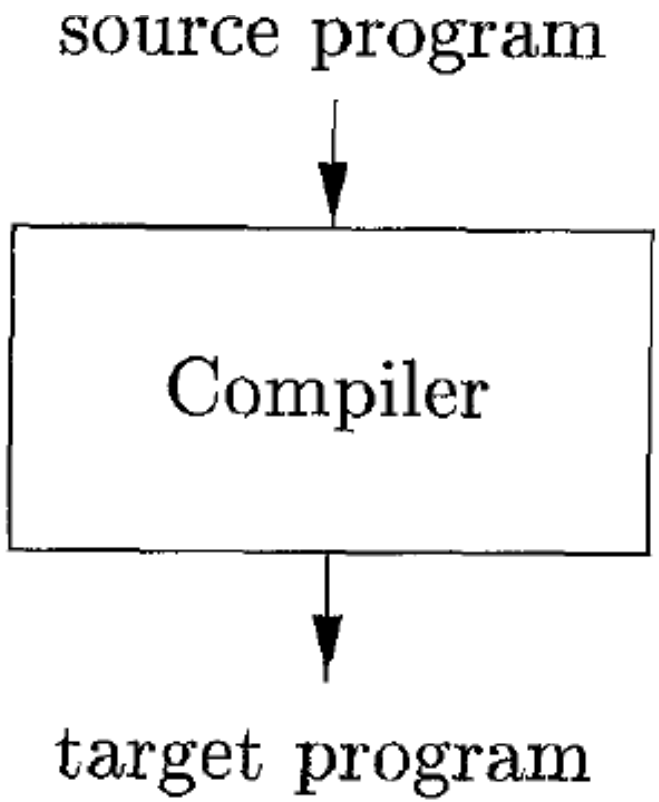
The Structure of a Compiler — *continued*



- What would happen if we didn't separate the Front End and Back End?

The Structure of a Compiler — *continued*

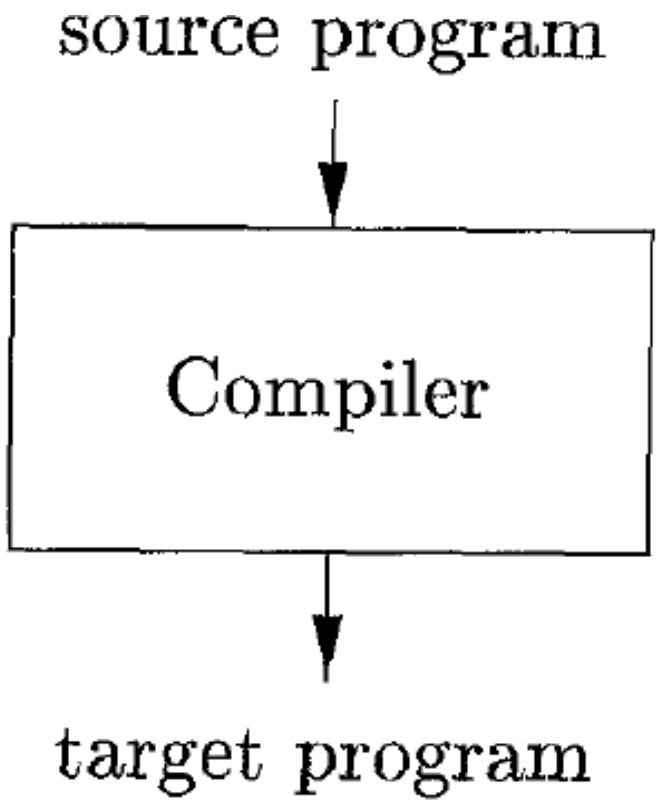
- Up to this point we have treated a compiler as a single box that maps a source program into a semantically equivalent target program.
- If we open up this box a little, we see that there are two parts to this mapping: analysis and synthesis.



A compiler

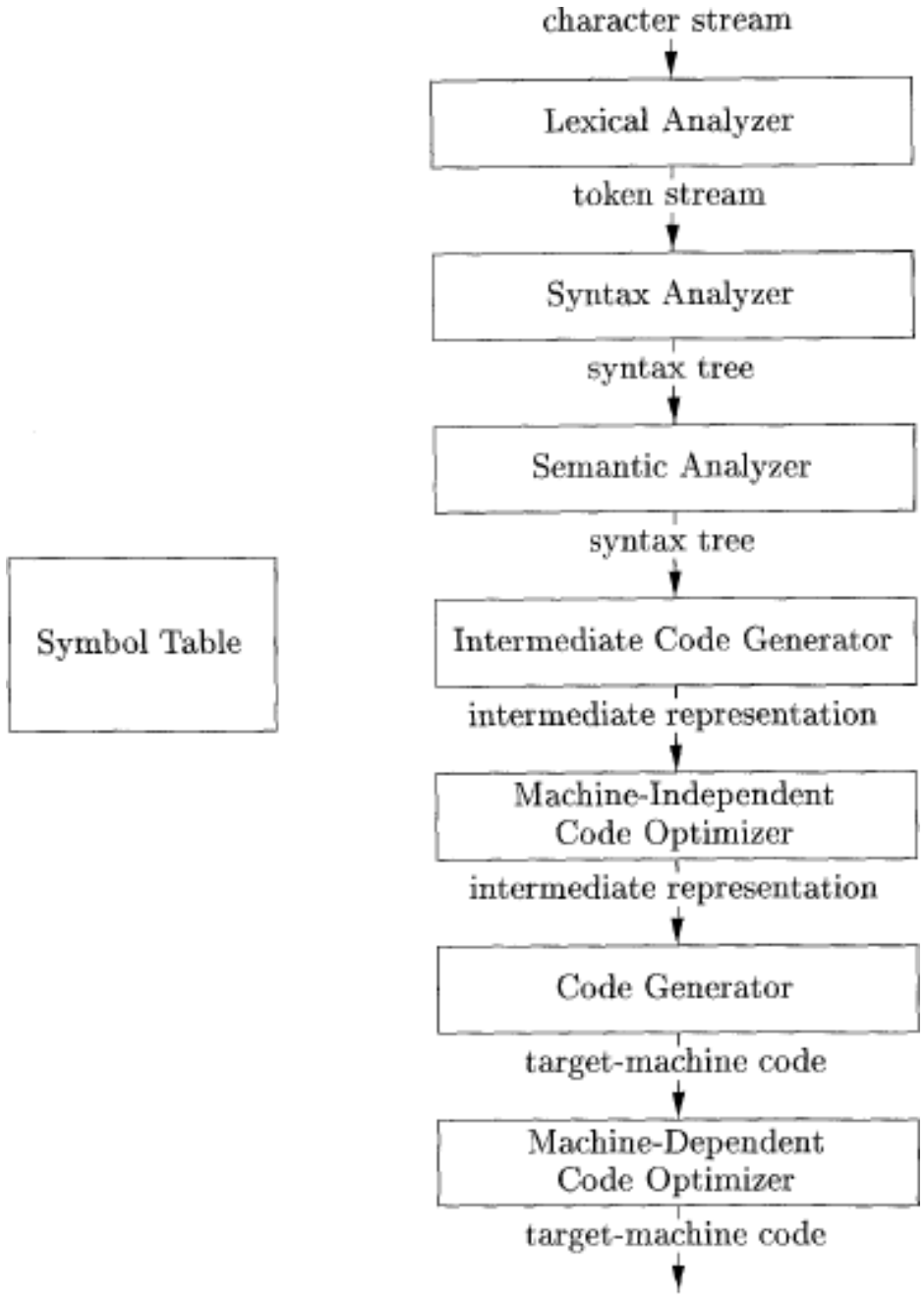
The Structure of a Compiler — *continued*

- Up to this point we have treated a compiler as a single box that maps a source program into a semantically equivalent target program.
- If we open up this box a little, we see that there are two parts to this mapping: analysis and synthesis.



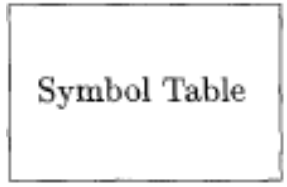
A compiler

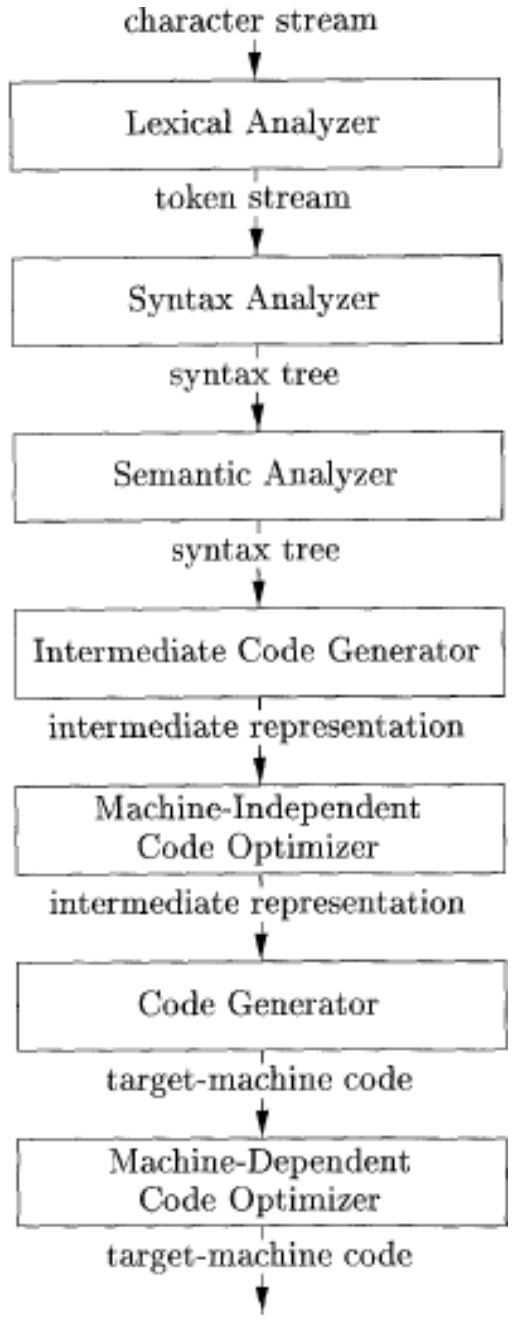
- The analysis part breaks up the source program into constituent pieces and imposes a grammatical structure on them.
- It then uses this structure to create an intermediate representation of the source program.
- If the analysis part detects that the source program is either syntactically ill formed or semantically unsound, then it must provide informative messages, so the user can take corrective action.



Phases of a compiler

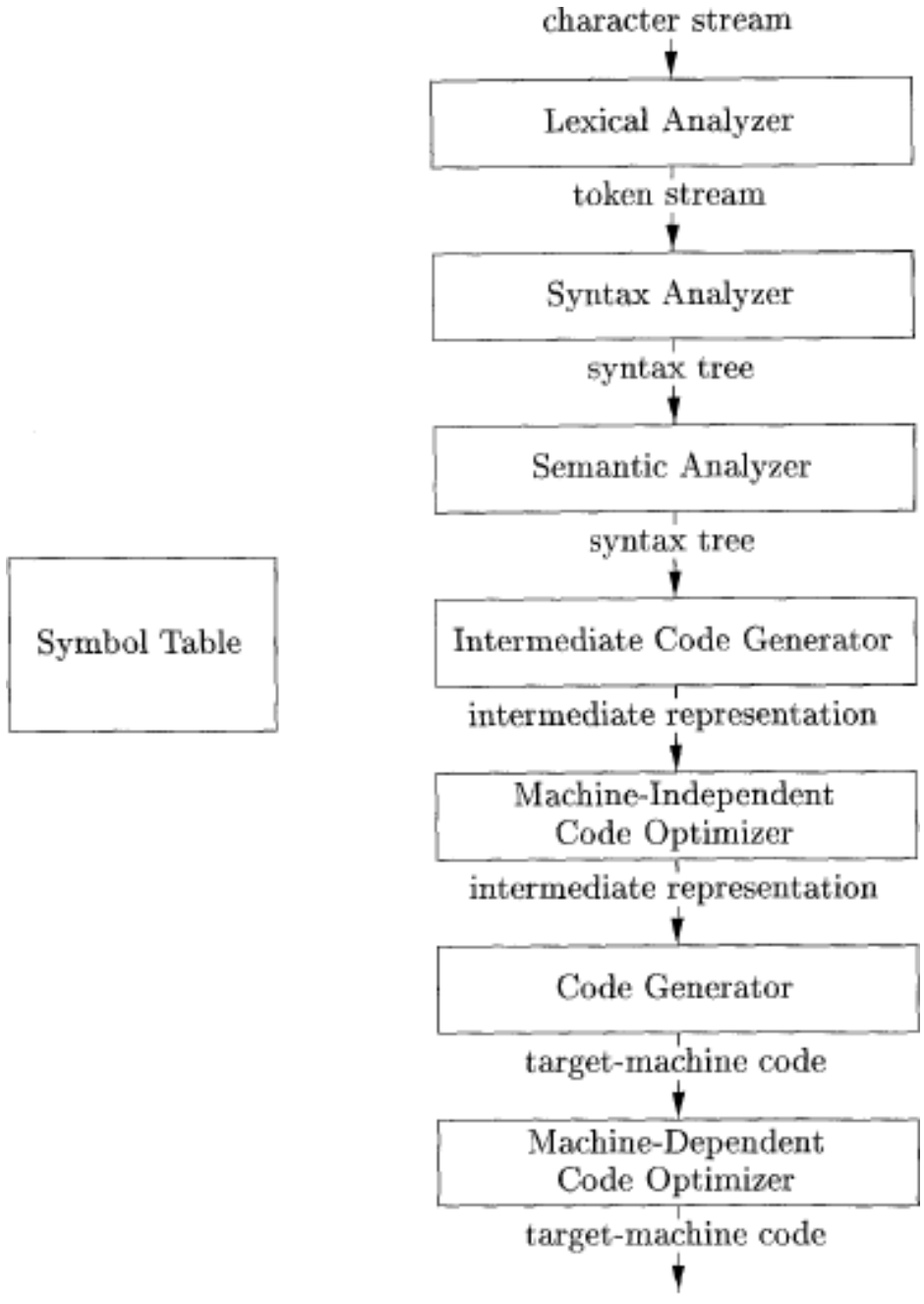
- The analysis part also collects information about the source program and stores it in a data structure called a symbol table, which is passed along with the intermediate representation to the synthesis part.
- The synthesis part constructs the desired target program from the intermediate representation and the information in the symbol table.
- The analysis part is often called the front end of the compiler; the synthesis part is the back end.





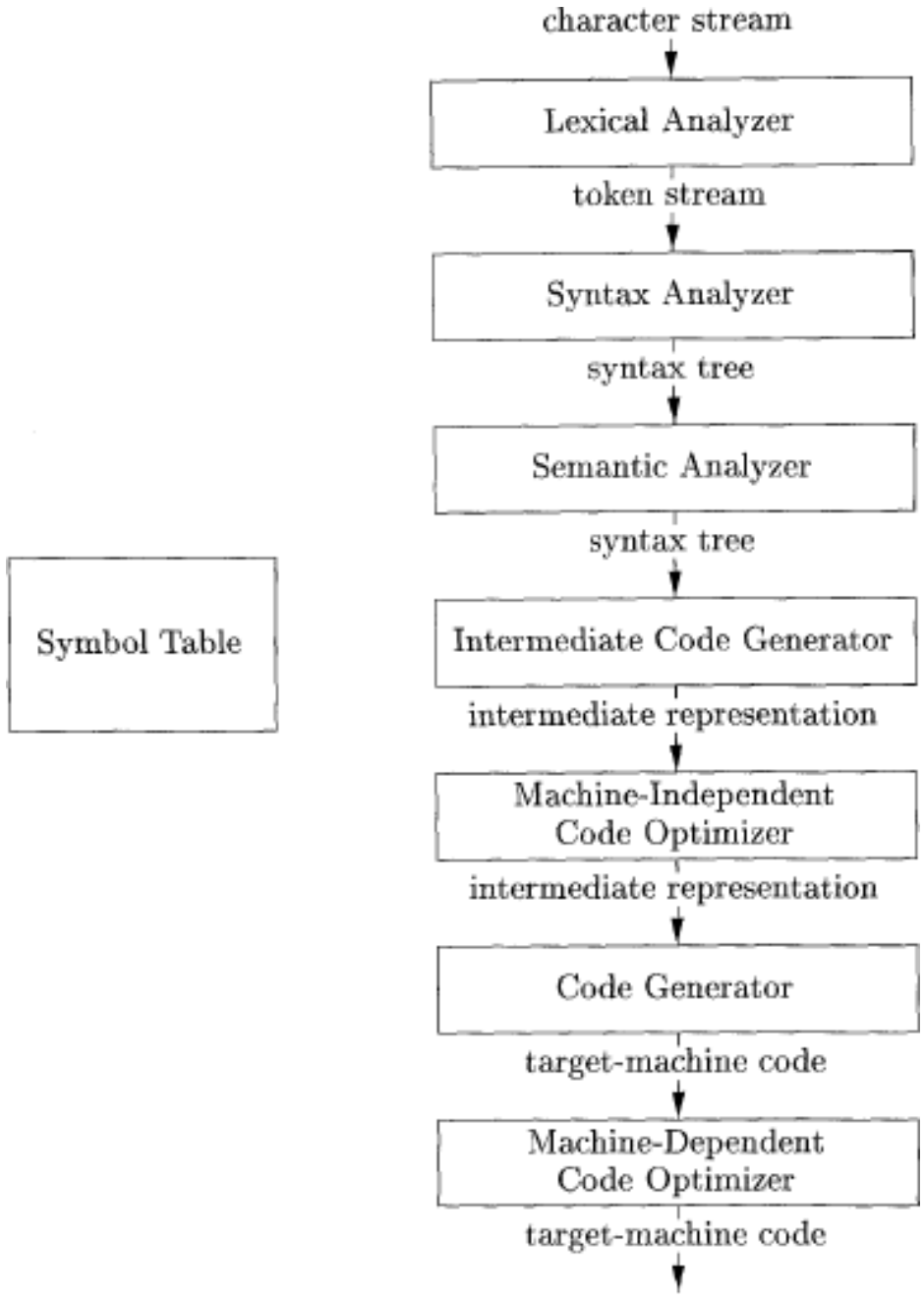
Phases of a compiler

- If we examine the compilation process in more detail, we see that it operates as a sequence of phases, each of which transforms one representation of the source program to another.
- In practice, several phases may be grouped together, and the intermediate representations between the grouped phases need not be constructed explicitly.



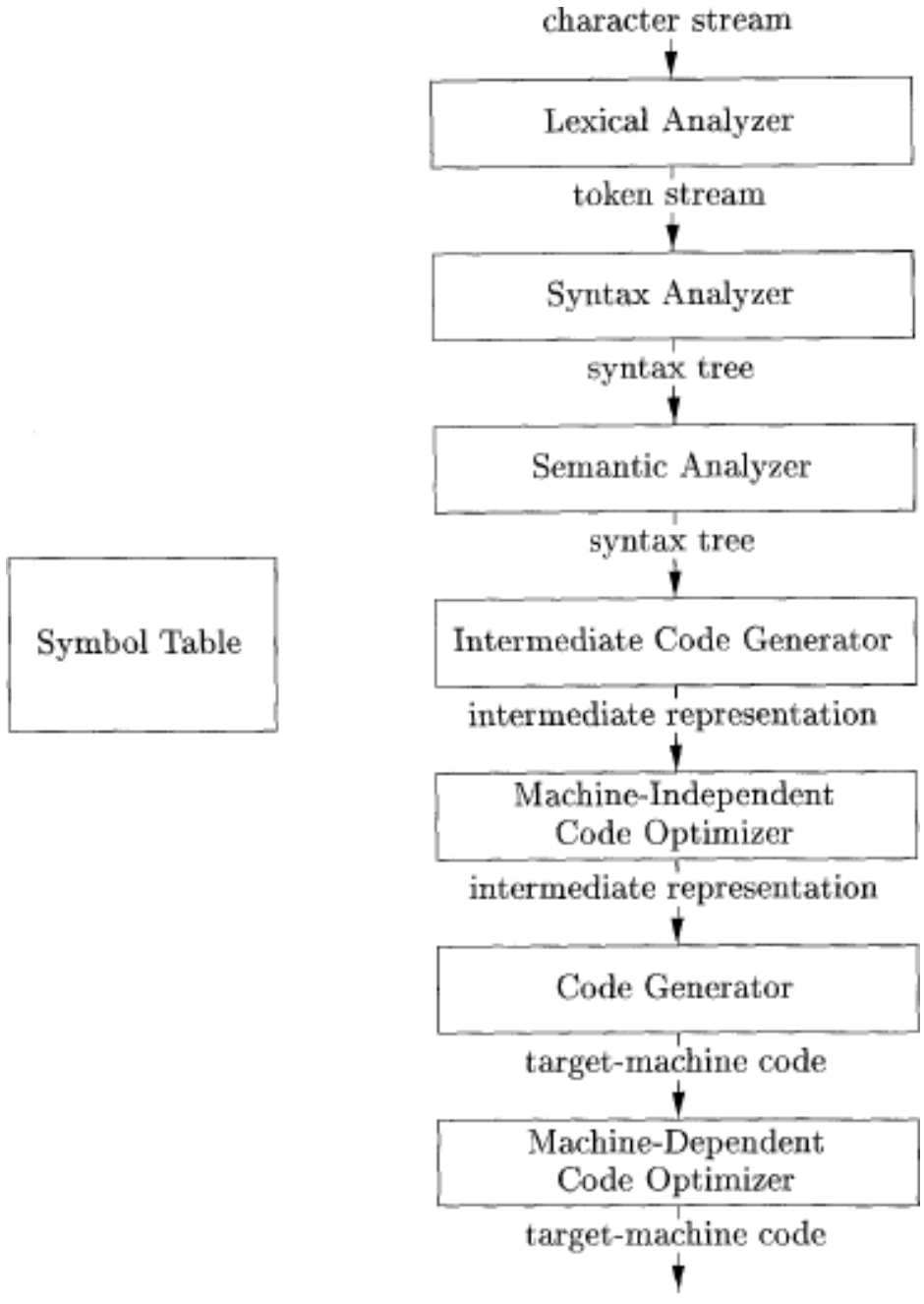
Phases of a compiler

- The symbol table, which stores information about the entire source program, is used by all phases of the compiler.



Phases of a compiler

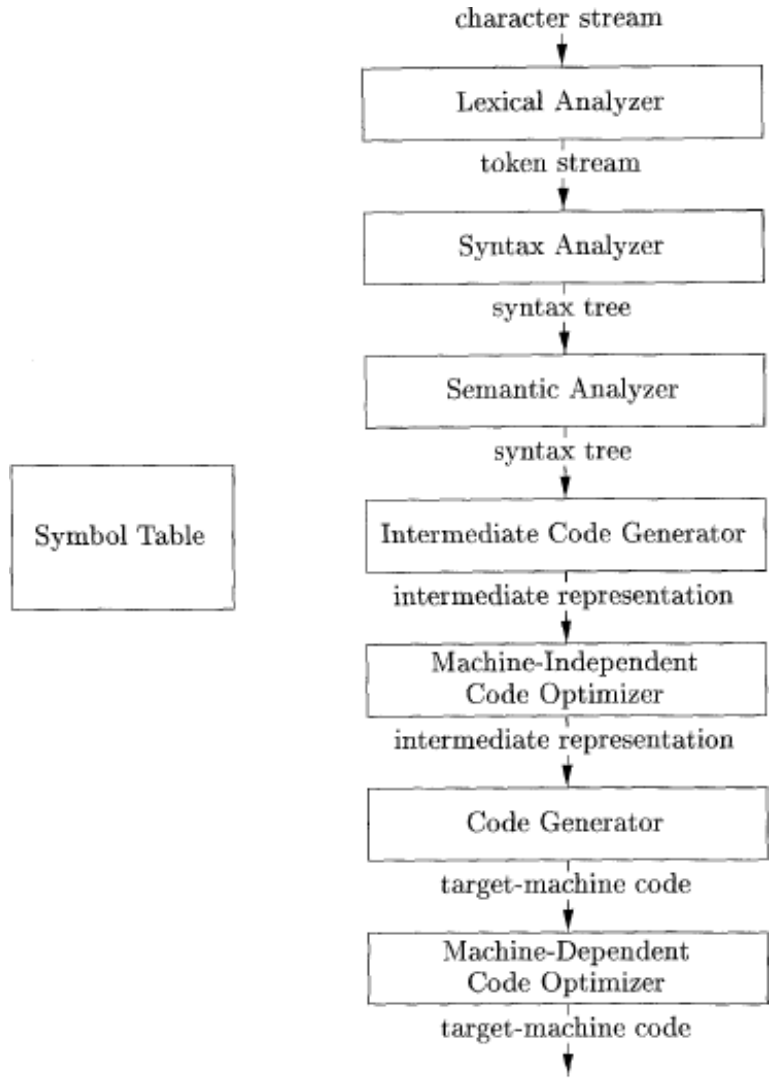
- Some compilers have a machine-independent optimization phase between the front end and the back end.
- The purpose of this optimization phase is to perform transformations on the intermediate representation, so that the back end can produce a better target program than it would have otherwise produced from an unoptimized intermediate representation.
- Since optimization is optional, one or the other of the two optimization phases shown may be missing.



Phases of a compiler

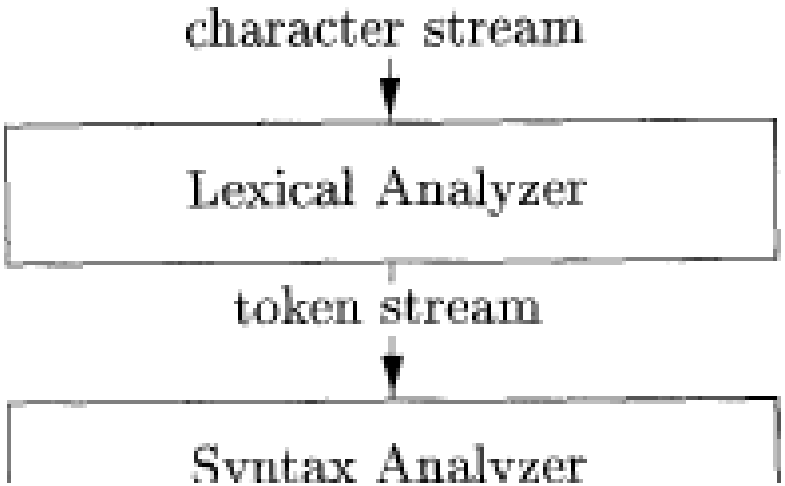
Lexical Analysis

- The first phase of a compiler is called lexical analysis or scanning.



Phases of a compiler

Lexical Analysis — *continued*



- The lexical analyzer reads the stream of characters making up the source program and groups the characters into meaningful sequences called lexemes.

Lexical Analysis — *continued*

- For each lexeme, the lexical analyzer produces as output a token of the form

  $$<\textit{token-name, attribute-value}>$$

  that it passes on to the subsequent phase, syntax analysis.
- In the token, the first component token-name is an abstract symbol that is used during syntax analysis, and the second component attribute-value points to an entry in the symbol table for this token.
- Information from the symbol-table entry is needed for semantic analysis and code generation.

Lexical Analysis — *continued*

$$\langle \textit{token-name}, \textit{attribute-value} \rangle$$

$$\texttt{position} = \texttt{initial} + \texttt{rate} * 60$$

- We consider a source program that contains the assignment statement
  `position = initial + rate * 60`
- The characters in this assignment could be grouped into the lexemes and mapped into the tokens passed on to the syntax analyzer

Lexical Analysis — *continued*

$$\langle \textit{token-name}, \textit{attribute-value} \rangle$$

$$\texttt{position} = \texttt{initial} + \texttt{rate} * 60$$

1. `position` is a lexeme that would be mapped into a token $\langle \textbf{id}, 1 \rangle$.
   - **id** is an abstract symbol standing for identifier,
   - 1 points to the symbol-table entry for `position`,
   - The symbol-table entry for an identifier holds information about the identifier, such as its name and type.

Lexical Analysis — *continued*

$$\langle \textit{token-name}, \textit{attribute-value} \rangle$$

$$\texttt{position} = \texttt{initial} + \texttt{rate} * 60$$

2. The assignment symbol = is a lexeme that is mapped into the token $\langle = \rangle$.
   - Since this token needs no attribute-value, we have omitted the second component.
   - We could have used any abstract symbol such as assign for the token-name, but for notational convenience we have chosen to use the lexeme itself as the name of the abstract symbol.

Lexical Analysis — *continued*

$$\langle \textit{token-name}, \textit{attribute-value} \rangle$$

$$\texttt{position} = \texttt{initial} + \texttt{rate} * 60$$

3. `initial` is a lexeme that is mapped into the token $\langle \textbf{id}, 2 \rangle$,
   - 2 points to the symbol-table entry for initial.

Lexical Analysis — *continued*

$$\langle \textit{token-name}, \textit{attribute-value} \rangle$$

$$\texttt{position} = \texttt{initial} + \texttt{rate} * 60$$

4. $+$ is a lexeme that is mapped into the token $\langle + \rangle$.

Lexical Analysis — *continued*

$\langle$*token-name*, *attribute-value*$\rangle$

`position = initial + rate * 60`

5. `rate` is a lexeme that is mapped into the token $\langle$**id**, $3\rangle$, where 3 points to the symbol-table entry for `rate`.

Lexical Analysis — *continued*

$$\langle \textit{token-name}, \textit{attribute-value} \rangle$$

$$\texttt{position} = \texttt{initial} + \texttt{rate} * 60$$

6. $*$ is a lexeme that is mapped into the token $\langle * \rangle$.

Lexical Analysis — *continued*

$$\langle \textit{token-name}, \textit{attribute-value} \rangle$$

$$\texttt{position} = \texttt{initial} + \texttt{rate} * \texttt{60}$$

7. `60` is a lexeme that is mapped into the token $\langle 60 \rangle$.

Lexical Analysis — *continued*

$$\langle \textit{token-name}, \textit{attribute-value} \rangle$$

$$\texttt{position} = \texttt{initial} + \texttt{rate} * 60$$

- Blanks separating the lexemes would be discarded by the lexical analyzer.

Lexical Analysis — *continued*

`position = initial + rate * 60`

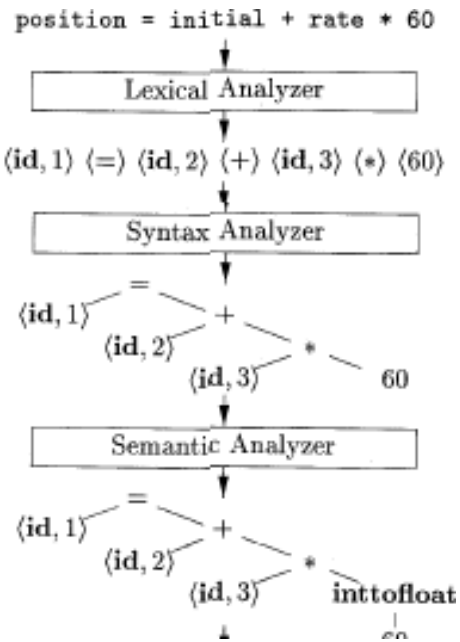
- Figure shows the representation of the assignment statement after lexical analysis as the sequence of tokens,

$$\langle \textbf{id}, 1\rangle \langle = \rangle \langle \textbf{id}, 2\rangle \langle + \rangle \langle \textbf{id}, 3\rangle \langle * \rangle \langle 60 \rangle$$

- In this representation, the token names $=$, $+$, and $*$ are abstract symbols for the assignment, addition, and multiplication operators, respectively.

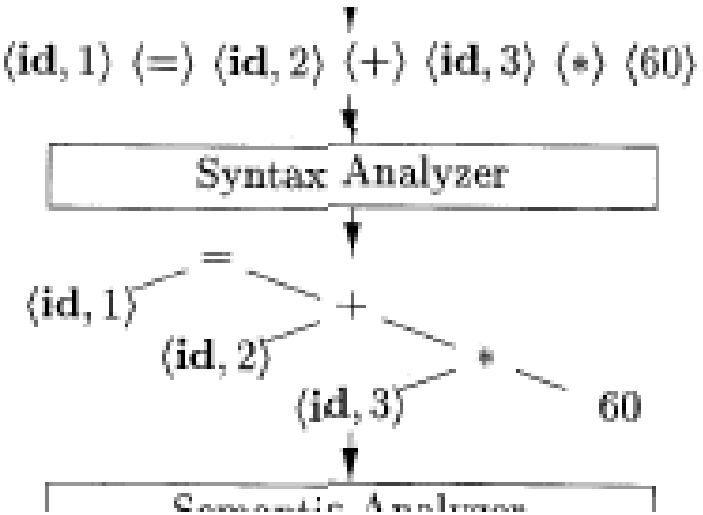
1	position	···
2	initial	···
3	rate	···

SYMBOL TABLE



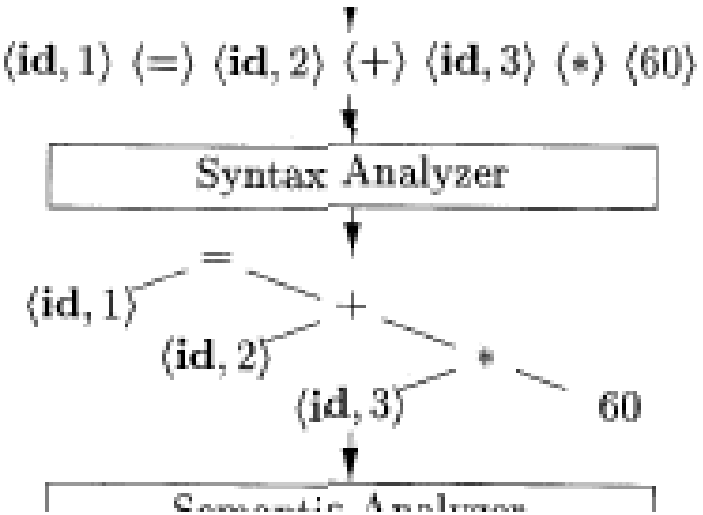
Syntax Analysis

- The second phase of the compiler is syntax analysis or parsing.
- The parser uses the first components of the tokens produced by the lexical analyzer to create a tree-like intermediate representation that depicts the grammatical structure of the token stream.



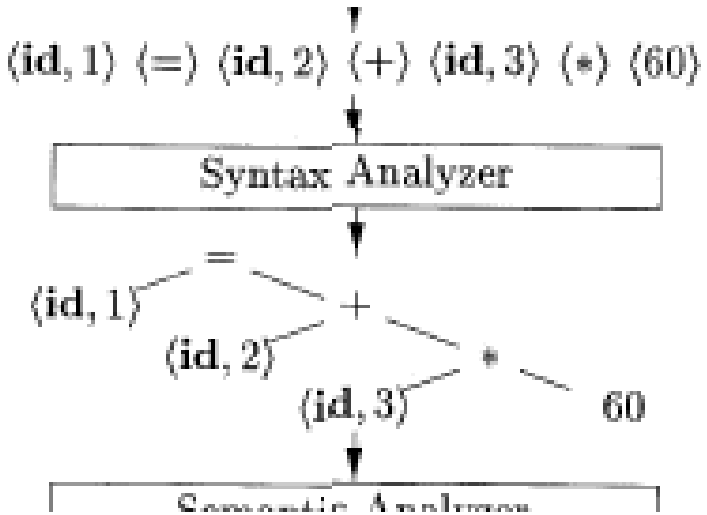
Syntax Analysis

- A typical representation is a syntax tree in which each interior node represents an operation and the children of the node represent the arguments of the operation.



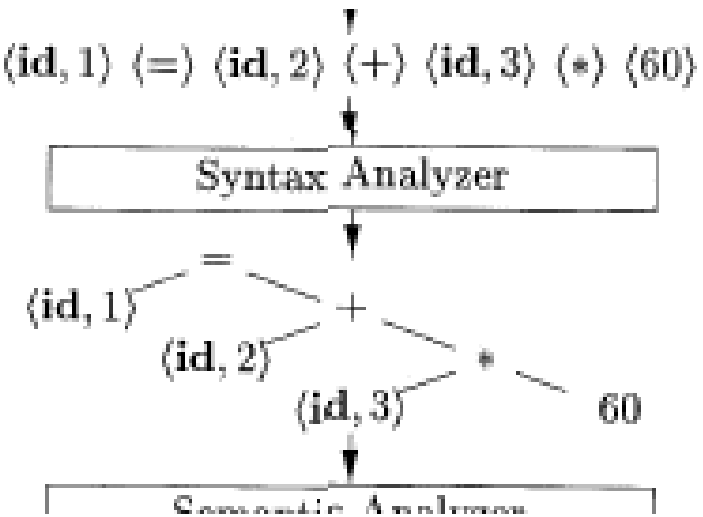
Syntax Analysis

- This tree shows the order in which the operations in the assignment
  `position = initial + rate * 60`
  are to be performed.



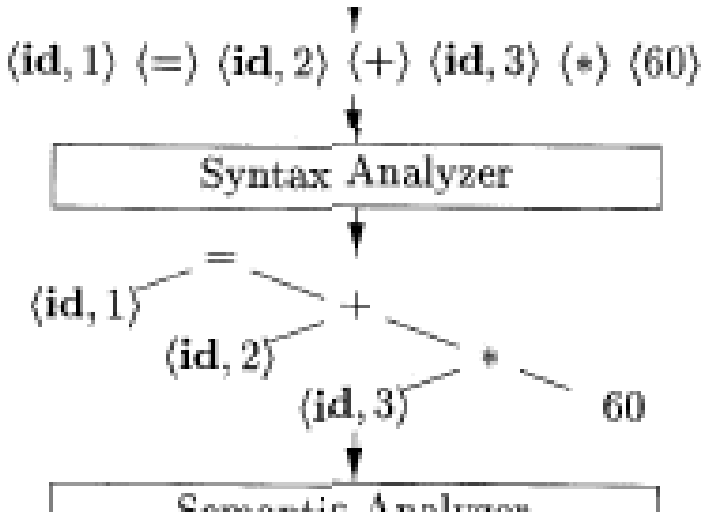
Syntax Analysis

- The tree has an interior node labeled $*$ with $\langle \textbf{id}, 3 \rangle$ as its left child and the integer 60 as its right child. The node $\langle \textbf{id}, 3 \rangle$ represents the identifier `rate`.
- The node labeled $*$ makes it explicit that we must first multiply the value of `rate` by `60`.

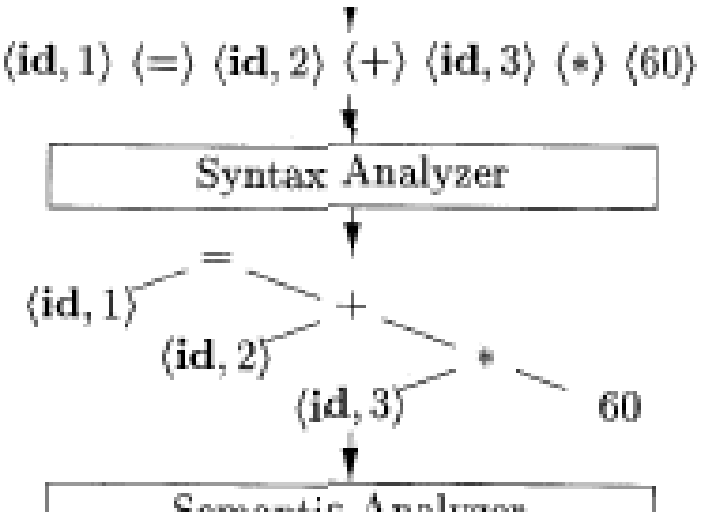


Syntax Analysis

- The node labeled + indicates that we must add the result of this multiplication to the value of `initial`.

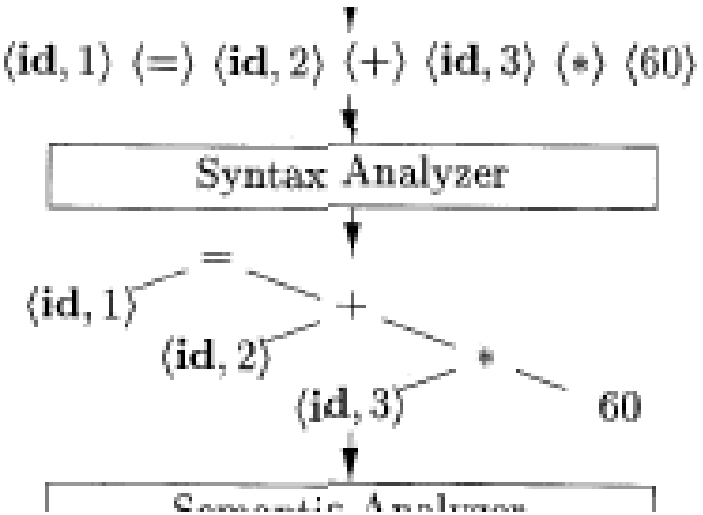


- The root of the tree, labeled =, indicates that we must store the result of this addition into the location for the identifier `position`.



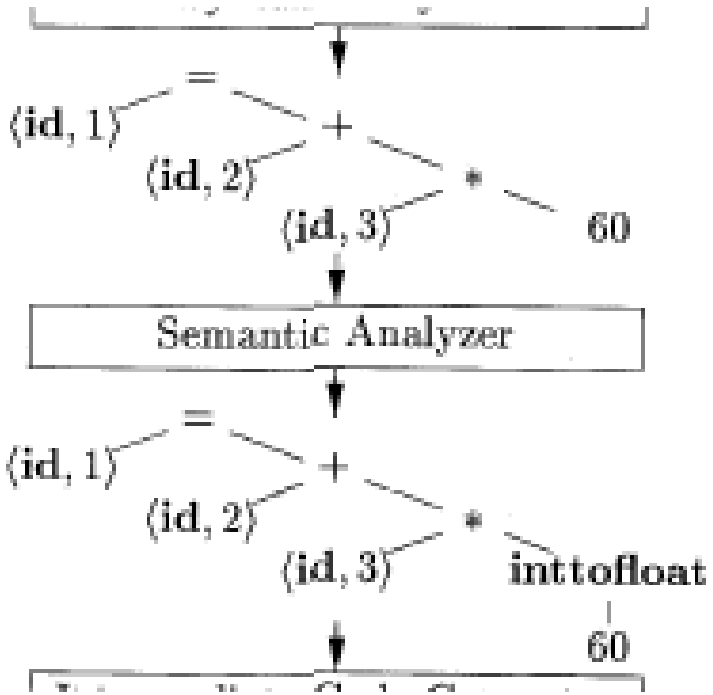
Syntax Analysis

- This ordering of operations is consistent with the usual conventions of arithmetic which tell us that multiplication has higher precedence than addition, and hence that the multiplication is to be performed before the addition.



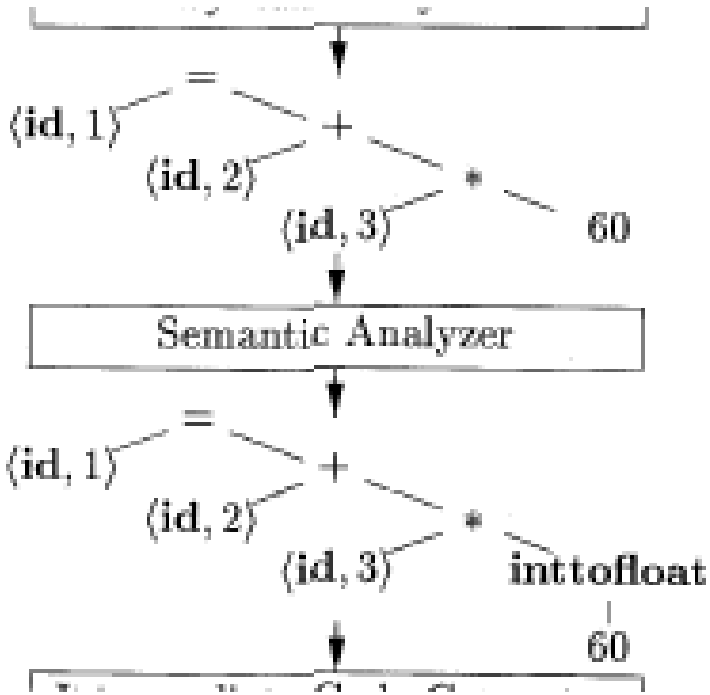
Semantic Analysis

- The semantic analyzer uses the syntax tree and the information in the symbol table to check the source program for semantic consistency with the language definition.
- It also gathers type information and saves it in either the syntax tree or the symbol table, for subsequent use during intermediate-code generation.



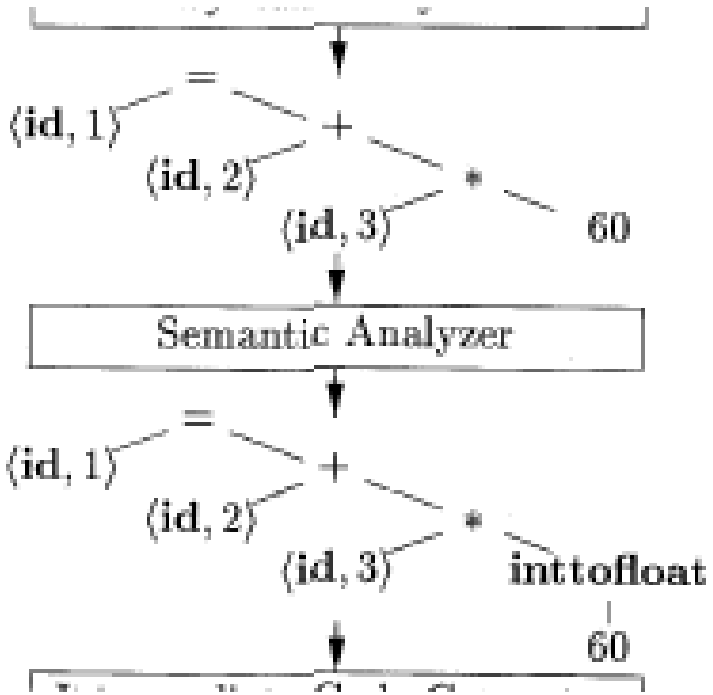
Semantic Analysis

- An important part of semantic analysis is type checking, where the compiler checks that each operator has matching operands.
- For example, many programming language definitions require an array index to be an integer.
- The compiler must report an error if a floating-point number is used to index an array.



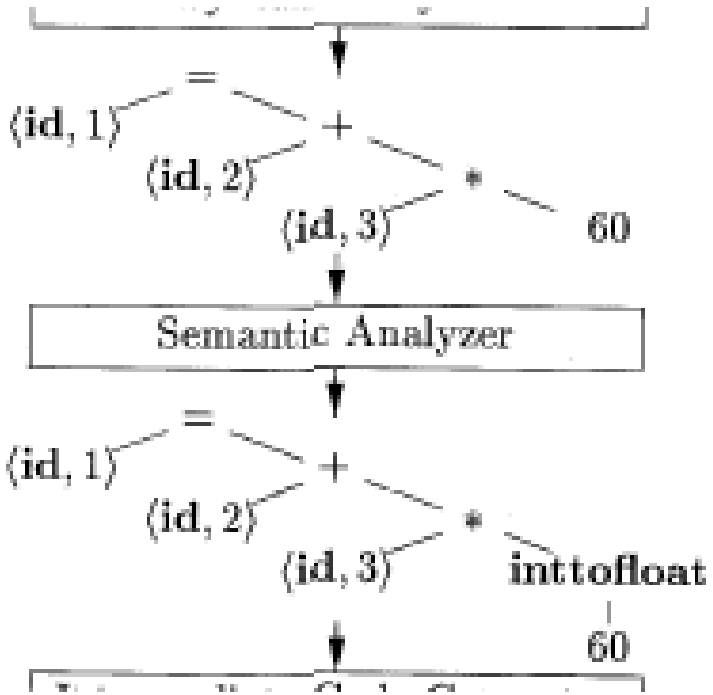
Semantic Analysis

- The language specification may permit some type conversions called coercions.
- For example, a binary arithmetic operator may be applied to either a pair of integers or to a pair of floating-point numbers.
- If the operator is applied to a floating-point number and an integer, the compiler may convert or coerce the integer into a floating-point number.



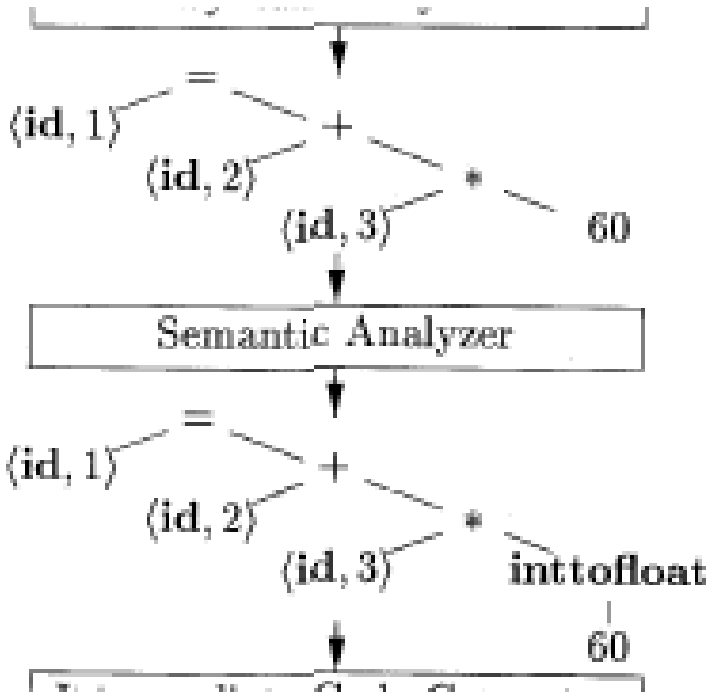
Semantic Analysis

- Suppose that `position`, `initial`, and `rate` have been declared to be floating-point numbers, and that the lexeme 60 by itself forms an integer.
- The type checker in the semantic analyzer discovers that the operator $*$ is applied to a floating-point number rate and an integer 60.
- In this case, the integer may be converted into a floating-point number.



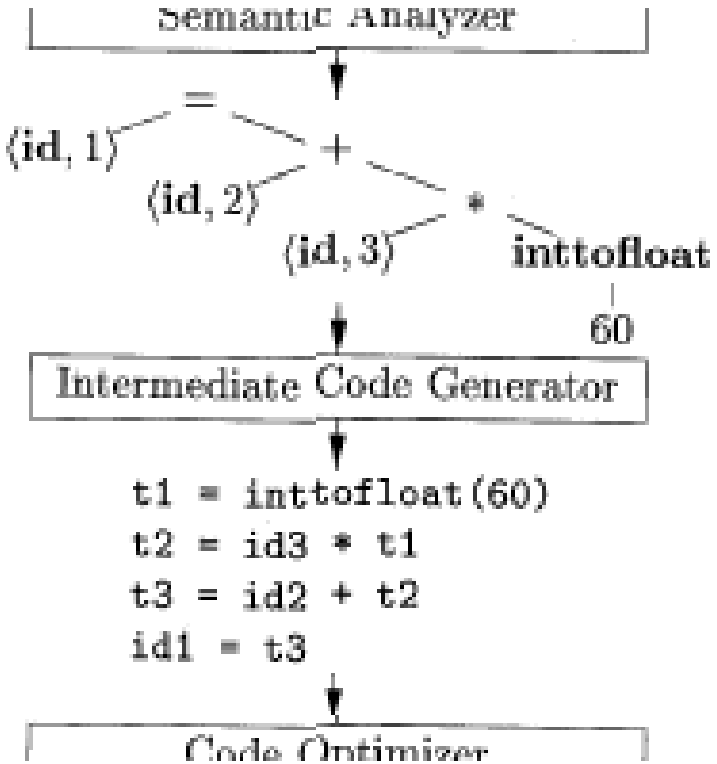
Semantic Analysis

- Notice that the output of the semantic analyzer has an extra node for the operator `inttofloat`, which explicitly converts its integer argument into a floating-point number.



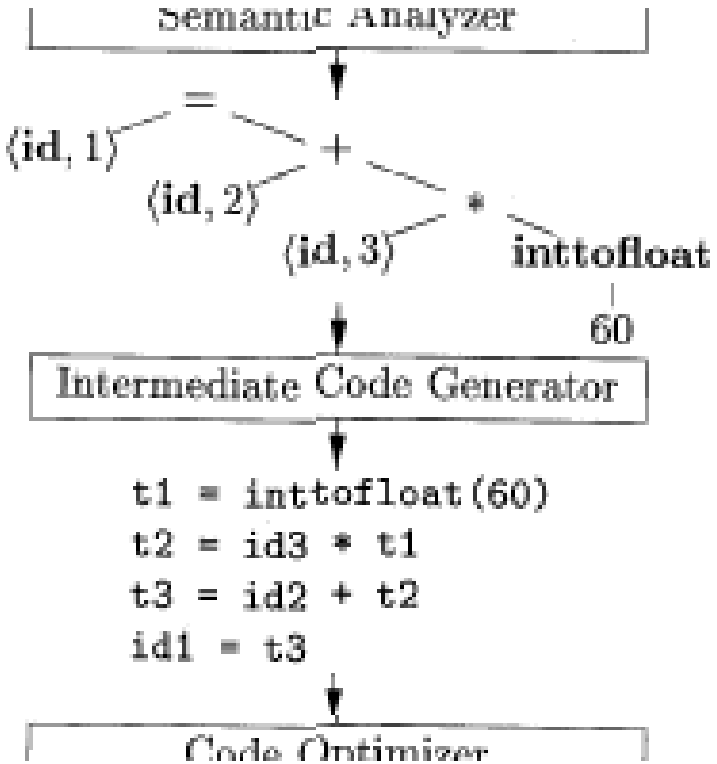
Intermediate Code Generation

- In the process of translating a source program into target code, a compiler may construct one or more intermediate representations, which can have a variety of forms.
- Syntax trees are a form of intermediate representation.
- They are commonly used during syntax and semantic analysis.



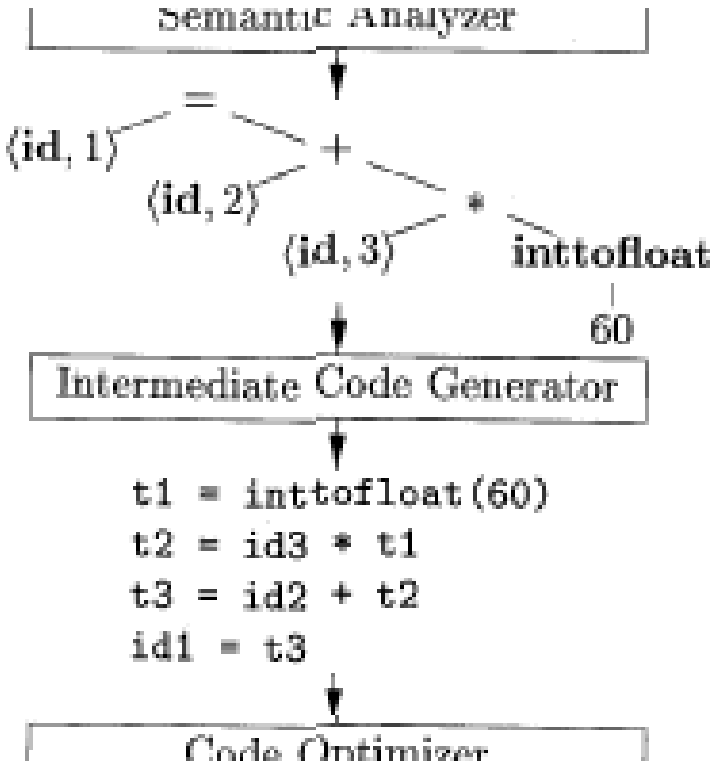
Intermediate Code Generation

- After syntax and semantic analysis of the source program, many compilers generate an explicit low-level or machine-like intermediate representation, which we can think of as a program for an abstract machine.
- This intermediate representation should have two important properties:
  - it should be easy to produce and
  - it should be easy to translate into the target machine.



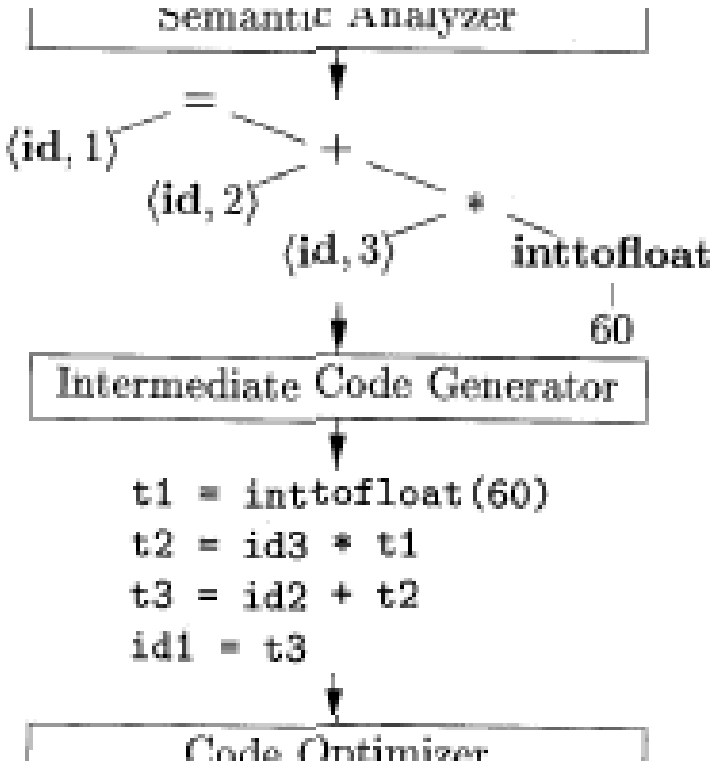
Intermediate Code Generation

- We consider an intermediate form called “three-address code”.
- Like the assembly language for a machine in which every memory location can act like a register.
- Three-address code consists of a sequence of instructions, each of which has at most three operands.



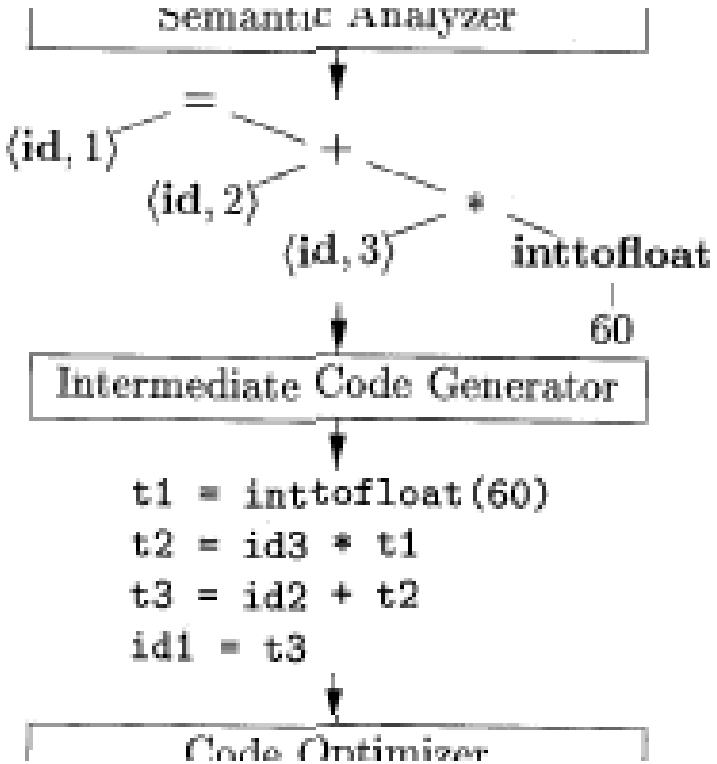
Intermediate Code Generation

```
t1 = inttofloat(60)
t2 = id3 * t1
t3 = id2 + t2
id1 = t3
```



Intermediate Code Generation

- Each three-address instruction has at most one operator in addition to the assignment.
  - Has to decide on the order in which operations are to be done.
  - Multiplication precedes the addition in the source program.
- Must generate a temporary name to hold the value computed by each instruction.
- Some “three-address” instructions have fewer than three operands.



Intermediate Codes in Modern Compilers

- In real modern compilers, intermediate code often appears in more advanced forms.
- Examples include SSA form and LLVM IR.
- Three-address code is still relevant because it teaches the core idea of intermediate representation.
- It shows how complex source programs can be represented using simpler compiler-friendly instructions.

Examples of Intermediate Representations

- LLVM IR
- GIMPLE in GCC
- MIR in the Rust compiler
- Java bytecode
- .NET IL
- SSA-based IR

- These representations are usually richer than basic three-address code.

Compiler IR: Independence Properties

Machine independent It is not tied to a specific CPU such as x86 or ARM.

Language independent Different source languages can be translated into a similar IR.

Can represent constructs Assignments, expressions, loops, conditionals, function calls, array access, and other constructs can be represented.

Compiler IR: Simple Instructions

- Complex expressions are broken into small operations.
- Each instruction usually has at most one operator.
- Temporary variables store intermediate results.

Example

```
t1 = a + b
t2 = t1 * c
x  = t2
```

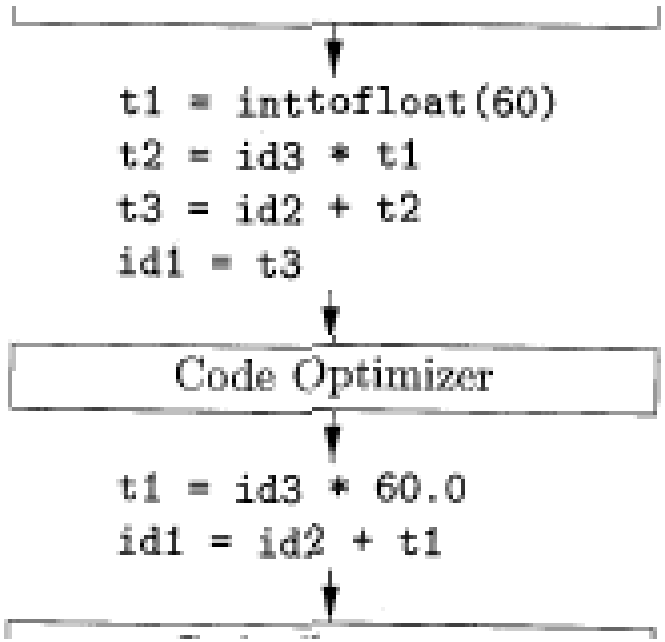
- This is preferred over a complex expression such as `x = (a + b) * c`.

Compiler IR: Control Flow and Optimization

- Branches, jumps, and labels are represented explicitly.
- This makes the control flow of the program clearer to the compiler.
- Optimizations become easier, for example:
  - dead code elimination,
  - constant folding,
  - common subexpression elimination.
- Since the instructions are simple, translating IR to machine code also becomes easier.

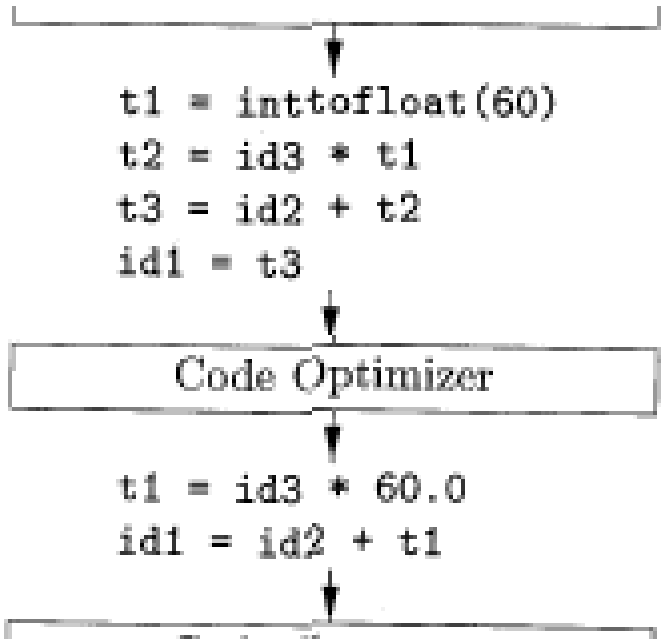
Code Optimization

- The machine-independent code-optimization phase attempts to improve the intermediate code so that better target code will result.
- Usually better means faster, but other objectives may be desired, such as shorter code, or target code that consumes less power.



Code Optimization

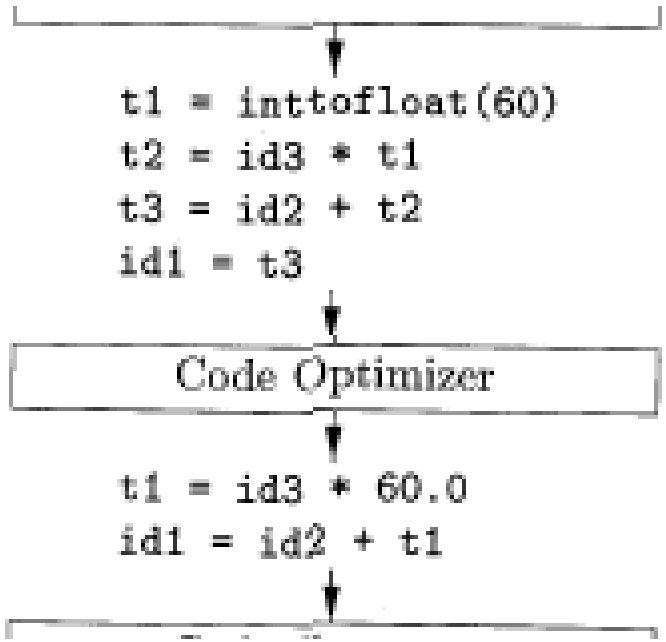
- A simple intermediate code generation algorithm followed by code optimization is a reasonable way to generate good target code.
- The optimizer can deduce that the conversion of 60 from integer to floating point can be done once and for all at compile time.
- So the `inttofloat` operation can be eliminated by replacing the integer 60 by the floating-point number 60.0.
- Moreover, `t3` is used only once to transmit its value to $\langle \textbf{id}, 1 \rangle$.



Code Optimization

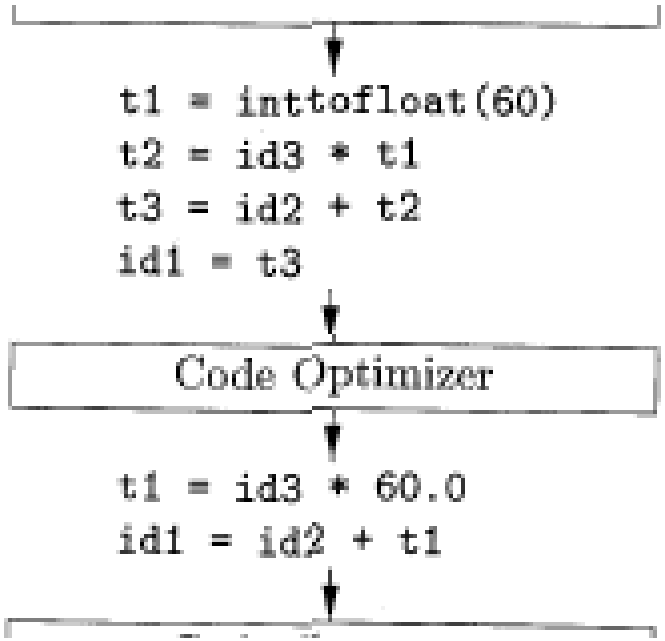
- So the optimizer can transform the code into the shorter sequence,

```
t1 = id3 * 60.0
id1 = id2 + t1
```



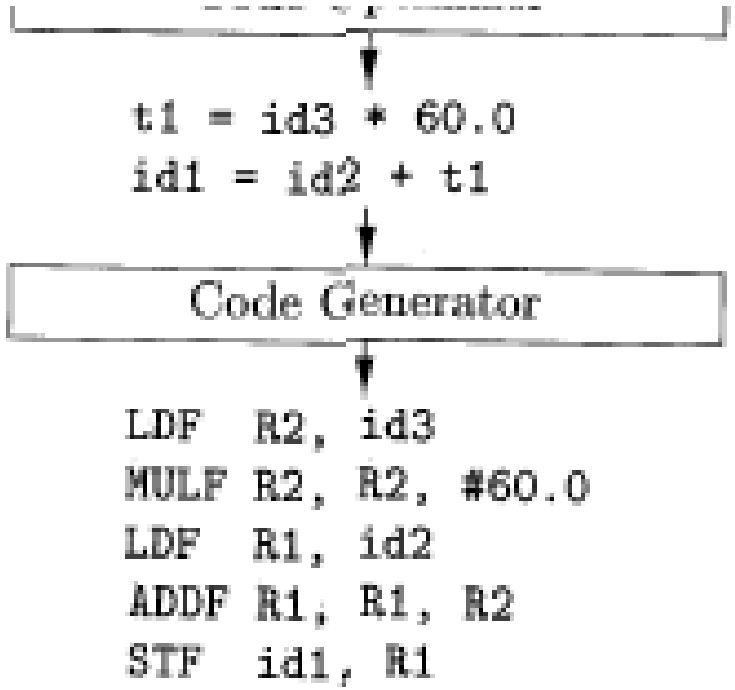
Code Optimization

- There is a great variation in the amount of code optimization different compilers perform.
- In those that do the most, the so-called "optimizing compilers," a significant amount of time is spent on this phase.
- There are simple optimizations that significantly improve the running time of the target program without slowing down compilation too much.



Code Generation

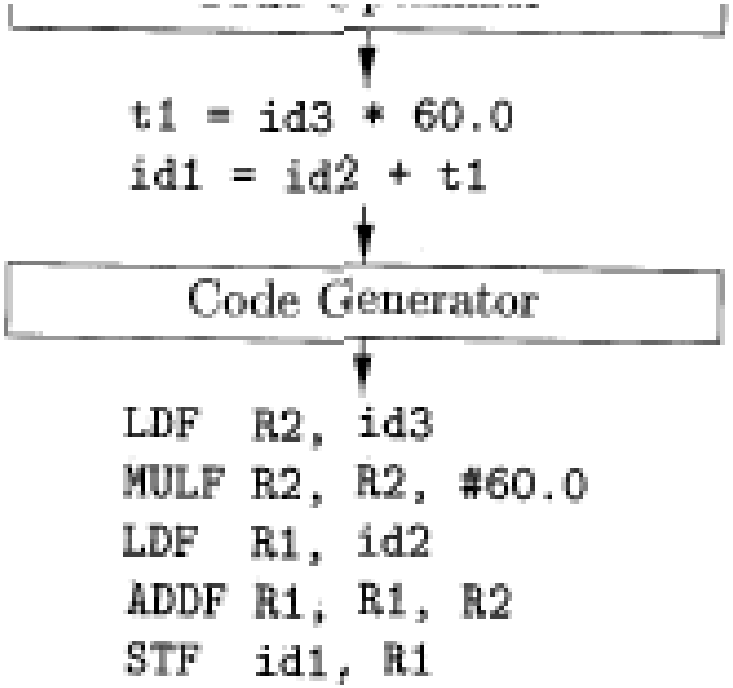
- The code generator takes as input an intermediate representation of the source program and maps it into the target language.
- If the target language is machine code, registers or memory locations are selected for each of the variables used by the program.
- Then, the intermediate instructions are translated into sequences of machine instructions that perform the same task.
- A crucial aspect of code generation is the judicious assignment of registers to hold variables.



Code Generation

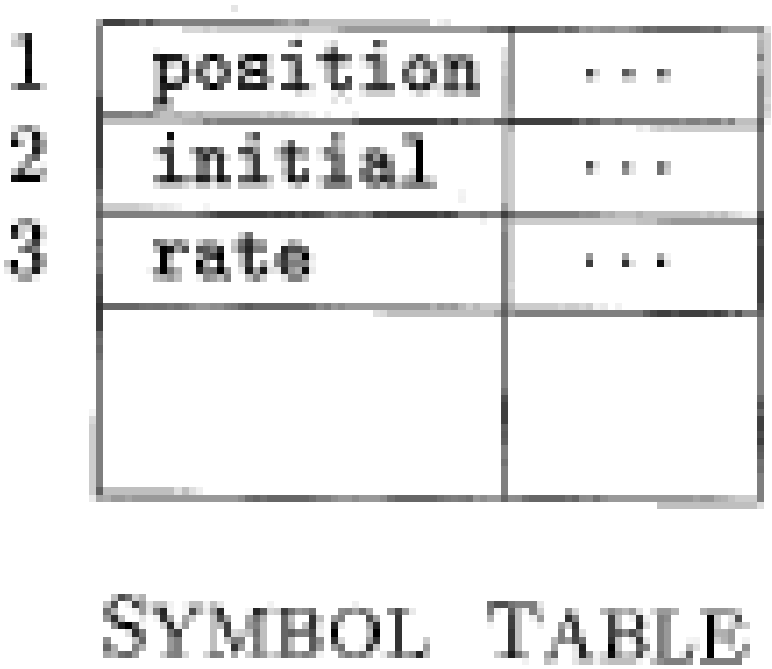
- For example, using registers `R1` and `R2`, the intermediate code might get translated into the machine code

```
LDF R2, id3
MULF R2, R2, #60.0
LDF R1, id2
ADDF R1, R1, R2
STF id1, R1
```



Symbol-Table Management

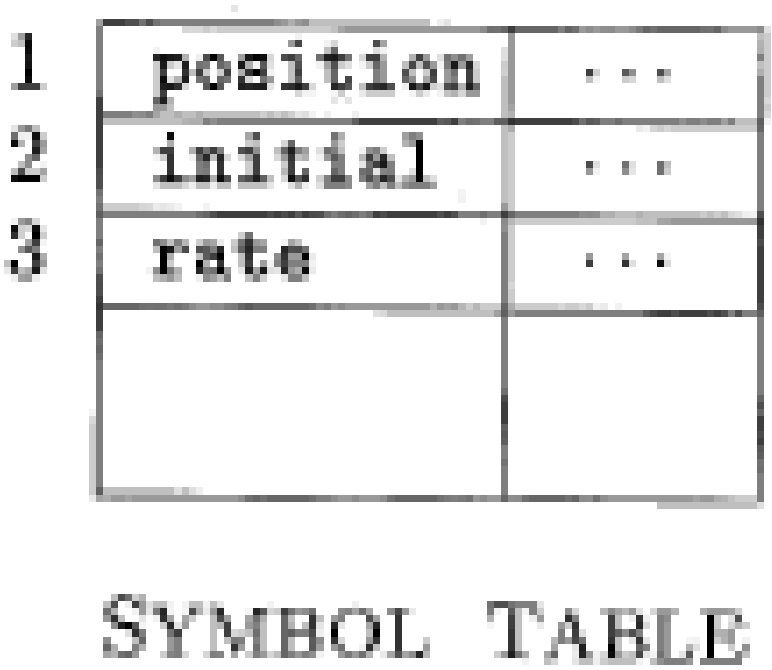
- An essential function of a compiler is to record the variable names used in the source program and collect information about various attributes of each name.



SYMBOL TABLE

Symbol-Table Management

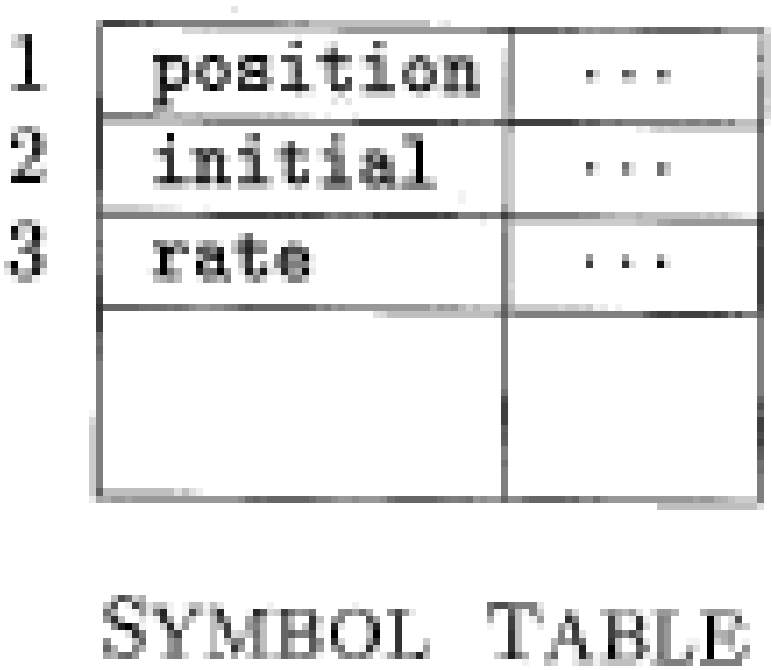
- These attributes may provide information about the storage allocated for a name, its type, its scope (where in the program its value may be used)
- In the case of procedure names, such things as the number and types of its arguments, the method of passing each argument (for example, by value or by reference), and the type returned.



SYMBOL TABLE

Symbol-Table Management

- The symbol table is a data structure containing a record for each variable name, with fields for the attributes of the name.
- The data structure should be designed to allow the compiler to find the record for each name quickly and to store or retrieve data from that record quickly.



SYMBOL TABLE

The Grouping of Phases into Passes

- The discussion of phases deals with the logical organization of a compiler.
- In an implementation, activities from several phases may be grouped together into a pass that reads an input file and writes an output file.
- For example, the front-end phases of lexical analysis, syntax analysis, semantic analysis, and intermediate code generation might be grouped together into one pass.
- Code optimization might be an optional pass.
- Then there could be a back-end pass consisting of code generation for a particular target machine.

The Grouping of Phases into Passes

- The discussion of phases deals with the logical organization of a compiler.
- In an implementation, activities from several phases may be grouped together into a pass that reads an input file and writes an output file.
- For example, the front-end phases of lexical analysis, syntax analysis, semantic analysis, and intermediate code generation might be grouped together into one pass.
- Code optimization might be an optional pass.
- Then there could be a back-end pass consisting of code generation for a particular target machine.

The Grouping of Phases into Passes — *continued*

- Some compiler collections have been created around carefully designed intermediate representations that allow the front end for a particular language to interface with the back end for a certain target machine.
- With these collections, we can produce compilers for different source languages for one target machine by combining different front ends with the back end for that target machine.
- Similarly, we can produce compilers for different target machines, by combining a front end with back ends for different target machines.

The Grouping of Phases into Passes — *continued*

- Some compiler collections have been created around carefully designed intermediate representations that allow the front end for a particular language to interface with the back end for a certain target machine.
- With these collections, we can produce compilers for different source languages for one target machine by combining different front ends with the back end for that target machine.
- Similarly, we can produce compilers for different target machines, by combining a front end with back ends for different target machines.

The Grouping of Phases into Passes — *continued*

- Some compiler collections have been created around carefully designed intermediate representations that allow the front end for a particular language to interface with the back end for a certain target machine.
- With these collections, we can produce compilers for different source languages for one target machine by combining different front ends with the back end for that target machine.
- Similarly, we can produce compilers for different target machines, by combining a front end with back ends for different target machines.

Compiler-Construction Tools

- The compiler writer, like any software developer, can profitably use modern software development environments containing tools such as language editors, debuggers, version managers, profilers, test harnesses, and so on.
- In addition to these general software-development tools, other more specialized tools have been created to help implement various phases of a compiler.

Compiler-Construction Tools — *continued*

- These tools use specialized languages for specifying and implementing specific components, and many use quite sophisticated algorithms.
- The most successful tools are those that hide the details of the generation algorithm and produce components that can be easily integrated into the remainder of the compiler.

Compiler-Construction Tools — *continued*

Some commonly used compiler-construction tools include

1. Parser generators that automatically produce syntax analyzers from a grammatical description of a programming language.
2. Scanner generators that produce lexical analyzers from a regular-expression description of the tokens of a language.
3. Syntax-directed translation engines that produce collections of routines for walking a parse tree and generating intermediate code.

Compiler-Construction Tools — *continued*

Some commonly used compiler-construction tools include

4. Code-generator generators that produce a code generator from a collection of rules for translating each operation of the intermediate language into the machine language for a target machine.
5. Data-flow analysis engines that facilitate the gathering of information about how values are transmitted from one part of a program to each other part. Data-flow analysis is a key part of code optimization.
6. Compiler-construction toolkits that provide an integrated set of routines for constructing various phases of a compiler.

AI in Compiler Design

Can AI or LLMs do the task of a state-of-the art compiler?

AI in Compiler Design

- Parts of the compilation process can be assisted by AI and LLM models.
- There is active research on using LLMs for compiler-related tasks.
- However, LLMs do not yet replace full traditional compilers.
- A compiler must be exact, deterministic, and semantics-preserving.
- LLMs can still generate invalid code, change semantics, or hallucinate.

Recent initiatives

- A major example is Large Language Models for Compiler Optimization by Cummins et al.
- It trained a 7B-parameter transformer for LLVM assembly and code-size optimization.
- The model predicts optimization options, instruction counts, and optimized code.
- The paper reports a 3.0% instruction-count improvement over the compiler.
- Generated code was compilable 91% of the time and perfectly emulated compiler output 70% of the time.

Recent Initiative: Meta LLM Compiler

- Meta introduced LLM Compiler, a family of models trained for compiler-related tasks.
- The models are built on Code Llama.
- They were trained on 546 billion tokens of LLVM IR and assembly code.
- The released model sizes include 7B and 13B parameters.
- The goal is to train LLMs on compiler representations, not only ordinary source code.

Meta LLM Compiler: Tasks

- Understanding LLVM IR.
- Understanding assembly.
- Optimizing code size.
- Predicting compiler behavior.

Compiler-Generated Feedback for LLMs

- Another recent direction uses the compiler itself to guide the LLM.
- The LLM proposes optimized IR, optimization passes, and instruction counts.
- The compiler checks whether the generated IR is compilable and whether predictions are correct.
- The feedback is sent back to the LLM for another attempt.

LLM proposes $\rightarrow$ Compiler checks $\rightarrow$ Feedback $\rightarrow$ LLM tries again

Recent Initiative: LLM4Decompile

- Decompilation is the reverse of compilation:

  machine code / assembly $\rightarrow$ high-level source code
- LLM4Decompile introduced open-source LLMs from 1.3B to 33B parameters trained for decompilation.
- The goal is to convert binary code into readable and executable C-like source code.
- This requires understanding assembly instructions, control flow, data flow, function structure, and high-level program reconstruction.

Recent AI Compiler Initiatives

- Large Language Models for Compiler Optimization
  - LLMs for LLVM assembly and optimization choices.
- Meta LLM Compiler
  - Code Llama-based models trained on LLVM IR and assembly.
- Compiler-generated feedback for LLMs
  - Compiler checks LLM-generated IR and provides feedback.
- LLM4Decompile
  - LLMs trained to decompile binaries into C-like code.

CSE 309 (Compilers)

Lexical Analysis

Dr. Muhammad Masroor Ali

Professor
Department of Computer Science and Engineering
Bangladesh University of Engineering and Technology
Dhaka-1205, Bangladesh

January 2026

Adapted by
Ahmed Mahir Sultan Rumi
Lecturer, Department of CSE, BUET

Version: 1.2, Last modified: June 30, 2026

The Role of the Lexical Analyzer

- As the first phase of a compiler, the main task of the lexical analyzer is to,
  - read the input characters of the source program,
  - group them into lexemes,
  - and produce as output a sequence of tokens for each lexeme in the source program.

The Role of the Lexical Analyzer — *continued*

- The stream of tokens is sent to the parser for syntax analysis.
- It is common for the lexical analyzer to interact with the symbol table as well.

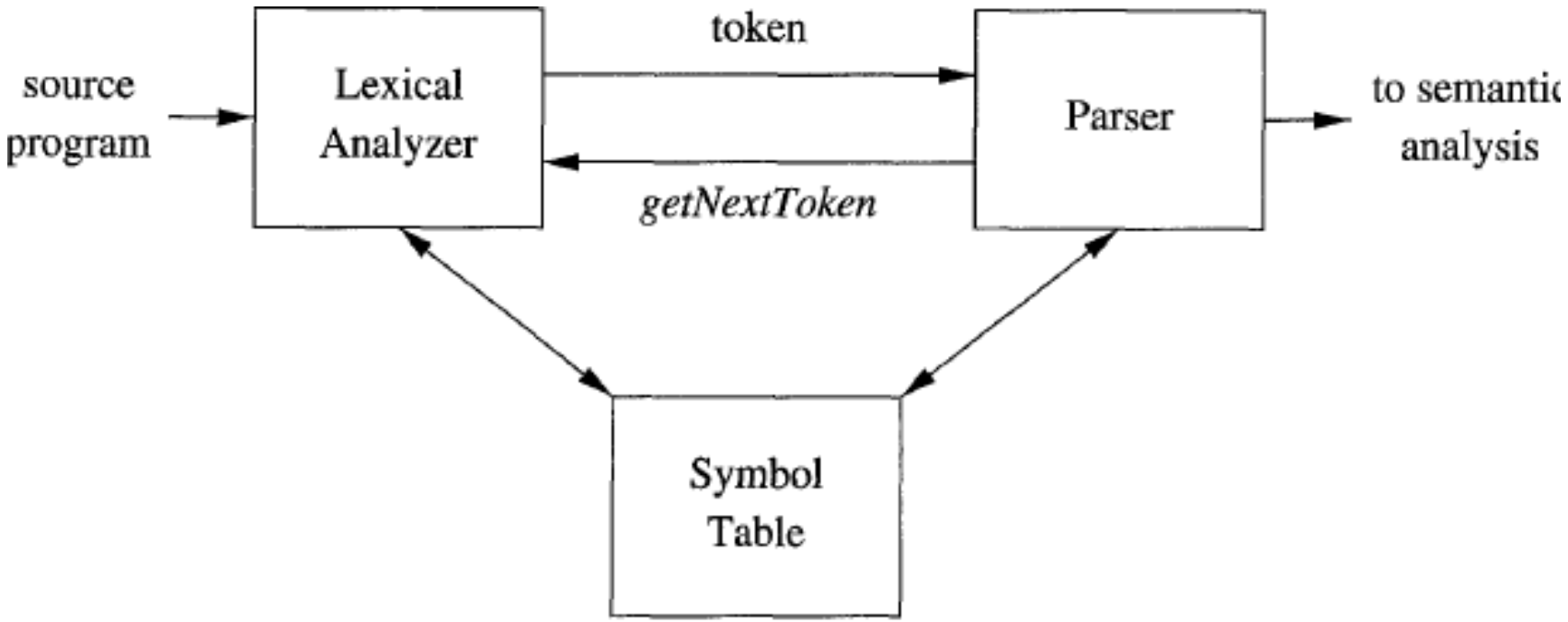
The Role of the Lexical Analyzer — *continued*

- The stream of tokens is sent to the parser for syntax analysis.
- It is common for the lexical analyzer to interact with the symbol table as well.

The Role of the Lexical Analyzer — *continued*

- When the lexical analyzer discovers a lexeme constituting an identifier, it needs to enter that lexeme into the symbol table.
- In some cases, information regarding the kind of identifier may be read from the symbol table by the lexical analyzer to assist it in determining the proper token it must pass to the parser.

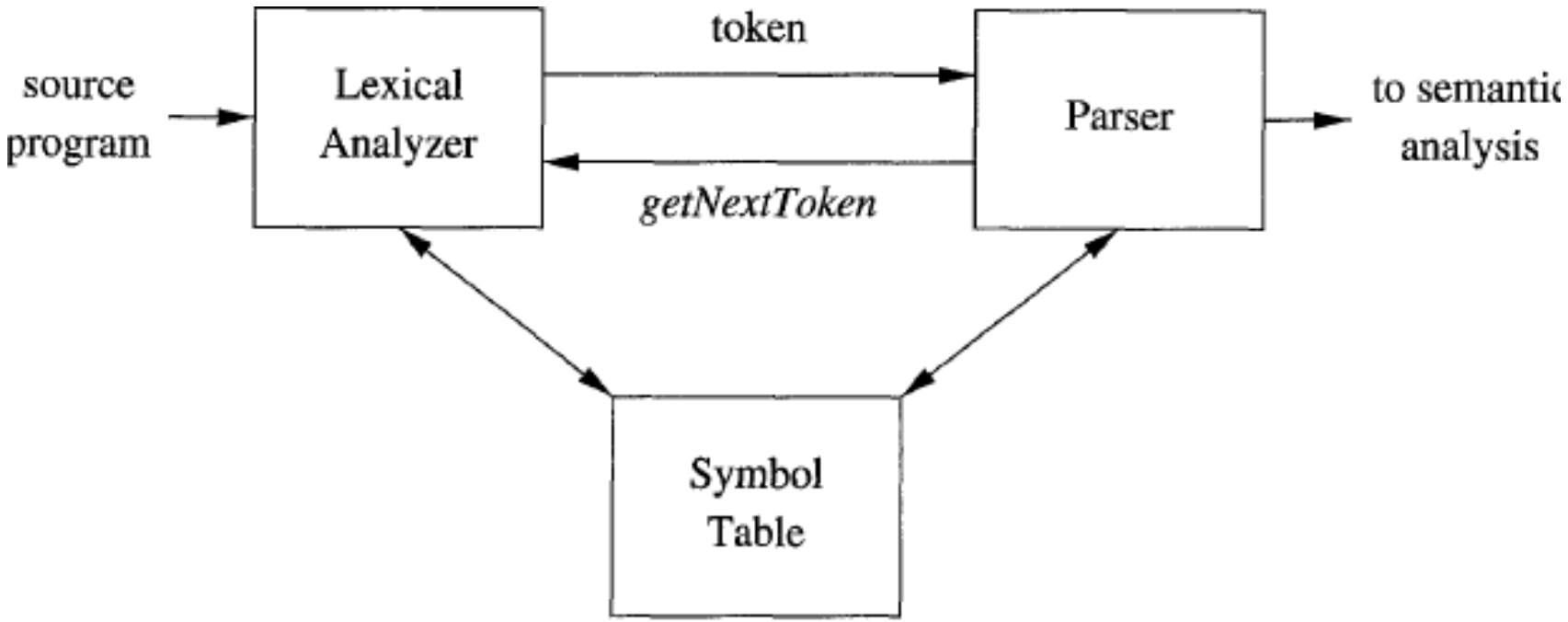
The Role of the Lexical Analyzer — *continued*



Interactions between the lexical analyzer and the parser

- These interactions are suggested in figure.
- Commonly, the interaction is implemented by having the parser call the lexical analyzer.

The Role of the Lexical Analyzer — *continued*



Interactions between the lexical analyzer and the parser

- The call, suggested by the *getNextToken* command, causes the lexical analyzer to read characters from its input until it can identify the next lexeme and produce for it the next token, which it returns to the parser.

The Role of the Lexical Analyzer — *continued*

- The lexical analyzer is the part of the compiler that reads the source text.
- It may perform certain other tasks besides identification of lexemes.
- One such task is stripping out comments and whitespace (blank, newline, tab, and perhaps other characters that are used to separate tokens in the input).

The Role of the Lexical Analyzer — *continued*

- Another task is correlating error messages generated by the compiler with the source program.
- For instance, the lexical analyzer may keep track of the number of newline characters seen, so it can associate a line number with each error message.

The Role of the Lexical Analyzer — *continued*

- In some compilers, the lexical analyzer makes a copy of the source program with the error messages inserted at the appropriate positions.
- If the source program uses a macro-preprocessor, the expansion of macros may also be performed by the lexical analyzer.

The Role of the Lexical Analyzer — *continued*

- Sometimes, lexical analyzers are divided into a cascade of two processes:
  a) *Scanning* consists of the simple processes that do not require tokenization of the input, such as deletion of comments and compaction of consecutive whitespace characters into one.
  b) *Lexical analysis proper* is the more complex portion, where the scanner produces the sequence of tokens as output.

The Role of the Lexical Analyzer — *continued*

- Sometimes, lexical analyzers are divided into a cascade of two processes:
  a) *Scanning* consists of the simple processes that do not require tokenization of the input, such as deletion of comments and compaction of consecutive whitespace characters into one.
  b) *Lexical analysis proper* is the more complex portion, where the scanner produces the sequence of tokens as output.

Lexical Analysis Versus Parsing

- There are a number of reasons why the analysis portion of a compiler is normally separated into lexical analysis and parsing (syntax analysis) phases.

Lexical Analysis Versus Parsing — *continued*

1. Simplicity of design is the most important consideration.
   - The separation of lexical and syntactic analysis often allows us to simplify at least one of these tasks.
   - For example, a parser that had to deal with comments and whitespace as syntactic units would be considerably more complex than one that can assume comments and whitespace have already been removed by the lexical analyzer.
   - If we are designing a new language, separating lexical and syntactic concerns can lead to a cleaner overall language design.

Lexical Analysis Versus Parsing — *continued*

2. Compiler efficiency is improved.
   - A separate lexical analyzer allows us to apply specialized techniques that serve only the lexical task, not the job of parsing.
   - In addition, specialized buffering techniques for reading input characters can speed up the compiler significantly.

Lexical Analysis Versus Parsing — *continued*

3. Compiler portability is enhanced.
   - Input-device-specific peculiarities can be restricted to the lexical analyzer.

Lexical Analysis Versus Parsing — *continued*

3. Compiler portability is enhanced.
   - Input-device-specific peculiarities can be restricted to the lexical analyzer.
   - Input-device-specific peculiarities include:
     - newline representation
     - character encoding
     - tabs and spaces
     - end-of-file markers
     - case conventions
     - special characters

Tokens, Patterns, and Lexemes

- When discussing lexical analysis, we use three related but distinct terms:

A token is a pair consisting of a token name and an optional attribute value.

- The token name is an abstract symbol representing a kind of lexical unit, e.g., a particular keyword, or a sequence of input characters denoting an identifier.
- The token names are the input symbols that the parser processes.
- We shall generally write the name of a token in boldface.
- We will often refer to a token by its token name.

Tokens, Patterns, and Lexemes

- When discussing lexical analysis, we use three related but distinct terms:

A pattern is a description of the form that the lexemes of a token may take.

- In the case of a keyword as a token, the pattern is just the sequence of characters that form the keyword.
- For identifiers and some other tokens, the pattern is a more complex structure that is matched by many strings.

Tokens, Patterns, and Lexemes

- When discussing lexical analysis, we use three related but distinct terms:

A lexeme is a sequence of characters in the source program that matches the pattern for a token and is identified by the lexical analyzer as an instance of that token.

TOKEN	INFORMAL DESCRIPTION	SAMPLE LEXEMES
if	characters `i`, `f`	`if`
else	characters `e`, `l`, `s`, `e`	`else`
comparison	< or > or <= or >= or == or !=	`<=`, `!=`
id	letter followed by letters and digits	`pi`, `score`, `D2`
number	any numeric constant	`3.14159`, `0`, `6.02e23`
literal	anything but ", surrounded by "'s	`"core dumped"`

Examples of tokens

- Figure gives some typical tokens, their informally described patterns, and some sample lexemes.

TOKEN	INFORMAL DESCRIPTION	SAMPLE LEXEMES
if	characters `i`, `f`	`if`
else	characters `e`, `l`, `s`, `e`	`else`
comparison	< or > or <= or >= or == or !=	`<=`, `!=`
id	letter followed by letters and digits	`pi`, `score`, `D2`
number	any numeric constant	`3.14159`, `0`, `6.02e23`
literal	anything but ", surrounded by "'s	`"core dumped"`

Examples of tokens

- To see how these concepts are used in practice, in the C statement `printf("Total = %d\n", score);` both `printf` and `score` are lexemes matching the pattern for token **id**, and `"Total = %d\n"` a lexeme matching **literal**.

Tokens, Patterns, and Lexemes — *continued*

In many programming languages, the following classes cover most or all of the tokens:

1. One token for each keyword.
   - The pattern for a keyword is the same as the keyword itself.
2. Tokens for the operators, either individually or in classes such as the token `comparison` mentioned.
3. One token representing all identifiers.
4. One or more tokens representing constants, such as numbers and literal strings.
5. Tokens for each punctuation symbol, such as left and right parentheses, comma, and semicolon.

Attributes for Tokens

- When more than one lexeme can match a pattern, the lexical analyzer must provide the subsequent compiler phases additional information about the particular lexeme that matched.
- For example, the pattern for token **number** matches both 0 and 1, but it is extremely important for the code generator to know which lexeme was found in the source program.

Attributes for Tokens — *continued*

- Thus, in many cases the lexical analyzer returns to the parser not only a token name, but an attribute value that describes the lexeme represented by the token.
- The token name influences parsing decisions, while the attribute value influences translation of tokens after the parse.

Attributes for Tokens — *continued*

- We shall assume that tokens have at most one associated attribute, although this attribute may have a structure that combines several pieces of information.
- The most important example is the token **id**, where we need to associate with the token a great deal of information.

Attributes for Tokens — *continued*

- Normally, information about an identifier e.g.,
  - its lexeme,
  - its type,
  - and the location at which it is first found (in case an error message about that identifier must be issued)

  — is kept in the symbol table.
- Thus, the appropriate attribute value for an identifier is a pointer to the symbol-table entry for that identifier.

Example

- The token names and associated attribute values for the Fortran statement

  `E = M * C ** 2`

  are written below as a sequence of pairs.

  <**id**, pointer to symbol-table entry for `E`>
  <**assign-op**>
  <**id**, pointer to symbol-table entry for `M`>
  <**mult-op**>
  <**id**, pointer to symbol-table entry for `C`>
  <**exp-op**>
  <**number**, integer value 2>

Example — *continued*

- In certain pairs, especially operators, punctuation, and keywords, there is no need for an attribute value.
- In this example, the token **number** has been given an integer-valued attribute.
- In practice, a typical compiler would instead store a character string representing the constant and use as an attribute value for **number** as pointer to that string.

Lexical Errors

- It is hard for a lexical analyzer to tell, without the aid of other components, that there is a source-code error.
- For instance, if the string `fi` is encountered for the first time in a C program in the context:
  `fi ( a == f(x)){};`
  a lexical analyzer cannot tell whether `fi` is a misspelling of the keyword `if` or an undeclared function identifier.
- Since `fi` is a valid lexeme for the token **id**, the lexical analyzer must return the token **id** to the parser and let some other phase of the compiler — probably the parser in this case — handle an error due to transposition of the letters.

Lexical Errors

- It is hard for a lexical analyzer to tell, without the aid of other components, that there is a source-code error.
- For instance, if the string `fi` is encountered for the first time in a C program in the context:
  `fi ( a == f(x)){};`
  a lexical analyzer cannot tell whether `fi` is a misspelling of the keyword `if` or an undeclared function identifier.
- Since `fi` is a valid lexeme for the token **id**, the lexical analyzer must return the token **id** to the parser and let some other phase of the compiler — probably the parser in this case — handle an error due to transposition of the letters.

Lexical Errors — *continued*

- Suppose a situation does arise in which the lexical analyzer is unable to proceed because none of the patterns for tokens matches a prefix of the remaining input.
- Perhaps the simplest recovery strategy is “panic mode” recovery.
- We delete successive characters from the remaining input until the lexical analyzer can find a well-formed token.
- This recovery technique may occasionally confuse the parser, but in an interactive computing environment it may be quite adequate.

Lexical Errors — *continued*

Other possible error-recovery actions are:

1. deleting an extraneous character,
2. inserting a missing character,
3. replacing an incorrect character by a correct character,
4. transposing two adjacent characters.

Lexical Errors — *continued*

lexerror1.cpp

```
#include <iostream>
int main()
{
  `int i, j, k;
  return 0;
}
```

Response from gcc

```
lexerror1.cpp:4: error: stray ` in program
```

Lexical Errors — *continued*

lexerror2.cpp

```
int main()
{
  int 5test;

  return 0;
}
```

Response from gcc

```
lexerror2.cpp:3:7: error: invalid suffix
"test" on integer constant
```

Lexical Errors — *continued*

- Transformations like these may be tried in an attempt to repair the input.
- The simplest such strategy is to see whether a prefix of the remaining input can be transformed into a valid lexeme by a single transformation.
- This strategy makes sense, since in practice most lexical errors involve a single character.

Lexical Errors — *continued*

- A more general correction strategy is to find the smallest number of transformations needed to convert the source program into one that consists only of valid lexemes.
- But this approach is considered too expensive in practice to be worth the effort.

Input Buffering

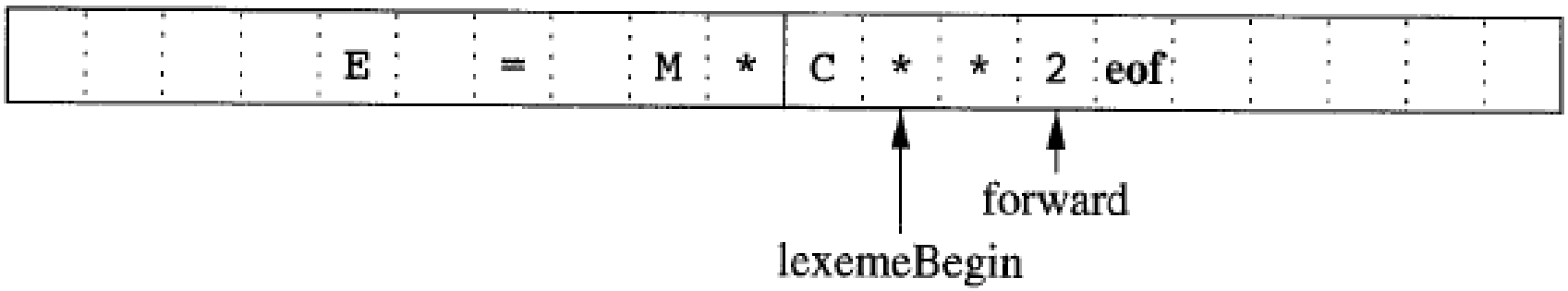
- Let us examine some ways that the simple but important task of reading the source program can be speeded.
- This task is made difficult by the fact that we often have to look one or more characters beyond the next lexeme before we can be sure we have the right lexeme.
- There are many situations where we need to look at least one additional character ahead.

Input Buffering — *continued*

- For instance, we cannot be sure we've seen the end of an identifier until we see a character that is not a letter or digit, and therefore is not part of the lexeme for id.
- In C, single-character operators like $-$, $=$, or $<$ could also be the beginning of a two-character operator like $->$, $==$, or $<=$.
- Thus, we shall introduce a two-buffer scheme that handles large lookaheads safely.
- We then consider an improvement involving "sentinels" that saves time checking for the ends of buffers.

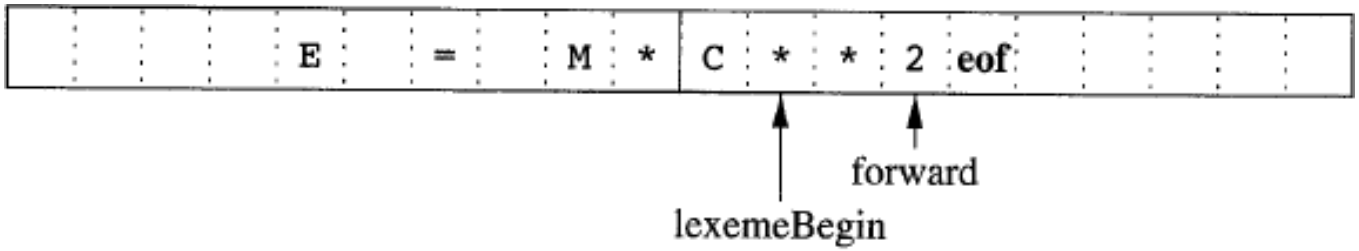
Buffer Pairs

- The amount of time taken is high to process characters of a large source program.
- Specialized buffering techniques have been developed to reduce the amount of overhead required to process a single input character.
- An important scheme involves two buffers that are alternately reloaded, as suggested in figure.



Using a pair of input buffers

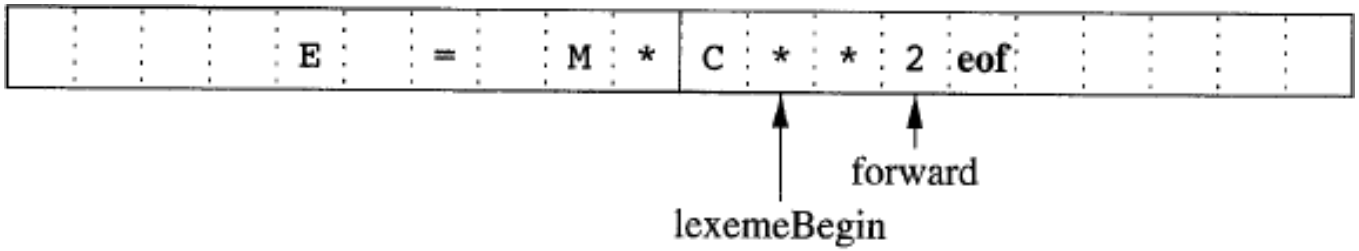
Buffer Pairs — *continued*



Using a pair of input buffers

- Each buffer is of the same size N.
- N is usually the size of a disk block, e.g., 4096 bytes.
- Using one system read command we can read N characters into a buffer, rather than using one system call per character.

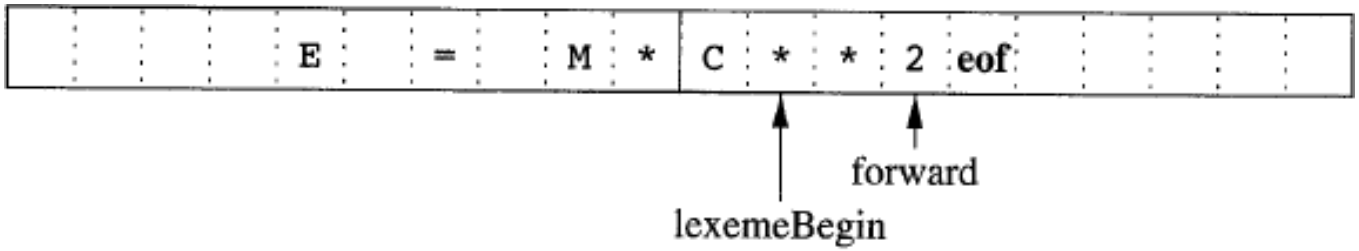
Buffer Pairs — *continued*



Using a pair of input buffers

- If fewer than N characters remain in the input file, then a special character, represented by **eof**, marks the end of the source file.
- This **eof** is different from any possible character of the source program.

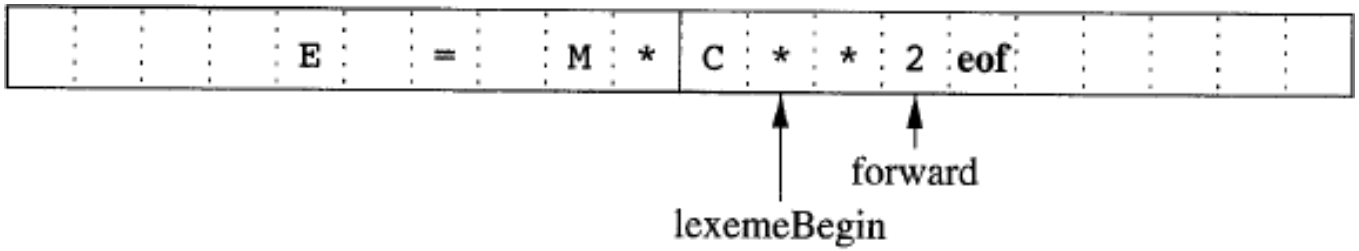
Buffer Pairs — *continued*



Using a pair of input buffers

- Two pointers to the input are maintained:
  1. Pointer `lexemeBegin`, marks the beginning of the current lexeme, whose extent we are attempting to determine.
  2. Pointer `forward` scans ahead until a pattern match is found.

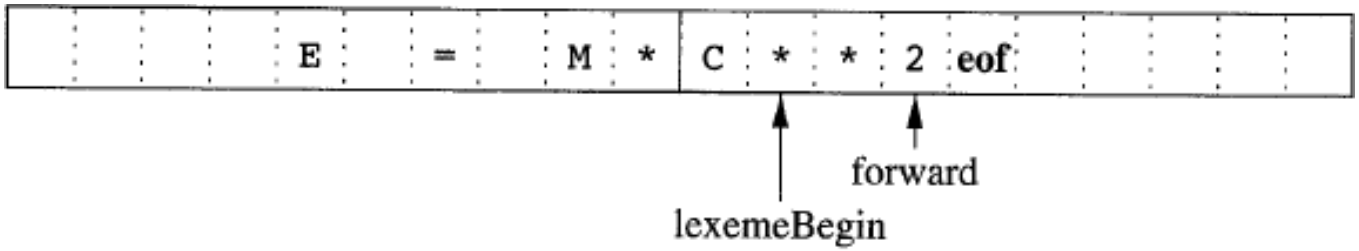
Buffer Pairs — *continued*



Using a pair of input buffers

- Once the next lexeme is determined, `forward` is set to the character at its right end.
- Then, after the lexeme is recorded as an attribute value of a token returned to the parser, `lexemeBegin` is set to the character immediately after the lexeme just found.
- In figure, we see `forward` has passed the end of the next lexeme, `**` (the Fortran exponentiation operator), and must be retracted one position to its left.

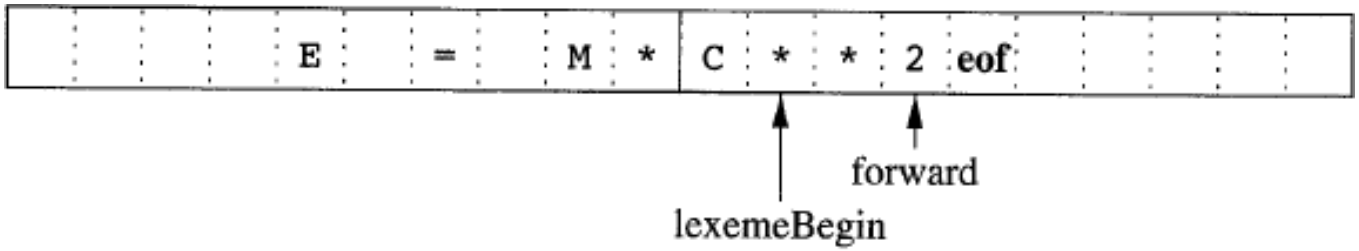
Buffer Pairs — *continued*



Using a pair of input buffers

- Advancing `forward` requires that we first test whether we have reached the end of one of the buffers.
- If so, we must reload the other buffer from the input.
- And move `forward` to the beginning of the newly loaded buffer.

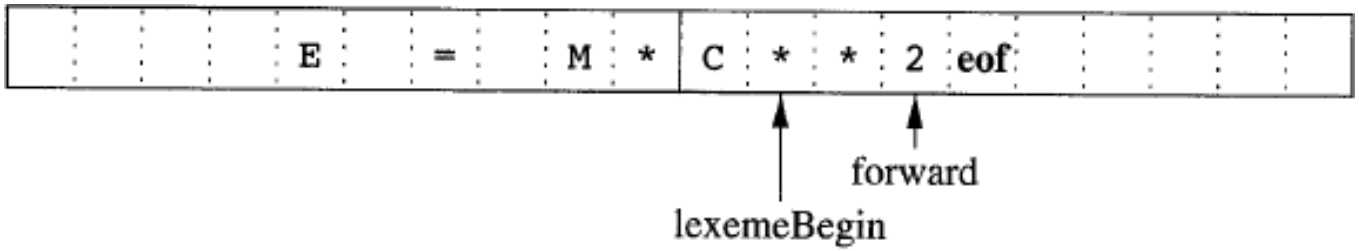
Buffer Pairs — *continued*



Using a pair of input buffers

- As long as we never need to look so far ahead of the actual lexeme that the sum of the lexeme's length plus the distance we look ahead is greater than N, we shall never overwrite the lexeme in its buffer before determining it.

Sentinels



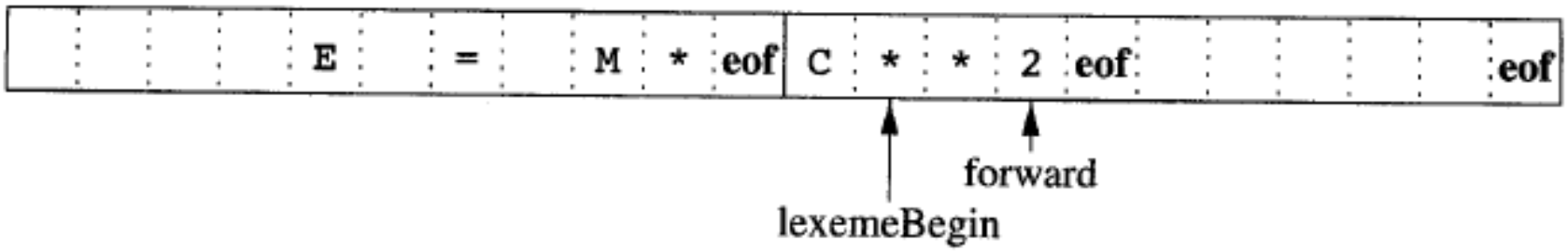
Using a pair of input buffers

- If we use the previous scheme as described, we must check, each time we advance forward, that we have not moved off one of the buffers.
- If we do, then we must also reload the other buffer.
- Thus, for each character read, we make two tests:
  - one for the end of the buffer.
  - And one to determine what character is read (the latter may be a multiway branch).

Sentinels — *continued*

- We can combine the buffer-end test with the test for the current character if we extend each buffer to hold a sentinel character at the end.
- The sentinel is a special character that cannot be part of the source program, and a natural choice is the character **eof**.

Sentinels — *continued*



Sentinels at the end of each buffer

- Figure shows the same arrangement as previous, but with the sentinels added.
- Note that **eof** retains its use as a marker for the end of the entire input.
- Any **eof** that appears other than at the end of a buffer means that the input is at an end.

```
switch ( *forward++ ) {
        case eof:
                if (forward is at end of first buffer ) {
                        reload second buffer;
                        forward = beginning of second buffer;
                }
                else if (forward is at end of second buffer ) {
                        reload first buffer;
                        forward = beginning of first buffer;
                }
                else /* eof within a buffer marks the end of input */
                        terminate lexical analysis;
                break;
        Cases for the other characters
}
```

Lookahead code with sentinels

- Figure summarizes the algorithm for advancing forward.
- Notice how the first test, which can be part of a multiway branch based on the character pointed to by forward, is the only test we make, except in the case where we actually are at the end of a buffer or the end of the input.

Buffering in Modern Lexers

- Conceptually, modern lexers still use buffered input.
- Reading character-by-character from disk is inefficient.
- Modern systems may use practical alternatives such as:
  - large input buffers,
  - memory-mapped files,
  - stream buffers,
  - runtime library buffering,
  - custom scanner buffers.
- The exact two equal halves with sentinels method is mostly a classical teaching model.
- The core idea is still important: read input in chunks, not one character at a time.

Buffering in Modern Lexers

- It explains efficient input scanning, lookahead, lexeme begin pointer, and forward pointer.
- It also explains how tokens can cross buffer boundaries.

```
Buffer 1:  abc veryLongIdenti
Buffer 2:  fierName xyz
```

- The lexer should still recognize the token `veryLongIdentifierName`.
- Modern compilers may not literally use the textbook technique, but they solve the same problem: efficient character feeding with lookahead and token-boundary handling.

Recognition of Tokens

- We can express patterns using regular expressions.

Recognition of Tokens

- Our discussion will make use of the following running example.

stmt	→	**if** *expr* **then** *stmt*
	|	**if** *expr* **then** *stmt* **else** *stmt*
	|	ϵ
expr	→	*term* **relop** *term*
	|	*term*
term	→	**id**
	|	**number**

A grammar for branching statements

Example

- The grammar fragment describes a simple form of branching statements and conditional expressions.
- This syntax is similar to that of the language Pascal, in that **then** appears explicitly after conditions.

stmt	→	**if** *expr* **then** *stmt*
	|	**if** *expr* **then** *stmt* **else** *stmt*
	|	ϵ
expr	→	*term* **relop** *term*
	|	*term*
term	→	**id**
	|	**number**

A grammar for branching statements

Example

- For **relop**, we use the comparison operators of languages like Pascal or SQL, where $=$ is "equals" and $<>$ is "not equals," because it presents an interesting structure of lexemes.

stmt	$\rightarrow$	**if** *expr* **then** *stmt*
	|	**if** *expr* **then** *stmt* **else** *stmt*
	|	ϵ
expr	$\rightarrow$	*term* **relop** *term*
	|	*term*
term	$\rightarrow$	**id**
	|	**number**

A grammar for branching statements

Example

- The terminals of the grammar, which are **if**, **then**, **else**, **relop**, **id**, and **number**, are the names of tokens as far as the lexical analyzer is concerned.

stmt	→	**if** *expr* **then** *stmt*
	|	**if** *expr* **then** *stmt* **else** *stmt*
	|	ϵ
expr	→	*term* **relop** *term*
	|	*term*
term	→	**id**
	|	**number**

A grammar for branching statements

Example — *continued*

digit	→	`[0-9]`
digits	→	*digit*$^+$
number	→	*digits* (. *digits*)? (`E` `[+-]`? *digits*)?
letter	→	`[A-Za-z]`
id	→	*letter* (*letter* | *digit*)*
if	→	`if`
then	→	`then`
else	→	`else`
relop	→	`<` | `>` | `<=` | `>=` | `=` | `<>`

Patterns for tokens of Example

- The patterns for these tokens are described using regular definitions.

Example — *continued*

digit	→	`[0-9]`
digits	→	*digit*$^+$
number	→	*digits* (`.` *digits*)? (`E` `[+-]`? *digits*)?
letter	→	`[A-Za-z]`
id	→	*letter* (*letter* | *digit*)*
if	→	`if`
then	→	`then`
else	→	`else`
relop	→	`<` | `>` | `<=` | `>=` | `=` | `<>`

Patterns for tokens of Example

- For this language, the lexical analyzer will recognize the keywords **if**, **then**, and **else**, as well as lexemes that match the patterns for *relop*, *id*, and *number*.

Example — *continued*

digit	→	`[0-9]`
digits	→	*digit*$^+$
number	→	*digits* (. *digits*)? (`E` `[+-]`? *digits*)?
letter	→	`[A-Za-z]`
id	→	*letter* (*letter* | *digit*)*
if	→	`if`
then	→	`then`
else	→	`else`
relop	→	`<` | `>` | `<=` | `>=` | `=` | `<>`

Patterns for tokens of Example

- To simplify matters, we make the common assumption that keywords are also *reserved words*.
- They are not identifiers, even though their lexemes match the pattern for identifiers.

Example — *continued*

- In addition, we assign the lexical analyzer the job of stripping out white- space, by recognizing the "token" *ws* defined by:

$$ws \rightarrow (\textbf{blank} \mid \textbf{tab} \mid \textbf{newline})^{+}$$

- Here, **blank**, **tab**, and **newline** are abstract symbols that we use to express the ASCII characters of the same names.
- Token *ws* is different from the other tokens in that, when we recognize it, we do not return it to the parser.
- We rather restart the lexical analysis from the character that follows the whitespace.
- It is the following token that gets returned to the parser.

LEXEMES	TOKEN NAME	ATTRIBUTE VALUE
Any *ws*	–	–
`if`	**if**	–
`then`	**then**	–
`else`	**else**	–
Any *id*	**id**	Pointer to table entry
Any *number*	**number**	Pointer to table entry
<	**relop**	LT
<=	**relop**	LE
=	**relop**	EQ
<>	**relop**	NE
>	**relop**	GT
>=	**relop**	GE

Tokens, their patterns, and attribute values

- Our goal for the lexical analyzer is summarized in figure.

LEXEMES	TOKEN NAME	ATTRIBUTE VALUE
Any *ws*	–	–
`if`	**if**	–
`then`	**then**	–
`else`	**else**	–
Any *id*	**id**	Pointer to table entry
Any *number*	**number**	Pointer to table entry
<	**relop**	LT
<=	**relop**	LE
=	**relop**	EQ
<>	**relop**	NE
>	**relop**	GT
>=	**relop**	GE

Tokens, their patterns, and attribute values

- That table shows, for each lexeme or family of lexemes, which token name is returned to the parser and what attribute value is returned.

LEXEMES	TOKEN NAME	ATTRIBUTE VALUE
Any *ws*	–	–
`if`	**if**	–
`then`	**then**	–
`else`	**else**	–
Any *id*	**id**	Pointer to table entry
Any *number*	**number**	Pointer to table entry
<	**relop**	LT
<=	**relop**	LE
=	**relop**	EQ
<>	**relop**	NE
>	**relop**	GT
>=	**relop**	GE

Tokens, their patterns, and attribute values

- Note that for the six relational operators, symbolic constants `LT`, `LE`, and so on are used as the attribute value, in order to indicate which instance of the token relop we have found.

LEXEMES	TOKEN NAME	ATTRIBUTE VALUE
Any *ws*	–	–
`if`	**if**	–
`then`	**then**	–
`else`	**else**	–
Any *id*	**id**	Pointer to table entry
Any *number*	**number**	Pointer to table entry
<	**relop**	LT
<=	**relop**	LE
=	**relop**	EQ
<>	**relop**	NE
>	**relop**	GT
>=	**relop**	GE

Tokens, their patterns, and attribute values

- The particular operator found will influence the code that is output from the compiler.

Exercise: Identifying Tokens

For the following code fragment, write the sequence of tokens produced by the lexical analyzer.

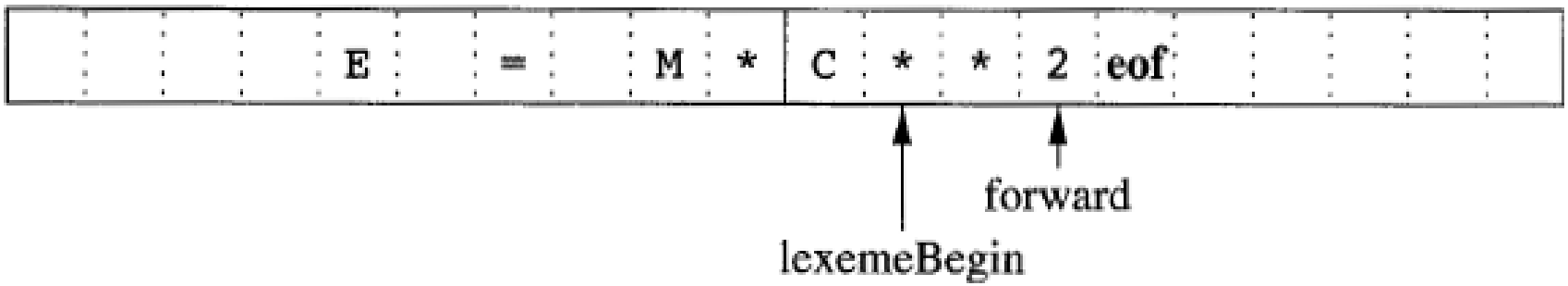
```
long long int x = 1;
printf("value = %d ", x);
```

Transition Diagrams

- As an intermediate step in the construction of a lexical analyzer, we first convert patterns into stylized flowcharts, called “transition diagrams.”
- In this section, we perform the conversion from regular-expression patterns to transition diagrams by hand.
- Later we shall see that there is a mechanical way to construct these diagrams from collections of regular expressions.

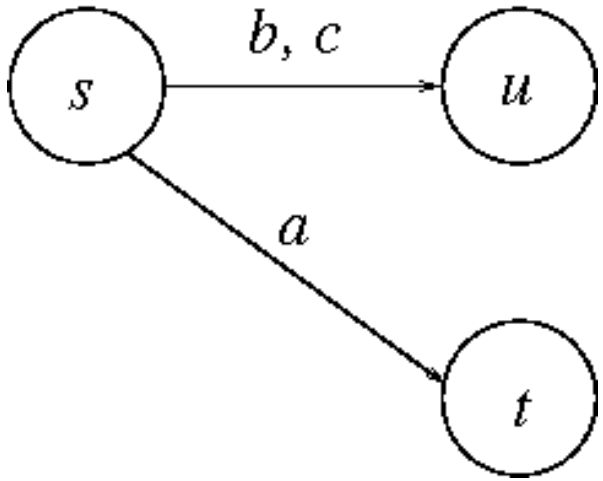
Transition Diagrams — *continued*

- Transition diagrams have a collection of nodes or circles, called states.
- Each state represents a condition that could occur during the process of scanning the input looking for a lexeme that matches one of several patterns.
- We may think of a state as summarizing all we need to know about what characters we have seen between the `lexemeBegin` pointer and the `forward` pointer.



Transition Diagrams — *continued*

- Edges are directed from one state of the transition diagram to another.
- Each edge is labeled by a symbol or set of symbols.
- If we are in some state s, and the next input symbol is a, we look for an edge out of state s labeled by a (and perhaps by other symbols, as well).
- If we find such an edge, we advance the forward pointer and enter the state of the transition diagram to which that edge leads.



Transition Diagrams — *continued*

- We shall assume that all our transition diagrams are deterministic, meaning that there is never more than one edge out of a given state with a given symbol among its labels.
- Later we shall relax the condition of determinism, making life much easier for the designer of a lexical analyzer, although trickier for the implementer.

Transition Diagrams — *continued*

Some important conventions about transition diagrams are:

1. Certain states are said to be accepting, or final.

- These states indicate that a lexeme has been found, although the actual lexeme may not consist of all positions between the `lexemeBegin` and `forward` pointers.
- We always indicate an accepting state by a double circle.
- If there is an action to be taken — typically returning a token and an attribute value to the parser - we shall attach that action to the accepting state.

Transition Diagrams — *continued*

Some important conventions about transition diagrams are:

2. In addition, if it is necessary to retract the forward pointer one position (i.e., the lexeme does not include the symbol that got us to the accepting state), then we shall additionally place a $*$ near that accepting state.

- In our example, it is never necessary to retract forward by more than one position, but if it were, we could attach any number of $*$'s to the accepting state.

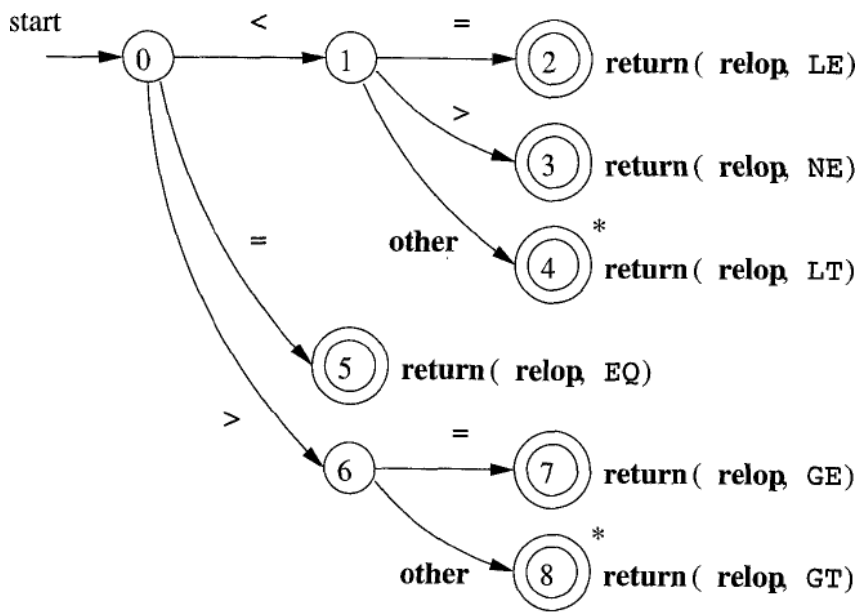
Transition Diagrams — *continued*

Some important conventions about transition diagrams are:

3. One state is designated the start state, or initial state it is indicated by an edge, labeled “start ,” entering from nowhere.

- The transition diagram always begins in the start state before any input symbols have been read.

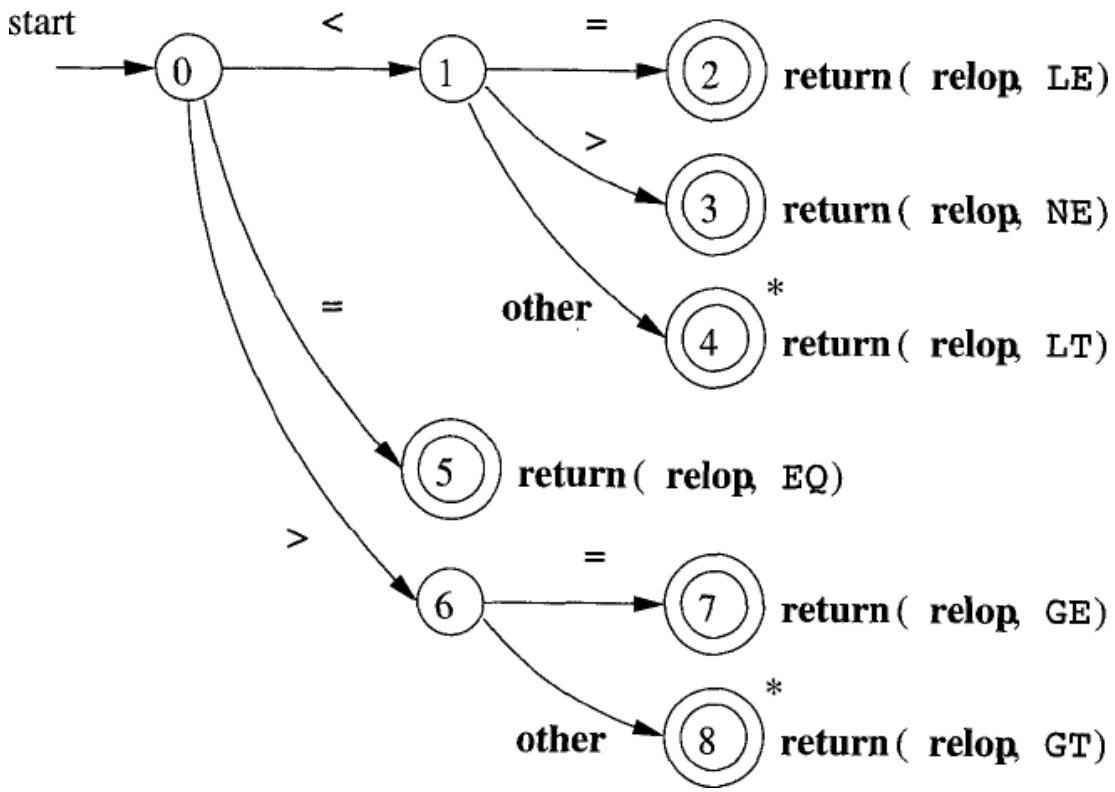
1. Certain states are said to be accepting, or final.
   - These states indicate that a lexeme has been found, although the actual lexeme may not consist of all positions between the `lexemeBegin` and `forward` pointers.
   - We always indicate an accepting state by a double circle.
2. In addition, if it is necessary to retract the forward pointer one position (i.e., the lexeme does not include the symbol that got us to the accepting state), then we shall additionally place a $*$ near that accepting state.
3. One state is designated the start state, or initial state it is indicated by an edge, labeled "start ," entering from nowhere.
   - The transition diagram always begins in the start state before any input symbols have been read.



Transition diagram for **relop**

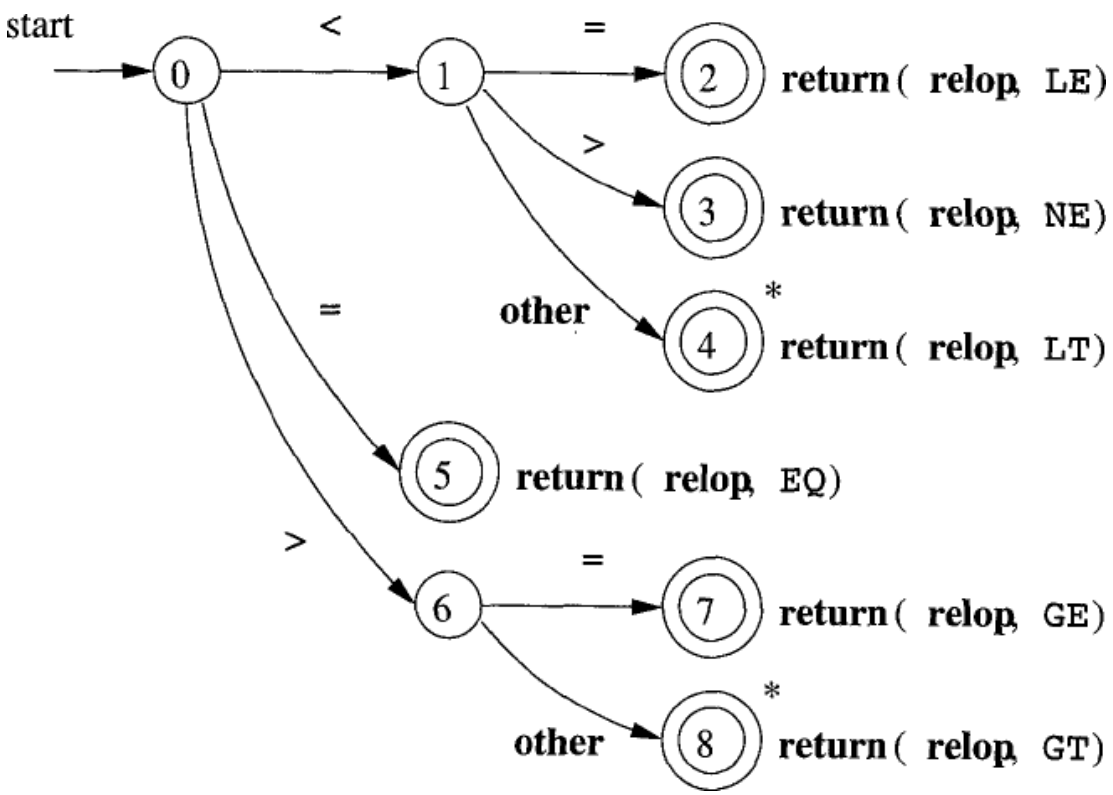
Example — *continued*

- We begin in state 0, the start state.
- If we see $<$ as the first input symbol, then among the lexemes that match the pattern for **relop** we can only be looking at $<$, $<>$, or $<=$.
- We therefore go to state 1, and look at the next character.



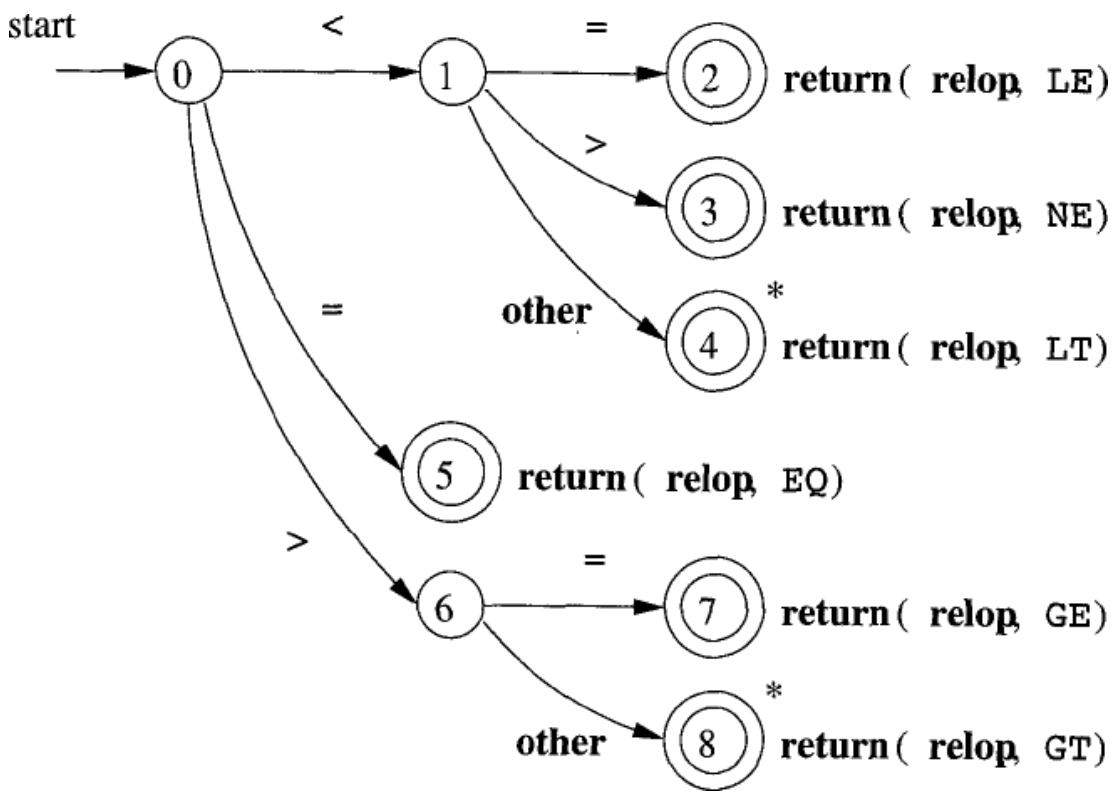
Example — *continued*

- If it is $=$, then we recognize lexeme $<=$, enter state 2, and return the token **relop** with attribute LE, the symbolic constant representing this particular comparison operator.
- If in state 1 the next character is $>$, then instead we have lexeme $<>$, and enter state 3 to return an indication that the not-equals operator has been found.



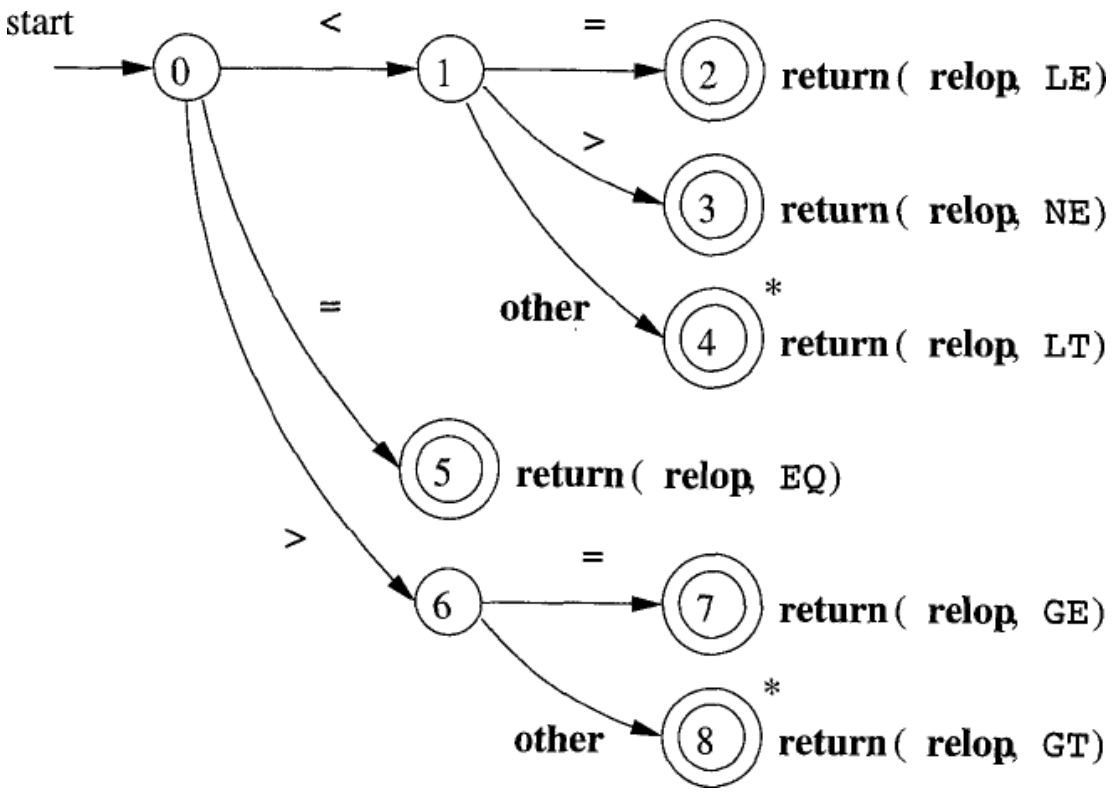
Example — *continued*

- On any other character, the lexeme is $<$, and we enter state 4 to return that information.
- Note, however, that state 4 has a $*$ to indicate that we must retract the input one position.



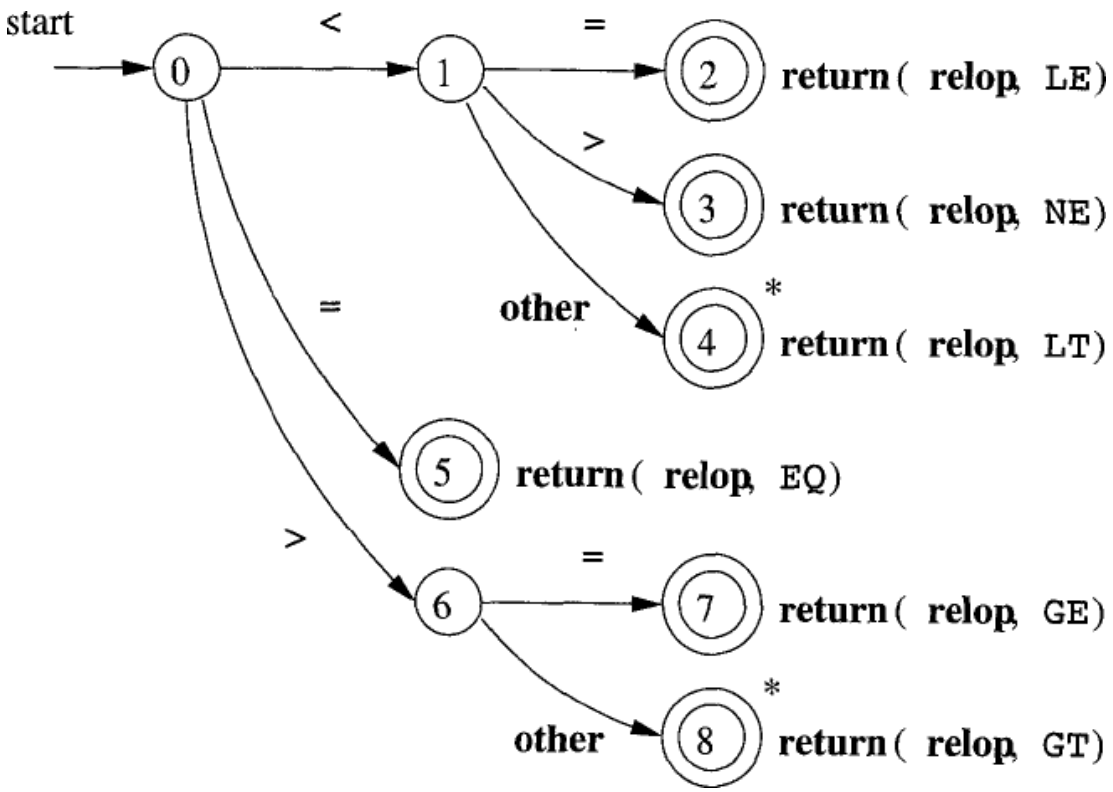
Example — *continued*

- On the other hand, if in state 0 the first character we see is $=$, then this one character must be the lexeme.
- We immediately return that fact from state 5.



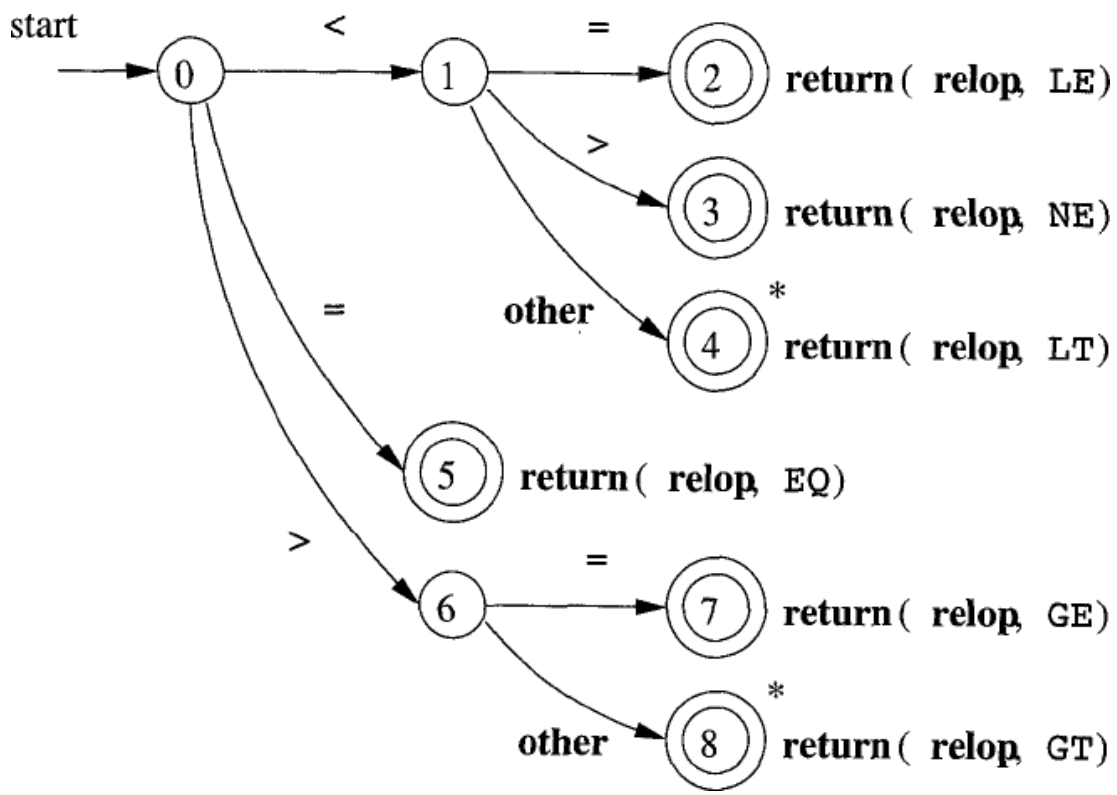
Example — *continued*

- The remaining possibility is that the first character is $>$.
- Then, we must enter state 6 and decide, on the basis of the next character, whether the lexeme is $>=$ (if we next see the $=$ sign), or just $>$ (on any other character).



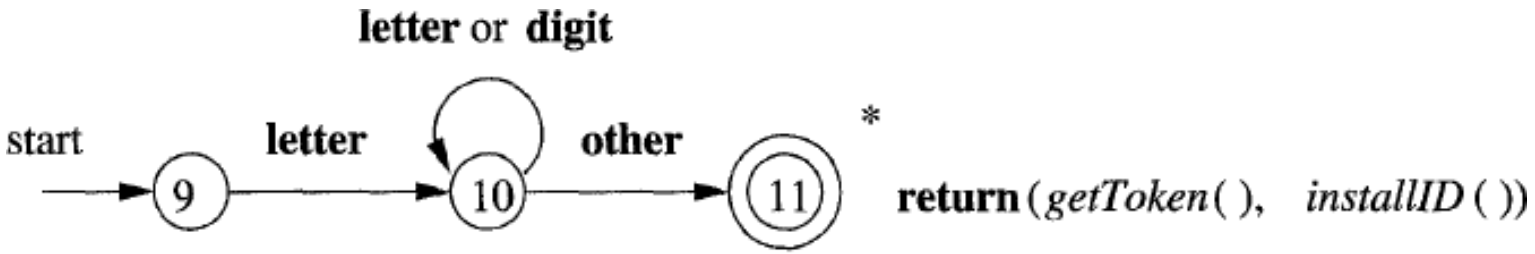
Example — *continued*

- Note that if, in state 0, we see any character besides $<$, $=$, or $>$, we can not possibly be seeing a **relop** lexeme, so this transition diagram will not be used.



Recognition of Reserved Words and Identifiers

- Recognizing keywords and identifiers presents a problem.
- Usually, keywords like `if` or `then` are reserved, so they are not identifiers even though they look like identifiers.
- Thus, we typically use a transition diagram like the following to search for identifier lexemes.



A transition diagram for **id**'s and keywords

- But, this diagram will also recognize the keywords `if`, `then`, and `else`.

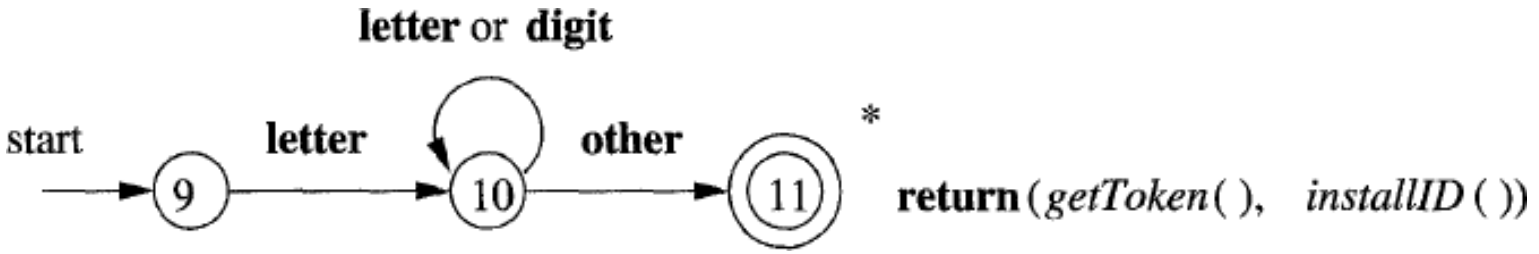
Recognition of Reserved Words and Identifiers — *continued*

- There are two ways that we can handle reserved words that look like identifiers.

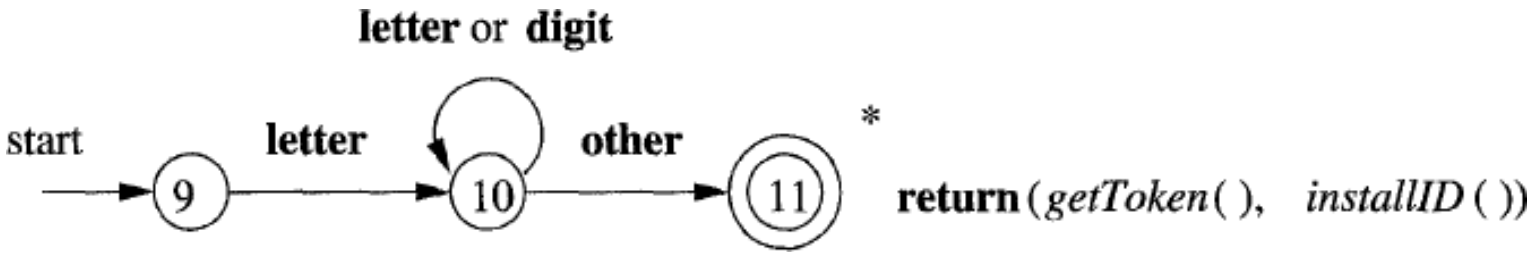
Recognition of Reserved Words and Identifiers — *continued*

1. Install the reserved words in the symbol table initially.

- A field of the symbol-table entry indicates that these strings are never ordinary identifiers, and tells which token they represent.
- This method is applicable below.



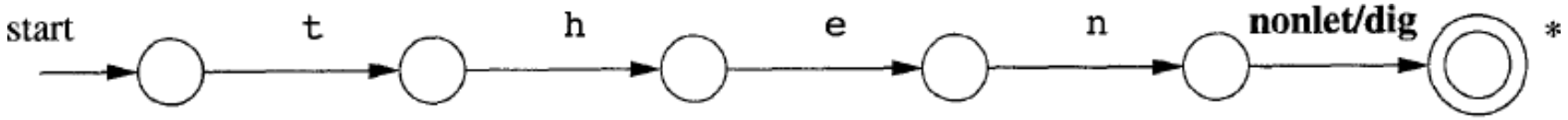
Recognition of Reserved Words and Identifiers — *continued*



- When we find an identifier, a call to `installID` places it in the symbol table if it is not already there and returns a pointer to the symbol-table entry for the lexeme found.
- Of course, any identifier not in the symbol table during lexical analysis cannot be a reserved word, so its token is **id**.
- The function `getToken` examines the symbol table entry for the lexeme found, and returns whatever token name the symbol table says this lexeme represents - either **id** or one of the keyword tokens that was initially installed in the table.

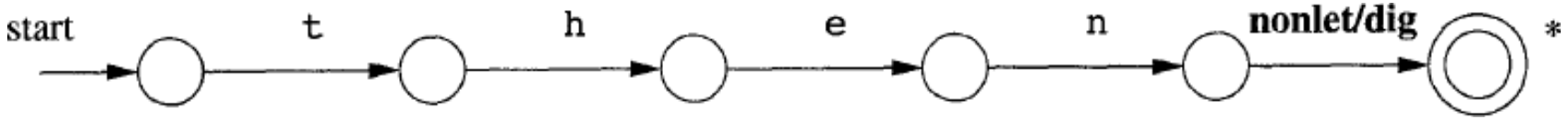
Recognition of Reserved Words and Identifiers — *continued*

2. Create separate transition diagrams for each keyword.
   - An example for the keyword `then` is shown in figure.



Hypothetical transition diagram for the keyword **then**

Recognition of Reserved Words and Identifiers — *continued*



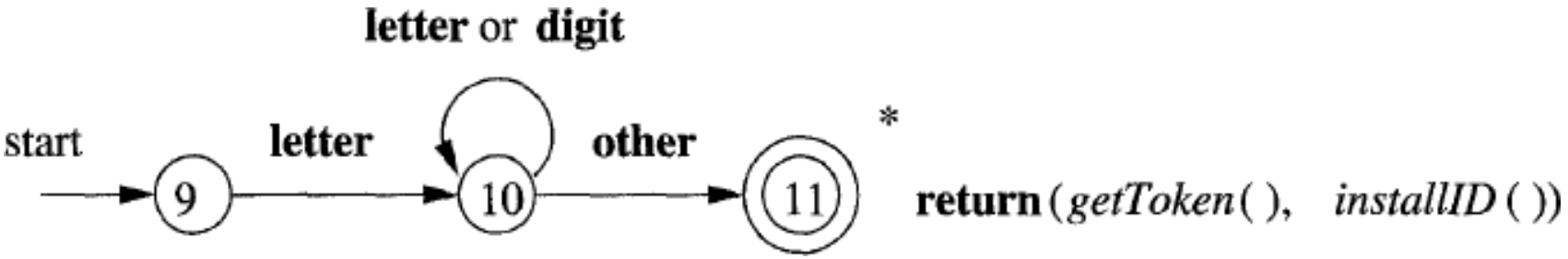
- Note that such a transition diagram consists of states representing the situation after each successive letter of the keyword is seen, followed by a test for a "nonletter-or-digit," i.e., any character that cannot be the continuation of an identifier.
- It is necessary to check that the identifier has ended, or else we would return token then in situations where the correct token was **id**, with a lexeme like `thenextvalue` that has `then` as a proper prefix.

Recognition of Reserved Words and Identifiers — *continued*

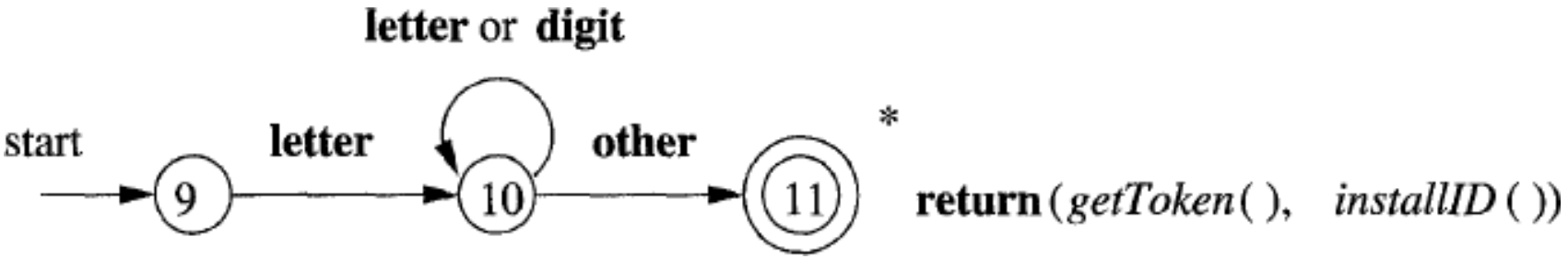
- If we adopt this approach, then we must prioritize the tokens so that the reserved-word tokens are recognized in preference to **id**, when the lexeme matches both patterns.

Completion of the Running Example

- The transition diagram for **id**'s in the figure below has a simple structure.



Completion of the Running Example — *continued*



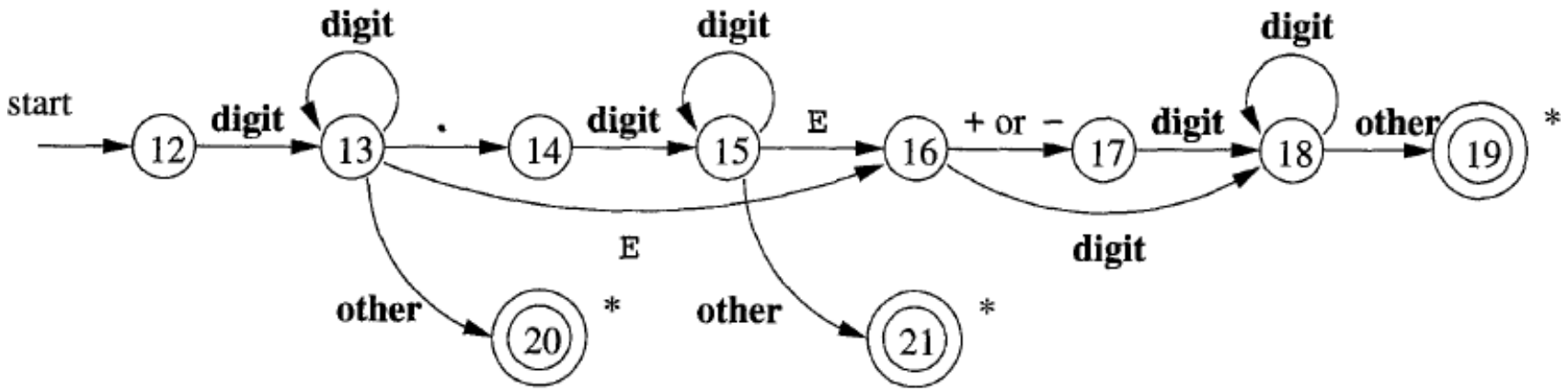
- Starting in state 9, it checks that the lexeme begins with a letter and goes to state 10 if so.
- We stay in state 10 as long as the input contains letters and digits.
- When we first encounter anything but a letter or digit, we go to state 11 and accept the lexeme found.
- Since the last character is not part of the identifier, we must retract the input one position.
- We enter what we have found in the symbol table and determine whether we have a keyword or a true identifier.

Completion of the Running Example — *continued*

- Draw the transition diagram for token **number**.

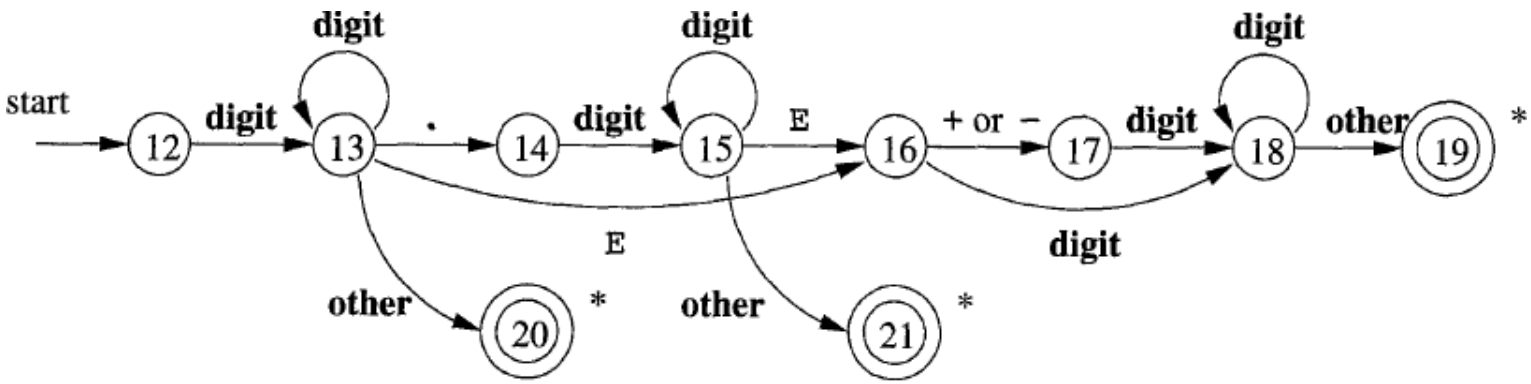
Completion of the Running Example — *continued*

- The transition diagram for token **number** is shown in figure.



A transition diagram for unsigned numbers

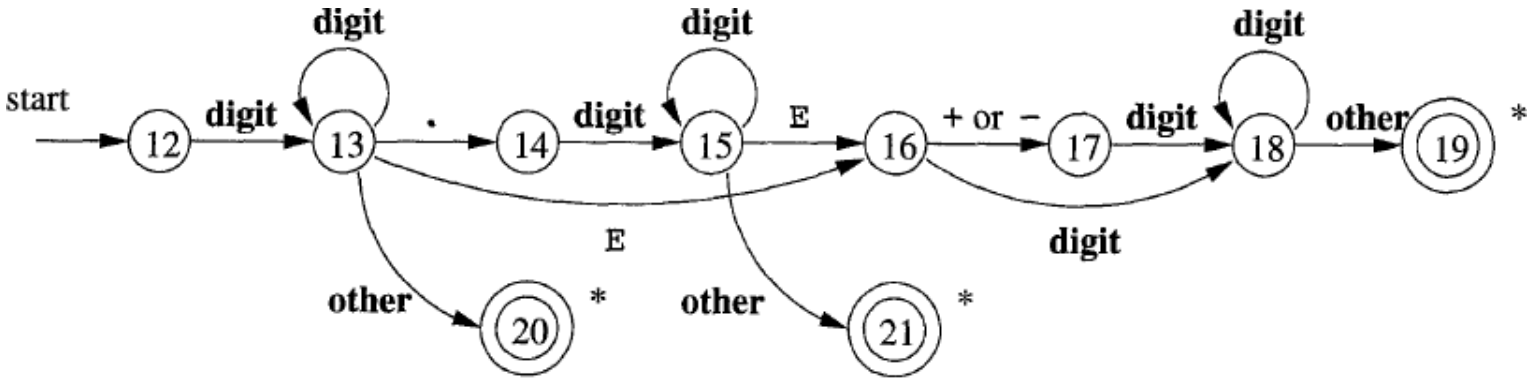
Completion of the Running Example — *continued*



123

- Beginning in state 12, if we see a digit, we go to state 13.
- In that state, we can read any number of additional digits.
- However, if we see anything but a digit or a dot, we have seen a number in the form of an integer.
- That case is handled by entering state 20, where we return token number and a pointer to a table of constants where the found lexeme is entered.

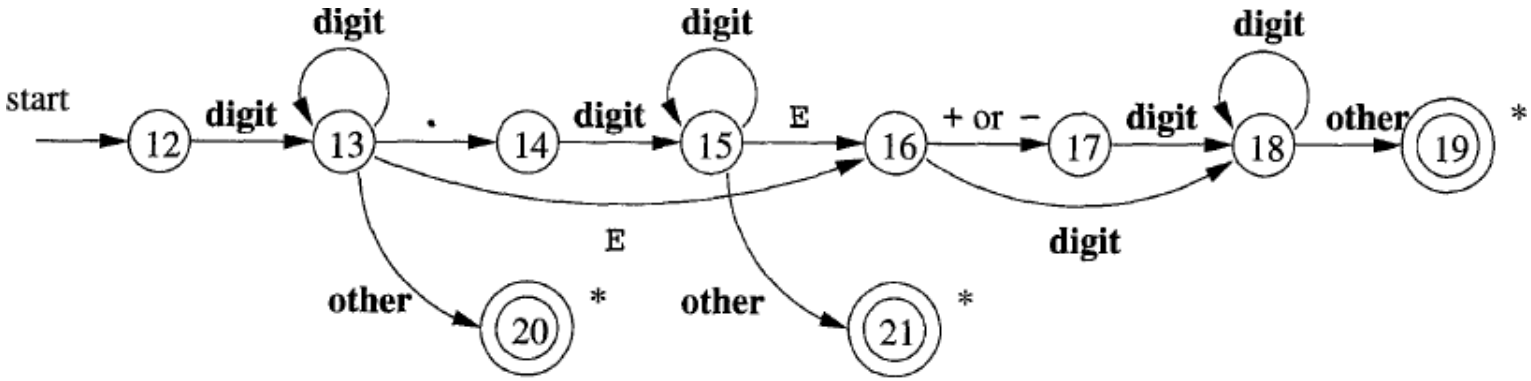
Completion of the Running Example — *continued*



123

- These mechanics are not shown on the diagram but are analogous to the way we handled identifiers.

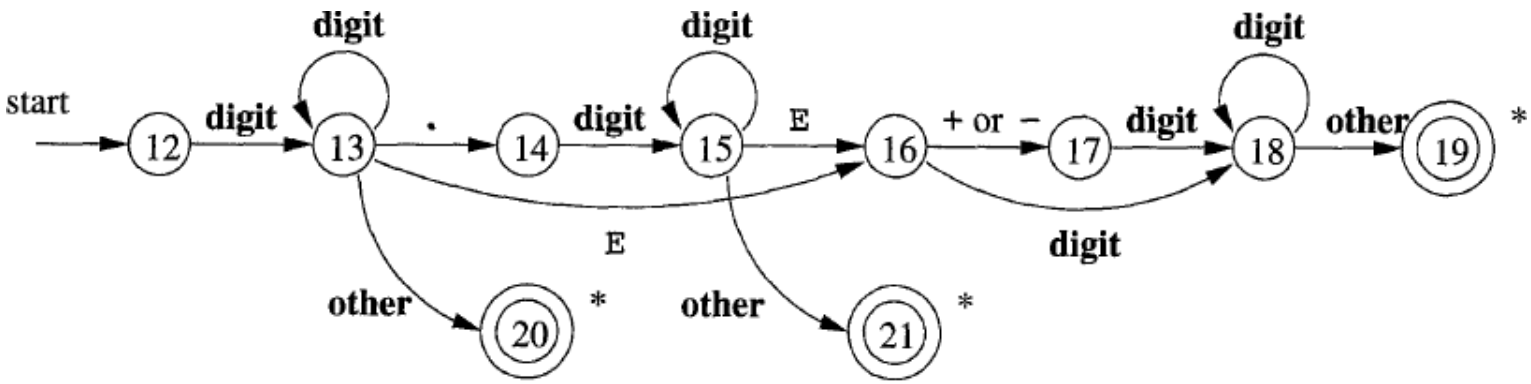
Completion of the Running Example — *continued*



123.456

- If we instead see a dot in state 13, then we have an "optional fraction."
- State 14 is entered, and we look for one or more additional digits; state 15 is used for that purpose.

Completion of the Running Example — *continued*

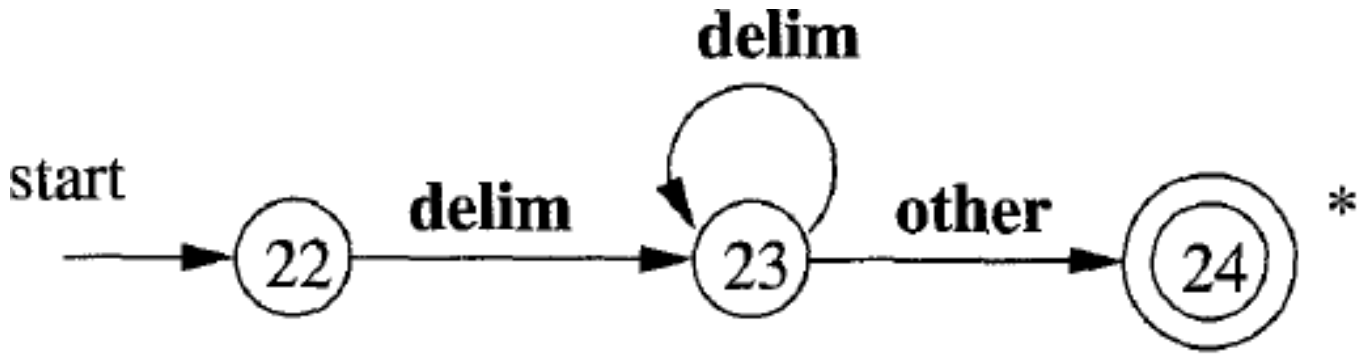


123.456E789 123.456E+789 123.456E-789

- If we see an E, then we have an "optional exponent," whose recognition is the job of states 16 through 19.
- Should we, in state 15, instead see anything but E or a digit, then we have come to the end of the fraction, there is no exponent, and we return the lexeme found, via state 21.

Completion of the Running Example — *continued*

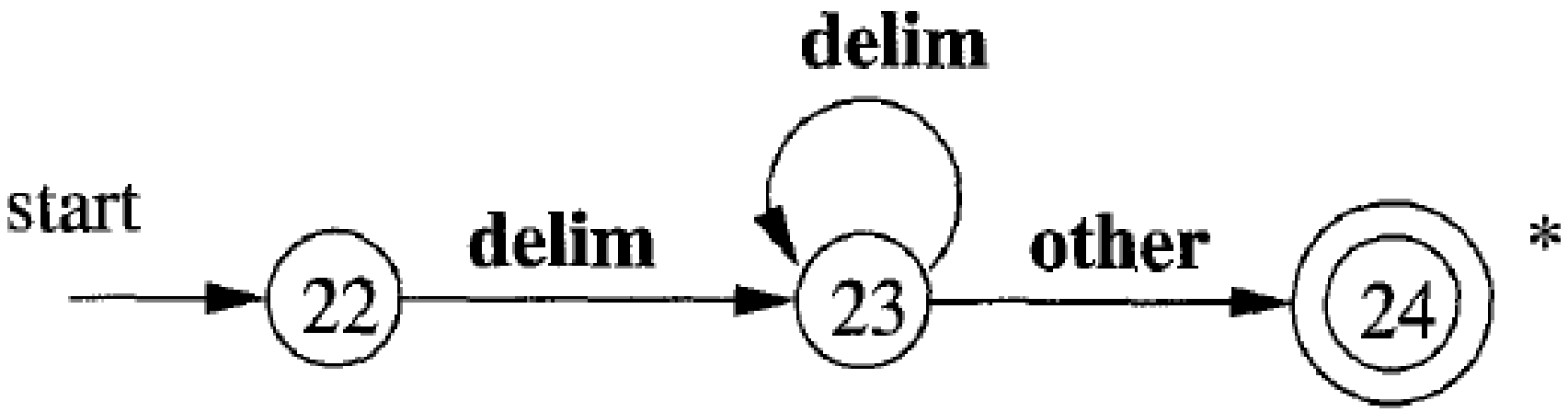
- Transition diagram for whitespace is shown.



A transition diagram for whitespace

- In that diagram, we look for one or more “whitespace” characters, represented by **delim** in that diagram.
- Typically these characters would be blank, tab, newline, and perhaps other characters that are not considered by the language design to be part of any token.

Completion of the Running Example — *continued*



- Note that in state 24, we have found a block of consecutive whitespace characters, followed by a nonwhitespace character.
- We retract the input to begin at the nonwhitespace, but we do not return to the parser.
- Rather, we must restart the process of lexical analysis after the whitespace.

Architecture of a Transition-Diagram-Based Lexical Analyzer

- There are several ways that a collection of transition diagrams can be used to build a lexical analyzer.
- Regardless of the overall strategy, each state is represented by a piece of code.
- We may imagine a variable `state` holding the number of the current state for a transition diagram.
- A switch based on the value of `state` takes us to code for each of the possible states, where we find the action of that state.
- Often, the code for a state is itself a switch statement or multiway branch that determines the next state by reading and examining the next input character.

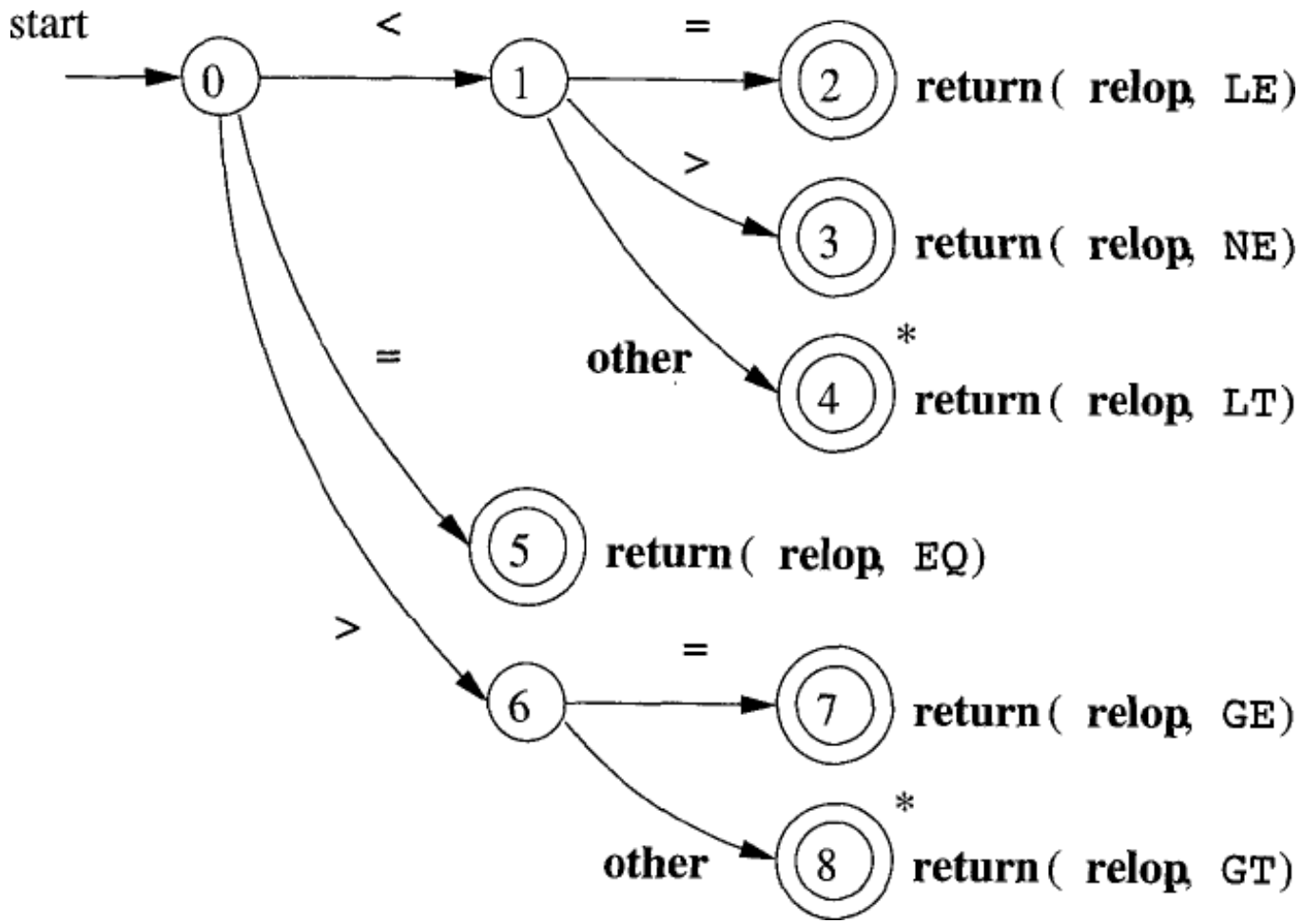
Example

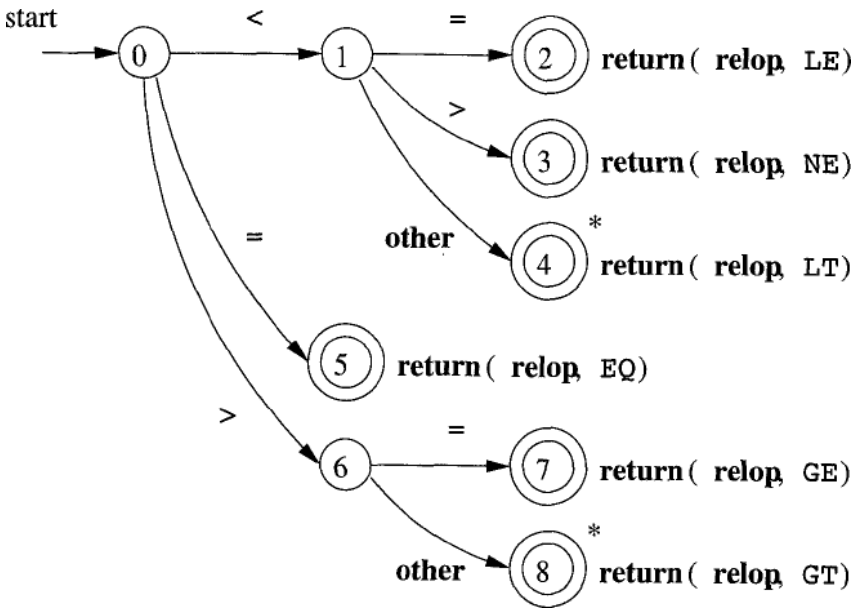
- In figure we see a sketch of `getRelop()`, a C++ function whose job is to simulate the transition diagram for **relop**.

```
TOKEN getRelop()
{
    TOKEN retToken = new(RELOP);
    while(1) { /* repeat character processing until a return
                  or failure occurs */
        switch(state) {
            case 0: c = nextChar();
                    if ( c == '<' ) state = 1;
                    else if ( c == '=' ) state = 5;
                    else if ( c == '>' ) state = 6;
                    else fail(); /* lexeme is not a relop */
                    break;
            case 1: ...
            ...
            case 8: retract();
                    retToken.attribute = GT;
                    return(retToken);
        }
    }
}
```

Example

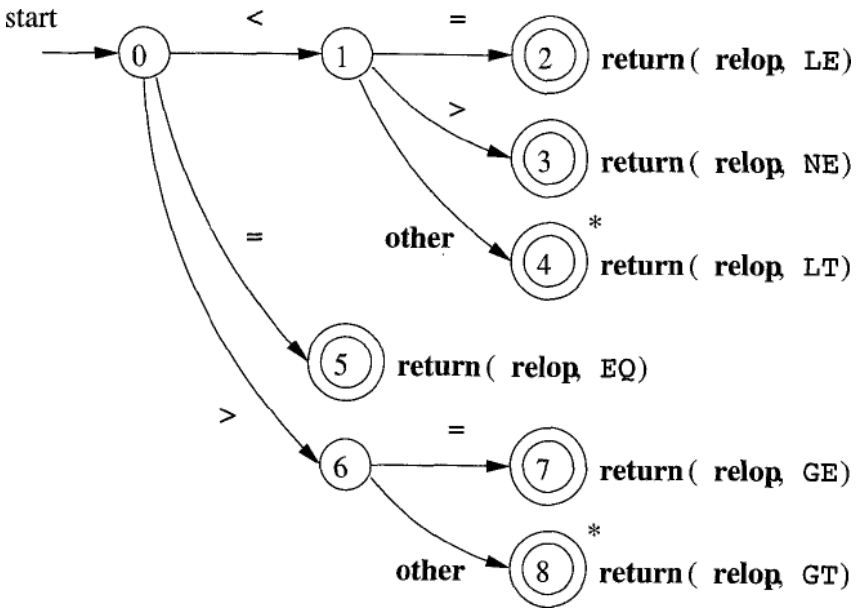
- In figure we see a sketch of `getRelop()`, a C++ function whose job is to simulate the transition diagram for **relop**.





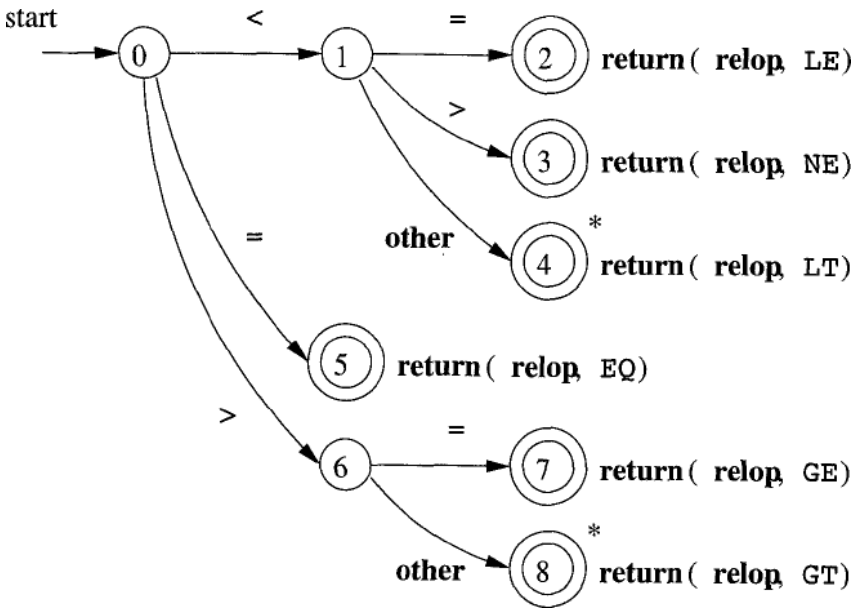

```
TOKEN getRelop()
{
    TOKEN retToken = new(RELOP);
    while(1) { /* repeat character processing until a return
                  or failure occurs */
        switch(state) {
            case 0: c = nextChar();
                    if ( c == '<' ) state = 1;
                    else if ( c == '=' ) state = 5;
                    else if ( c == '>' ) state = 6;
                    else fail(); /* lexeme is not a relop */
                    break;
            case 1: ...
            ...
            case 8: retract();
                    retToken.attribute = GT;
                    return(retToken);
        }
    }
}
```

- Function `getRelop()` returns an object of type `TOKEN`, that is, a pair consisting of the token name **relop** and an attribute value (the code for one of the six comparison operators in this case).
- `getRelop()` first creates a new object `retToken` and initializes its first component to RELOP, the symbolic code for token **relop**.



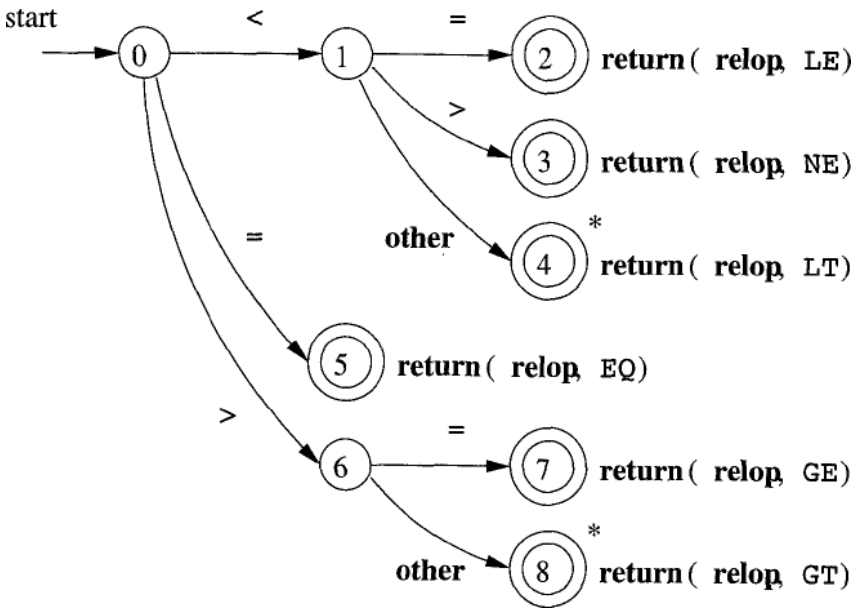

```
TOKEN getRelop()
{
    TOKEN retToken = new(RELOP);
    while(1) { /* repeat character processing until a return
                  or failure occurs */
        switch(state) {
            case 0: c = nextChar();
                    if ( c == '<' ) state = 1;
                    else if ( c == '=' ) state = 5;
                    else if ( c == '>' ) state = 6;
                    else fail(); /* lexeme is not a relop */
                    break;
            case 1: ...
            ...
            case 8: retract();
                    retToken.attribute = GT;
                    return(retToken);
        }
    }
}
```

- We see the typical behavior of a state in case 0, the case where the current state is 0.
- A function `nextchar()` obtains the next character from the input and assigns it to local variable `c`.



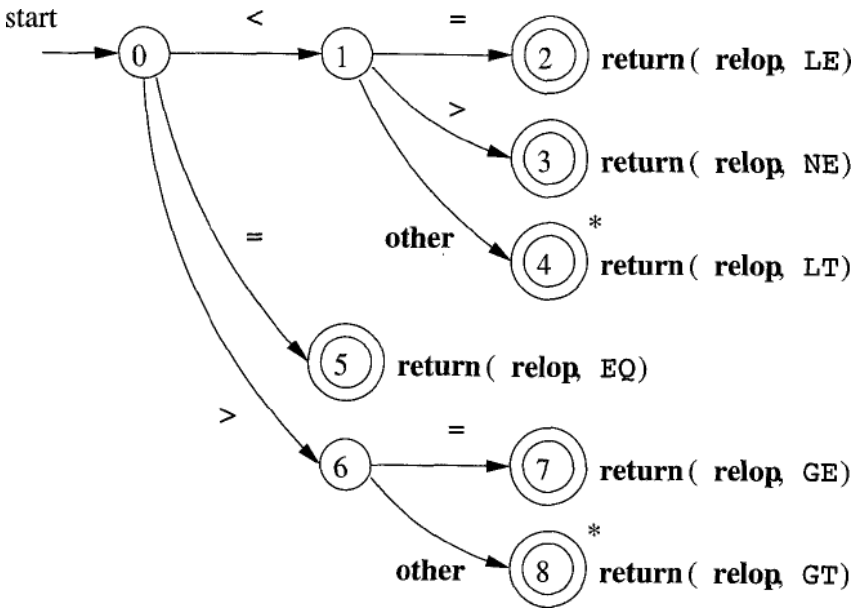

```
TOKEN getRelop()
{
    TOKEN retToken = new(RELOP);
    while(1) { /* repeat character processing until a return
                  or failure occurs */
        switch(state) {
            case 0: c = nextChar();
                    if ( c == '<' ) state = 1;
                    else if ( c == '=' ) state = 5;
                    else if ( c == '>' ) state = 6;
                    else fail(); /* lexeme is not a relop */
                    break;
            case 1: ...
            ...
            case 8: retract();
                    retToken.attribute = GT;
                    return(retToken);
        }
    }
}
```

- We then check c for the three characters we expect to find, making the state transition dictated by the transition diagram in each case.
- For example, if the next input character is $=$, we go to state 5.



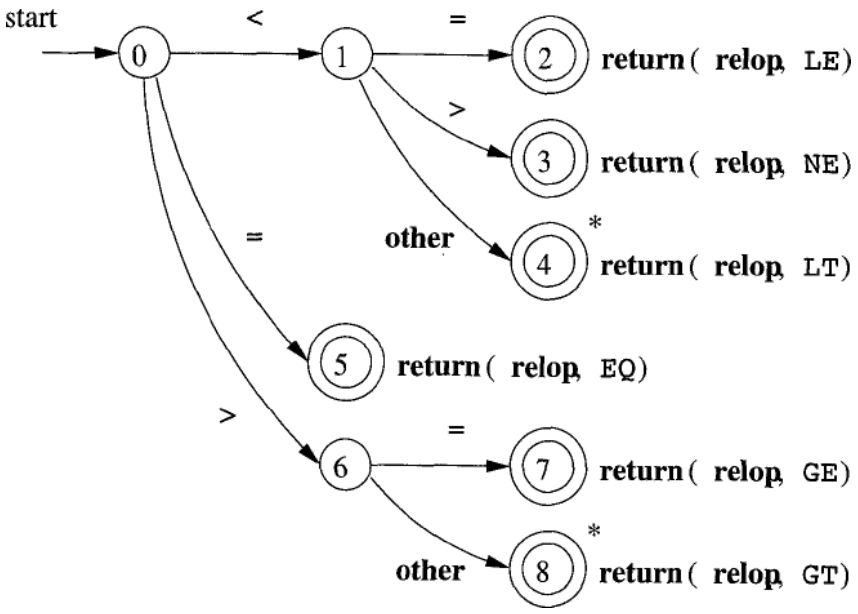

```
TOKEN getRelop()
{
    TOKEN retToken = new(RELOP);
    while(1) { /* repeat character processing until a return
                  or failure occurs */
        switch(state) {
            case 0: c = nextChar();
                    if ( c == '<' ) state = 1;
                    else if ( c == '=' ) state = 5;
                    else if ( c == '>' ) state = 6;
                    else fail(); /* lexeme is not a relop */
                    break;
            case 1: ...
            ...
            case 8: retract();
                    retToken.attribute = GT;
                    return(retToken);
        }
    }
}
```

- If the next input character is not one that can begin a comparison operator, then a function `fail()` is called.
- What `fail()` does depends on the global error recovery strategy of the lexical analyzer.



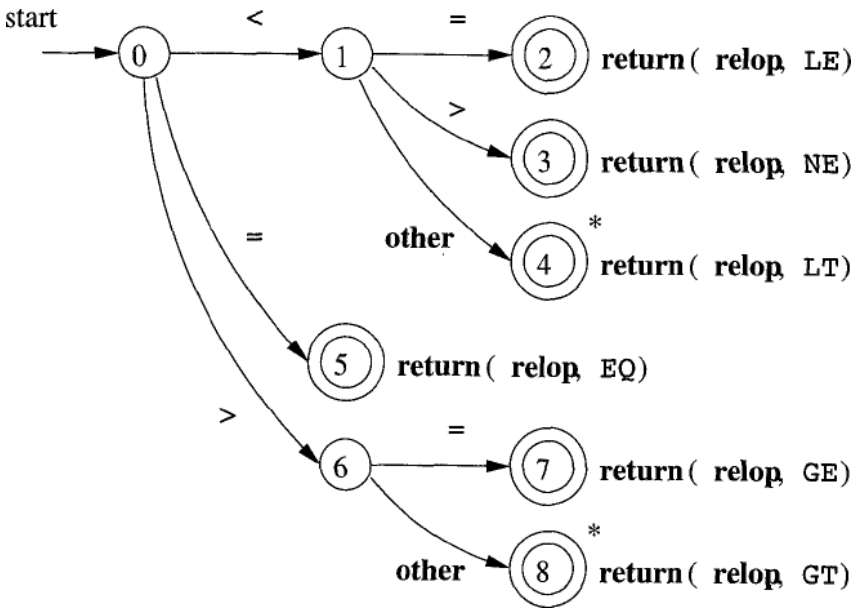

```
TOKEN getRelop()
{
    TOKEN retToken = new(RELOP);
    while(1) { /* repeat character processing until a return
                  or failure occurs */
        switch(state) {
            case 0: c = nextChar();
                    if ( c == '<' ) state = 1;
                    else if ( c == '=' ) state = 5;
                    else if ( c == '>' ) state = 6;
                    else fail(); /* lexeme is not a relop */
                    break;
            case 1: ...
            ...
            case 8: retract();
                    retToken.attribute = GT;
                    return(retToken);
        }
    }
}
```

- It should reset the forward pointer to `lexemeBegin`, in order to allow another transition diagram to be applied to the true beginning of the unprocessed input.
- It might then change the value of `state` to be the start state for another transition diagram, which will search for another token.



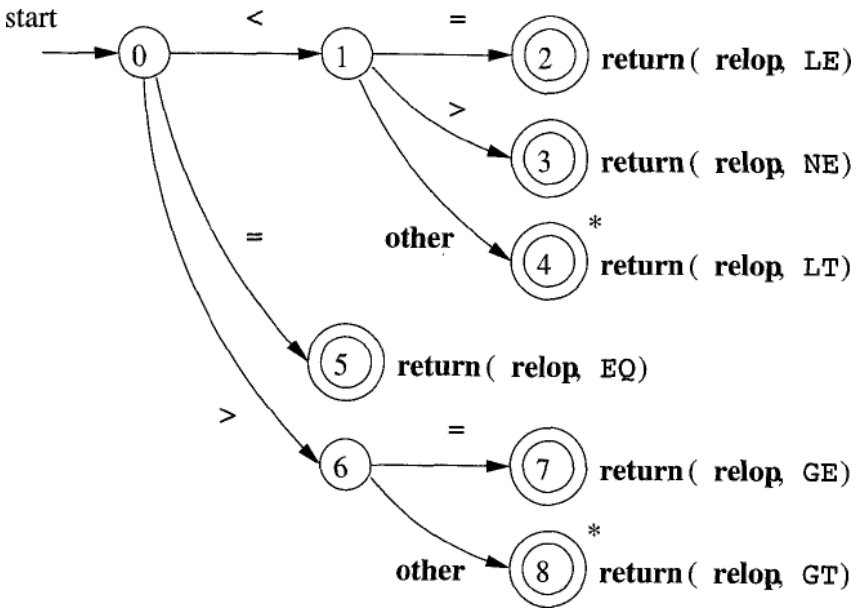

```
TOKEN getRelop()
{
    TOKEN retToken = new(RELOP);
    while(1) { /* repeat character processing until a return
                  or failure occurs */
        switch(state) {
            case 0: c = nextChar();
                    if ( c == '<' ) state = 1;
                    else if ( c == '=' ) state = 5;
                    else if ( c == '>' ) state = 6;
                    else fail(); /* lexeme is not a relop */
                    break;
            case 1: ...
            ...
            case 8: retract();
                    retToken.attribute = GT;
                    return(retToken);
        }
    }
}
```

- Alternatively, if there is no other transition diagram that remains unused, `fail()` could initiate an error-correction phase that will try to repair the input and find a lexeme.




```
TOKEN getRelop()
{
    TOKEN retToken = new(RELOP);
    while(1) { /* repeat character processing until a return
                  or failure occurs */
        switch(state) {
            case 0: c = nextChar();
                    if ( c == '<' ) state = 1;
                    else if ( c == '=' ) state = 5;
                    else if ( c == '>' ) state = 6;
                    else fail(); /* lexeme is not a relop */
                    break;
            case 1: ...
            ...
            case 8: retract();
                    retToken.attribute = GT;
                    return(retToken);
        }
    }
}
```

- We also show the action for state 8.
- Because state 8 bears a $*$, we must retract the input pointer one position (i.e., put `c` back on the input stream).
- That task is accomplished by the function `retract()`.




```
TOKEN getRelop()
{
    TOKEN retToken = new(RELOP);
    while(1) { /* repeat character processing until a return
                  or failure occurs */
        switch(state) {
            case 0: c = nextChar();
                    if ( c == '<' ) state = 1;
                    else if ( c == '=' ) state = 5;
                    else if ( c == '>' ) state = 6;
                    else fail(); /* lexeme is not a relop */
                    break;
            case 1: ...
            ...
            case 8: retract();
                    retToken.attribute = GT;
                    return(retToken);
        }
    }
}
```

- Since state 8 represents the recognition of lexeme $>=$, we set the second component of the returned object, which we suppose is named `attribute`, to `GT`, the code for this operator.

Architecture of a . . . Lexical Analyzer — *continued*

- To place the simulation of one transition diagram in perspective, let us consider the ways code could fit into the entire lexical analyzer.

```
TOKEN getRelop()
{
    TOKEN retToken = new(RELOP);
    while(1) { /* repeat character processing until a return
                  or failure occurs */
        switch(state) {
            case 0: c = nextChar();
                    if ( c == '<' ) state = 1;
                    else if ( c == '=' ) state = 5;
                    else if ( c == '>' ) state = 6;
                    else fail(); /* lexeme is not a relop */
                    break;
            case 1: ...
            ...
            case 8: retract();
                    retToken.attribute = GT;
                    return(retToken);
        }
    }
}
```

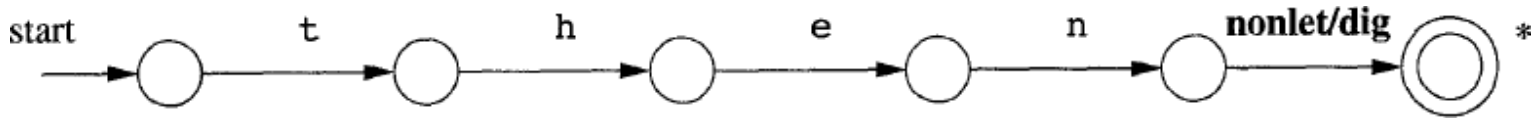
Architecture of a . . . Lexical Analyzer — *continued*

1. We could arrange for the transition diagrams for each token to be tried sequentially.
   - Then, the function `fail()` resets the pointer forward and starts the next transition diagram, each time it is called.

```
TOKEN getRelop()
{
    TOKEN retToken = new(RELOP);
    while(1) { /* repeat character processing until a return
                  or failure occurs */
        switch(state) {
            case 0: c = nextChar();
                    if ( c == '<' ) state = 1;
                    else if ( c == '=' ) state = 5;
                    else if ( c == '>' ) state = 6;
                    else fail(); /* lexeme is not a relop */
                    break;
            case 1: ...
            ...
            case 8: retract();
                    retToken.attribute = GT;
                    return(retToken);
        }
    }
}
```

Architecture of a . . . Lexical Analyzer — *continued*

- This method allows us to use transition diagrams for the individual keywords.



- We have only to use these before we use the diagram for **id**, in order for the keywords to be reserved words.

Architecture of a . . . Lexical Analyzer — *continued*

2. We could run the various transition diagrams "in parallel," feeding the next input character to all of them and allowing each one to make whatever transitions it required.

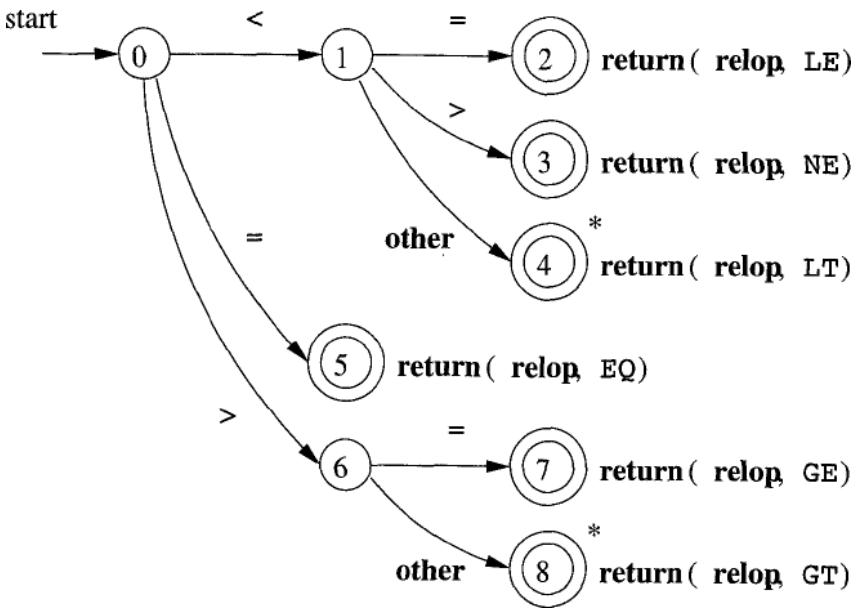
- If we use this strategy, we must be careful to resolve the case where one diagram finds a lexeme that matches its pattern, while one or more other diagrams are still able to process input.
- The normal strategy is to take the longest prefix of the input that matches any pattern.
- That rule allows us to prefer identifier `thenext` to keyword `then`, or the operator `->` to `-`, for example.

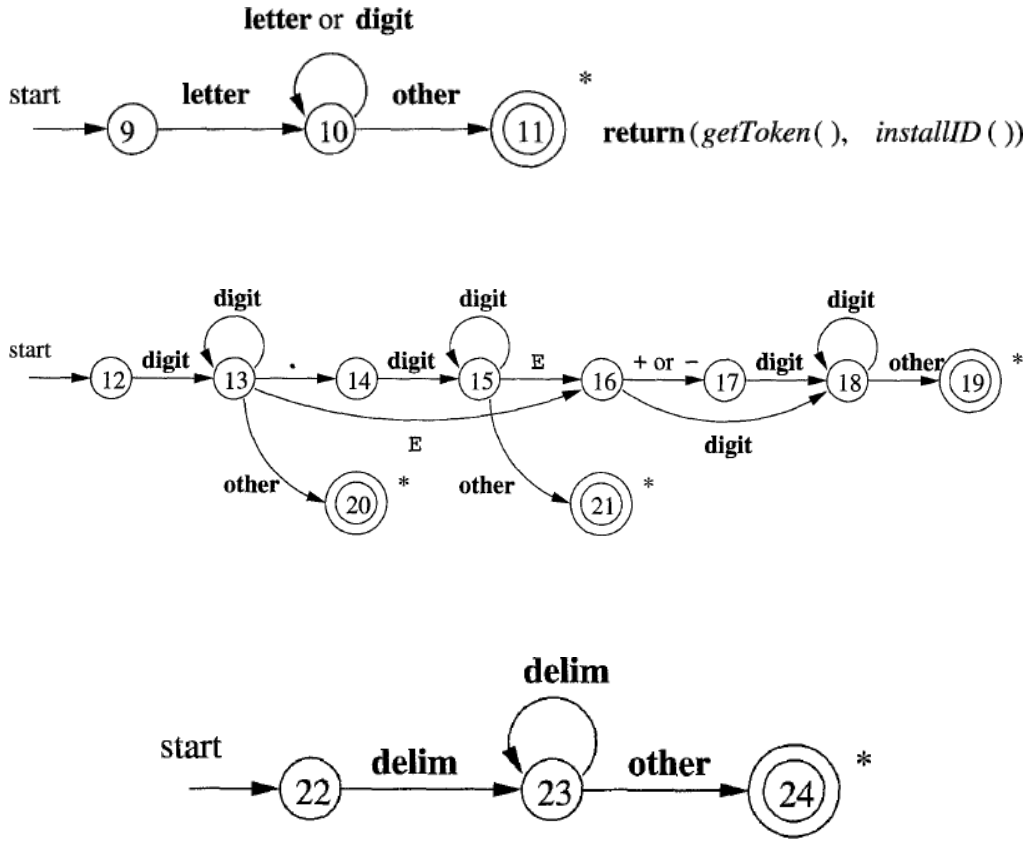
Architecture of a . . . Lexical Analyzer — *continued*

3. The preferred approach, is to combine all the transition diagrams into one.
   - We allow the transition diagram to read input until there is no possible next state.
   - And then take the longest lexeme that matched any pattern, as we discussed in item (2) above.

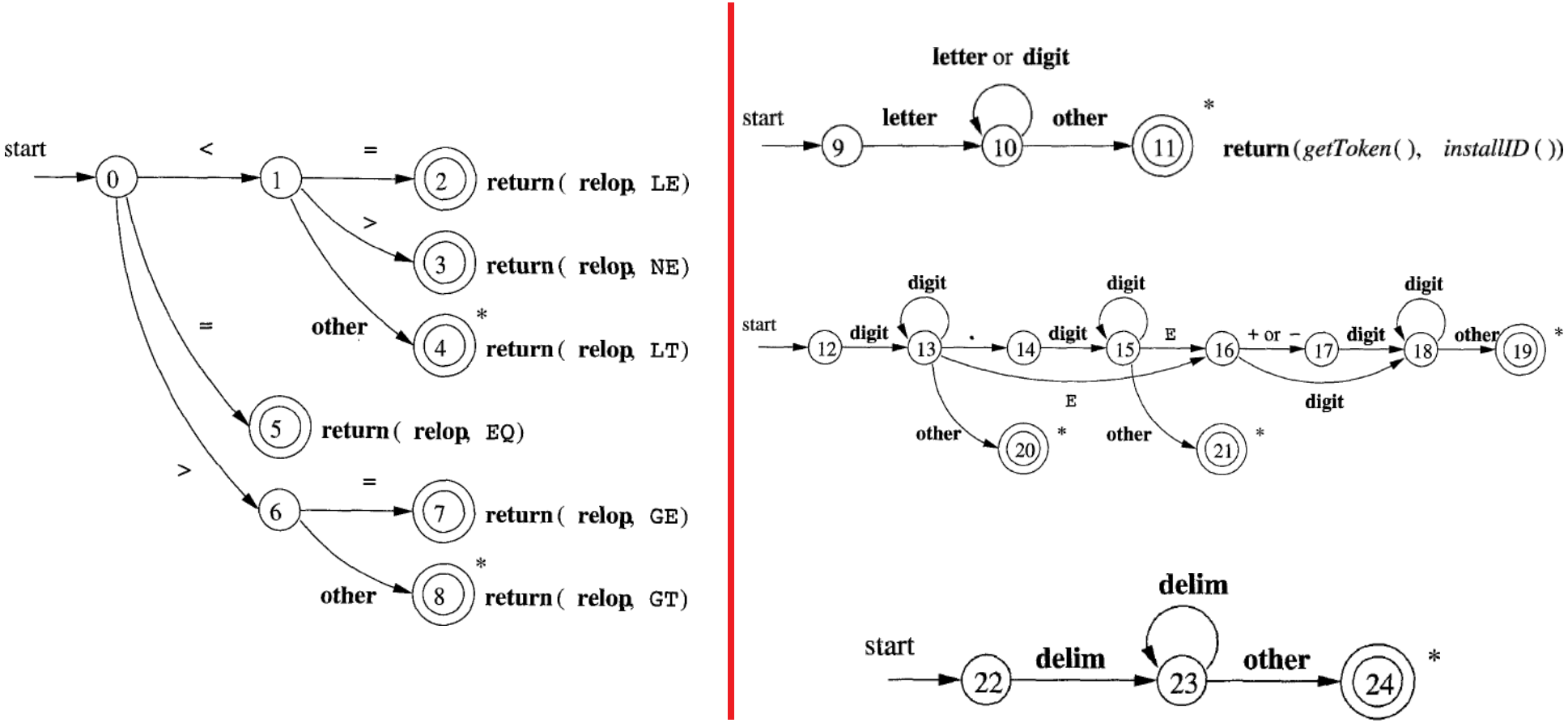
Architecture of a . . . Lexical Analyzer — *continued*

- In our running example, this combination is easy, because no two tokens can start with the same character.
- The first character immediately tells us which token we are looking for.





Architecture of a . . . Lexical Analyzer — *continued*



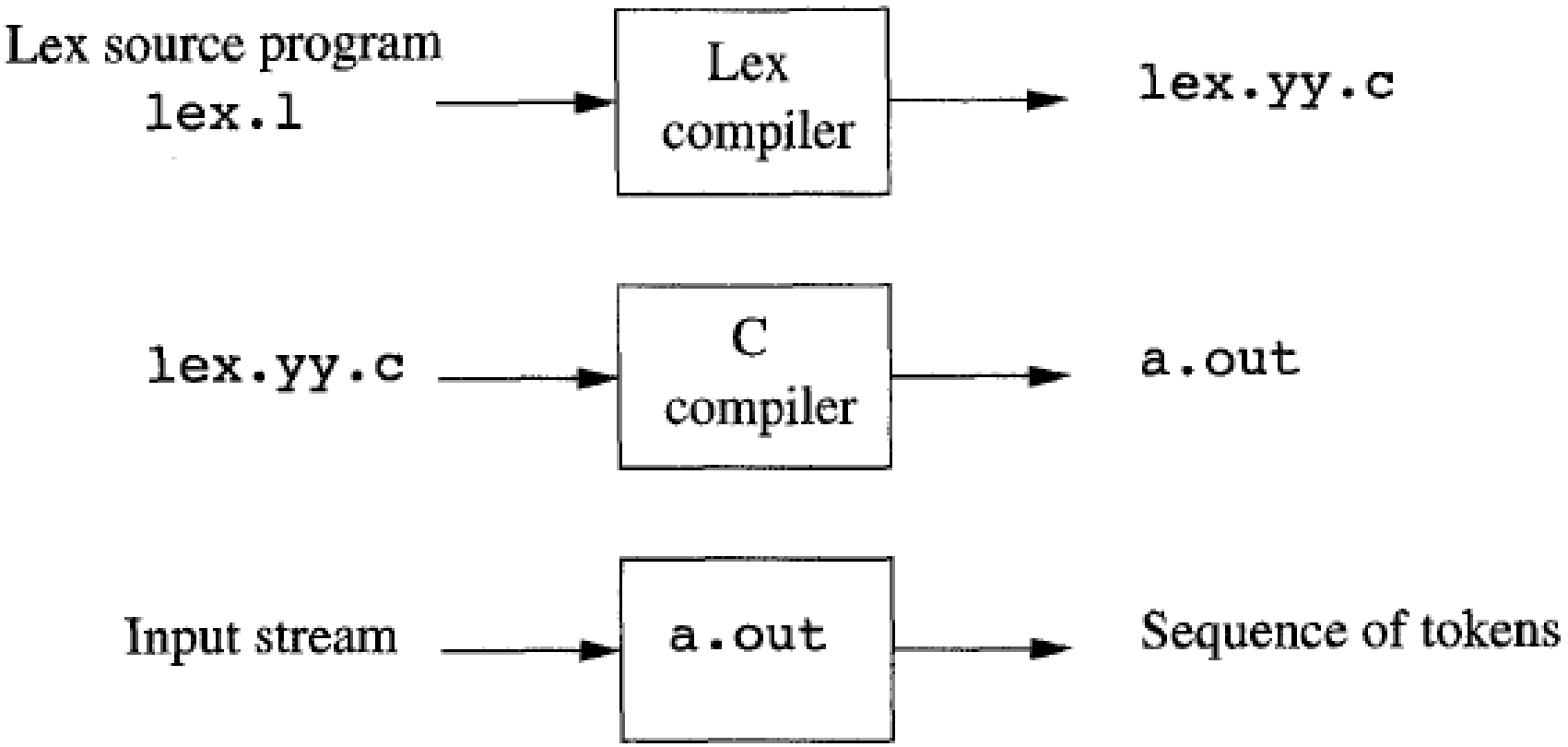
- Thus, we could simply combine states 0, 9, 12, and 22 into one start state, leaving other transitions intact.
- However, in general, the problem of combining transition diagrams for several tokens is more complex.

The Lexical- Analyzer Generator `Lex`

- We introduce a tool called `Lex`, or in a more recent implementation `Flex`.
- This allows one to specify a lexical analyzer by specifying regular expressions to describe patterns for tokens.
- The input notation for the `Lex` tool is referred to as the *Lex language* and the tool itself is the *Lex compiler*.
- Behind the scenes, the Lex compiler transforms the input patterns into a transition diagram and generates code, in a file called `lex.yy.c`, that simulates this transition diagram.

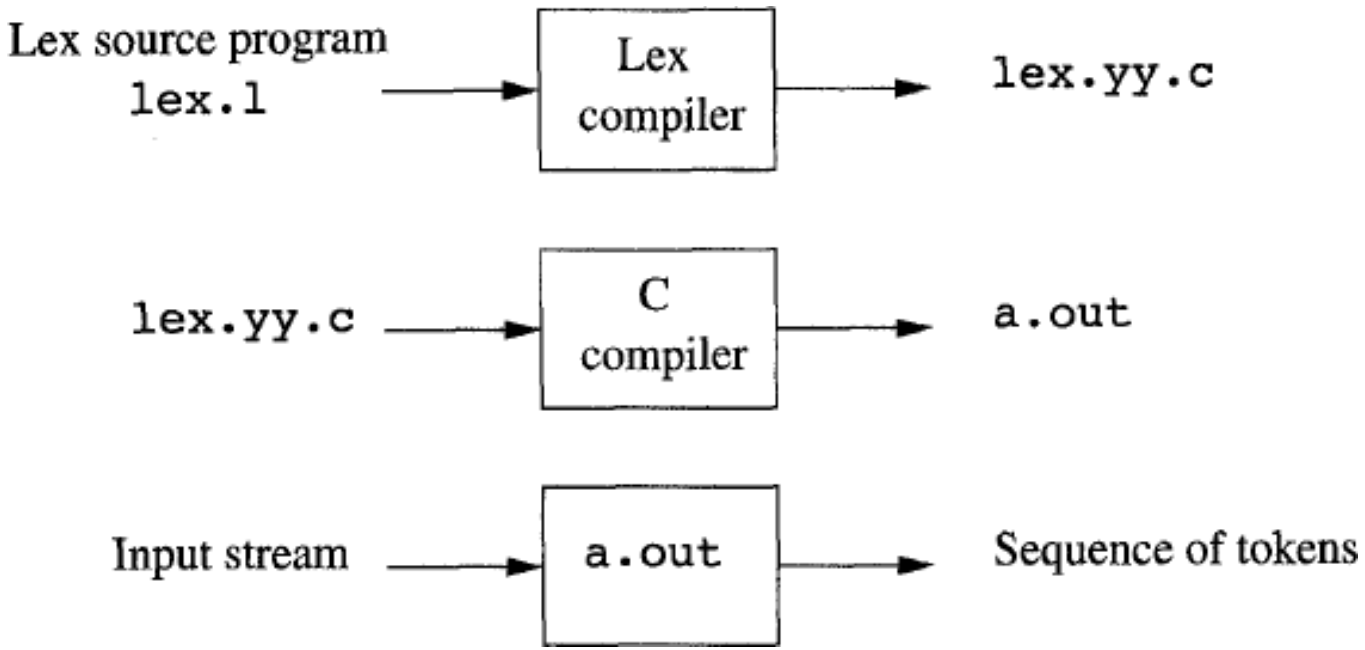
Use of Lex

- Figure suggests how Lex is used.



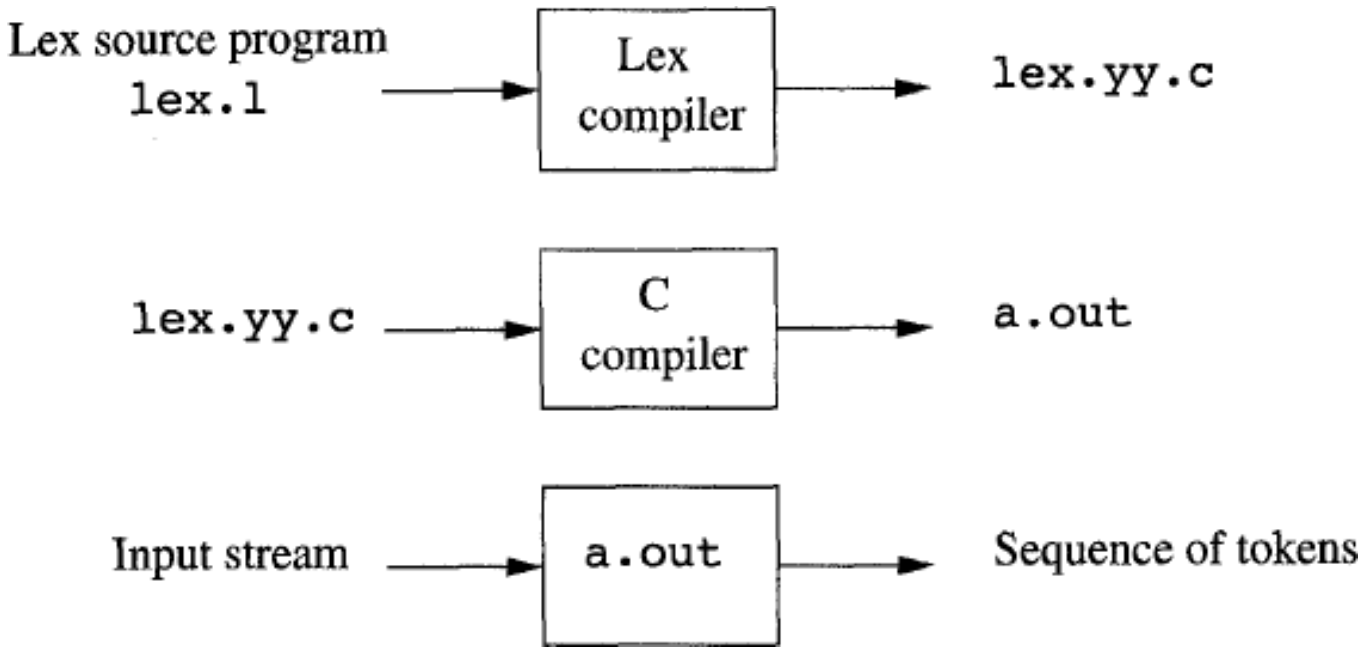
Creating a lexical analyzer with Lex

Use of Lex — *continued*



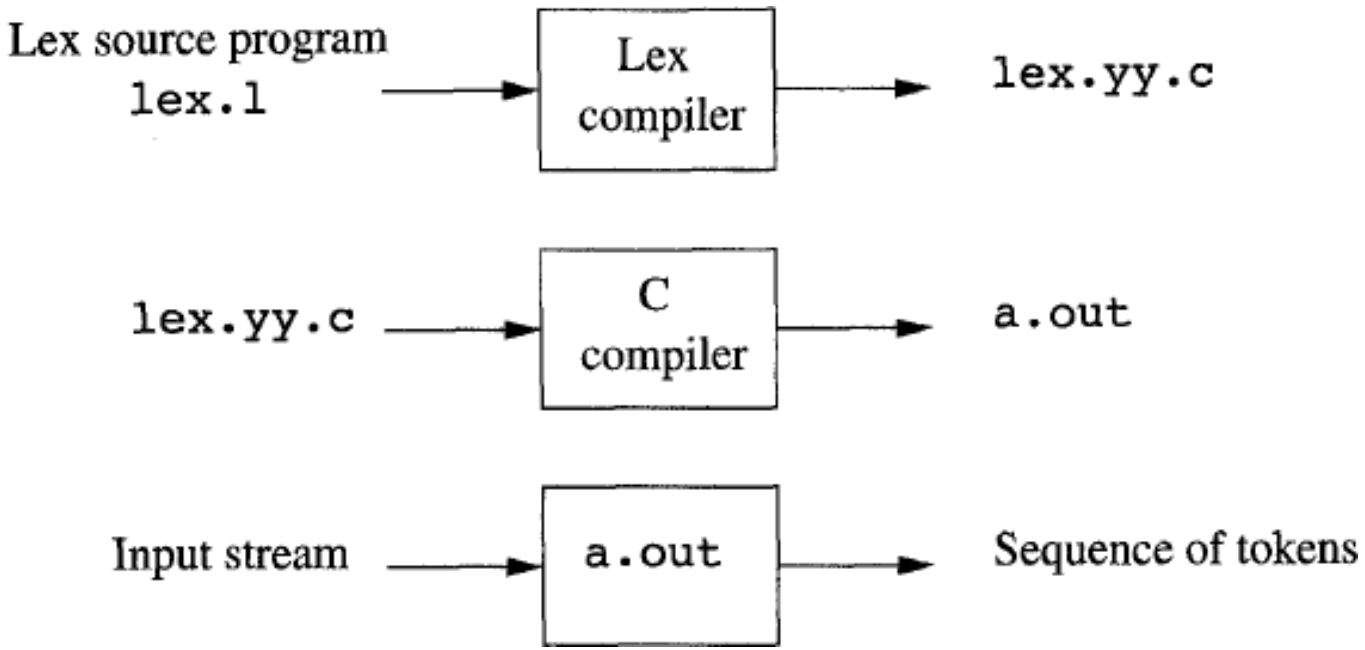
- An input file, which we call `lex.l`, is written in the Lex language and describes the lexical analyzer to be generated.
- The Lex compiler transforms `lex.l` to a C program, in a file that is always named `lex.yy.c`.

Use of Lex — *continued*



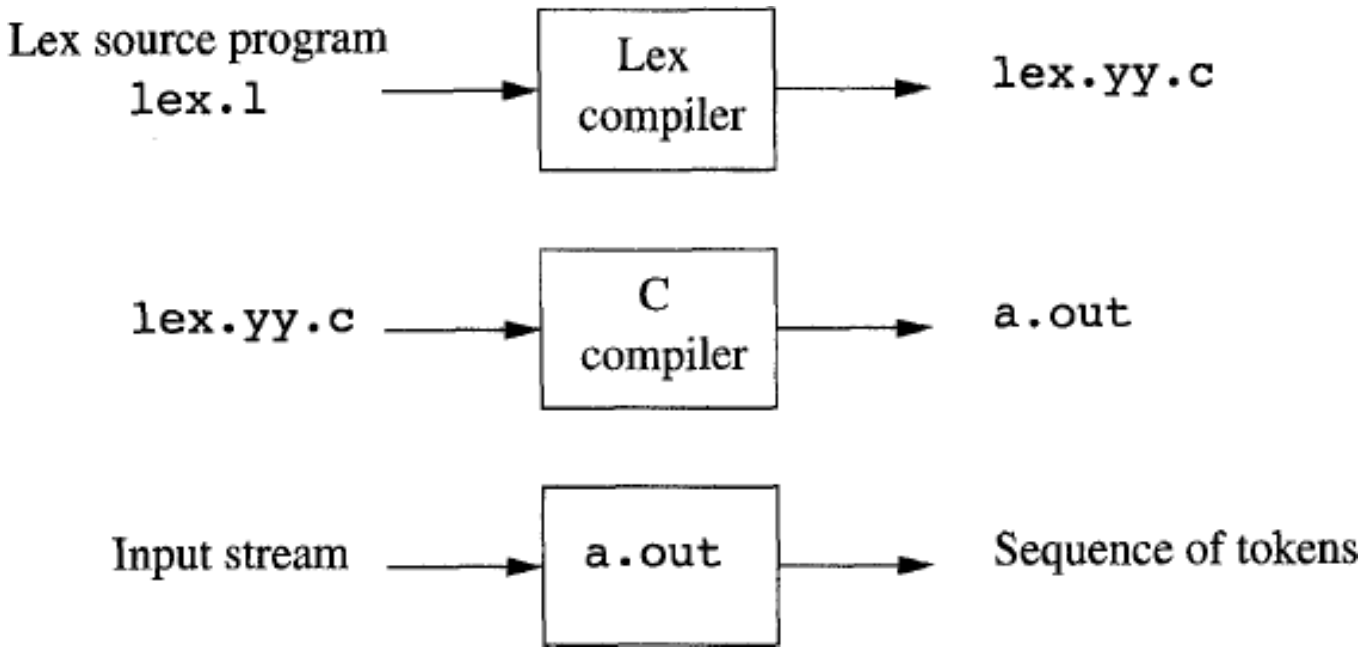
- The latter file is compiled by the C compiler into a file called `a.out`, as always.
- The C-compiler output is a working lexical analyzer that can take a stream of input characters and produce a stream of tokens.

Use of Lex — *continued*



- The normal use of the compiled C program, referred to as `a.out`, is as a subroutine of the parser.
- It is a C function that returns an integer, which is a code for one of the possible token names.

Use of Lex — *continued*



- The attribute value, whether it be another numeric code, a pointer to the symbol table, or nothing, is placed in a global variable `yylval` which is shared between the lexical analyzer and parser, thereby making it simple to return both the name and an attribute value of a token.

Structure of Lex Programs

- A Lex program has the following form:

```
declarations
%%
translation rules
%%
auxiliary functions
```

Structure of Lex Programs — *continued*

```
declarations
%%
translation rules
%%
auxiliary functions
```

- The declarations section includes
  - declarations of variables,
  - manifest constants (identifiers declared to stand for a constant, e.g., the name of a token),
  - and regular definitions.

Structure of Lex Programs — *continued*

```
declarations
%%
translation rules
%%
auxiliary functions
```

- The translation rules each have the form

  ```
  Pattern { Action }
  ```

- Each pattern is a regular expression, which may use the regular definitions of the declaration section.
- The actions are fragments of code, typically written in C, although many variants of Lex using other languages have been created.

Structure of Lex Programs — *continued*

```
declarations
%%
translation rules
%%
auxiliary functions
```

- The third section holds whatever additional functions are used in the actions.
- Alternatively, these functions can be compiled separately and loaded with the lexical analyzer.

Structure of Lex Programs — *continued*

- The lexical analyzer created by Lex behaves in concert with the parser as follows.
- When called by the parser, the lexical analyzer begins reading its remaining input, one character at a time, until it finds the longest prefix of the input that matches one of the patterns P_i.
- It then executes the associated action A_i.
- Typically, A_i will return to the parser.
- But if it does not (e.g., because P_i describes whitespace or comments), then the lexical analyzer proceeds to find additional lexemes, until one of the corresponding actions causes a return to the parser.
- The lexical analyzer returns a single value, the token name, to the parser, but uses the shared, integer variable `yylval` to pass additional information about the lexeme found, if needed.

Structure of Lex Programs — *continued*

- The lexical analyzer created by Lex behaves in concert with the parser as follows.
- When called by the parser, the lexical analyzer begins reading its remaining input, one character at a time, until it finds the longest prefix of the input that matches one of the patterns P_i.
- It then executes the associated action A_i.
- Typically, A_i will return to the parser.
- But if it does not (e.g., because P_i describes whitespace or comments), then the lexical analyzer proceeds to find additional lexemes, until one of the corresponding actions causes a return to the parser.
- The lexical analyzer returns a single value, the token name, to the parser, but uses the shared, integer variable `yylval` to pass additional information about the lexeme found, if needed.

Example

- Figure is a Lex program that recognizes the tokens of the definitions and returns the token found.

```
%{
    /* definitions of manifest constants
    LT, LE, EQ, NE, GT, GE,
    IF, THEN, ELSE, ID, NUMBER, RELOP */
%}

/* regular definitions */
delim     [ \t\n]
ws        {delim}+
letter    [A-Za-z]
digit     [0-9]
id        {letter}({letter}|{digit})*
number    {digit}+(\.{digit}+)?(E[+-]?{digit}+)?

%%

{ws}      {/* no action and no return */}
if        {return(IF);}
then      {return(THEN);}
else      {return(ELSE);}
{id}      {yylval = (int) installID(); return(ID);}
{number}  {yylval = (int) installNum(); return(NUMBER);}
"<"       {yylval = LT; return(RELOP);}
"<="      {yylval = LE; return(RELOP);}
"="       {yylval = EQ; return(RELOP);}
"<>"      {yylval = NE; return(RELOP);}
">"       {yylval = GT; return(RELOP);}
">="      {yylval = GE; return(RELOP);}

%%

int installID() {/* function to install the lexeme, whose
                    first character is pointed to by yytext,
                    and whose length is yyleng, into the
                    symbol table and return a pointer
                    thereto */
}

int installNum() {/* similar to installID, but puts numer-
                     ical constants into a separate table */
}
```

Figure 3.23: Lex program for the tokens of Fig. 3.12

LEXEMES	TOKEN NAME	ATTRIBUTE VALUE
Any *ws*	–	–
if	if	–
then	then	–
else	else	–
Any *id*	id	Pointer to table entry
Any *number*	number	Pointer to table entry
<	relop	LT
<=	relop	LE
=	relop	EQ
<>	relop	NE
>	relop	GT
>=	relop	GE

```
%{
    /* definitions of manifest constants
    LT, LE, EQ, NE, GT, GE,
    IF, THEN, ELSE, ID, NUMBER, RELOP */
%}
```

- In the declarations section we see a pair of special brackets, `%{` and `%}`.
- Anything within these brackets is copied directly to the file `lex.yy.c`, and is not treated as a regular definition.
- It is common to place there the definitions of the manifest constants, using C `#define` statements to associate unique integer codes with each of the manifest constants.
- In our example, we have listed in a comment the names of the manifest constants, `LT`, `IF`, and so on, but have not shown them defined to be particular integer.

```
%{
    /* definitions of manifest constants
    LT, LE, EQ, NE, GT, GE,
    IF, THEN, ELSE, ID, NUMBER, RELOP */
%}
```

- In the declarations section we see a pair of special brackets, `%{` and `%}`.
- Anything within these brackets is copied directly to the file `lex.yy.c`, and is not treated as a regular definition.
- It is common to place there the definitions of the manifest constants, using C `#define` statements to associate unique integer codes with each of the manifest constants.
- In our example, we have listed in a comment the names of the manifest constants, `LT`, `IF`, and so on, but have not shown them defined to be particular integer.

Example — *continued*

```
/* regular definitions */
delim       [ \t\n]
ws          {delim}+
letter      [A-Za-z]
digit       [0-9]
id          {letter}({letter}|{digit})*
number      {digit}+(\.{digit}+)?(E[+-]?{digit}+)?

%%
```

- Also in the declarations section is a sequence of regular definitions.
- These use the extended notation for regular expressions.

Example — *continued*

```
/* regular definitions */
delim      [ \t\n]
ws         {delim}+
letter     [A-Za-z]
digit      [0-9]
id         {letter}({letter}|{digit})*
number     {digit}+(\.{digit}+)?(E[+-]?{digit}+)?

%%
```

- Regular definitions that are used in later definitions or in the patterns of the translation rules are surrounded by curly braces.
- Thus, for instance, *delim* is defined to be a shorthand for the character class consisting of the blank, the tab, and the newline.

Example — *continued*

```
/* regular definitions */
delim       [ \t\n]
ws          {delim}+
letter      [A-Za-z]
digit       [0-9]
id          {letter}({letter}|{digit})*
number      {digit}+(\.{digit}+)?(E[+-]?{digit}+)?

%%
```

- The latter two are represented, as in all UNIX commands, by backslash followed by `t` or `n`, respectively.
- Then, `ws` is defined to be one or more delimiters, by the regular expression `{delim}`.

Example — *continued*

```
/* regular definitions */
delim       [ \t\n]
ws          {delim}+
letter      [A-Za-z]
digit       [0-9]
id          {letter}({letter}|{digit})*
number      {digit}+(\.{digit}+)?(E[+-]?{digit}+)?

%%
```

- Notice that in the definition of *id* and *number*, parentheses are used as grouping metasymbols and do not stand for themselves.
- In contrast, `E` in the definition of *number* stands for itself.

Example — *continued*

```
/* regular definitions */
delim      [ \t\n]
ws         {delim}+
letter     [A-Za-z]
digit      [0-9]
id         {letter}({letter}|{digit})*
number     {digit}+(\.{digit}+)?(E[+-]?{digit}+)?

%%
```

- If we wish to use one of the Lex metasymbols, such as any of the parentheses, `+`, `*`, or `?`, to stand for themselves, we may precede them with a backslash.

Example — *continued*

```
/* regular definitions */
delim       [ \t\n]
ws          {delim}+
letter      [A-Za-z]
digit       [0-9]
id          {letter}({letter}|{digit})*
number      {digit}+(\.{digit}+)?(E[+-]?{digit}+)?

%%
```

- For instance, we see `\.` in the definition of number, to represent the dot, since that character is a metasymbol representing “any character,” as usual in UNIX regular expressions.

```
int installID() {/* function to install the lexeme, whose
                    first character is pointed to by yytext,
                    and whose length is yyleng, into the
                    symbol table and return a pointer
                    thereto */
}

int installNum() {/* similar to installID, but puts numer-
                     ical constants into a separate table */
}
```

- In the auxiliary-function section, we see two such functions, `installID()` and `installNum()`.
- Like the portion of the declaration section that appears between `%{...%}` everything in the auxiliary section is copied directly to file `lex.yy.c`, but may be used in the actions.

```
{ws}      {/* no action and no return */}
if        {return(IF);}
then      {return(THEN);}
else      {return(ELSE);}
{id}      {yylval = (int) installID(); return(ID);}
{number}  {yylval = (int) installNum(); return(NUMBER);}
"<"       {yylval = LT; return(RELOP);}
"<="      {yylval = LE; return(RELOP);}
"="       {yylval = EQ; return(RELOP);}
"<>"      {yylval = NE; return(RELOP);}
">"       {yylval = GT; return(RELOP);}
">="      {yylval = GE; return(RELOP);}

%%
```

- Let us examine some of the patterns and rules in the middle section.

```
{ws}      {/* no action and no return */}
if        {return(IF);}
then      {return(THEN);}
else      {return(ELSE);}
{id}      {yylval = (int) installID(); return(ID);}
{number}  {yylval = (int) installNum(); return(NUMBER);}
"<"       {yylval = LT; return(RELOP);}
"<="      {yylval = LE; return(RELOP);}
"="       {yylval = EQ; return(RELOP);}
"<>"      {yylval = NE; return(RELOP);}
">"       {yylval = GT; return(RELOP);}
">="      {yylval = GE; return(RELOP);}

%%
```

- First, `ws`, an identifier declared in the first section, has an associated empty action.
- If we find whitespace, we do not return to the parser, but look for another lexeme.

```
{ws}      {/* no action and no return */}
if        {return(IF);}
then      {return(THEN);}
else      {return(ELSE);}
{id}      {yylval = (int) installID(); return(ID);}
{number}  {yylval = (int) installNum(); return(NUMBER);}
"<"       {yylval = LT; return(RELOP);}
"<="      {yylval = LE; return(RELOP);}
"="       {yylval = EQ; return(RELOP);}
"<>"      {yylval = NE; return(RELOP);}
">"       {yylval = GT; return(RELOP);}
">="      {yylval = GE; return(RELOP);}

%%
```

- The second token has the simple regular expression pattern `if`.

```
{ws}      {/* no action and no return */}
if        {return(IF);}
then      {return(THEN);}
else      {return(ELSE);}
{id}      {yylval = (int) installID(); return(ID);}
{number}  {yylval = (int) installNum(); return(NUMBER);}
"<"       {yylval = LT; return(RELOP);}
"<="      {yylval = LE; return(RELOP);}
"="       {yylval = EQ; return(RELOP);}
"<>"      {yylval = NE; return(RELOP);}
">"       {yylval = GT; return(RELOP);}
">="      {yylval = GE; return(RELOP);}

%%
```

- Should we see the two letters `if` on the input, and they are not followed by another letter or digit, then the lexical analyzer consumes these two letters from the input and returns the token name `IF`, that is, the integer for which the manifest constant `IF` stands.

```
{ws}      {/* no action and no return */}
if        {return(IF);}
then      {return(THEN);}
else      {return(ELSE);}
{id}      {yylval = (int) installID(); return(ID);}
{number}  {yylval = (int) installNum(); return(NUMBER);}
"<"       {yylval = LT; return(RELOP);}
"<="      {yylval = LE; return(RELOP);}
"="       {yylval = EQ; return(RELOP);}
"<>"      {yylval = NE; return(RELOP);}
">"       {yylval = GT; return(RELOP);}
">="      {yylval = GE; return(RELOP);}

%%
```

- If we see letters or digits after `if` this would cause the lexical analyzer to find a longer prefix of the input matching the pattern for **id**.
- Keywords `then` and `else` are treated similarly.

```
{ws}      {/* no action and no return */}
if        {return(IF);}
then      {return(THEN);}
else      {return(ELSE);}
{id}      {yylval = (int) installID(); return(ID);}
{number}  {yylval = (int) installNum(); return(NUMBER);}
"<"       {yylval = LT; return(RELOP);}
"<="      {yylval = LE; return(RELOP);}
"="       {yylval = EQ; return(RELOP);}
"<>"      {yylval = NE; return(RELOP);}
">"       {yylval = GT; return(RELOP);}
">="      {yylval = GE; return(RELOP);}

%%
```

- The fifth token has the pattern defined by **id**.
- Note that, although keywords like `if` match this pattern as well as an earlier pattern, Lex chooses whichever pattern is listed first in situations where the longest matching prefix matches two or more patterns.

```
{ws}      {/* no action and no return */}
if        {return(IF);}
then      {return(THEN);}
else      {return(ELSE);}
{id}      {yylval = (int) installID(); return(ID);}
{number}  {yylval = (int) installNum(); return(NUMBER);}
"<"       {yylval = LT; return(RELOP);}
"<="      {yylval = LE; return(RELOP);}
"="       {yylval = EQ; return(RELOP);}
"<>"      {yylval = NE; return(RELOP);}
">"       {yylval = GT; return(RELOP);}
">="      {yylval = GE; return(RELOP);}

%%
```

- The action taken when id is matched is threefold.

```
{ws}      {/* no action and no return */}
if        {return(IF);}
then      {return(THEN);}
else      {return(ELSE);}
{id}      {yylval = (int) installID(); return(ID);}
{number}  {yylval = (int) installNum(); return(NUMBER);}
"<"       {yylval = LT; return(RELOP);}
"<="      {yylval = LE; return(RELOP);}
"="       {yylval = EQ; return(RELOP);}
"<>"      {yylval = NE; return(RELOP);}
">"       {yylval = GT; return(RELOP);}
">="      {yylval = GE; return(RELOP);}

%%
```

1. Function `installID()` is called to place the lexeme found in the symbol table.

```
{ws}      {/* no action and no return */}
if        {return(IF);}
then      {return(THEN);}
else      {return(ELSE);}
{id}      {yylval = (int) installID(); return(ID);}
{number}  {yylval = (int) installNum(); return(NUMBER);}
"<"       {yylval = LT; return(RELOP);}
"<="      {yylval = LE; return(RELOP);}
"="       {yylval = EQ; return(RELOP);}
"<>"      {yylval = NE; return(RELOP);}
">"       {yylval = GT; return(RELOP);}
">="      {yylval = GE; return(RELOP);}

%%
```

2. This function returns a pointer to the symbol table, which is placed in global variable `yylval`, where it can be used by the parser or a later component of the compiler.

```
{ws}      {/* no action and no return */}
if        {return(IF);}
then      {return(THEN);}
else      {return(ELSE);}
{id}      {yylval = (int) installID(); return(ID);}
{number}  {yylval = (int) installNum(); return(NUMBER);}
"<"       {yylval = LT; return(RELOP);}
"<="      {yylval = LE; return(RELOP);}
"="       {yylval = EQ; return(RELOP);}
"<>"      {yylval = NE; return(RELOP);}
">"       {yylval = GT; return(RELOP);}
">="      {yylval = GE; return(RELOP);}

%%
```

- Note that `installID()` has available to it two variables that are set automatically by the lexical analyzer that Lex generates:

```
{ws}      {/* no action and no return */}
if        {return(IF);}
then      {return(THEN);}
else      {return(ELSE);}
{id}      {yylval = (int) installID(); return(ID);}
{number}  {yylval = (int) installNum(); return(NUMBER);}
"<"       {yylval = LT; return(RELOP);}
"<="      {yylval = LE; return(RELOP);}
"="       {yylval = EQ; return(RELOP);}
"<>"      {yylval = NE; return(RELOP);}
">"       {yylval = GT; return(RELOP);}
">="      {yylval = GE; return(RELOP);}

%%
```

(a) `yytext` is a pointer to the beginning of the lexeme, analogous to `lexemeBegin`.

(b) `yyleng` is the length of the lexeme found.

```
{ws}      {/* no action and no return */}
if        {return(IF);}
then      {return(THEN);}
else      {return(ELSE);}
{id}      {yylval = (int) installID(); return(ID);}
{number}  {yylval = (int) installNum(); return(NUMBER);}
"<"       {yylval = LT; return(RELOP);}
"<="      {yylval = LE; return(RELOP);}
"="       {yylval = EQ; return(RELOP);}
"<>"      {yylval = NE; return(RELOP);}
">"       {yylval = GT; return(RELOP);}
">="      {yylval = GE; return(RELOP);}

%%
```

3. The token name `ID` is returned to the parser.

```
{ws}      {/* no action and no return */}
if        {return(IF);}
then      {return(THEN);}
else      {return(ELSE);}
{id}      {yylval = (int) installID(); return(ID);}
{number}  {yylval = (int) installNum(); return(NUMBER);}
"<"       {yylval = LT; return(RELOP);}
"<="      {yylval = LE; return(RELOP);}
"="       {yylval = EQ; return(RELOP);}
"<>"      {yylval = NE; return(RELOP);}
">"       {yylval = GT; return(RELOP);}
">="      {yylval = GE; return(RELOP);}

%%
```

- The action taken when a lexeme matching the pattern number is similar, using the auxiliary function `installNum()`.

Conflict Resolution in `Lex`

- We have alluded to the two rules that `Lex` uses to decide on the proper lexeme to select, when several prefixes of the input match one or more patterns:
  1. Always prefer a longer prefix to a shorter prefix.
  2. If the longest possible prefix matches two or more patterns, prefer the pattern listed first in the Lex program.

Example

1. Always prefer a longer prefix to a shorter prefix.
2. If the longest possible prefix matches two or more patterns, prefer the pattern listed first in the Lex program.

- The first rule tells us to continue reading letters and digits to find the longest prefix of these characters to group as an identifier.
- It also tells us to treat $<=$ as a single lexeme, rather than selecting $<$ as one lexeme and $=$ as the next lexeme.
- The second rule makes keywords reserved, if we list the keywords before **id** in the program.

```
{ws}      {/* no action and no return */}
if        {return(IF);}
then      {return(THEN);}
else      {return(ELSE);}
{id}      {yylval = (int) installID(); return(ID);}
{number}  {yylval = (int) installNum(); return(NUMBER);}
"<"       {yylval = LT; return(RELOP);}
"<="      {yylval = LE; return(RELOP);}
"="       {yylval = EQ; return(RELOP);}
"<>"      {yylval = NE; return(RELOP);}
">"       {yylval = GT; return(RELOP);}
">="      {yylval = GE; return(RELOP);}

%%
```

- For instance, if `then` is determined to be the longest prefix of the input that matches any pattern, and the pattern `then` precedes **id**, then the token `THEN` is returned, rather than `ID`.



End of Slides

CSE 309 (Compilers)

Basic Concepts of Parsing in Compiler

Dr. Muhammad Masroor Ali

Professor
Department of Computer Science and Engineering
Bangladesh University of Engineering and Technology
Dhaka-1205, Bangladesh

January 2026

Adapted by
Ahmed Mahir Sultan Rumi
Lecturer, Department of CSE, BUET

Version: 1.2, Last modified: July 13, 2026

Top-Down Parsing

- We introduce top-down parsing by considering a grammar that is well-suited for this class of methods.

Top-Down Parsing — *continued*

$$
\begin{array}{lcl}
\textit{stmt} & \rightarrow & \textbf{expr ;} \\
 & | & \textbf{if (expr)} \ \textit{stmt} \\
 & | & \textbf{for (} \ \textit{optexpr} \ \textbf{;} \ \textit{optexpr} \ \textbf{;} \ \textit{optexpr} \ \textbf{)} \ \textit{stmt} \\
 & | & \textbf{other} \\
 & & \\
\textit{optexpr} & \rightarrow & \epsilon \\
 & | & \textbf{expr}
\end{array}
$$

A grammar for some statements in C and Java

- The grammar generates a subset of the statements of C or Java.
- We use the boldface terminals **if** and **for** for the keywords "if" and "for", respectively, to emphasize that these character sequences are treated as units, *i.e.*, as single terminal symbols.

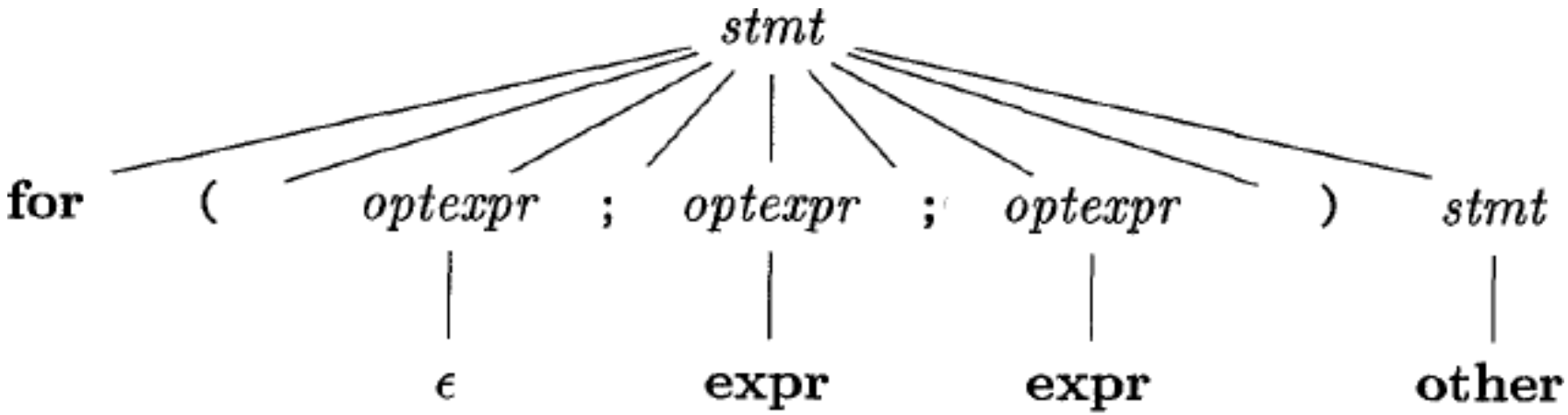
Top-Down Parsing — *continued*

$$
\begin{array}{lcl}
\textit{stmt} & \rightarrow & \textbf{expr ;} \\
 & | & \textbf{if (expr)} \ \textit{stmt} \\
 & | & \textbf{for (} \ \textit{optexpr} \ \textbf{;} \ \textit{optexpr} \ \textbf{;} \ \textit{optexpr} \ \textbf{)} \ \textit{stmt} \\
 & | & \textbf{other} \\
 & & \\
\textit{optexpr} & \rightarrow & \epsilon \\
 & | & \textbf{expr}
\end{array}
$$

A grammar for some statements in C and Java

- Further, the terminal **expr** represents expressions.
- A more complete grammar would use a nonterminal *expr* and have productions for nonterminal *expr*.
- Similarly, **other** is a terminal representing other statement constructs.

Top-Down Parsing — *continued*



A parse tree according to the grammar

- The top-down construction of a parse tree is done by starting with the root, labeled with the starting nonterminal *stmt*, and repeatedly performing the following two steps.

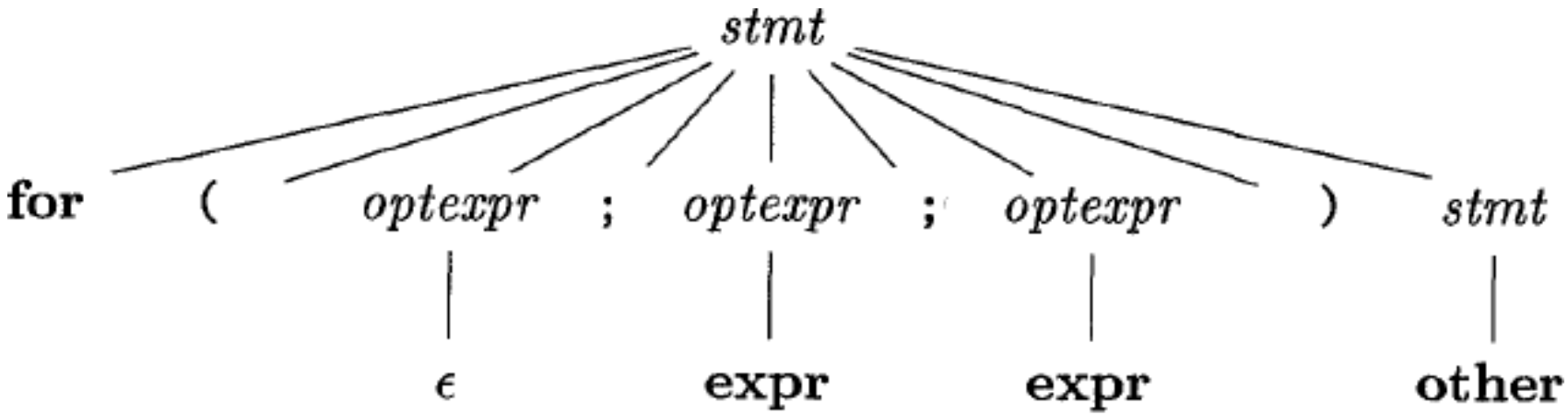
Top-Down Parsing — *continued*

$$
\begin{array}{lcl}
\textit{stmt} & \rightarrow & \textbf{expr ;} \\
 & | & \textbf{if (expr)} \ \textit{stmt} \\
 & | & \textbf{for (} \ \textit{optexpr} \ \textbf{;} \ \textit{optexpr} \ \textbf{;} \ \textit{optexpr} \ \textbf{)} \ \textit{stmt} \\
 & | & \textbf{other} \\
 & & \\
\textit{optexpr} & \rightarrow & \epsilon \\
 & | & \textbf{expr}
\end{array}
$$

A grammar for some statements in C and Java

- The top-down construction of a parse tree is done by starting with the root, labeled with the starting nonterminal *stmt*, and repeatedly performing the following two steps.

Top-Down Parsing — *continued*



A parse tree according to the grammar

1. At node N, labeled with nonterminal A, select one of the productions for A and construct children at N for the symbols in the production body.

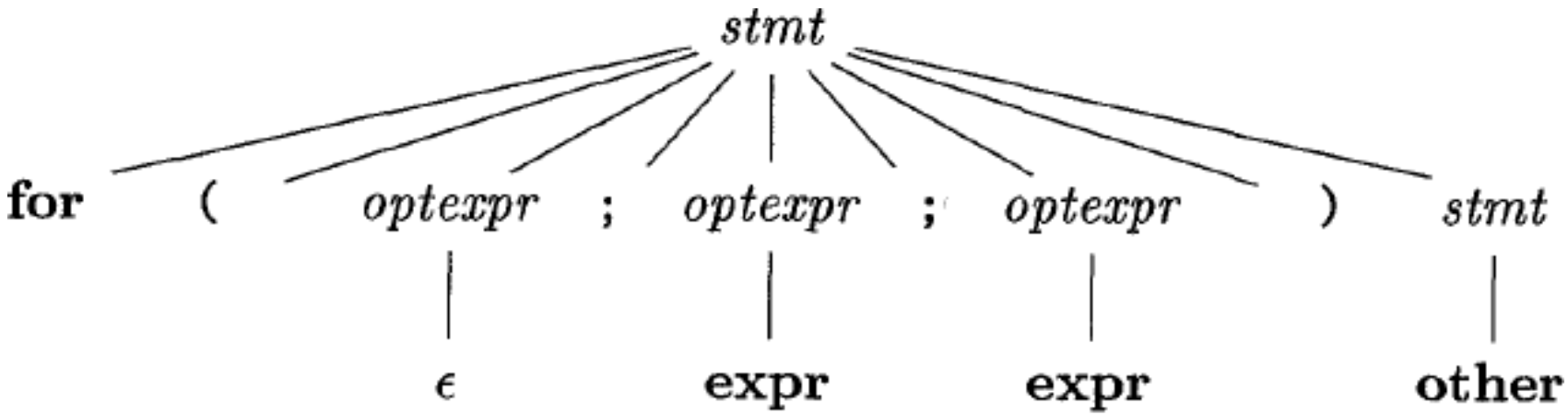
Top-Down Parsing — *continued*

$$
\begin{array}{lcl}
\mathit{stmt} & \rightarrow & \textbf{expr ;} \\
 & | & \textbf{if (expr)}\ \mathit{stmt} \\
 & | & \textbf{for (}\ \mathit{optexpr}\ \textbf{;}\ \mathit{optexpr}\ \textbf{;}\ \mathit{optexpr}\ \textbf{)}\ \mathit{stmt} \\
 & | & \textbf{other} \\
 & & \\
\mathit{optexpr} & \rightarrow & \epsilon \\
 & | & \textbf{expr}
\end{array}
$$

A grammar for some statements in C and Java

1. At node N, labeled with nonterminal A, select one of the productions for A and construct children at N for the symbols in the production body.

Top-Down Parsing — *continued*



A parse tree according to the grammar

2. Find the next node at which a subtree is to be constructed, typically the leftmost unexpanded nonterminal of the tree.

Top-Down Parsing — *continued*

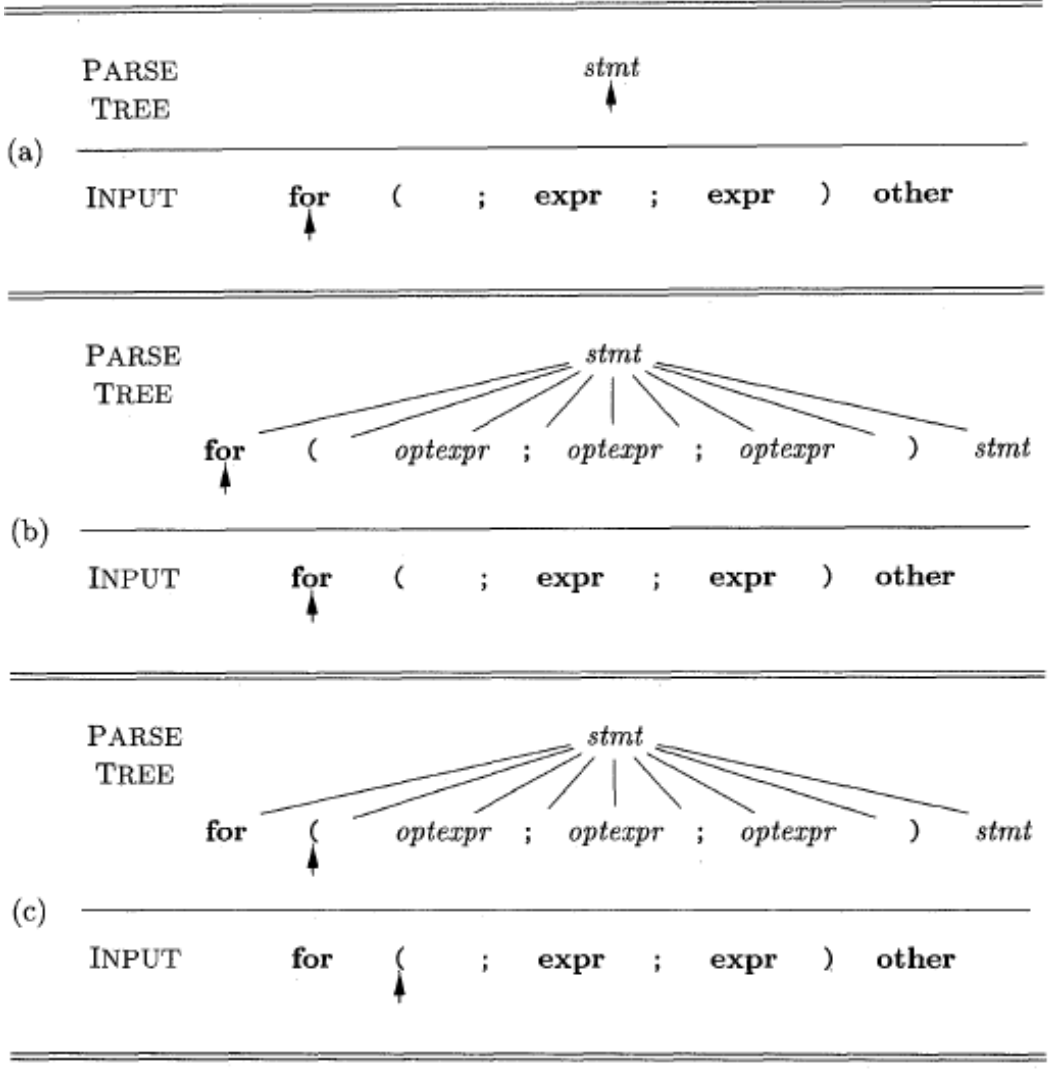
$$
\begin{array}{lcl}
\textit{stmt} & \rightarrow & \textbf{expr ;} \\
 & | & \textbf{if (expr)} \ \textit{stmt} \\
 & | & \textbf{for (} \ \textit{optexpr} \ \textbf{;} \ \textit{optexpr} \ \textbf{;} \ \textit{optexpr} \ \textbf{)} \ \textit{stmt} \\
 & | & \textbf{other} \\
 & & \\
\textit{optexpr} & \rightarrow & \epsilon \\
 & | & \textbf{expr}
\end{array}
$$

A grammar for some statements in C and Java

2. Find the next node at which a subtree is to be constructed, typically the leftmost unexpanded nonterminal of the tree.

Top-Down Parsing — *continued*

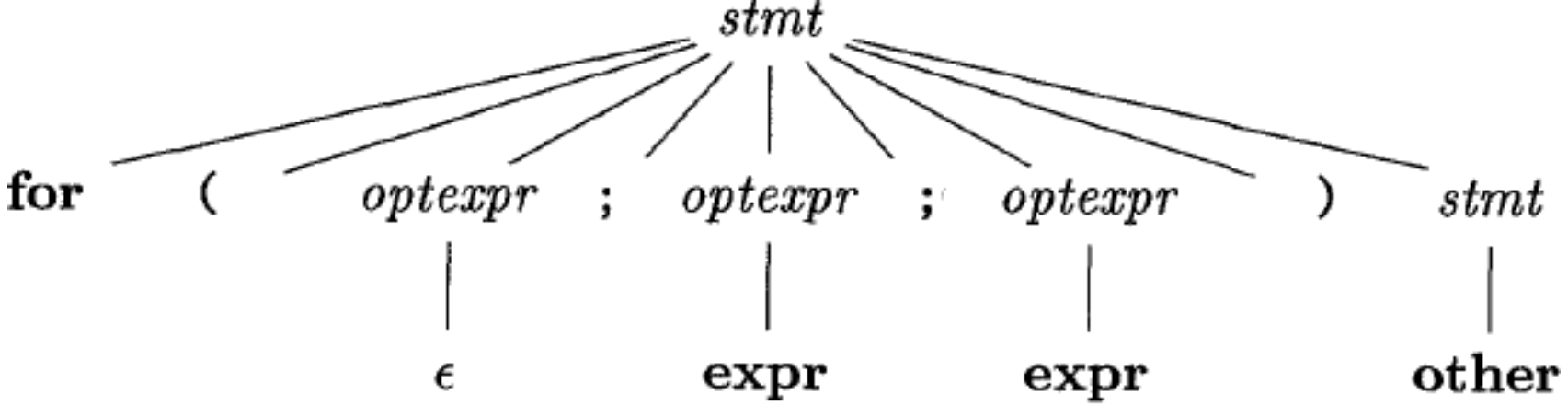
- For some grammars, the above steps can be implemented during a single left-to-right scan of the input string.
- The current terminal being scanned in the input is frequently referred to as the *lookahead* symbol.
- Initially, the lookahead symbol is the first, *i.e.*, leftmost terminal of the input string.



Top-down parsing while scanning the input from left to right

- Figure illustrates the construction of the parse tree in for the input string

  for (; **expr** ; **expr**) **other**

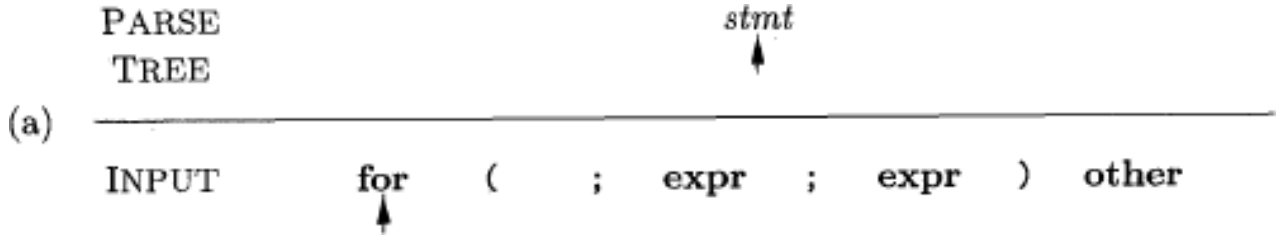


A parse tree according to the grammar

- Figure illustrates the construction of the parse tree in for the input string

  for (; **expr** ; **expr**) **other**

for (; expr ; expr) other

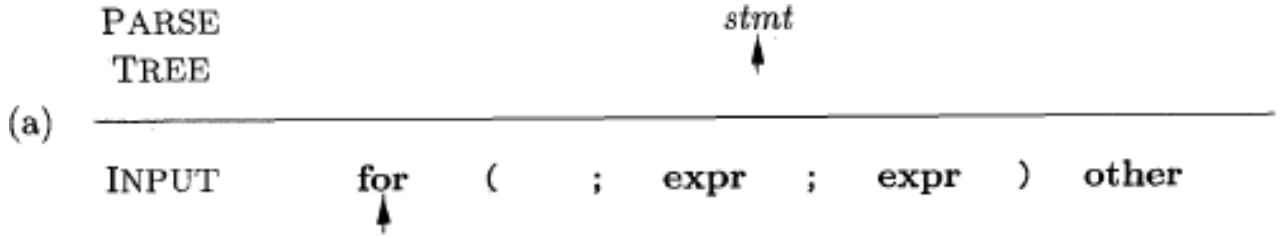


$$
\begin{array}{lcl}
stmt & \rightarrow & \textbf{expr ;} \\
 & | & \textbf{if (expr)} \ stmt \\
 & | & \textbf{for (} \ optexpr \ \textbf{;} \ optexpr \ \textbf{;} \ optexpr \ \textbf{)} \ stmt \\
 & | & \textbf{other} \\
 & & \\
optexpr & \rightarrow & \epsilon \\
 & | & \textbf{expr}
\end{array}
$$

A grammar for some statements in C and Java

- Initially, the terminal **for** is the lookahead symbol, and the known part of the parse tree consists of the root, labeled with the starting nonterminal *stmt*.
- The objective is to construct the remainder of the parse tree in such a way that the string generated by the parse tree matches the input string.

for (; expr ; expr) other

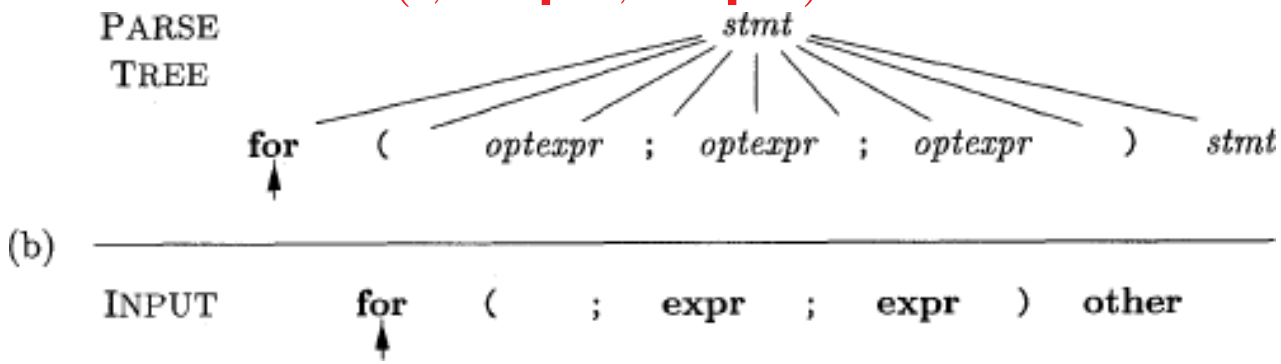


$$
\begin{array}{lcl}
stmt & \rightarrow & \textbf{expr ;} \\
 & | & \textbf{if (expr)} \ stmt \\
 & | & \textbf{for (} \ optexpr \ \textbf{;} \ optexpr \ \textbf{;} \ optexpr \ \textbf{)} \ stmt \\
 & | & \textbf{other} \\
 & & \\
optexpr & \rightarrow & \epsilon \\
 & | & \textbf{expr}
\end{array}
$$

A grammar for some statements in C and Java

- For a match to occur, the nonterminal *stmt* in must derive a string that starts with the lookahead symbol **for**.
- In the grammar, there is just one production for *stmt* that can derive such a string, so we select it, and construct the children of the root labeled with the symbols in the production body.
- This expansion of the parse tree is shown in (b).

for (; expr ; expr) other

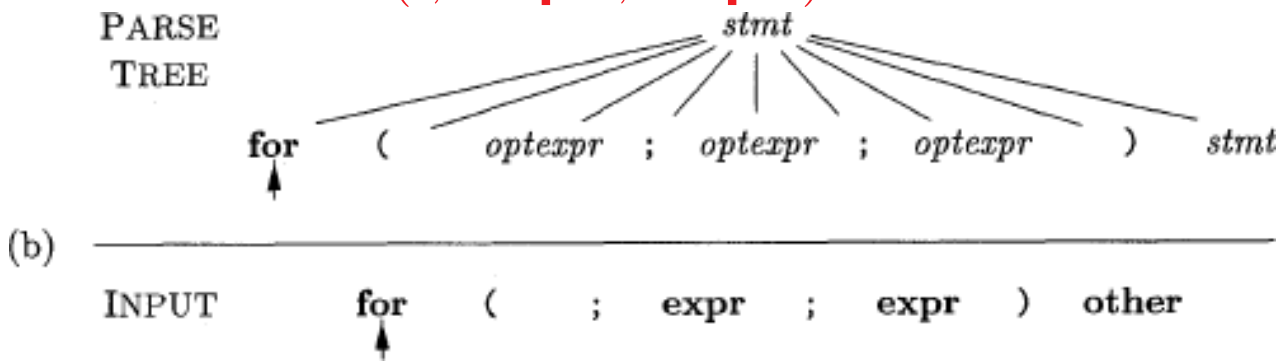


$$
\begin{array}{lcl}
stmt & \rightarrow & \textbf{expr ;} \\
 & | & \textbf{if (expr)} \ stmt \\
 & | & \textbf{for (} \ optexpr \ \textbf{;} \ optexpr \ \textbf{;} \ optexpr \ \textbf{)} \ stmt \\
 & | & \textbf{other} \\
 & & \\
optexpr & \rightarrow & \epsilon \\
 & | & \textbf{expr}
\end{array}
$$

A grammar for some statements in C and Java

- For a match to occur, the nonterminal *stmt* in must derive a string that starts with the lookahead symbol **for**.
- In the grammar, there is just one production for *stmt* that can derive such a string, so we select it, and construct the children of the root labeled with the symbols in the production body.
- This expansion of the parse tree is shown in (b).

for (; expr ; expr) other



- Each of the three snapshots has arrows marking the lookahead symbol in the input and the node in the parse tree that is being considered.
- Once children are constructed at a node, we next consider the leftmost child.
- In (b), children have just been constructed at the root, and the leftmost child labeled with for is being considered.

$$
\begin{array}{lcl}
\textit{stmt} & \rightarrow & \textbf{expr ;} \\
 & | & \textbf{if (expr)} \ \textit{stmt} \\
 & | & \textbf{for (} \ \textit{optexpr} \ \textbf{;} \ \textit{optexpr} \ \textbf{;} \ \textit{optexpr} \ \textbf{)} \ \textit{stmt} \\
 & | & \textbf{other} \\
 & & \\
\textit{optexpr} & \rightarrow & \epsilon \\
 & | & \textbf{expr}
\end{array}
$$

A grammar for some statements in C and Java

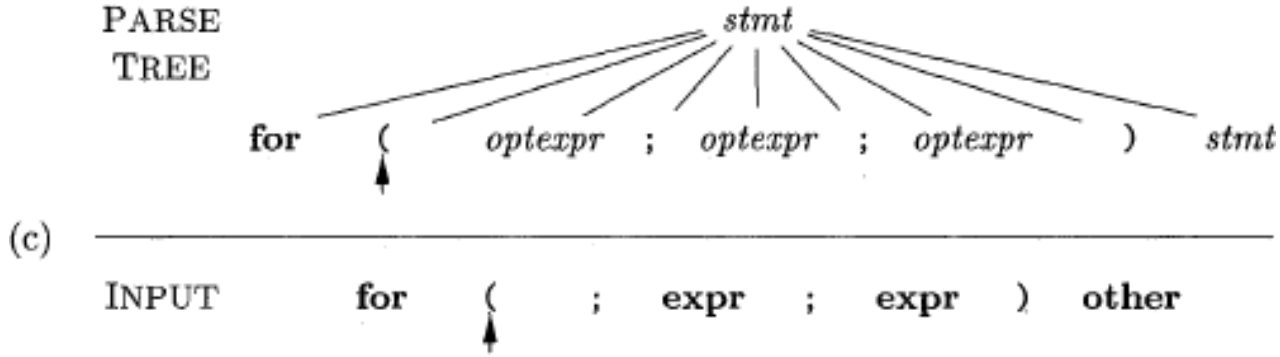
for (; expr ; expr) other

stmt	→	**expr ;**
	|	**if (expr)** *stmt*
	|	**for (** *optexpr* **;** *optexpr* **;** *optexpr* **)** *stmt*
	|	**other**
optexpr	→	ϵ
	|	**expr**

A grammar for some statements in C and Java

- When the node being considered in the parse tree is for a terminal, and the terminal matches the lookahead symbol, then we advance in both the parse tree and the input.
- The next terminal in the input becomes the new lookahead symbol, and the next child in the parse tree is considered.

for (; expr ; expr) other

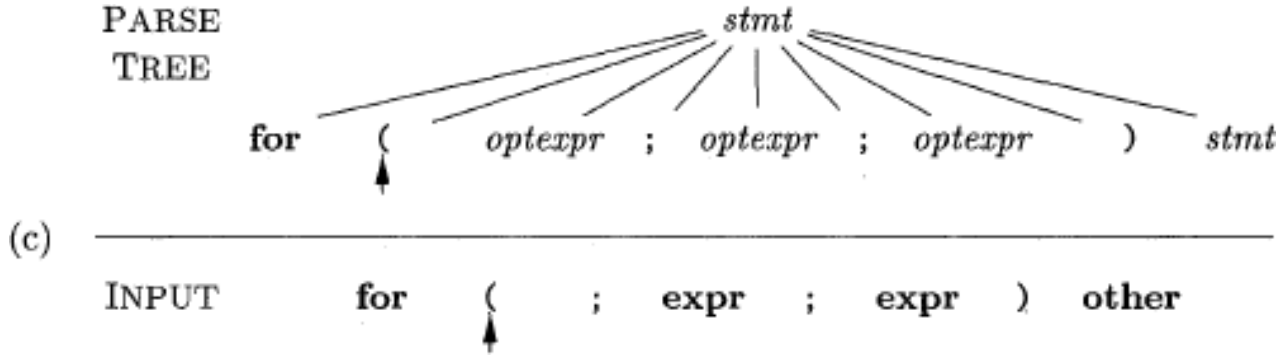


- In (c), the arrow in the parse tree has advanced to the next child of the root, and the arrow in the input has advanced to the next terminal, which is (.
- A further advance will take the arrow in the parse tree to the child labeled with nonterminal *optexpr* and take the arrow in the input to the terminal ;.

$$
\begin{array}{lcl}
\textit{stmt} & \rightarrow & \textbf{expr ;} \\
 & | & \textbf{if (expr)} \ \textit{stmt} \\
 & | & \textbf{for (} \ \textit{optexpr} \ \textbf{;} \ \textit{optexpr} \ \textbf{;} \ \textit{optexpr} \ \textbf{)} \ \textit{stmt} \\
 & | & \textbf{other} \\
 & & \\
\textit{optexpr} & \rightarrow & \epsilon \\
 & | & \textbf{expr}
\end{array}
$$

A grammar for some statements in C and Java

for (; expr ; expr) other

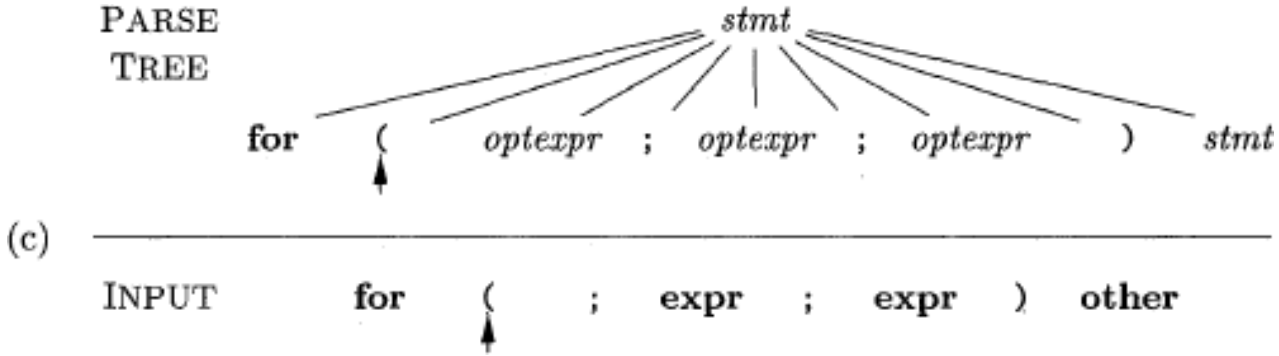


stmt	→	**expr ;**
	|	**if (expr)** *stmt*
	|	**for (** *optexpr* **;** *optexpr* **;** *optexpr* **)** *stmt*
	|	**other**
optexpr	→	ϵ
	|	**expr**

A grammar for some statements in C and Java

- At the nonterminal node labeled *optexpr*, we repeat the process of selecting a production for a nonterminal.
- Productions with ϵ as the body ("ϵ-productions") require special treatment.
- For the moment, we use them as a default when no other production can be used.

for (; expr ; expr) other



- With nonterminal *optexpr* and lookahead ;, the ϵ-production is used, since ; does not match the only other production for *optexpr*, which has terminal **expr** as its body.

$$
\begin{array}{lcl}
\textit{stmt} & \rightarrow & \textbf{expr ;} \\
 & | & \textbf{if (expr)} \ \textit{stmt} \\
 & | & \textbf{for (} \ \textit{optexpr} \ \textbf{;} \ \textit{optexpr} \ \textbf{;} \ \textit{optexpr} \ \textbf{)} \ \textit{stmt} \\
 & | & \textbf{other} \\
 & & \\
\textit{optexpr} & \rightarrow & \epsilon \\
 & | & \textbf{expr}
\end{array}
$$

A grammar for some statements in C and Java

Top-Down Parsing — *continued*

- In general, the selection of a production for a nonterminal may involve trial- and-error.
- That is, we may have to try a production and backtrack to try another production if the first is found to be unsuitable.
- A production is unsuitable if, after using the production, we cannot complete the tree to match the input string.
- Backtracking is not needed, however, in an important special case called predictive parsing.

Predictive Parsing

- Recursive-descent parsing is a top-down method of syntax analysis in which a set of recursive procedures is used to process the input.
- One procedure is associated with each nonterminal of a grammar.
- Here, we consider a simple form of recursive-descent parsing, called predictive parsing, in which the lookahead symbol unambiguously determines the flow of control through the procedure body for each nonterminal.
- The sequence of procedure calls during the analysis of an input string implicitly defines a parse tree for the input, and can be used to build an explicit parse tree, if desired.

Predictive Parsing

- Recursive-descent parsing is a top-down method of syntax analysis in which a set of recursive procedures is used to process the input.
- One procedure is associated with each nonterminal of a grammar.
- Here, we consider a simple form of recursive-descent parsing, called predictive parsing, in which the lookahead symbol unambiguously determines the flow of control through the procedure body for each nonterminal.
- The sequence of procedure calls during the analysis of an input string implicitly defines a parse tree for the input, and can be used to build an explicit parse tree, if desired.

Predictive Parsing

- Recursive-descent parsing is a top-down method of syntax analysis in which a set of recursive procedures is used to process the input.
- One procedure is associated with each nonterminal of a grammar.
- Here, we consider a simple form of recursive-descent parsing, called predictive parsing, in which the lookahead symbol unambiguously determines the flow of control through the procedure body for each nonterminal.
- The sequence of procedure calls during the analysis of an input string implicitly defines a parse tree for the input, and can be used to build an explicit parse tree, if desired.

```
void stmt() {
        switch ( lookahead ) {
        case expr:
                match(expr); match(';'); break;
        case if:
                match(if); match('('); match(expr); match(')'); stmt();
                break;
        case for:
                match(for); match('(');
                optexpr(); match(';'); optexpr(); match(';'); optexpr();
                match(')'); stmt(); break;
        case other;
                match(other); break;
        default:
                report("syntax error");
        }
}

void optexpr() {
        if ( lookahead == expr ) match(expr);
}

void match(terminal t) {
        if ( lookahead == t ) lookahead = nextTerminal;
        else report("syntax error");
}
```

Pseudocode for a predictive parser

- The predictive parser consists of procedures for the nonterminals *stmt* and *optexpr* of the grammar and an additional procedure *match*, used to simplify the code for *stmt* and *optexpr*.

$$
\begin{array}{lcl}
\textit{stmt} & \rightarrow & \textbf{expr ;} \\
 & | & \textbf{if (expr)} \ \textit{stmt} \\
 & | & \textbf{for (} \ \textit{optexpr} \ \textbf{;} \ \textit{optexpr} \ \textbf{;} \ \textit{optexpr} \ \textbf{)} \ \textit{stmt} \\
 & | & \textbf{other} \\
 & & \\
\textit{optexpr} & \rightarrow & \epsilon \\
 & | & \textbf{expr}
\end{array}
$$

A grammar for some statements in C and Java

- The predictive parser consists of procedures for the nonterminals *stmt* and *optexpr* of the grammar and an additional procedure *match*, used to simplify the code for *stmt* and *optexpr*.

Predictive Parsing — *continued*

```
void match(terminal t) {
        if ( lookahead == t ) lookahead = nextTerminal;
        else report("syntax error");
}
```

- Procedure *match*(t) compares its argument t with the lookahead symbol and advances to the next input terminal if they match.
- Thus match changes the value of variable *lookahead*, a global variable that holds the currently scanned input terminal.

```
void stmt() {
        switch ( lookahead ) {
        case expr:
                match(expr); match(';'); break;
        case if:
                match(if); match('('); match(expr); match(')'); stmt();
                break;
        case for:
                match(for); match('(');
                optexpr(); match(';'); optexpr(); match(';'); optexpr();
                match(')'); stmt(); break;
        case other;
                match(other); break;
        default:
                report("syntax error");
        }
}
```

- Parsing begins with a call of the procedure for the starting nonterminal stmt.

```
void stmt() {
        switch ( lookahead ) {
        case expr:
                match(expr); match(';'); break;
        case if:
                match(if); match('('); match(expr); match(')'); stmt();
                break;
        case for:
                match(for); match('(');
                optexpr(); match(';'); optexpr(); match(';'); optexpr();
                match(')'); stmt(); break;
        case other;
                match(other); break;
        default:
                report("syntax error");
        }
}
```

- With the input **for** (; **expr** ; **expr**) **other**, lookahead is initially the first terminal **for.**

```
void stmt() {
        switch ( lookahead ) {
        case expr:
                match(expr); match(';'); break;
        case if:
                match(if); match('('); match(expr); match(')'); stmt();
                break;
        case for:
                match(for); match('(');
                optexpr(); match(';'); optexpr(); match(';'); optexpr();
                match(')'); stmt(); break;
        case other;
                match(other); break;
        default:
                report("syntax error");
        }
}
```

- Procedure *stmt* executes code corresponding to the production

$$stmt \rightarrow \textbf{for}\ (\ optexpr\ ;\ optexpr\ ;\ optexpr\ )\ stmt$$

```
void stmt() {
        switch ( lookahead ) {
        case expr:
                match(expr); match(';'); break;
        case if:
                match(if); match('('); match(expr); match(')'); stmt();
                break;
        case for:
                match(for); match('(');
                optexpr(); match(';'); optexpr(); match(';'); optexpr();
                match(')'); stmt(); break;
        case other;
                match(other); break;
        default:
                report("syntax error");
        }
}
```

- In the code for the production body — that is, the **for** case of procedure *stmt*— each terminal is matched with the lookahead symbol.

```
void stmt() {
        switch ( lookahead ) {
        case expr:
                match(expr); match(';'); break;
        case if:
                match(if); match('('); match(expr); match(')'); stmt();
                break;
        case for:
                match(for); match('(');
                optexpr(); match(';'); optexpr(); match(';'); optexpr();
                match(')'); stmt(); break;
        case other;
                match(other); break;
        default:
                report("syntax error");
        }
}
```

- Each nonterminal leads to a call of its procedure, in the following sequence of calls:

  match(**for**); *match*('('); *optexpr*(); *match*(';'); *optexpr*(); *match*(';'); *optexpr*(); *match*(')'); *stmt*();

Predictive Parsing — *continued*

- Predictive parsing relies on information about the first symbols that can be generated by a production body.
- More precisely, let α be a string of grammar symbols (terminals and/or nonterminals).
- We define FIRST(α) to be the set of terminals that appear as the first symbols of one or more strings of terminals generated from α.
- If α is ϵ or can generate ϵ, then ϵ is also in FIRST(α).

Predictive Parsing — *continued*

- Here, we shall just use ad hoc reasoning to deduce the symbols in FIRST(α).
- Typically, α will either begin with a terminal, which is therefore the only symbol in FIRST(α).
- Or α will begin with a nonterminal whose production bodies begin with terminals, in which case these terminals are the only members of FIRST(α).

Predictive Parsing — *continued*

$$
\begin{array}{lcl}
\textit{stmt} & \rightarrow & \textbf{expr ;} \\
 & | & \textbf{if (expr)} \ \textit{stmt} \\
 & | & \textbf{for (} \ \textit{optexpr} \ \textbf{;} \ \textit{optexpr} \ \textbf{;} \ \textit{optexpr} \ \textbf{)} \ \textit{stmt} \\
 & | & \textbf{other} \\
 & & \\
\textit{optexpr} & \rightarrow & \epsilon \\
 & | & \textbf{expr}
\end{array}
$$

A grammar for some statements in C and Java

- With respect to the grammar, the following are correct calculations of FIRST.
  - FIRST(*stmt*) = {**expr**, **if**, **for**, **other**}
  - FIRST(**expr ;**) = {**expr**}

Predictive Parsing — *continued*

- The FIRST sets must be considered if there are two productions $A \to \alpha$, and $A \to \beta$.
- Ignoring ϵ-productions for the moment, predictive parsing requires FIRST(α) and FIRST(β) to be disjoint.
- The lookahead symbol can then be used to decide which production to use.
- If the lookahead symbol is in FIRST(α), then α is used.
- Otherwise, if the lookahead symbol is in FIRST(β), then β is used.

When to Use ϵ-Productions

```
void optexpr() {
        if ( lookahead == expr ) match(expr);
}
```

- Our predictive parser uses an ϵ-production as a default when no other production can be used.
- With the input **for** (; **expr** ; **expr**) **other**, after the terminals **for** and (are matched, the lookahead symbol is ;.

When to Use ϵ-Productions

```
void optexpr() {
        if ( lookahead == expr ) match(expr);
}
```

- At this point procedure *optexpr* is called, and the code
  if (*lookahead* == **expr**) *match*(**expr**);
  in its body is executed.

When to Use ϵ-Productions

```
void optexpr() {
        if ( lookahead == expr ) match(expr);
}
```

- Nonterminal *optexpr* has two productions, with bodies **expr** and ϵ.
- The lookahead symbol ";" does not match the terminal **expr**, so the production with body **expr** cannot apply.

When to Use ϵ-Productions

```
void optexpr() {
        if ( lookahead == expr ) match(expr);
}
```

- In fact, the procedure returns without changing the lookahead symbol or doing anything else.
- Doing nothing corresponds to applying an ϵ-production.

When to Use ϵ-Productions — *continued*

- More generally, consider a variant of the productions where *optexpr* generates an expression nonterminal instead of the terminal **expr**:

$$\begin{aligned} \textit{optexpr} &\rightarrow \textit{expr} \\ &\mid \epsilon \end{aligned}$$

- Thus, *optexpr* either generates an expression using nonterminal *expr* or it generates ϵ.
- While parsing *optexpr*, if the lookahead symbol is not in FIRST(*expr*) then the ϵ-production is used.

Designing a Predictive Parser

- A predictive parser is a program consisting of a procedure for every nonterminal.
- The procedure for nonterminal A does two things.

1. It decides which A-production to use by examining the lookahead symbol.

   The production with body α (where α is not ϵ, the empty string) is used if the lookahead symbol is in FIRST(α).

   If there is a conflict between two nonempty bodies for any lookahead symbol, then we cannot use this parsing method on this grammar.

   In addition, the ϵ-production for A, if it exists, is used if the lookahead symbol is not in the FIRST set for any other production body for A.

Designing a Predictive Parser

- A predictive parser is a program consisting of a procedure for every nonterminal.
- The procedure for nonterminal A does two things.

2. The procedure then mimics the body of the chosen production.

   That is, the symbols of the body are “executed” in turn, from the left.

   A nonterrninal is “executed” by a call to the procedure for that nonterminal, and a terminal matching the lookahead symbol is “executed” by reading the next input symbol.

   If at some point the terminal in the body does not match the lookahead symbol, a syntax error is reported.

$$
\begin{array}{lcl}
\textit{stmt} & \rightarrow & \textbf{expr ;} \\
 & | & \textbf{if (expr)} \textit{ stmt} \\
 & | & \textbf{for (} \textit{ optexpr ;} \textit{ optexpr ;} \textit{ optexpr)} \textit{ stmt} \\
 & | & \textbf{other} \\
 & & \\
\textit{optexpr} & \rightarrow & \epsilon \\
 & | & \textbf{expr}
\end{array}
$$

A grammar for some statements in C and Java

```
void stmt() {
        switch ( lookahead ) {
        case expr:
                match(expr); match(';'); break;
        case if:
                match(if); match('('); match(expr); match(')'); stmt();
                break;
        case for:
                match(for); match('(');
                optexpr(); match(';'); optexpr(); match(';'); optexpr();
                match(')'); stmt(); break;
        case other;
                match(other); break;
        default:
                report("syntax error");
        }
}

void optexpr() {
        if ( lookahead == expr ) match(expr);
}

void match(terminal t) {
        if ( lookahead == t ) lookahead = nextTerminal;
        else report("syntax error");
}
```

Pseudocode for a predictive parser

Designing a Predictive Parser — *continued*

- Just as a translation scheme is formed by extending a grammar, a syntax-directed translator can be formed by extending a predictive parser.
- The following limited construction suffices for the present:
  1. Construct a predictive parser, ignoring the actions in productions.
  2. Copy the actions from the translation scheme into the parser.
     If an action appears after grammar symbol X in production p, then it is copied after the implementation of X in the code for p.
     Otherwise, if it appears at the beginning of the production, then it is copied just before the code for the production body.

Designing a Predictive Parser — *continued*

- Just as a translation scheme is formed by extending a grammar, a syntax-directed translator can be formed by extending a predictive parser.
- The following limited construction suffices for the present:
  1. Construct a predictive parser, ignoring the actions in productions.
  2. Copy the actions from the translation scheme into the parser.
     If an action appears after grammar symbol X in production p, then it is copied after the implementation of X in the code for p.
     Otherwise, if it appears at the beginning of the production, then it is copied just before the code for the production body.

Designing a Predictive Parser — *continued*

- Just as a translation scheme is formed by extending a grammar, a syntax-directed translator can be formed by extending a predictive parser.
- The following limited construction suffices for the present:
  1. Construct a predictive parser, ignoring the actions in productions.
  2. Copy the actions from the translation scheme into the parser.
     If an action appears after grammar symbol X in production p, then it is copied after the implementation of X in the code for p.
     Otherwise, if it appears at the beginning of the production, then it is copied just before the code for the production body.

Left Recursion

- It is possible for a recursive-descent parser to loop forever.
- A problem arises with "left-recursive" productions like

$$expr \rightarrow expr + term$$

  where the leftmost symbol of the body is the same as the nonterminal at the head of the production.
- Suppose the procedure for expr decides to apply this production.
- The body begins with *expr* so the procedure for *expr* is called recursively.
- Since the lookahead symbol changes only when a terminal in the body is matched, no change to the input took place between recursive calls of *expr*.
- As a result, the second call to *expr* does exactly what the first call did, which means a third call to *expr*, and so on, forever.

Left Recursion

- It is possible for a recursive-descent parser to loop forever.
- A problem arises with "left-recursive" productions like

$$expr \rightarrow expr + term$$

  where the leftmost symbol of the body is the same as the nonterminal at the head of the production.
- Suppose the procedure for expr decides to apply this production.
- The body begins with *expr* so the procedure for *expr* is called recursively.
- Since the lookahead symbol changes only when a terminal in the body is matched, no change to the input took place between recursive calls of *expr*.
- As a result, the second call to *expr* does exactly what the first call did, which means a third call to *expr*, and so on, forever.

Left Recursion — *continued*

- A left-recursive production can be eliminated by rewriting the offending production.
- Consider a nonterminal A with two productions

$$A \to A\alpha \mid \beta$$

where α and β are sequences of terminals and nonterminals that do not start with A.

- For example, in

$$\textit{expr} \to \textit{expr} + \textit{term} \mid \textit{term}$$

nonterminal $A = \textit{expr}$, string $\alpha = +\textit{term}$, and string $\beta = \textit{term}$.

Left Recursion — *continued*

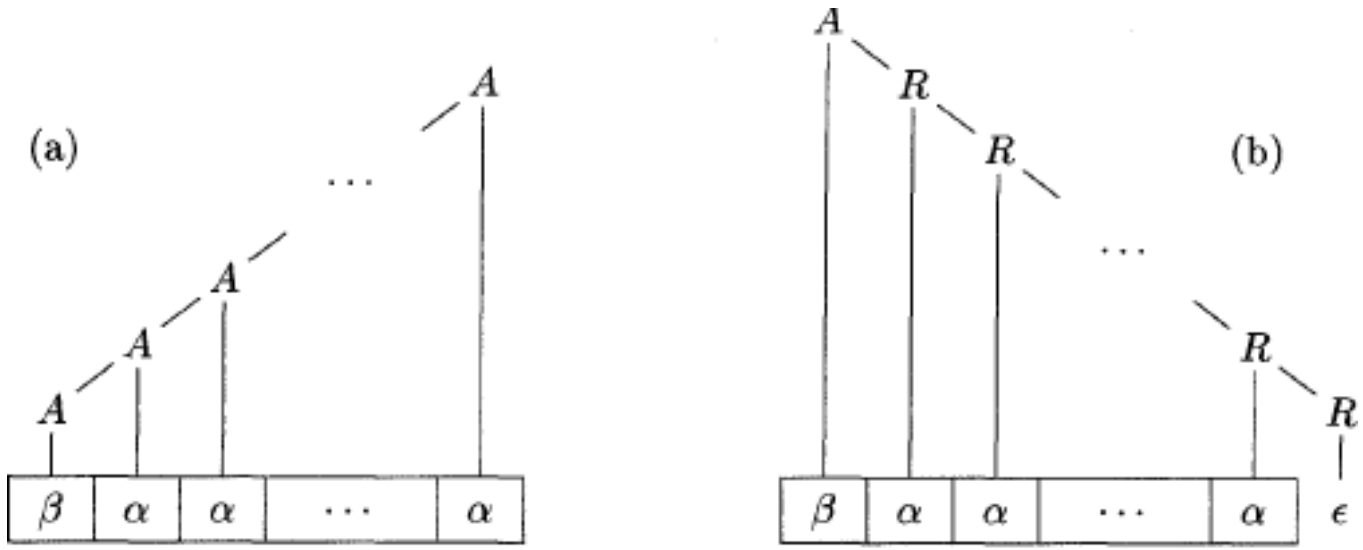
$$A \rightarrow A\alpha \mid \beta$$

- The nonterminal A and its production are said to be left recursive, because the production $A \rightarrow A\alpha$ has A itself as the leftmost symbol on the right side.

Left Recursion — *continued*

$$A \to A\alpha \mid \beta$$

- Repeated application of this production builds up a sequence of α's to the right of A.



Left- and right-recursive ways of generating a string

- When A is finally replaced by β, we have β followed by a sequence of zero or more α's.

Left Recursion — *continued*

$$A \rightarrow A\alpha \mid \beta$$

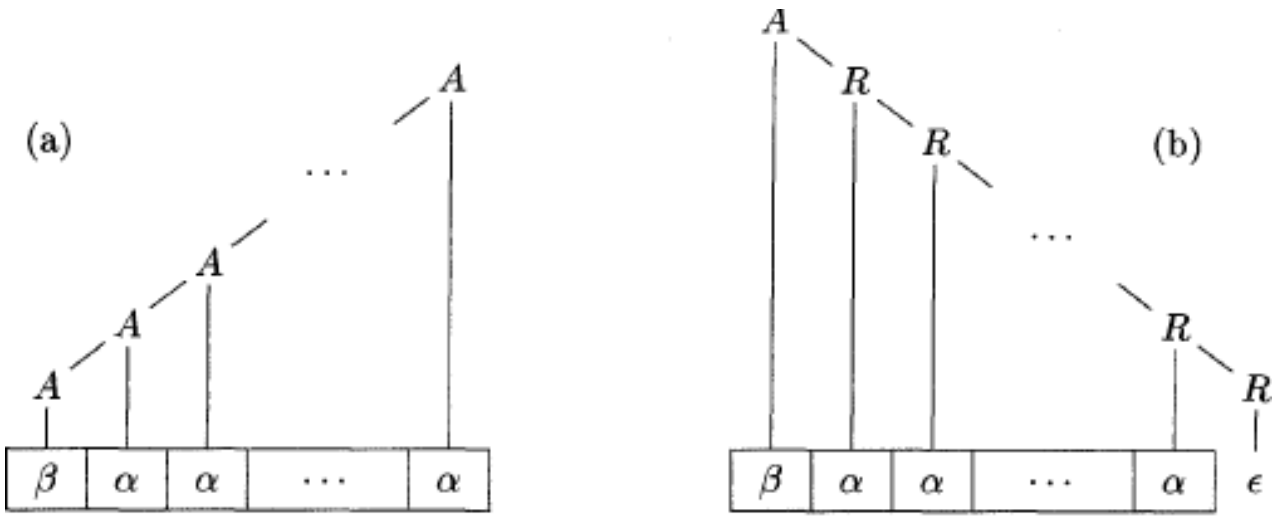
- The same effect can be achieved, by rewriting the productions for A in the following manner, using a new nonterminal R:

$$\begin{aligned} A &\rightarrow \beta R \\ R &\rightarrow \alpha R \mid \epsilon \end{aligned}$$

Left Recursion — *continued*

$$A \rightarrow \beta R$$
$$R \rightarrow \alpha R \mid \epsilon$$

- Right-recursive productions lead to trees that grow down towards the right.



Left- and right-recursive ways of generating a string

Practice

- Eliminate left recursion from this grammar.

$$
\begin{aligned}
A &\rightarrow ABd \mid Aa \mid a \\
R &\rightarrow Be \mid b
\end{aligned}
$$

End of Slides



CSE 309 (Compilers)

Syntax Analysis

Dr. Muhammad Masroor Ali

Professor
Department of Computer Science and Engineering
Bangladesh University of Engineering and Technology
Dhaka-1205, Bangladesh

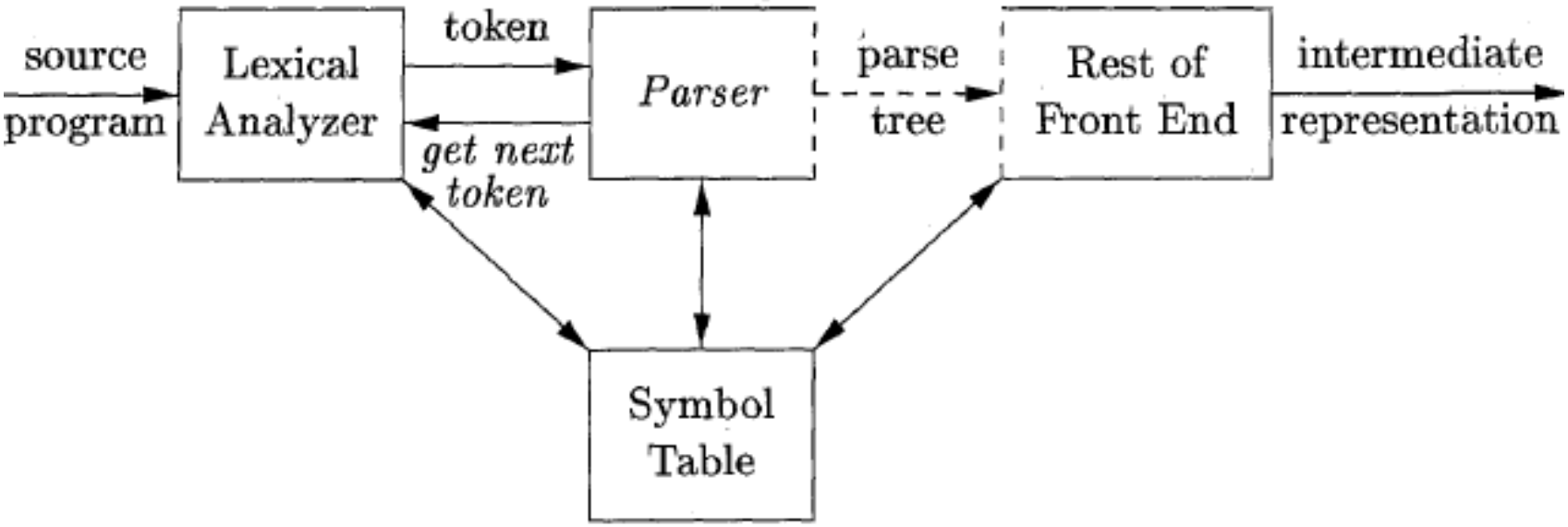
January 2026

Adapted by
Ahmed Mahir Sultan Rumi
Lecturer, Department of CSE, BUET

Version: 1.7, Last modified: July 19, 2026

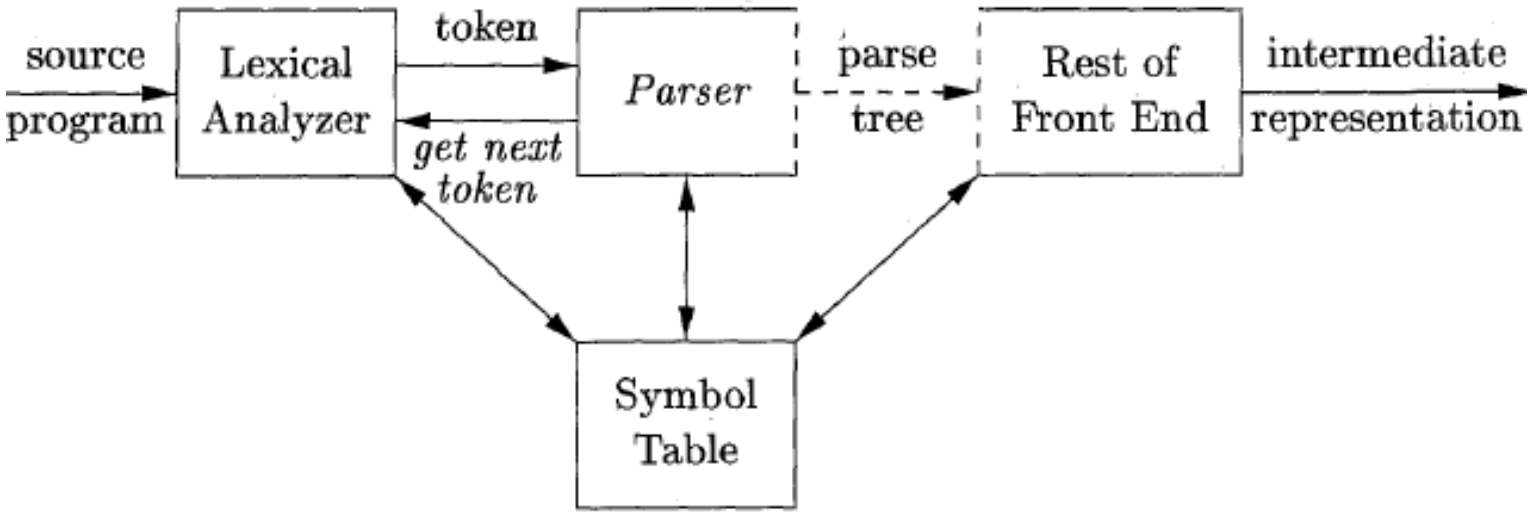
The Role of the Parser

- In our compiler model, the parser obtains a string of tokens from the lexical analyzer.
- It then verifies that the string of token names can be generated by the grammar for the source language.



Position of parser in compiler model

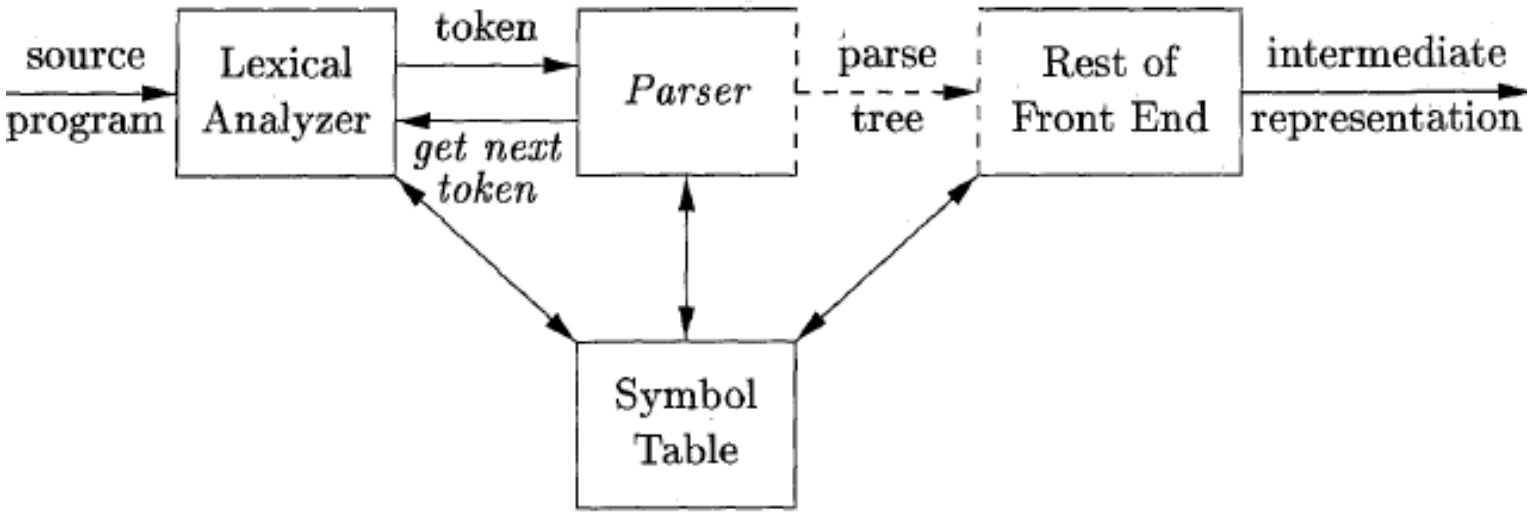
The Role of the Parser — *continued*



Position of parser in compiler model

- We expect the parser
  - to report any syntax errors in an intelligible fashion and
  - to recover from commonly occurring errors to continue processing the remainder of the program.

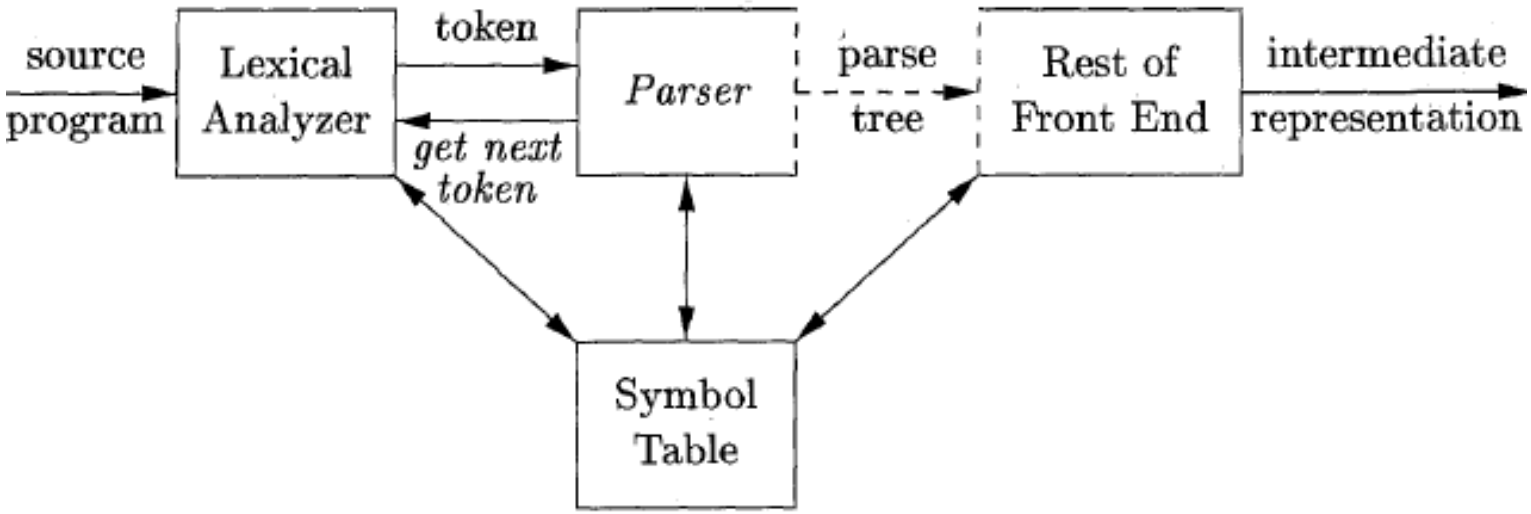
The Role of the Parser — *continued*



Position of parser in compiler model

- Conceptually, for well-formed programs, the parser constructs a parse tree and passes it to the rest of the compiler for further processing.
- In fact, the parse tree need not be constructed explicitly.
- Since checking and translation actions can be interspersed with parsing.

The Role of the Parser — *continued*



Position of parser in compiler model

- Thus, the parser and the rest of the front end could well be implemented by a single module.

The Role of the Parser — *continued*

- There are three general types of parsers for grammars:
  - universal,
  - top-down, and
  - bottom-up.
- Universal parsing methods such as the Cocke-Younger-Kasami algorithm and Earley's algorithm can parse any grammar.
- These general methods are, however, too inefficient to use in production compilers.

The Role of the Parser — *continued*

- The methods commonly used in compilers can be classified as being either top-down or bottom-up.
- As implied by their names, top-down methods build parse trees from the top (root) to the bottom (leaves).
- Bottom-up methods start from the leaves and work their way up to the root.
- In either case, the input to the parser is scanned from left to right, one symbol at a time.

The Role of the Parser — *continued*

- The most efficient top-down and bottom-up methods work only for subclasses of grammars.
- But several of these classes, particularly, LL and LR grammars, are expressive enough to describe most of the syntactic constructs in modern programming languages.
- Parsers implemented by hand often use LL grammars.
- The predictive-parsing approach works for LL grammars.
- Parsers for the larger class of LR grammars are usually constructed using automated tools.

Representative Grammars

- Some of the grammars that will be examined are presented here for ease of reference.
- Constructs that begin with keywords like **while** or **int**, are relatively easy to parse.
- The keyword guides the choice of the grammar production that must be applied to match the input.
- We therefore concentrate on expressions, which present more of challenge, because of the associativity and precedence of operators.

Representative Grammars

- Some of the grammars that will be examined are presented here for ease of reference.
- Constructs that begin with keywords like **while** or **int**, are relatively easy to parse.
- The keyword guides the choice of the grammar production that must be applied to match the input.
- We therefore concentrate on expressions, which present more of challenge, because of the associativity and precedence of operators.

Representative Grammars — *continued*

- Associativity and precedence are captured in the following grammar.
- E represents expressions consisting of terms separated by $+$ signs.
- T represents terms consisting of factors separated by $*$ signs.
- F represents factors that can be either parenthesized expressions or identifiers:

$$\begin{aligned} E &\rightarrow E + T \mid T \\ T &\rightarrow T * F \mid F \\ F &\rightarrow (E) \mid \textbf{id} \end{aligned}$$

Representative Grammars — *continued*

$$
\begin{aligned}
E &\rightarrow E + T \mid T \\
T &\rightarrow T * F \mid F \\
F &\rightarrow (\ E \ ) \mid \textbf{id}
\end{aligned}
$$

- The above grammar belongs to the class of LR grammars that are suitable for bottom-up parsing.
- This grammar can be adapted to handle additional operators and additional levels of precedence.
- However, it cannot be used for top-down parsing because it is left recursive.

Representative Grammars — *continued*

- The following non-left-recursive variant of the expression grammar will be used for top-down parsing:

$$
\begin{aligned}
E &\rightarrow TE' \\
E' &\rightarrow +TE' \mid \epsilon \\
T &\rightarrow FT' \\
T' &\rightarrow *FT' \mid \epsilon \\
F &\rightarrow (E) \mid \textbf{id}
\end{aligned}
$$

Representative Grammars — *continued*

- The following grammar treats $+$ and $*$ alike.

$$E \rightarrow E + E \quad | \quad E * E \quad | \quad (\; E \;) \quad | \quad \textbf{id}$$

- So it is useful for illustrating techniques for handling ambiguities during parsing.
- Here, E represents expressions of all types.
- This grammar permits more than one parse tree for expressions like $a + b * c$.

Syntax Error Handling

- If a compiler had to process only correct programs, its design and implementation would be simplified greatly.
- However, a compiler is expected to assist the programmer in locating and tracking down errors that inevitably creep into programs, despite the programmer's best efforts.
- Strikingly, few languages have been designed with error handling in mind, even though errors are so commonplace.

Syntax Error Handling — *continued*

- Our civilization would be radically different if spoken languages had the same requirements for syntactic accuracy as computer languages.
- Most programming language specifications do not describe how a compiler should respond to errors.
- Error handling is left to the compiler designer.
- Planning the error handling right from the start can both simplify the structure of a compiler and improve its handling of errors.

Syntax Error Handling — *continued*

Common programming errors can occur at many different levels.

Lexical errors include misspellings of identifiers, keywords, or operators — e.g., the use of an identifier `elipsesize` instead of `ellipsesize` — and missing quotes around text intended as a string.

Syntax Error Handling — *continued*

Common programming errors can occur at many different levels.

Syntactic errors include misplaced semicolons or extra or missing braces, that is, "{" or "}".

As another example, in C or Java, the appearance of a case statement without an enclosing switch is a syntactic error.

However, this situation is usually allowed by the parser and caught later in the processing, as the compiler attempts to generate code.

Syntax Error Handling — *continued*

Common programming errors can occur at many different levels.

Semantic errors include type mismatches between operators and operands.

An example is a `return` statement in a Java method with result type `void`.

Syntax Error Handling — *continued*

Common programming errors can occur at many different levels.

Logical errors can be anything from incorrect reasoning on the part of the programmer to the use in a C program of the assignment operator $=$ instead of the comparison operator $==$.

The program containing $=$ may be well formed; however, it may not reflect the programmer's intent.

Syntax Error Handling — *continued*

- The precision of parsing methods allows syntactic errors to be detected very efficiently.
- Several parsing methods, such as the LL and LR methods, detect an error as soon as possible.
- That is, when the stream of tokens from the lexical analyzer cannot be parsed further according to the grammar for the language.
- More precisely, they have the viable-prefix property, meaning that they detect that an error has occurred as soon as they see a prefix of the input that cannot be completed to form a string in the language.

Syntax Error Handling — *continued*

- Another reason for emphasizing error recovery during parsing is that many errors appear syntactic, whatever their cause, and are exposed when parsing cannot continue.
- A few semantic errors, such as type mismatches, can also be detected efficiently.
- However, accurate detection of semantic and logical errors at compile time is in general a difficult task.

Syntax Error Handling — *continued*

- The error handler in a parser has goals that are simple to state but challenging to realize:
  - Report the presence of errors clearly and accurately.
  - Recover from each error quickly enough to detect subsequent errors.
  - Add minimal overhead to the processing of correct programs.

Syntax Error Handling — *continued*

- Fortunately, common errors are simple ones.
- A relatively straightforward error-handling mechanism often suffices.
- How should an error handler report the presence of an error?
- At the very least, it must report the place in the source program where an error is detected.
- There is a good chance that the actual error occurred within the previous few tokens.
- A common strategy is to print the offending line with a pointer to the position at which an error is detected.

Writing a Grammar

- Grammars are capable of describing most, but not all, of the syntax of programming languages.
- For instance, the requirement that identifiers be declared before they are used, cannot be described by a context-free grammar.
- Therefore, the sequences of tokens accepted by a parser form a superset of the programming language.
- Subsequent phases of the compiler must analyze the output of the parser to ensure compliance with rules that are not checked by the parser.

Writing a Grammar

- Grammars are capable of describing most, but not all, of the syntax of programming languages.
- For instance, the requirement that identifiers be declared before they are used, cannot be described by a context-free grammar.
- Therefore, the sequences of tokens accepted by a parser form a superset of the programming language.
- Subsequent phases of the compiler must analyze the output of the parser to ensure compliance with rules that are not checked by the parser.

Elimination of Left Recursion

- A grammar is left recursive if it has a nonterminal A such that there is a derivation $A \stackrel{+}{\Longrightarrow} Aa$ for some string a.
- Top-down parsing methods cannot handle left-recursive grammars, so a transformation that eliminates left recursion is needed.
- In simple left recursion there was one production of the form $A \rightarrow A\alpha$.
- Here we study the general case.

Elimination of Left Recursion — *continued*

- Left-recursive pair of productions $A \to A\alpha \mid \beta$ can be replaced by the non-left-recursive productions

$$\begin{aligned} A &\to \beta A' \\ A' &\to \alpha A' \mid \epsilon \end{aligned}$$

  without changing the set of strings derivable from A.
- This rule by itself suffices in many grammars.

Example

$$A \rightarrow A\alpha \mid \beta$$

to be replaced by

$$\begin{aligned} A &\rightarrow \beta A' \\ A' &\rightarrow \alpha A' \mid \epsilon \end{aligned}$$

- Grammar for arithmetic expressions,

$$\begin{aligned} E &\rightarrow E + T \mid T \\ T &\rightarrow T * F \mid F \\ F &\rightarrow (E) \mid \textbf{id} \end{aligned}$$

- Eliminating the immediate left recursions we obtain,

$$\begin{aligned} E &\rightarrow TE' \\ E' &\rightarrow +TE' \mid \epsilon \\ T &\rightarrow FT' \\ T' &\rightarrow *FT' \mid \epsilon \\ F &\rightarrow (E) \mid \textbf{id} \end{aligned}$$

Elimination of Left Recursion — *continued*

- No matter how many A-productions there are, we can eliminate immediate left recursion from them.
- First, we group the A-productions as,

$$A \rightarrow A\alpha_1 \mid A\alpha_2 \mid A\alpha_3 \mid \ldots \mid A\alpha_m \mid \beta_1 \mid \beta_2 \mid \beta_3 \mid \ldots \mid \beta_n$$

  where no β_i, begins with an A.
- Then, we replace the A-productions by,

$$\begin{aligned} A &\rightarrow \beta_1 A' \mid \beta_2 A' \mid \beta_3 A' \mid \ldots \mid \beta_n A' \\ A' &\rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \alpha_3 A' \mid \ldots \mid \alpha_m A' \mid \epsilon \end{aligned}$$

- It does not eliminate left recursion involving derivations of two or more steps.

Elimination of Left Recursion — *continued*

- It does not eliminate left recursion involving derivations of two or more steps.
- Consider the grammar,

$$\begin{aligned} S &\rightarrow Aa \mid b \\ A &\rightarrow Ac \mid Sd \mid \epsilon \end{aligned}$$

- The nonterminal S is left-recursive because $S \Rightarrow Aa \Rightarrow Sda$, but it is not immediately left recursive.

Algorithm

Eliminating left recursion.

INPUT: Grammar G with no cycles or ϵ-productions.

OUTPUT: An equivalent grammar with no left recursion.

METHOD: Apply the algorithm to G. Note that the resulting non-left-recursive grammar may have ϵ-productions.

1) arrange the nonterminals in some order $A_1, A_2, \ldots, A_n$.
2) **for** (each i from 1 to n) {
3) **for** (each j from 1 to $i-1$) {
4) replace each production of the form $A_i \rightarrow A_j\gamma$ by the productions $A_i \rightarrow \delta_1\gamma \mid \delta_2\gamma \mid \ldots \mid \delta_k\gamma$ where $A_j \rightarrow \delta_1 \mid \delta_2 \mid \ldots \delta_k$ are all the current A_j-productions
5) }
6) eliminate the immediate left recursion among the A_i-productions;
7) }

Algorithm

Grammar with cycles: Grammar where derivations of the form $A \stackrel{+}{\Rightarrow} A$ occurs.

1) arrange the nonterminals in some order $A_1, A_2, \ldots, A_n$.
2) **for** (each i from 1 to n) {
3) **for** (each j from 1 to $i-1$) {
4) replace each production of the form $A_i \rightarrow A_j\gamma$ by the productions $A_i \rightarrow \delta_1\gamma \mid \delta_2\gamma \mid \ldots \mid \delta_k\gamma$ where $A_j \rightarrow \delta_1 \mid \delta_2 \mid \ldots \delta_k$ are all the current A_j-productions
5) }
6) eliminate the immediate left recursion among the A_i-productions;
7) }

- In the first iteration for $i = 1$, the outer `for`-loop of lines (2) through (7) eliminates any immediate left recursion among A_1-productions.

1) arrange the nonterminals in some order $A_1, A_2, \ldots, A_n$.
2) **for** (each i from 1 to n) {
3) **for** (each j from 1 to $i-1$) {
4) replace each production of the form $A_i \rightarrow A_j\gamma$ by the productions $A_i \rightarrow \delta_1\gamma \mid \delta_2\gamma \mid \ldots \mid \delta_k\gamma$ where $A_j \rightarrow \delta_1 \mid \delta_2 \mid \ldots \delta_k$ are all the current A_j-productions
5) }
6) eliminate the immediate left recursion among the A_i-productions;
7) }

- Any remaining A_1 productions of the form $A_1 \rightarrow A_l\alpha$ must therefore have $l > 1$.

1) arrange the nonterminals in some order $A_1, A_2, \ldots, A_n$.
2) **for** (each i from 1 to n) {
3) **for** (each j from 1 to $i-1$) {
4) replace each production of the form $A_i \rightarrow A_j\gamma$ by the productions $A_i \rightarrow \delta_1\gamma \mid \delta_2\gamma \mid \ldots \mid \delta_k\gamma$ where $A_j \rightarrow \delta_1 \mid \delta_2 \mid \ldots \delta_k$ are all the current A_j-productions
5) }
6) eliminate the immediate left recursion among the A_i-productions;
7) }

- After the $i-1$st iteration of the outer `for`- loop, all nonterminals A_k, where $k < i$, are "cleaned".
- That is, any production $A_k \rightarrow A_l\alpha$, must have $l > k$.

1) arrange the nonterminals in some order $A_1, A_2, \ldots, A_n$.
2) **for** (each i from 1 to n) {
3) **for** (each j from 1 to $i-1$) {
4) replace each production of the form $A_i \rightarrow A_j\gamma$ by the productions $A_i \rightarrow \delta_1\gamma \mid \delta_2\gamma \mid \ldots \mid \delta_k\gamma$ where $A_j \rightarrow \delta_1 \mid \delta_2 \mid \ldots \delta_k$ are all the current A_j-productions
5) }
6) eliminate the immediate left recursion among the A_i-productions;
7) }

- As a result, on the ith iteration, the inner loop of lines (3) through (5) progressively raises the lower limit in any production $A_i \rightarrow A_m\alpha$, until we have $m \geq i$.

1) arrange the nonterminals in some order $A_1, A_2, \ldots, A_n$.
2) **for** (each i from 1 to n) {
3) **for** (each j from 1 to $i-1$) {
4) replace each production of the form $A_i \rightarrow A_j\gamma$ by the productions $A_i \rightarrow \delta_1\gamma \mid \delta_2\gamma \mid \ldots \mid \delta_k\gamma$ where $A_j \rightarrow \delta_1 \mid \delta_2 \mid \ldots \delta_k$ are all the current A_j-productions
5) }
6) eliminate the immediate left recursion among the A_i-productions;
7) }

- Then, eliminating immediate left recursion for the A_i productions at line (6) forces m to be greater than i.

Example

Input Grammar G with no cycles or ϵ-productions.

- We apply the procedure to grammar,

$$\begin{aligned} S &\rightarrow Aa \mid b \\ A &\rightarrow Ac \mid Sd \mid \epsilon \end{aligned}$$

- Technically, the algorithm is not guaranteed to work, because of the ϵ-production.
- But in this case the production $A \rightarrow \epsilon$ turns out to be harmless.

Example — *continued*

1) arrange the nonterminals in some order $A_1, A_2, \ldots, A_n$.

Left-Recursive Grammar

$$
\begin{aligned}
S &\rightarrow Aa \mid b \\
A &\rightarrow Ac \mid Sd \mid \epsilon
\end{aligned}
$$

- We order the nonterminals S, A.
- $A_1 = S$, $A_2 = A$

Example — *continued*

1) arrange the nonterminals in some order $A_1, A_2, \ldots, A_n$.

Left-Recursive Grammar

$$
\begin{aligned}
S &\rightarrow Aa \mid b \\
A &\rightarrow Ac \mid Sd \mid \epsilon
\end{aligned}
$$

- We order the nonterminals S, A.
- $A_1 = S$, $A_2 = A$

Example — *continued*

1) arrange the nonterminals in some order $A_1, A_2, \ldots, A_n$.
2) **for** (each i from 1 to n) {
3) **for** (each j from 1 to $i - 1$) {
4) replace each production of the form $A_i \to A_j\gamma$ by the productions $A_i \to \delta_1\gamma \mid \delta_2\gamma \mid \ldots \mid \delta_k\gamma$ where $A_j \to \delta_1 \mid \delta_2 \mid \ldots \delta_k$ are all the current A_j-productions
5) }
6) eliminate the immediate left recursion among the A_i-productions;
7) }

- $i = 1$, $A_1 = S$
- $j = 1$ to $j = 1 - 1 = 0$, the loop is *not* entered

Example — *continued*

1) arrange the nonterminals in some order $A_1, A_2, \ldots, A_n$.
2) **for** (each i from 1 to n) {
3) **for** (each j from 1 to $i - 1$) {
4) replace each production of the form $A_i \rightarrow A_j\gamma$ by the productions $A_i \rightarrow \delta_1\gamma \mid \delta_2\gamma \mid \ldots \mid \delta_k\gamma$ where $A_j \rightarrow \delta_1 \mid \delta_2 \mid \ldots \delta_k$ are all the current A_j-productions
5) }
6) eliminate the immediate left recursion among the A_i-productions;
7) }

- $i = 1$, $A_1 = S$
- $j = 1$ to $j = 1 - 1 = 0$, the loop is *not* entered

Example — *continued*

6) eliminate the immediate left recursion among the A_i-productions;

Left-Recursive Grammar

$$
\begin{aligned}
S &\rightarrow Aa \mid b \\
A &\rightarrow Ac \mid Sd \mid \epsilon
\end{aligned}
$$

- There is no immediate left recursion among the S-productions, so nothing happens for the case $i = 1$. $(A_1 = S)$

Example — *continued*

1) arrange the nonterminals in some order $A_1, A_2, \ldots, A_n$.
2) **for** (each i from 1 to n) {
3) **for** (each j from 1 to $i - 1$) {
4) replace each production of the form $A_i \to A_j\gamma$ by the productions $A_i \to \delta_1\gamma \mid \delta_2\gamma \mid \ldots \mid \delta_k\gamma$ where $A_j \to \delta_1 \mid \delta_2 \mid \ldots \delta_k$ are all the current A_j-productions
5) }
6) eliminate the immediate left recursion among the A_i-productions;
7) }

- $i = 2$, $A_2 = A$
- $j = 1$ to $j = 2 - 1 = 1$, the loop is entered

Example — *continued*

1) arrange the nonterminals in some order $A_1, A_2, \ldots, A_n$.
2) **for** (each i from 1 to n) {
3) **for** (each j from 1 to $i - 1$) {
4) replace each production of the form $A_i \rightarrow A_j\gamma$ by the productions $A_i \rightarrow \delta_1\gamma \mid \delta_2\gamma \mid \ldots \mid \delta_k\gamma$ where $A_j \rightarrow \delta_1 \mid \delta_2 \mid \ldots \delta_k$ are all the current A_j-productions
5) }
6) eliminate the immediate left recursion among the A_i-productions;
7) }

- $i = 2$, $A_2 = A$
- $j = 1$ to $j = 2 - 1 = 1$, the loop is entered

Example — *continued*

4) replace each production of the form $A_i \rightarrow A_j\gamma$ by the productions $A_i \rightarrow \delta_1\gamma \mid \delta_2\gamma \mid \ldots \mid \delta_k\gamma$ where $A_j \rightarrow \delta_1 \mid \delta_2 \mid \ldots \delta_k$ are all the current A_j-productions

Left-Recursive Grammar

$$\begin{aligned} S &\rightarrow Aa \mid b \\ A &\rightarrow Ac \mid Sd \mid \epsilon \end{aligned}$$

- $i = 2$, $A_2 = A$, $j = 1$, $A_1 = S$
- We need to
  - put productions of the form $S \rightarrow \delta_1 \mid \delta_2 \mid \ldots \mid \delta_k$
  - in productions of the form $A \rightarrow S\gamma$
- Production(s) with S at the left-hand-side, $S \rightarrow Aa \mid b$
- Productions(s) with A at the left side and right side beginning with S is (are), $A \rightarrow Sd$

Example — *continued*

4) replace each production of the form $A_i \rightarrow A_j\gamma$ by the productions $A_i \rightarrow \delta_1\gamma \mid \delta_2\gamma \mid \ldots \mid \delta_k\gamma$ where $A_j \rightarrow \delta_1 \mid \delta_2 \mid \ldots \delta_k$ are all the current A_j-productions

Left-Recursive Grammar

$$S \rightarrow Aa \mid b$$
$$A \rightarrow Ac \mid Sd \mid \epsilon$$

- $i = 2$, $A_2 = A$, $j = 1$, $A_1 = S$
- We need to
  - put productions of the form $S \rightarrow \delta_1 \mid \delta_2 \mid \ldots \mid \delta_k$
  - in productions of the form $A \rightarrow S\gamma$
- Production(s) with S at the left-hand-side, $S \rightarrow Aa \mid b$
- Productions(s) with A at the left side and right side beginning with S is (are), $A \rightarrow Sd$

Example — *continued*

4) replace each production of the form $A_i \rightarrow A_j\gamma$ by the productions $A_i \rightarrow \delta_1\gamma \mid \delta_2\gamma \mid \ldots \mid \delta_k\gamma$ where $A_j \rightarrow \delta_1 \mid \delta_2 \mid \ldots \delta_k$ are all the current A_j-productions

Left-Recursive Grammar

$$\begin{aligned} S &\rightarrow Aa \mid b \\ A &\rightarrow Ac \mid Sd \mid \epsilon \end{aligned}$$

- $i = 2$, $A_2 = A$, $j = 1$, $A_1 = S$
- We need to
  - put productions of the form $S \rightarrow \delta_1 \mid \delta_2 \mid \ldots \mid \delta_k$
  - in productions of the form $A \rightarrow S\gamma$
- Production(s) with S at the left-hand-side, $S \rightarrow Aa \mid b$
- Productions(s) with A at the left side and right side beginning with S is (are), $A \rightarrow Sd$

Example — *continued*

4) replace each production of the form $A_i \rightarrow A_j\gamma$ by the productions $A_i \rightarrow \delta_1\gamma \mid \delta_2\gamma \mid \ldots \mid \delta_k\gamma$ where $A_j \rightarrow \delta_1 \mid \delta_2 \mid \ldots \delta_k$ are all the current A_j-productions

Left-Recursive Grammar

$$\begin{aligned} S &\rightarrow Aa \mid b \\ A &\rightarrow Ac \mid Sd \mid \epsilon \end{aligned}$$

- $i = 2$, $A_2 = A$, $j = 1$, $A_1 = S$
- We need to
  - put productions of the form $S \rightarrow \delta_1 \mid \delta_2 \mid \ldots \mid \delta_k$
  - in productions of the form $A \rightarrow S\gamma$
- Production(s) with S at the left-hand-side, $S \rightarrow Aa \mid b$
- Productions(s) with A at the left side and right side beginning with S is (are), $A \rightarrow Sd$

Example — *continued*

4) replace each production of the form $A_i \rightarrow A_j\gamma$ by the productions $A_i \rightarrow \delta_1\gamma \mid \delta_2\gamma \mid \ldots \mid \delta_k\gamma$ where $A_j \rightarrow \delta_1 \mid \delta_2 \mid \ldots \delta_k$ are all the current A_j-productions

Left-Recursive Grammar

$$\begin{aligned} S &\rightarrow Aa \mid b \\ A &\rightarrow Ac \mid Sd \mid \epsilon \end{aligned}$$

- $i = 2$, $A_2 = A$, $j = 1$, $A_1 = S$
- We need to
  - put productions of the form $S \rightarrow \delta_1 \mid \delta_2 \mid \ldots \mid \delta_k$
  - in productions of the form $A \rightarrow S\gamma$
- Production(s) with S at the left-hand-side, $S \rightarrow Aa \mid b$
- Productions(s) with A at the left side and right side beginning with S is (are), $A \rightarrow Sd$

Example — *continued*

4) replace each production of the form $A_i \rightarrow A_j\gamma$ by the productions $A_i \rightarrow \delta_1\gamma \mid \delta_2\gamma \mid \ldots \mid \delta_k\gamma$ where $A_j \rightarrow \delta_1 \mid \delta_2 \mid \ldots \delta_k$ are all the current A_j-productions

Left-Recursive Grammar

$$\begin{aligned} S &\rightarrow Aa \mid b \\ A &\rightarrow Ac \mid Sd \mid \epsilon \end{aligned}$$

- $i = 2$, $A_2 = A$, $j = 1$, $A_1 = S$
- We need to
  - put productions of the form $S \rightarrow \delta_1 \mid \delta_2 \mid \ldots \mid \delta_k$
  - in productions of the form $A \rightarrow S\gamma$
- Production(s) with S at the left-hand-side, $S \rightarrow Aa \mid b$
- Productions(s) with A at the left side and right side beginning with S is (are), $A \rightarrow Sd$

Example — *continued*

4) replace each production of the form $A_i \rightarrow A_j\gamma$ by the productions $A_i \rightarrow \delta_1\gamma \mid \delta_2\gamma \mid \ldots \mid \delta_k\gamma$ where $A_j \rightarrow \delta_1 \mid \delta_2 \mid \ldots \delta_k$ are all the current A_j-productions

Left-Recursive Grammar

$$\begin{aligned} S &\rightarrow Aa \mid b \\ A &\rightarrow Ac \mid Sd \mid \epsilon \end{aligned}$$

- $S \rightarrow Aa \mid b$ to be put in A, $A \rightarrow Sd$
- We substitute $S \rightarrow Aa \mid b$ in $A \rightarrow Sd$ to get the following A-productions,

$$A \rightarrow Aad \mid bd$$

Example — *continued*

4) replace each production of the form $A_i \rightarrow A_j\gamma$ by the productions $A_i \rightarrow \delta_1\gamma \mid \delta_2\gamma \mid \ldots \mid \delta_k\gamma$ where $A_j \rightarrow \delta_1 \mid \delta_2 \mid \ldots \delta_k$ are all the current A_j-productions

Left-Recursive Grammar

$$\begin{aligned} S &\rightarrow Aa \mid b \\ A &\rightarrow Ac \mid Sd \mid \epsilon \end{aligned}$$

- $S \rightarrow Aa \mid b$ to be put in A, $A \rightarrow Sd$
- We substitute $S \rightarrow Aa \mid b$ in $A \rightarrow Sd$ to get the following A-productions,

$$A \rightarrow Aad \mid bd$$

Example — *continued*

1) arrange the nonterminals in some order $A_1, A_2, \ldots, A_n$.
2) **for** (each i from 1 to n) {
3) **for** (each j from 1 to $i - 1$) {
4) replace each production of the form $A_i \rightarrow A_j\gamma$ by the productions $A_i \rightarrow \delta_1\gamma \mid \delta_2\gamma \mid \ldots \mid \delta_k\gamma$ where $A_j \rightarrow \delta_1 \mid \delta_2 \mid \ldots \delta_k$ are all the current A_j-productions
5) }
6) ⇒ eliminate the immediate left recursion among the A_i-productions;
7) }

Example — *continued*

6) eliminate the immediate left recursion among the A_i-productions;

- All $A_i = A_2 = A$-productions together,

$$A \rightarrow Ac \mid Aad \mid bd \mid \epsilon$$

- Eliminating the immediate left recursion among the A-productions yields the following,

$$\begin{aligned} A &\rightarrow bdA' \mid A' \\ A' &\rightarrow cA' \mid adA' \mid \epsilon \end{aligned}$$

Example — *continued*

6) eliminate the immediate left recursion among the A_i-productions;

- All $A_i = A_2 = A$-productions together,

$$A \rightarrow Ac \mid Aad \mid bd \mid \epsilon$$

- Eliminating the immediate left recursion among the A-productions yields the following,

$$\begin{aligned} A &\rightarrow bdA' \mid A' \\ A' &\rightarrow cA' \mid adA' \mid \epsilon \end{aligned}$$

Example — *continued*

1) arrange the nonterminals in some order $A_1, A_2, \ldots, A_n$.
2) **for** (each i from 1 to n) {
3) **for** (each j from 1 to $i-1$) {
4) replace each production of the form $A_i \rightarrow A_j\gamma$ by the productions $A_i \rightarrow \delta_1\gamma \mid \delta_2\gamma \mid \ldots \mid \delta_k\gamma$ where $A_j \rightarrow \delta_1 \mid \delta_2 \mid \ldots \delta_k$ are all the current A_j-productions
5) }
6) eliminate the immediate left recursion among the A_i-productions;
7) }

i has attained the value of $n = 2$ and the loops are no more entered.

Example — *continued*

Left-Recursive Grammar

$$
\begin{aligned}
S &\rightarrow Aa \mid b \\
A &\rightarrow Ac \mid Sd \mid \epsilon
\end{aligned}
$$

- Put together we get the following non-left-recursive grammar,

$$
\begin{aligned}
S &\rightarrow Aa \mid b \\
A &\rightarrow bdA' \mid A' \\
A' &\rightarrow cA' \mid adA' \mid \epsilon
\end{aligned}
$$

1) arrange the nonterminals in some order $A_1, A_2, \ldots, A_n$.
2) **for** (each i from 1 to n) {
3) **for** (each j from 1 to $i-1$) {
4) replace each production of the form $A_i \rightarrow A_j\gamma$ by
the productions $A_i \rightarrow \delta_1\gamma \mid \delta_2\gamma \mid \ldots \mid \delta_k\gamma$
where $A_j \rightarrow \delta_1 \mid \delta_2 \mid \ldots \delta_k$ are all the
current A_j-productions
5) }
6) eliminate the immediate left recursion among
the A_i-productions;
7) }

Conceptual Technique Summary (AGAIN)

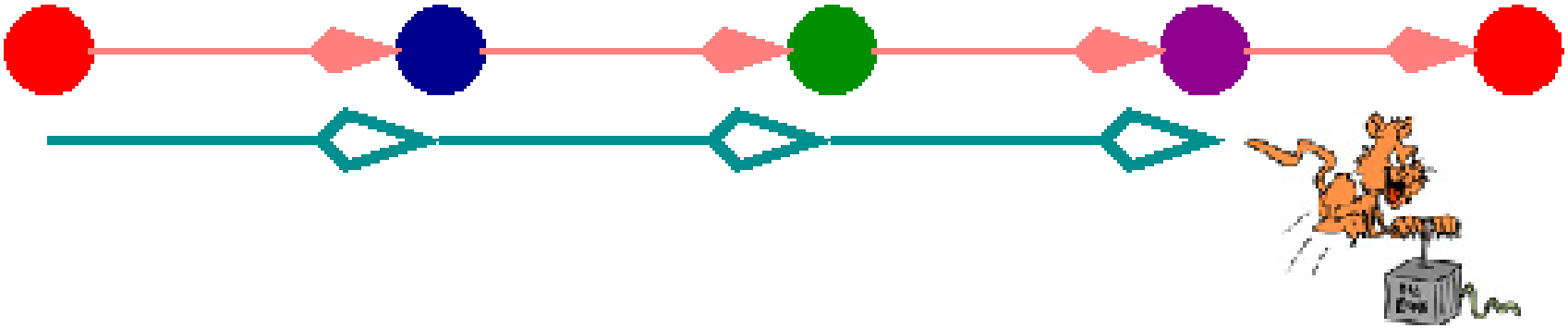
- Put some order in the nonterminals.
- Start by making first nonterminal productions left-recursion-free.
- Put the first nonterminal left-recursion-free productions into those of the second one.
- Now make the productions of second nonterminal left-recursion-free.
- Thus keep on growing the set of left-recursion-free productions.

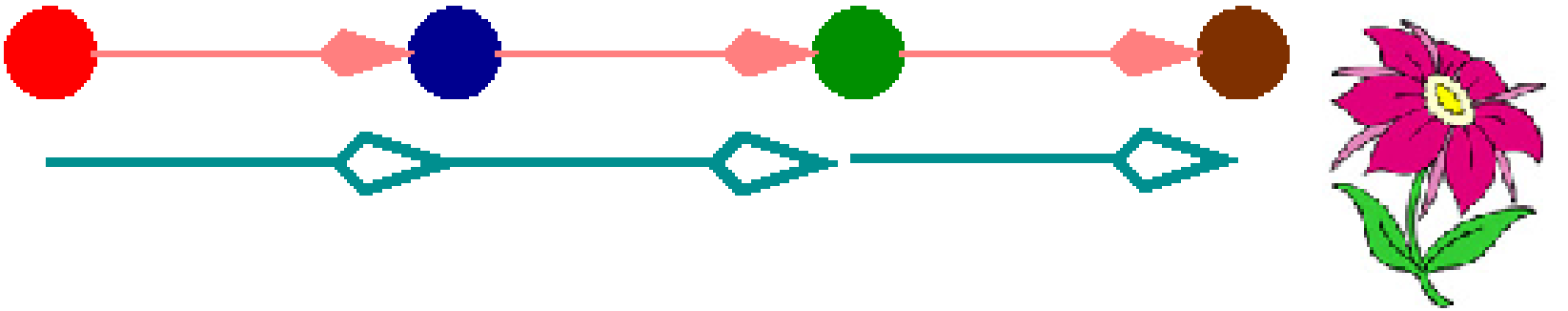
Left Factoring

- Left factoring is a grammar transformation that is useful for producing a grammar suitable for predictive parsing.
- The basic idea is that sometimes it is not clear which of two alternative productions to use to expand a nonterminal A.
- We may be able to rewrite the A-productions to defer the decision until we have seen enough of the input to make the right choice.

Left Factoring — *continued*

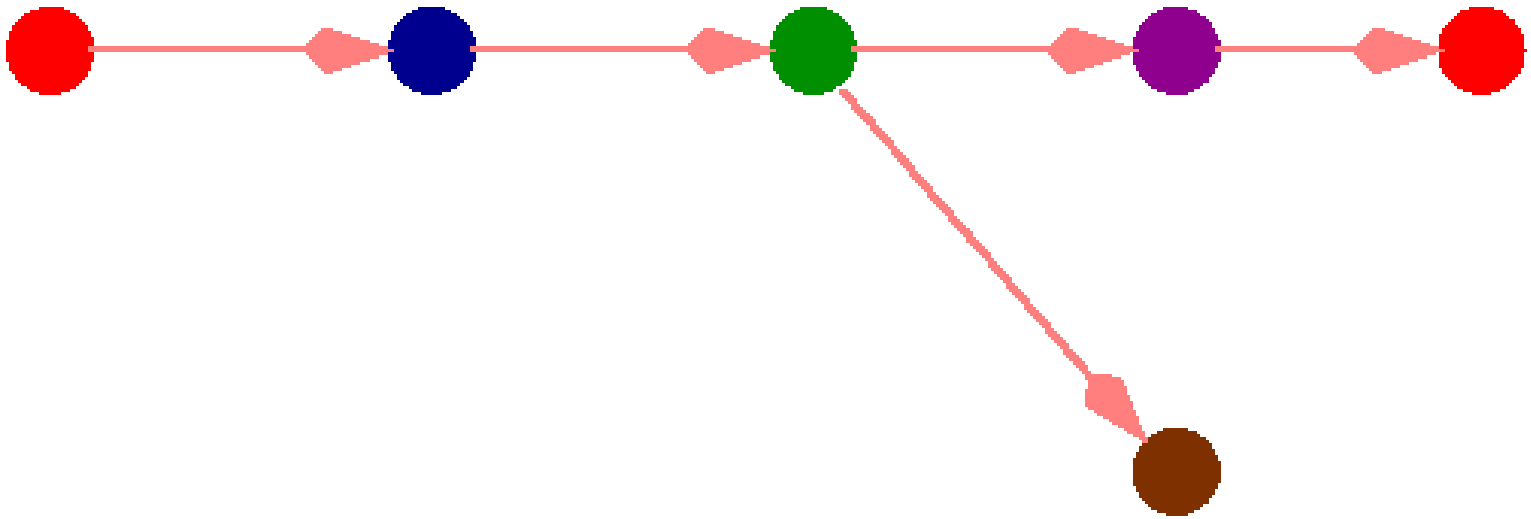
Road Direction: $darkred \rightarrow bluepigment \rightarrow Green \rightarrow Brown$





Left Factoring — *continued*

Defer the decision until we have seen enough of the input to make the right choice.



Left Factoring — *continued*

- We have the two productions,

$$\begin{aligned} \textit{stmt} &\rightarrow \textbf{if}\ \textit{expr}\ \textbf{then}\ \textit{stmt}\ \textbf{else}\ \textit{stmt} \\ &\mid \textbf{if}\ \textit{expr}\ \textbf{then}\ \textit{stmt} \end{aligned}$$

- On seeing the input token **if**, we cannot immediately tell which production to choose to expand *stmt*.

Left Factoring — *continued*

- $A \to \alpha\beta_1 \mid \alpha\beta_2$ are two A-productions.
- The input begins with a nonempty string derived from α.
- We do not know whether to expand A to $\alpha\beta_1$ or $\alpha\beta_2$.
- However, we may defer the decision by expanding A to $\alpha A'$.
- Then, after seeing the input derived from α we expand A' to β_1 or β_2.
- Left-factored, the original productions become,

$$\begin{aligned} A &\to \alpha A' \\ A' &\to \beta_1 \mid \beta_2 \end{aligned}$$

Left Factoring Algorithm

INPUT. Grammar G.

OUTPUT An equivalent left-factored grammar.

Left Factoring Algorithm — *continued*

Method.

- For each nonterminal A find the longest prefix α common to two or more of its alternatives.
- If $\alpha \neq \epsilon$ (there is a nontrivial common prefix), replace all the A productions $A \to \alpha\beta_1 \mid \alpha\beta_2 \mid \ldots \mid \alpha\beta_n \mid \gamma$ where γ represents all alternatives that do not begin with α by

$$\begin{aligned} A &\to \alpha A' \mid \gamma \\ A' &\to \beta_1 \mid \beta_2 \mid \ldots \mid \beta_n \end{aligned}$$

  where A' is a new nonterminal.
- Repeatedly apply this transformation until no two alternatives for a nonterminal have a common prefix.

Example

- The following grammar abstracts the dangling-else problem:

$$\begin{aligned} S &\rightarrow iEtS \mid iEtSeS \mid a \\ E &\rightarrow b \end{aligned}$$

- Here i, t, and e stand for **if**, **then** and **else**, E and S for "expression" and "statement."
- Left-factored, this grammar becomes:

$$\begin{aligned} S &\rightarrow iEtSS' \mid a \\ S' &\rightarrow eS \mid \epsilon \\ E &\rightarrow b \end{aligned}$$

Example — *continued*

$$
\begin{aligned}
S &\rightarrow iEtSS' \mid a \\
S' &\rightarrow eS \mid \epsilon \\
E &\rightarrow b
\end{aligned}
$$

- Thus, we may expand S to $iEtSS'$ on input i, and wait until $iEtS$ has been seen to decide whether to expand S' to eS or to ϵ.
- Of course, both of the grammars are ambiguous.
- On input e, it will not be clear which alternative for S should be chosen.

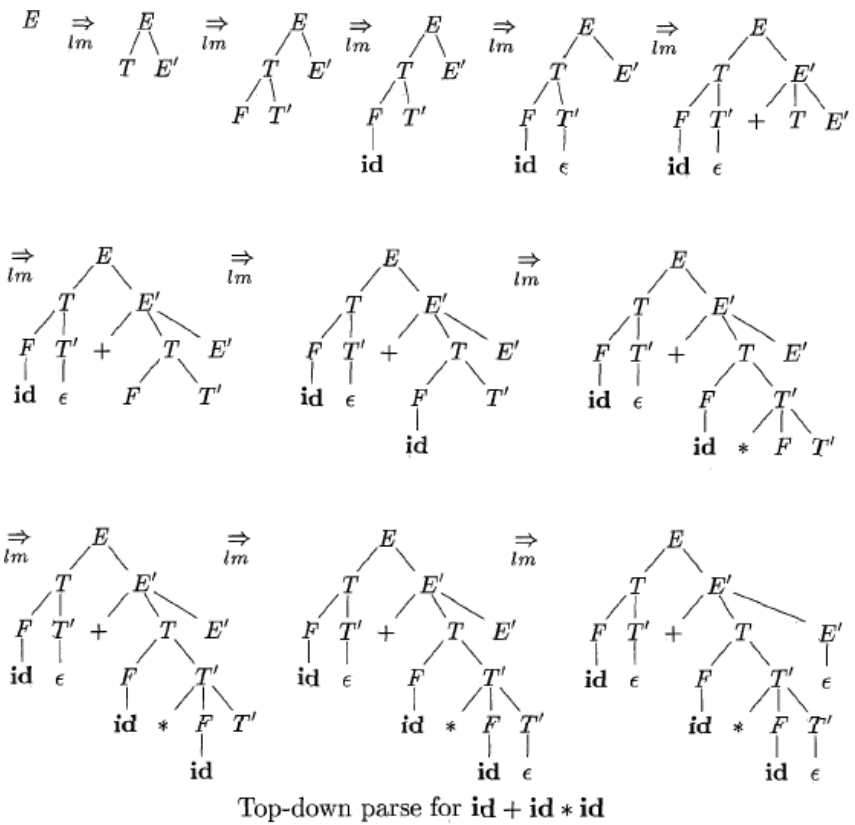
Top-Down Parsing

- Top-down parsing can be viewed as the problem of
  - constructing a parse tree for the input string,
  - starting from the root and
  - creating the nodes of the parse tree in preorder (depth-first).
- Equivalently, top-down parsing can be viewed as finding a leftmost derivation for an input string.

Example

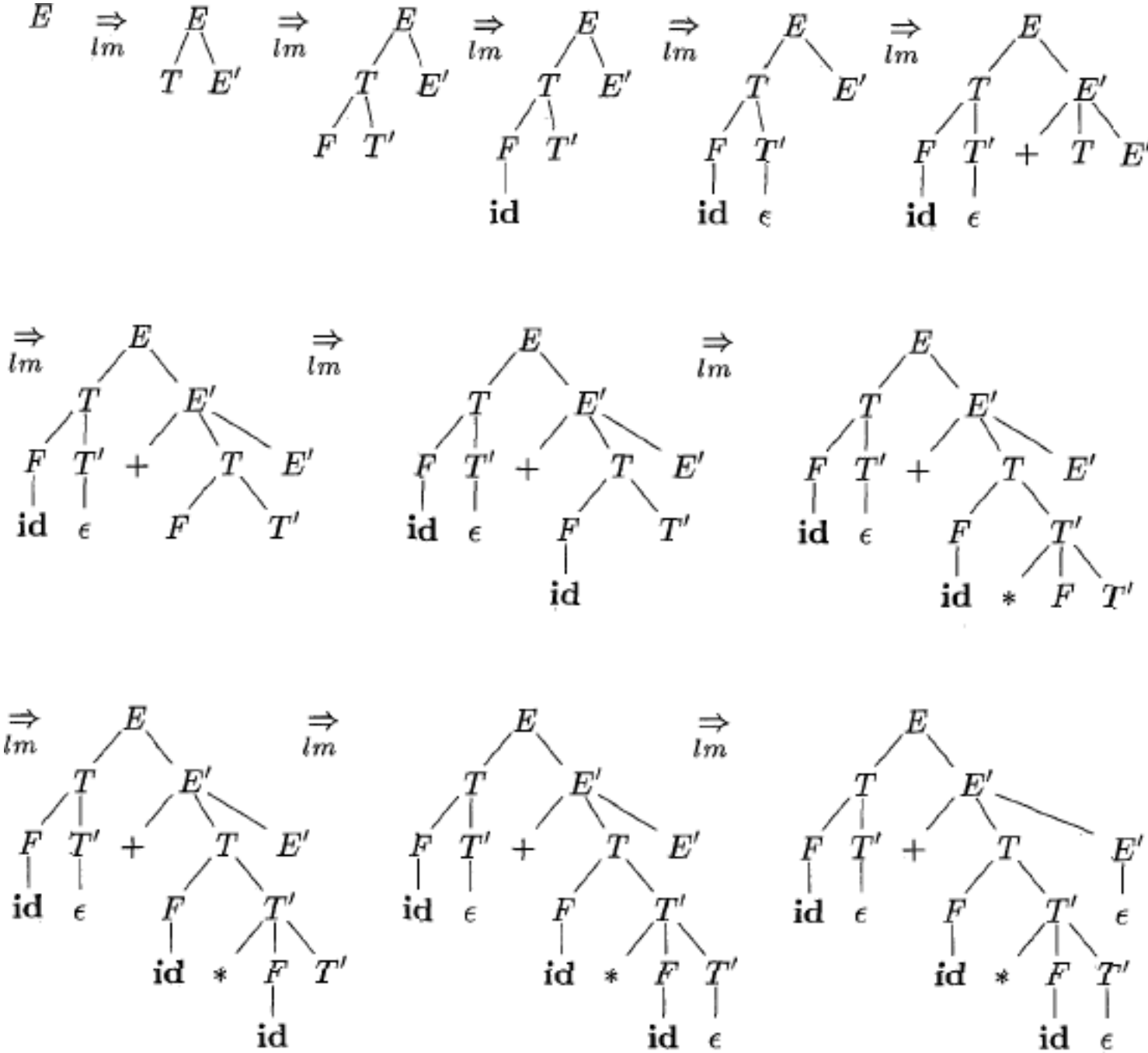
- The sequence of parse trees for the input **id** $+$ **id** $*$ **id** is a top-down parse according to grammar.

$$
\begin{aligned}
E &\rightarrow TE' \\
E' &\rightarrow +TE' \mid \epsilon \\
T &\rightarrow FT' \\
T' &\rightarrow *FT' \mid \epsilon \\
F &\rightarrow (E) \mid \textbf{id}
\end{aligned}
$$



Top-down parse for **id** + **id** * **id**

- This sequence of trees corresponds to a leftmost derivation of the input.

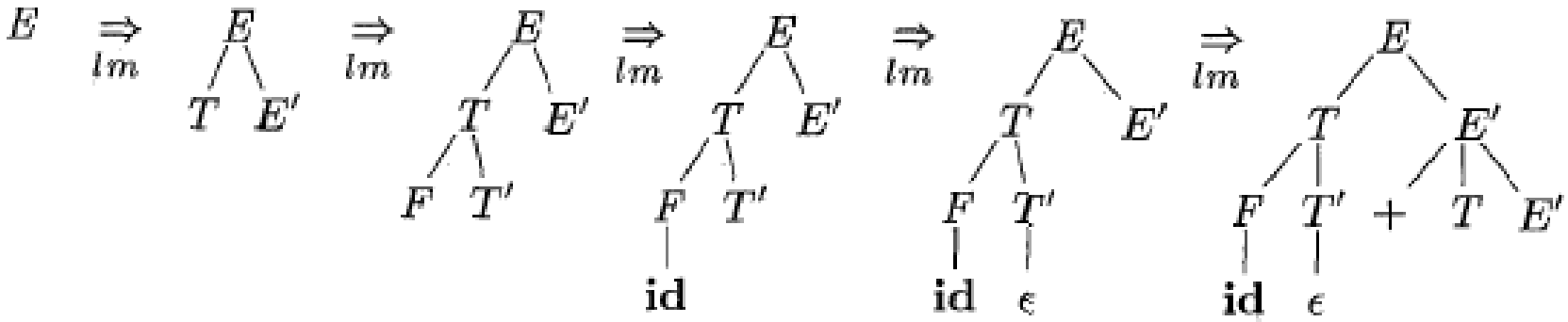


Top-down parse for **id + id * id**

Top-Down Parsing — *continued*

- At each step of a top-down parse, the key problem is that of determining the production to be applied for a nonterminal, say A.
- Once an A-production is chosen, the rest of the parsing process consists of "matching" the terminal symbols in the production body with the input string.

Top-Down Parsing — *continued*



- Consider the top-down parse in figure.
- This constructs a tree with two nodes labeled E'.
- At the first E' node (in preorder), the production $E' \to +TE'$ is chosen.
- At the second E' node, the production $E' \to \epsilon$ is chosen.
- A predictive parser can choose between E'-productions by looking at the next input symbol.

Top-Down Parsing — *continued*

- The class of grammars for which we can construct predictive parsers looking k symbols ahead in the input is sometimes called the LL(k) class.

FIRST and FOLLOW

- The construction of both top-down and bottom-up parsers is aided by two functions, FIRST and FOLLOW, associated with a grammar G.
- During top-down parsing, FIRST and FOLLOW allow us to choose which production to apply, based on the next input symbol.
- During panic-mode error recovery, sets of tokens produced by FOLLOW can be used as synchronizing tokens.

FIRST and FOLLOW

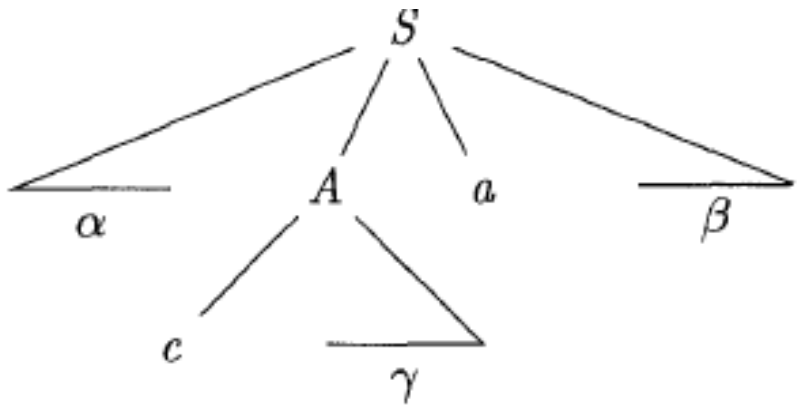
- The construction of both top-down and bottom-up parsers is aided by two functions, FIRST and FOLLOW, associated with a grammar G.
- During top-down parsing, FIRST and FOLLOW allow us to choose which production to apply, based on the next input symbol.
- During panic-mode error recovery, sets of tokens produced by FOLLOW can be used as synchronizing tokens.

FIRST and FOLLOW

- The construction of both top-down and bottom-up parsers is aided by two functions, FIRST and FOLLOW, associated with a grammar G.
- During top-down parsing, FIRST and FOLLOW allow us to choose which production to apply, based on the next input symbol.
- During panic-mode error recovery, sets of tokens produced by FOLLOW can be used as synchronizing tokens.

FIRST and FOLLOW — *continued*

- Define FIRST(α), where α is any string of grammar symbols, to be the set of terminals that begin strings derived from α.
- If $\alpha \Rightarrow \epsilon$, then ϵ is also in FIRST(α).
- For example, in figure $A \Rightarrow c\gamma$, so c is in FIRST(A).



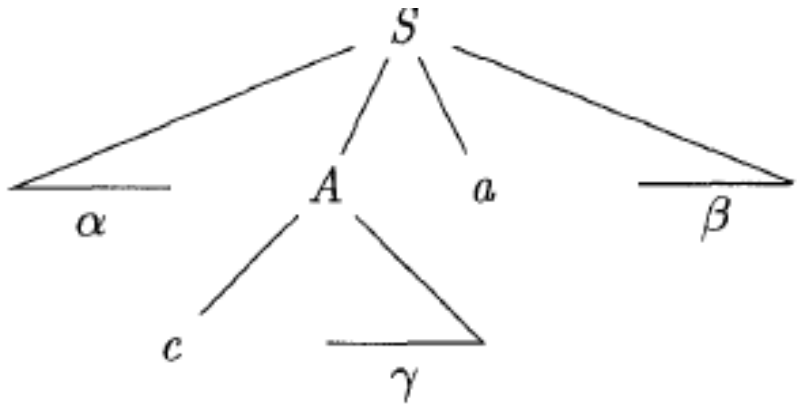
Terminal c is in FIRST(A) and a is in FOLLOW(A)

FIRST and FOLLOW — *continued*

- Let us see how FIRST can be used during predictive parsing.
- Consider two A-productions $A \to \alpha \mid \beta$, where FIRST(α) and FIRST(β) are disjoint sets.
- We can then choose between these A-productions by looking at the next input symbol a, since a can be in at most one of FIRST(α) and FIRST(β), not both.
- For instance, if a is in FIRST(β) choose the production $A \to \beta$.

FIRST and FOLLOW — *continued*

- Define FOLLOW(A), nonterminal A, to be the set of terminals a that can appear immediately to the right of A in some sentential form.
- That is, the set of terminals a such that there exists a derivation of the form $S \Rightarrow \alpha A a \beta$, for some α and β.
- Note that there may have been symbols between A and α, at some time during the derivation, but if so, they derived ϵ and disappeared.



Terminal c is in FIRST(A) and a is in FOLLOW(A)

FIRST and FOLLOW — *continued*

- In addition, if A can be the rightmost symbol in some sentential form, then $\$$ is in FOLLOW(A).
- Recall that $\$$ is a special "endmarker" symbol that is assumed not to be a symbol of any grammar.

FIRST and FOLLOW — *continued*

To compute FIRST(X) for all grammar symbols X, apply the following rules until no more terminals or ϵ can be added to any FIRST set.

1. If X is terminal, then FIRST(X) is $\{X\}$.

FIRST and FOLLOW — *continued*

2.
   - If X is nonterminal and $X \to Y_1 Y_2 \dots Y_k$ is a production for some $k \geq 1$, then place a in FIRST(X) if for some i, a is in FIRST(Y_i), and ϵ is in all of FIRST$(Y_1), \dots,$ FIRST(Y_{i-1}); that is, $Y_1, \dots, Y_{i-1} \stackrel{*}{\Rightarrow} \epsilon$.
   - If ϵ is in FIRST(Y_j) for all $j = 1, 2, \dots, k$ then add ϵ to FIRST(X).
3. If $X \to \epsilon$ is a production, then add ϵ to FIRST(X).

- For example, everything in FIRST(Y_1) is surely in FIRST(X).
- If Y_1, does not derive ϵ, then we add nothing more to FIRST(X), but if $Y_1 \stackrel{*}{\Rightarrow} \epsilon$, then we add FIRST$(Y_2)$ and so on.

FIRST and FOLLOW — *continued*

Now, we can compute FIRST for any string $X_1 X_2 \ldots X_n$ as follows.

- Add to FIRST$(X_1 X_2 \ldots X_n)$ all the non-ϵ symbols of FIRST(X_1).
- Also add the non-ϵ symbols of FIRST(X_2) if ϵ is in FIRST(X_1), the non-ϵ symbols of FIRST(X_3) if ϵ is in both FIRST(X_1) and FIRST(X_2) and so on.
- Finally, add ϵ to FIRST$(X_1 X_2 \ldots X_n)$ if, for all i, ϵ is in FIRST(X_i).

FIRST and FOLLOW — *continued*

To compute FOLLOW(A) for all nonterminals A, apply the following rules until nothing can be added to any FOLLOW set.

1. Place \$ in FOLLOW(S), where S is the start symbol and \$ is the input right endmarker.
2. If there is a production $A \to \alpha B \beta$, then everything in FIRST(β) except ϵ is in FOLLOW(B).
3. If there is a production $A \to \alpha B$, or a production $A \to \alpha B \beta$ where FIRST(β) contains ϵ, then everything in FOLLOW(A) is in FOLLOW(B).

Example

1. If X is terminal, then FIRST(X) is $\{X\}$.
2. If $X \to \epsilon$ is a production, then add ϵ to FIRST(X).
3. If X is nonterminal and $X \to Y_1 Y_2 \dots Y_k$ is a production for some $k \geq 1$, then place a in FIRST(X) if for some i, a is in FIRST(Y_i), and ϵ is in all of FIRST$(Y_1), \dots,$ FIRST(Y_{i-1}); that is, $Y_1, \dots, Y_{i-1} \stackrel{*}{\Rightarrow} \epsilon$. If ϵ is in FIRST(Y_j) for all $j = 1, 2, \dots, k$ then add ϵ to FIRST(X).

Grammar,

$$
\begin{aligned}
E &\to TE' \\
E' &\to +TE' \mid \epsilon \\
T &\to FT' \\
T' &\to *FT' \mid \epsilon \\
F &\to (E) \mid \textbf{id}
\end{aligned}
$$

Then,

$$\text{FIRST}(E) = \text{FIRST}(T) = \text{FIRST}(F) = \{(, \textbf{id}\}$$

$$\text{FIRST}(E') = \{+, \epsilon\}$$

$$\text{FIRST}(T') = \{*, \epsilon\}$$

Example

1. FIRST(F) = FIRST(T) = FIRST$(E) = \{($, **id**$\}$.
   - To see why, note that the two productions for F have bodies that start with these two terminal symbols, **id** and the left parenthesis.
   - T has only one production, and its body starts with F.
   - Since F does not derive ϵ, FIRST(T) must be the same as FIRST(F).
   - The same argument covers FIRST(E).

Grammar,

$$
\begin{aligned}
E &\rightarrow TE' \\
E' &\rightarrow +TE' \mid \epsilon \\
T &\rightarrow FT' \\
T' &\rightarrow *FT' \mid \epsilon \\
F &\rightarrow (E) \mid \textbf{id}
\end{aligned}
$$

Then,

$$\text{FIRST}(E) = \text{FIRST}(T) = \text{FIRST}(F) = \{(, \textbf{id}\}$$

$$\text{FIRST}(E') = \{+, \epsilon\}$$

$$\text{FIRST}(T') = \{*, \epsilon\}$$

Example

2. FIRST$(E') = \{+, \epsilon)$.
   - The reason is that one of the two productions for E' has a body that begins with terminal $+$, and the other's body is ϵ.
   - Whenever a nonterminal derives ϵ, we place ϵ in FIRST for that nonterminal.

Grammar,

$$
\begin{aligned}
E &\rightarrow TE' \\
E' &\rightarrow +TE' \mid \epsilon \\
T &\rightarrow FT' \\
T' &\rightarrow *FT' \mid \epsilon \\
F &\rightarrow (E) \mid \textbf{id}
\end{aligned}
$$

Then,

$$\text{FIRST}(E) = \text{FIRST}(T) = \text{FIRST}(F) = \{(, \textbf{id}\}$$

$$\text{FIRST}(E') = \{+, \epsilon\}$$

$$\text{FIRST}(T') = \{*, \epsilon\}$$

Example

3. FIRST$(T') = \{*, \epsilon)$.
   - The reasoning is analogous to that for FIRST(E').

Grammar,

$$
\begin{aligned}
E &\rightarrow TE' \\
E' &\rightarrow +TE' \mid \epsilon \\
T &\rightarrow FT' \\
T' &\rightarrow *FT' \mid \epsilon \\
F &\rightarrow (E) \mid \textbf{id}
\end{aligned}
$$

Then,

$$\text{FIRST}(E) = \text{FIRST}(T) = \text{FIRST}(F) = \{(, \textbf{id}\}$$

$$\text{FIRST}(E') = \{+, \epsilon\}$$

$$\text{FIRST}(T') = \{*, \epsilon\}$$

Example — *continued*

- Grammar:

$$E \rightarrow TE' \qquad T \rightarrow FT' \qquad F \rightarrow (E) \mid \textbf{id}$$
$$E' \rightarrow +TE' \mid \epsilon \qquad T' \rightarrow *FT' \mid \epsilon$$

- Computation of FOLLOW:

FOLLOW(E)	FOLLOW(E')	FOLLOW(T)	FOLLOW(T')	FOLLOW(F)

Initially all sets are empty

Put $ in FOLLOW(E) by rule (1) *(Place $ in FOLLOW(S), where S is the start symbol and $ is the input right endmarker)*

$				

Example — *continued*

$$E \rightarrow TE' \qquad T \rightarrow FT' \qquad F \rightarrow (E) \mid \textbf{id}$$
$$E' \rightarrow +TE' \mid \epsilon \qquad T' \rightarrow *FT' \mid \epsilon$$

FIRST(E) = FIRST(T) = FIRST(F) = $\{($, **id**$\}$, FIRST(E') = $\{+, \epsilon\}$,

FIRST(T') = $\{*, \epsilon\}$

By rule (2) (If there is a production $A \rightarrow \alpha B \beta$, then everything in FIRST(β) except for ϵ is placed in FOLLOW(B)) applied to,

- $E \rightarrow TE'$ *: FIRST(E') except ϵ i.e. $\{+\}$ are in FOLLOW(T)*
- $E' \rightarrow +TE'$ *: FIRST(E') except ϵ i.e. $\{+\}$ are in FOLLOW(T)*
- $T \rightarrow FT'$ *: FIRST(T') except ϵ i.e. $\{*\}$ are in FOLLOW(F)*
- $T \rightarrow *FT'$ *: FIRST(T') except ϵ i.e. $\{*\}$ are in FOLLOW(F)*
- $F \rightarrow (E)$*: FIRST$())$ i.e. $\{)\}$ are in FOLLOW(E)*

FOLLOW(E)	FOLLOW(E')	FOLLOW(T)	FOLLOW(T')	FOLLOW(F)
$\$,)$		$+$		$*$

Rule (2) is not applicable any more since it depends only on FIRST, which are now stable sets.

Example — *continued*

Application of rule (3) *(If there is a production $A \to \alpha B$, or a production $A \to \alpha B \beta$ where FIRST(β) contains ϵ (i.e., $\beta \stackrel{*}{\Longrightarrow} \epsilon$), then everything in FOLLOW(A) is in FOLLOW(B))*

$$E \to TE' \qquad T \to FT' \qquad F \to (E) \mid \textbf{id}$$
$$E' \to +TE' \mid \epsilon \qquad T' \to *FT' \mid \epsilon$$

FOLLOW(E)	FOLLOW(E')	FOLLOW(T)	FOLLOW(T')	FOLLOW(F)
$\$,)$		$+$		$*$

$E \to TE'$: Everything in FOLLOW(E) are in FOLLOW(E')

$\$,)$	$\$,)$	$+$		$*$

$E' \to +TE'$ (also $\epsilon \in$ FIRST(E')): Everything in FOLLOW(E') are in FOLLOW(T)

$\$,)$	$\$,)$	$+, \$,)$		$*$

$T \to FT'$: Everything in FOLLOW(T) are in FOLLOW(T')

$\$,)$	$\$,)$	$+, \$,)$	$+, \$,)$	$*$

Example — *continued*

Application of rule (3) *(If there is a production $A \to \alpha B$, or a production $A \to \alpha B\beta$ where FIRST(β) contains ϵ (i.e., $\beta \stackrel{*}{\Longrightarrow} \epsilon$), then everything in FOLLOW(A) is in FOLLOW(B))*

$$E \to TE' \qquad T \to FT' \qquad F \to (E) \mid \textbf{id}$$
$$E' \to +TE' \mid \epsilon \qquad T' \to *FT' \mid \epsilon$$

FOLLOW(E)	FOLLOW(E')	FOLLOW(T)	FOLLOW(T')	FOLLOW(F)
$\$,)$		$+$		$*$

$E \to TE'$: Everything in FOLLOW(E) are in FOLLOW(E')

$\$,)$	$\$,)$	$+$		$*$

$E' \to +TE'$ (also $\epsilon \in$ FIRST(E')): Everything in FOLLOW(E') are in FOLLOW(T)

$\$,)$	$\$,)$	$+, \$,)$		$*$

$T \to FT'$: Everything in FOLLOW(T) are in FOLLOW(T')

$\$,)$	$\$,)$	$+, \$,)$	$+, \$,)$	$*$

Example — *continued*

Application of rule (3) *(If there is a production $A \to \alpha B$, or a production $A \to \alpha B\beta$ where FIRST(β) contains ϵ (i.e., $\beta \stackrel{*}{\Longrightarrow} \epsilon$), then everything in FOLLOW(A) is in FOLLOW(B))*

$$E \to TE' \qquad T \to FT' \qquad F \to (E) \mid \textbf{id}$$
$$E' \to +TE' \mid \epsilon \qquad T' \to *FT' \mid \epsilon$$

FOLLOW(E)	FOLLOW(E')	FOLLOW(T)	FOLLOW(T')	FOLLOW(F)
$,)		+		*

$E \to TE'$: Everything in FOLLOW(E) are in FOLLOW(E')

$,)	$,)	+		*

$E' \to +TE'$ (also $\epsilon \in$ FIRST(E')): Everything in FOLLOW(E') are in FOLLOW(T)

$,)	$,)	+, $,)		*

$T \to FT'$: Everything in FOLLOW(T) are in FOLLOW(T')

$,)	$,)	+, $,)	+, $,)	*

Example — *continued*

Application of rule (3) *(If there is a production $A \to \alpha B$, or a production $A \to \alpha B\beta$ where FIRST(β) contains ϵ (i.e., $\beta \stackrel{*}{\Longrightarrow} \epsilon$), then everything in FOLLOW(A) is in FOLLOW(B))*

$$E \to TE' \qquad T \to FT' \qquad F \to (E) \mid \textbf{id}$$
$$E' \to +TE' \mid \epsilon \qquad T' \to *FT' \mid \epsilon$$

FOLLOW(E)	FOLLOW(E')	FOLLOW(T)	FOLLOW(T')	FOLLOW(F)
$\$,)$		$+$		$*$

$E \to TE'$: Everything in FOLLOW(E) are in FOLLOW(E')

$\$,)$	$\$,)$	$+$		$*$

$E' \to +TE'$ (also $\epsilon \in$ FIRST(E')): Everything in FOLLOW(E') are in FOLLOW(T)

$\$,)$	$\$,)$	$+, \$,)$		$*$

$T \to FT'$: Everything in FOLLOW(T) are in FOLLOW(T')

$\$,)$	$\$,)$	$+, \$,)$	$+, \$,)$	$*$

Example — *continued*

Application of rule (3) *(If there is a production $A \to \alpha B$, or a production $A \to \alpha B\beta$ where FIRST(β) contains ϵ (i.e., $\beta \stackrel{*}{\Longrightarrow} \epsilon$), then everything in FOLLOW(A) is in FOLLOW(B))*

$$E \to TE' \qquad T \to FT' \qquad F \to (E) \mid \textbf{id}$$
$$E' \to +TE' \mid \epsilon \qquad T' \to *FT' \mid \epsilon$$

FOLLOW(E)	FOLLOW(E')	FOLLOW(T)	FOLLOW(T')	FOLLOW(F)
$\$,)$		$+$		$*$

$E \to TE'$: Everything in FOLLOW(E) are in FOLLOW(E')

$\$,)$	$\$,)$	$+$		$*$

$E' \to +TE'$ (also $\epsilon \in$ FIRST(E')): Everything in FOLLOW(E') are in FOLLOW(T)

$\$,)$	$\$,)$	$+, \$,)$		$*$

$T \to FT'$: Everything in FOLLOW(T) are in FOLLOW(T')

$\$,)$	$\$,)$	$+, \$,)$	$+, \$,)$	$*$

Example — *continued*

Application of rule (3) *(If there is a production $A \to \alpha B$, or a production $A \to \alpha B \beta$ where FIRST(β) contains ϵ (i.e., $\beta \stackrel{*}{\Longrightarrow} \epsilon$), then everything in FOLLOW(A) is in FOLLOW(B))*

$$E \to TE' \qquad T \to FT' \qquad F \to (E) \mid \textbf{id}$$
$$E' \to +TE' \mid \epsilon \qquad T' \to *FT' \mid \epsilon$$

FOLLOW(E)	FOLLOW(E')	FOLLOW(T)	FOLLOW(T')	FOLLOW(F)
$\$,)$		$+$		$*$

$E \to TE'$: Everything in FOLLOW(E) are in FOLLOW(E')

$\$,)$	$\$,)$	$+$		$*$

$E' \to +TE'$ (also $\epsilon \in$ FIRST(E')): Everything in FOLLOW(E') are in FOLLOW(T)

$\$,)$	$\$,)$	$+, \$,)$		$*$

$T \to FT'$: Everything in FOLLOW(T) are in FOLLOW(T')

$\$,)$	$\$,)$	$+, \$,)$	$+, \$,)$	$*$

Example — *continued*

Application of rule (3) — continued *(If there is a production $A \to \alpha B$, or a production $A \to \alpha B \beta$ where FIRST(β) contains ϵ (i.e., $\beta \stackrel{*}{\Longrightarrow} \epsilon$), then everything in FOLLOW(A) is in FOLLOW(B))*

$$E \to TE' \qquad T \to FT' \qquad F \to (E) \mid \textbf{id}$$
$$E' \to +TE' \mid \epsilon \qquad T' \to *FT' \mid \epsilon$$

FOLLOW(E)	FOLLOW(E')	FOLLOW(T)	FOLLOW(T')	FOLLOW(F)
$\$,)$	$\$,)$	$+, \$,)$	$+, \$,)$	$*$

$T' \to *FT'$ (also $\epsilon \in$ FIRST(T')): Everything in FOLLOW(T') are in FOLLOW(F)

$\$,)$	$\$,)$	$+, \$,)$	$+, \$,)$	$*, +, \$,)$

We can try applying Rule (3) again, but will find that the sets have stabilized (nothing can be added to any FOLLOW set).

Example — *continued*

Application of rule (3) — continued *(If there is a production $A \to \alpha B$, or a production $A \to \alpha B \beta$ where FIRST(β) contains ϵ (i.e., $\beta \stackrel{*}{\Longrightarrow} \epsilon$), then everything in FOLLOW(A) is in FOLLOW(B))*

$$E \to TE' \qquad T \to FT' \qquad F \to (E) \mid \textbf{id}$$
$$E' \to +TE' \mid \epsilon \qquad T' \to *FT' \mid \epsilon$$

FOLLOW(E)	FOLLOW(E')	FOLLOW(T)	FOLLOW(T')	FOLLOW(F)
$\$,)$	$\$,)$	$+, \$,)$	$+, \$,)$	$*$

$T' \to *FT'$ (also $\epsilon \in$ FIRST(T')): Everything in FOLLOW(T') are in FOLLOW(F)

$\$,)$	$\$,)$	$+, \$,)$	$+, \$,)$	$*, +, \$,)$

We can try applying Rule (3) again, but will find that the sets have stabilized (nothing can be added to any FOLLOW set).

Example — *continued*

Application of rule (3) — continued *(If there is a production $A \to \alpha B$, or a production $A \to \alpha B \beta$ where FIRST(β) contains ϵ (i.e., $\beta \stackrel{*}{\Longrightarrow} \epsilon$), then everything in FOLLOW(A) is in FOLLOW(B))*

$$E \to TE' \qquad T \to FT' \qquad F \to (E) \mid \textbf{id}$$
$$E' \to +TE' \mid \epsilon \qquad T' \to *FT' \mid \epsilon$$

FOLLOW(E)	FOLLOW(E')	FOLLOW(T)	FOLLOW(T')	FOLLOW(F)
$\$,)$	$\$,)$	$+, \$,)$	$+, \$,)$	$*$

$T' \to *FT'$ (also $\epsilon \in$ FIRST(T')): Everything in FOLLOW(T') are in FOLLOW(F)

$\$,)$	$\$,)$	$+, \$,)$	$+, \$,)$	$*, +, \$,)$

We can try applying Rule (3) again, but will find that the sets have stabilized (nothing can be added to any FOLLOW set).

Example — *continued*

Application of rule (3) — continued *(If there is a production $A \to \alpha B$, or a production $A \to \alpha B \beta$ where FIRST(β) contains ϵ (i.e., $\beta \stackrel{*}{\Longrightarrow} \epsilon$), then everything in FOLLOW(A) is in FOLLOW(B))*

$$E \to TE' \qquad T \to FT' \qquad F \to (E) \mid \textbf{id}$$
$$E' \to +TE' \mid \epsilon \qquad T' \to *FT' \mid \epsilon$$

FOLLOW(E)	FOLLOW(E')	FOLLOW(T)	FOLLOW(T')	FOLLOW(F)
$\$,)$	$\$,)$	$+, \$,)$	$+, \$,)$	$*$

$T' \to *FT'$ (also $\epsilon \in$ FIRST(T')): Everything in FOLLOW(T') are in FOLLOW(F)

$\$,)$	$\$,)$	$+, \$,)$	$+, \$,)$	$*, +, \$,)$

We can try applying Rule (3) again, but will find that the sets have stabilized (nothing can be added to any FOLLOW set).

LL(1) Grammars

- Predictive parsers, that is, recursive-descent parsers needing no backtracking, can be constructed for a class of grammars called LL(1).
- The first “L” in LL(1) stands for scanning the input from left to right.
- The second “L” for producing a leftmost derivation.
- And the “1” for using one input symbol of lookahead at each step to make parsing action decisions.

LL(1) Grammars — *continued*

- The class of LL(1) grammars is rich enough to cover most programming constructs.
- Although care is needed in writing a suitable grammar for the source language.
- For example, no left-recursive or ambiguous grammar can be LL(1).

LL(1) Grammars — *continued*

A grammar G is LL(1) if and only if whenever $A \to \alpha \mid \beta$ are two distinct productions of G the following conditions hold:

1. For no terminal a do both α and β derive strings beginning with a.
2. At most one of α and β can derive the empty string.
3. If $\beta \stackrel{*}{\Rightarrow} \epsilon$, then α does not derive any string beginning with a terminal in FOLLOW(A).
   Likewise, $\alpha \stackrel{*}{\Rightarrow} \epsilon$, then β does not derive any string beginning with a terminal in FOLLOW(A).

LL(1) Grammars — *continued*

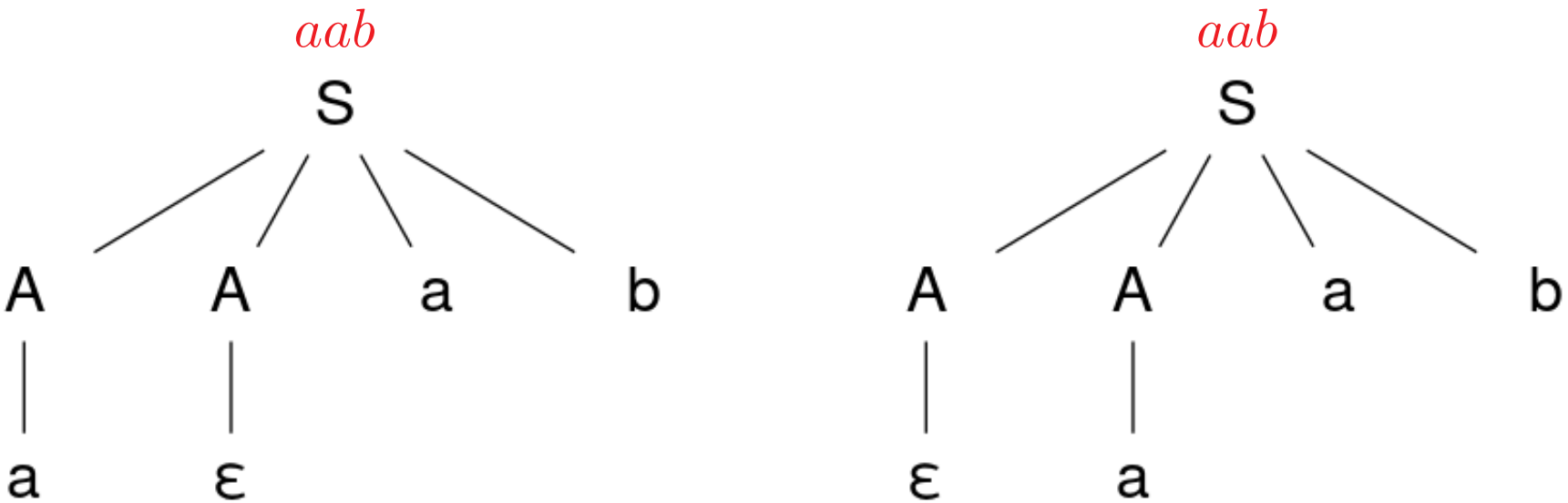
1. For no terminal a do both α and β derive strings beginning with a.
2. At most one of α and β can derive the empty string.
3. If $\beta \stackrel{*}{\Rightarrow} \epsilon$, then α does not derive any string beginning with a terminal in FOLLOW(A).

   Likewise, $\alpha \stackrel{*}{\Rightarrow} \epsilon$, then β does not derive any string beginning with a terminal in FOLLOW(A).

- The first two conditions are equivalent to the statement that FIRST(α) and FIRST(β) are disjoint sets.
- The third condition is equivalent to stating that if ϵ is in FIRST(β), then FIRST(α) and FOLLOW(A) are disjoint sets, and likewise if ϵ is in FIRST(α).

A Case of a non-LL(1) Grammar

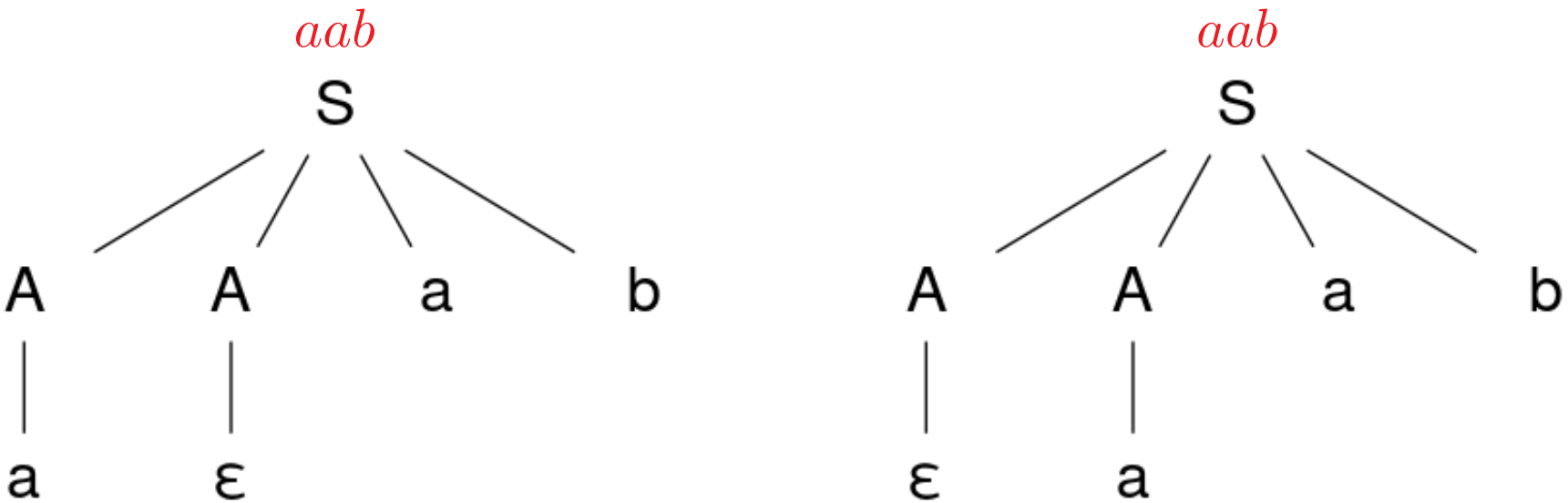
$$
\begin{aligned}
S &\rightarrow AAab \mid BBba \\
A &\rightarrow a \mid \epsilon \\
B &\rightarrow b \mid \epsilon
\end{aligned}
$$



- Now, when expanding the first A, we can not decide which of the two production rules to be used.
- Both give us the promise of an a.

A Case of a non-LL(1) Grammar

$$
\begin{aligned}
S &\rightarrow AAab \mid BBba \\
A &\rightarrow a \mid \epsilon \\
B &\rightarrow b \mid \epsilon
\end{aligned}
$$



- Two derivations for the same string.

LL(1) Grammars — *continued*

- Predictive parsers can be constructed for LL(1) grammars since the proper production to apply for a nonterminal can be selected by looking only at the current input symbol.
- Flow-of-control constructs, with their distinguishing key-words, generally satisfy the LL(1) constraints.

LL(1) Grammars — *continued*

- For instance, if we have the productions,

$$\begin{aligned} \textit{stmt} \quad &\rightarrow \quad \textbf{if} \ (\ \textit{expr}\ )\ \textit{stmt}\ \textbf{else}\ \textit{stmt} \\ &\mid \quad \textbf{while}\ (\ \textit{expr}\ )\ \textit{stmt} \\ &\mid \quad \{\ \textit{stmt_list}\ \} \end{aligned}$$

  then the keywords **if**, **while**, and the symbol { tell us which alternative is the one that could possibly succeed if we are to find a statement.

LL(1) Grammars — *continued*

- The next algorithm collects the information from FIRST and FOLLOW sets into a predictive parsing table $M[A, a]$, a two dimensional array, where A is a nonterminal, and a is a terminal or the symbol $\$$, the input endmarker.
- The idea behind the algorithm is the following.
- Suppose $A \to \alpha$ is a production with a in FIRST(α).
- Then, the parser will expand A by α when the current input symbol is a.
- The only complication occurs when $\alpha = \epsilon$ or $\alpha \stackrel{*}{\Rightarrow} \epsilon$.
- In this case, we should again expand A by α if the current input symbol is in FOLLOW(A), or if the $\$$ on the input has been reached and $\$$ is in FOLLOW(A).

Algorithm for Construction of a Predictive Parsing Table

INPUT: Grammar G.

OUTPUT: Parsing table M.

METHOD: For each production $A \to \alpha$ of the grammar, do the following:

1. For each terminal a in FIRST(α), add $A \to \alpha$ to $M[A, a]$.
2. If ϵ is in FIRST(α), then for each terminal b in FOLLOW(A), add $A \to \alpha$ to $M[A, b]$.
   If ϵ is in FIRST(α) and \$ is in FOLLOW(A), add $A \to \alpha$ to $M[A, \$]$ as well.

If, after performing the above, there is no production at all in $M[A, a]$, then set $M[A, a]$ to **error** (which we normally represent by an empty entry in the table).

Example

- For the expression grammar below,

$$E \rightarrow TE' \qquad T \rightarrow FT' \qquad F \rightarrow (E) \mid \mathbf{id}$$
$$E' \rightarrow +TE' \mid \epsilon \qquad T' \rightarrow *FT' \mid \epsilon$$

the algorithm produces the parsing table in figure.

NON-TERMINAL	INPUT SYMBOL					
	id	+	*	(	)	$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow \mathbf{id}$			$F \rightarrow (E)$		

- Blanks are error entries.
- Nonblanks indicate a production with which to expand a nonterminal.

Example — *continued*

For each production $A \to \alpha$ of the grammar, do the following:

1. For each terminal a in FIRST(α), add $A \to \alpha$ to $M[A, a]$.
2. If ϵ is in FIRST(α), then for each terminal b in FOLLOW(A), add $A \to \alpha$ to $M[A, b]$.
   If ϵ is in FIRST(α) and \$ is in FOLLOW(A), add $A \to \alpha$ to $M[A, \$]$ as well.

NON-TERMINAL	INPUT SYMBOL					
	id	+	*	(	)	\$
E	$E \to TE'$			$E \to TE'$		
E'		$E' \to +TE'$			$E' \to \epsilon$	$E' \to \epsilon$
T	$T \to FT'$			$T \to FT'$		
T'		$T' \to \epsilon$	$T' \to *FT'$		$T' \to \epsilon$	$T' \to \epsilon$
F	$F \to$ **id**			$F \to (E)$		

- Consider production $E \to TE'$.
- Since,

$$\text{FIRST}(TE') = \text{FIRST}(T) = \{(, \textbf{id}\}$$

this production is added to $M[E, (]$ and $M[E,$ **id**$]$.

Example — *continued*

For each production $A \to \alpha$ of the grammar, do the following:

1. For each terminal a in FIRST(α), add $A \to \alpha$ to $M[A, a]$.
2. If ϵ is in FIRST(α), then for each terminal b in FOLLOW(A), add $A \to \alpha$ to $M[A, b]$.
   If ϵ is in FIRST(α) and \$ is in FOLLOW(A), add $A \to \alpha$ to $M[A, \$]$ as well.

NON-TERMINAL	INPUT SYMBOL					
	id	+	*	(	)	\$
E	$E \to TE'$			$E \to TE'$		
E'		$E' \to +TE'$			$E' \to \epsilon$	$E' \to \epsilon$
T	$T \to FT'$			$T \to FT'$		
T'		$T' \to \epsilon$	$T' \to *FT'$		$T' \to \epsilon$	$T' \to \epsilon$
F	$F \to$ **id**			$F \to (E)$		

- Production $E' \to +TE'$ is added to $M[E', +]$ since FIRST$(+TE') = \{+\}$.

Example — *continued*

For each production $A \to \alpha$ of the grammar, do the following:

1. For each terminal a in FIRST(α), add $A \to \alpha$ to $M[A, a]$.
2. If ϵ is in FIRST(α), then for each terminal b in FOLLOW(A), add $A \to \alpha$ to $M[A, b]$.
   If ϵ is in FIRST(α) and \$ is in FOLLOW(A), add $A \to \alpha$ to $M[A, \$]$ as well.

NON-TERMINAL	INPUT SYMBOL					
	id	+	*	(	)	\$
E	$E \to TE'$			$E \to TE'$		
E'		$E' \to +TE'$			$E' \to \epsilon$	$E' \to \epsilon$
T	$T \to FT'$			$T \to FT'$		
T'		$T' \to \epsilon$	$T' \to *FT'$		$T' \to \epsilon$	$T' \to \epsilon$
F	$F \to$ **id**			$F \to (E)$		

- Since FOLLOW(E') $= \{), \$\}$, production $E' \to \epsilon$ is added to $M[E',)]$ and $M[E', \$]$.

Algorithm . . . Predictive Parsing Table — *continued*

- The aforementioned algorithm can be applied to any grammar G to produce a parsing table M.
- For every LL(1) grammar, each parsing-table entry uniquely identifies a production or signals an error.

Algorithm . . . Predictive Parsing Table — *continued*

- For some grammars, however, M may have some entries that are multiply defined.
- For example, if G is left-recursive or ambiguous, then M will have at least one multiply defined entry.
- Although left-recursion elimination and left factoring are easy to do, there are some grammars for which no amount of alteration will produce an LL(1) grammar.
- The language in the following example has no LL(1) grammar at all.

Example

- The following grammar, which abstracts the dangling-else problem, is repeated here:
- The grammar,

$$\begin{aligned} S &\rightarrow iEtSS' \mid a \\ S' &\rightarrow eS \mid \epsilon \\ E &\rightarrow b \end{aligned}$$

- The grammar is ambiguous.
- On input e, it will not be clear which alternative for S' should be chosen.

Example — *continued*

$$
\begin{aligned}
S &\rightarrow iEtSS' \mid a \\
S' &\rightarrow eS \mid \epsilon \\
E &\rightarrow b
\end{aligned}
$$

NON - TERMINAL	INPUT SYMBOL					
	a	b	e	i	t	\$
S	$S \rightarrow a$			$S \rightarrow iEtSS'$		
S'			$S' \rightarrow \epsilon$ $S' \rightarrow eS$			$S' \rightarrow \epsilon$
E		$E \rightarrow b$				

Example — *continued*

NON - TERMINAL	INPUT SYMBOL					
	a	b	e	i	t	\$
S	$S \to a$			$S \to iEtSS'$		
S'			$S' \to \epsilon$ $S' \to eS$			$S' \to \epsilon$
E		$E \to b$				

$$
\begin{aligned}
S &\to iEtSS' \mid a \\
S' &\to eS \mid \epsilon \\
E &\to b
\end{aligned}
$$

- The entry for $M[S', e]$ contains both $S' \to eS$ and $S' \to \epsilon$, since FOLLOW$(S') = \{e, \$\}$

Example — *continued*

NON - TERMINAL	INPUT SYMBOL					
	a	b	e	i	t	\$
S	$S \to a$			$S \to iEtSS'$		
S'			$S' \to \epsilon$ $S' \to eS$			$S' \to \epsilon$
E		$E \to b$				

- The grammar is ambiguous and the ambiguity is manifested by a choice in what production to use when an e (**else**) is seen.
- We can resolve the ambiguity if we choose $S' \to eS$.
- This choice corresponds to associating **else**'s with the closest previous **then**'s.

Example — *continued*

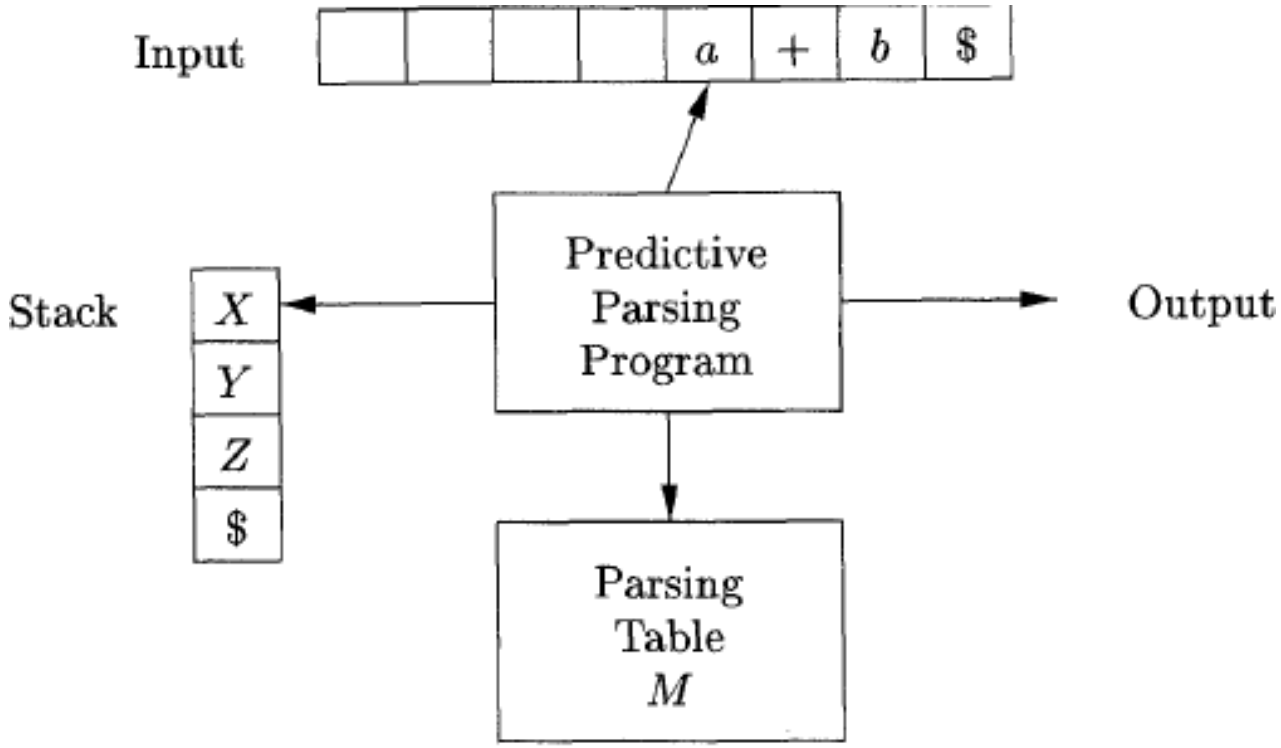
NON - TERMINAL	INPUT SYMBOL					
	a	b	e	i	t	\$
S	$S \to a$			$S \to iEtSS'$		
S'			$S' \to \epsilon$ $S' \to eS$			$S' \to \epsilon$
E		$E \to b$				

- Note that the choice $S' \to \epsilon$ would prevent e from ever being put on the stack or removed from the input, and is therefore surely wrong.

Nonrecursive Predictive Parsing

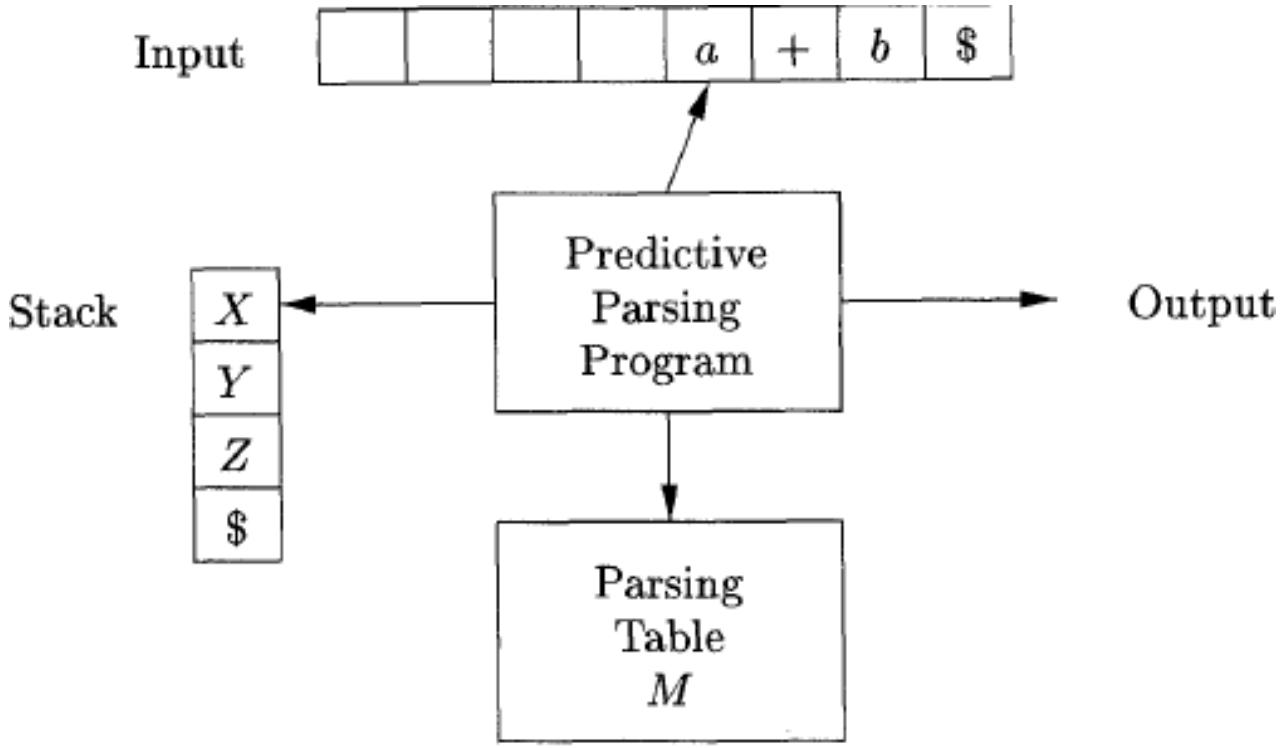
- A nonrecursive predictive parser can be built by maintaining a stack explicitly, rather than implicitly via recursive calls.
- The parser mimics a leftmost derivation.
- If w is the input that has been matched so far, then the stack holds a sequence of grammar symbols α such that

$$S \underset{lm}{\overset{*}{\Longrightarrow}} w\alpha$$



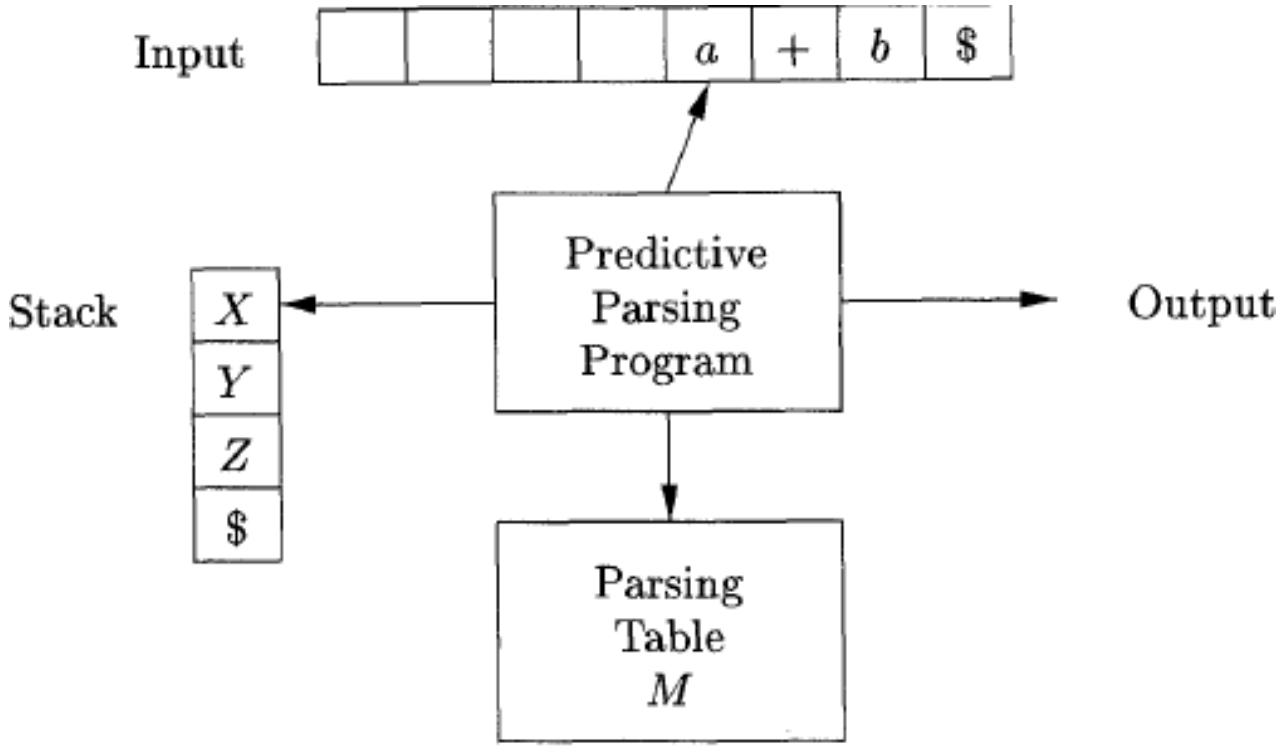
Model of a table-driven predictive parser

- The table-driven parser in figure has an input buffer, a stack containing a sequence of grammar symbols, a parsing table constructed by algorithm, and an output stream.



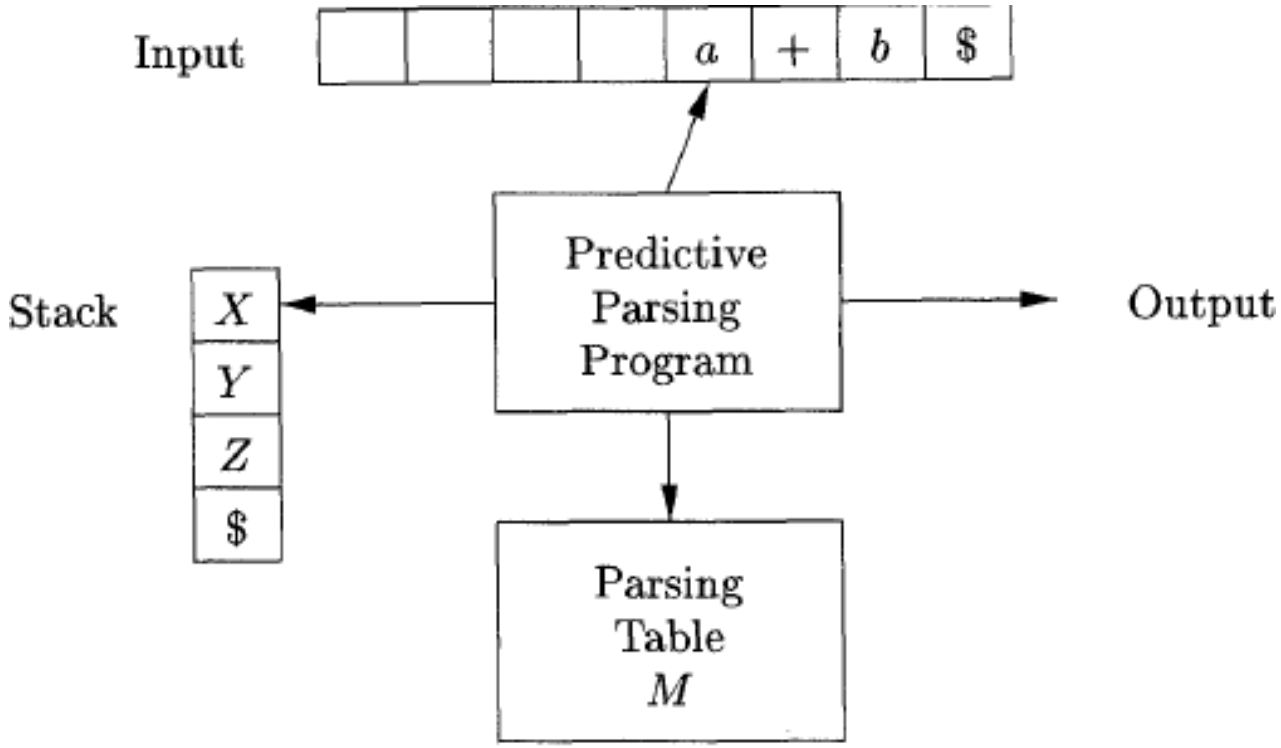
Model of a table-driven predictive parser

- The input buffer contains the string to be parsed, followed by the endmarker $.
- We reuse the symbol $ to mark the bottom of the stack, which initially contains the start symbol of the grammar on top of $.



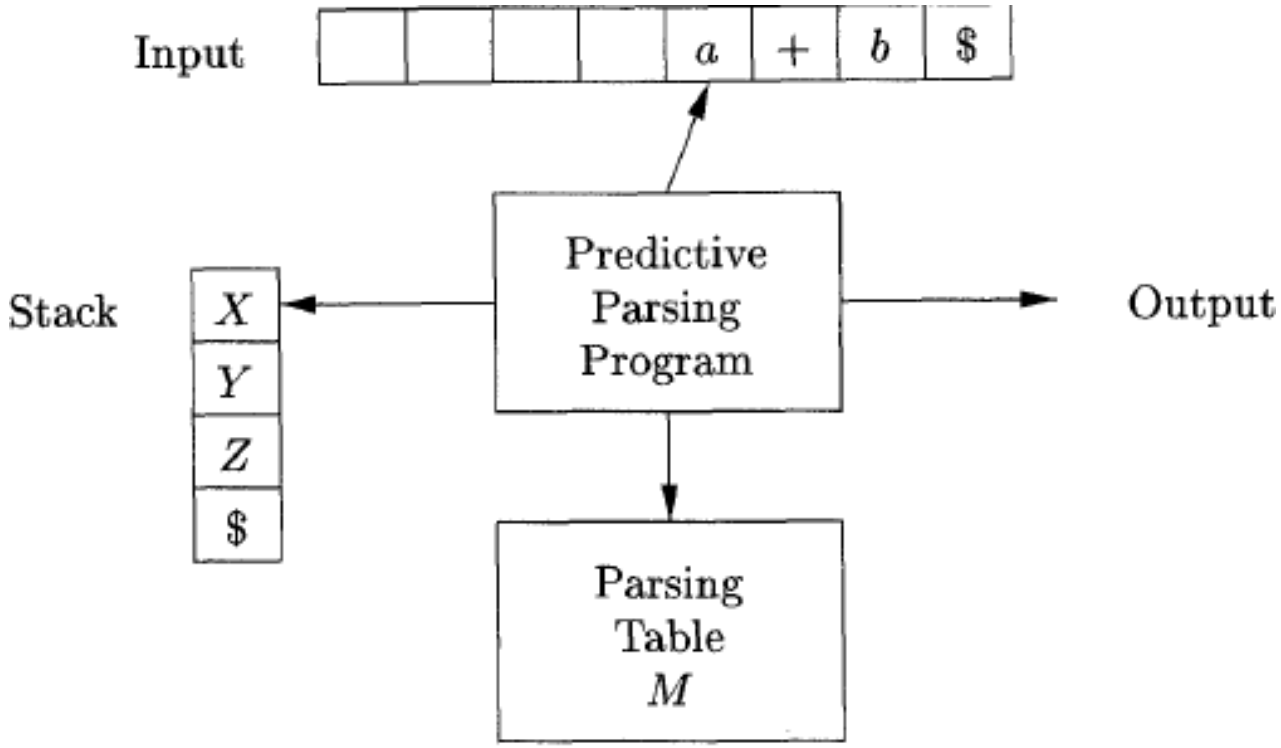
Model of a table-driven predictive parser

- The parser is controlled by a program that considers X, the symbol on top of the stack, and a, the current input symbol.



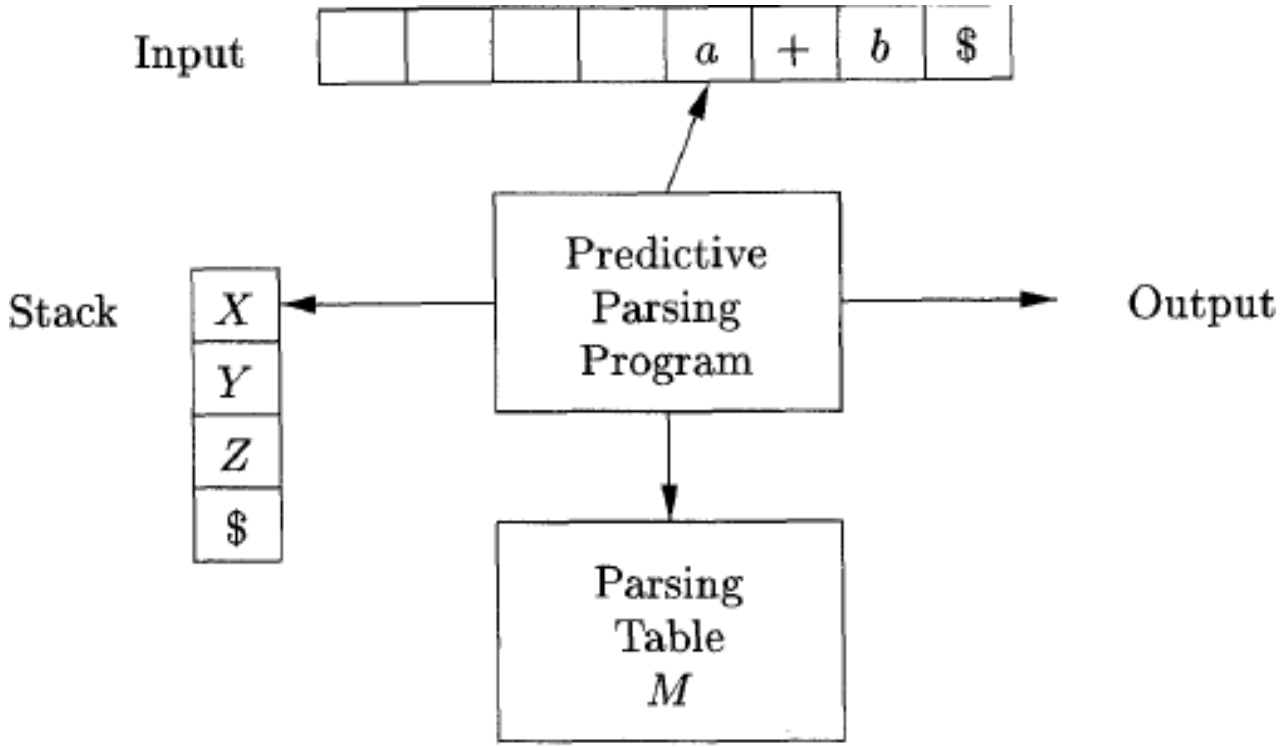
Model of a table-driven predictive parser

- If X is a nonterminal, the parser chooses an X-production by consulting entry $M[X, a]$ of the parsing table M.
- Additional code could be executed here, for example, code to construct a node in a parse tree.



Model of a table-driven predictive parser

- Otherwise, it checks for a match between the terminal X and current input symbol a.



Model of a table-driven predictive parser

- The behavior of the parser can be described in terms of its configurations, which give the stack contents and the remaining input.
- The next algorithm describes how configurations are manipulated.

Nonrecursive Predictive Parsing — *continued*

Nonrecursive predictive parsing.

INPUT. A string w and a parsing table M for grammar G.

OUTPUT.
- If w is in $L(G)$, a leftmost derivation of w;
- otherwise, an error indication.

METHOD.
- Initially, the parser is in a configuration in which it has $\$S$ on the stack with S, the start symbol of G on top, and w in the input buffer.
- The program that utilizes the predictive parsing table M to produce a parse for the input is shown here.

Nonrecursive Predictive Parsing — *continued*

set ip to point to the first symbol of w;
set X to the top stack symbol;
while ($X \neq \$$) { /* stack is not empty */
    if (X is a) pop the stack and advance ip;
    else if (X is a terminal) *error*();
    else if ($M[X, a]$ is an error entry) *error*();
    else if ($M[X, a] = X \rightarrow Y_1 Y_2 \cdots Y_k$) {
        output the production $X \rightarrow Y_1 Y_2 \cdots Y_k$;
        pop the stack;
        push $Y_k, Y_{k-1}, \ldots, Y_1$ onto the stack, with Y_1 on top;
    }
    set X to the top stack symbol;
}

Predictive parsing algorithm

Example

- We consider grammar:

$$
\begin{aligned}
E &\rightarrow TE' \\
E' &\rightarrow +TE' \mid \epsilon \\
T &\rightarrow FT' \\
T' &\rightarrow *FT' \mid \epsilon \\
F &\rightarrow (E) \mid \textbf{id}
\end{aligned}
$$

- We have already seen its parsing table.

NON-TERMINAL	INPUT SYMBOL					
	id	+	*	(	)	$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow \textbf{id}$			$F \rightarrow (E)$		

- On input **id** + **id** * **id**, the nonrecursive predictive parser algorithm makes the sequence of moves,

MATCHED	STACK	INPUT	ACTION
	$E\$$	**id** + **id** * **id**$	
	$TE'\$$	**id** + **id** * **id**$	output $E \to TE'$
	$FT'E'\$$	**id** + **id** * **id**$	output $T \to FT'$
	id $T'E'\$$	**id** + **id** * **id**$	output $F \to$ **id**
id	$T'E'\$$	+ **id** * **id**$	match **id**
id	$E'\$$	+ **id** * **id**$	output $T' \to \epsilon$
id	$+ TE'\$$	+ **id** * **id**$	output $E' \to + TE'$
id +	$TE'\$$	**id** * **id**$	match +
id +	$FT'E'\$$	**id** * **id**$	output $T \to FT'$
id +	**id** $T'E'\$$	**id** * **id**$	output $F \to$ **id**
id + **id**	$T'E'\$$	* **id**$	match **id**
id + **id**	$* FT'E'\$$	* **id**$	output $T' \to * FT'$
id + **id** *	$FT'E'\$$	**id**$	match *
id + **id** *	**id** $T'E'\$$	**id**$	output $F \to$ **id**
id + **id** * **id**	$T'E'\$$	$	match **id**
id + **id** * **id**	$E'\$$	$	output $T' \to \epsilon$
id + **id** * **id**	$	$	output $E' \to \epsilon$

Moves made by a predictive parser on input **id** + **id** * **id**

- These moves correspond to a leftmost derivation,

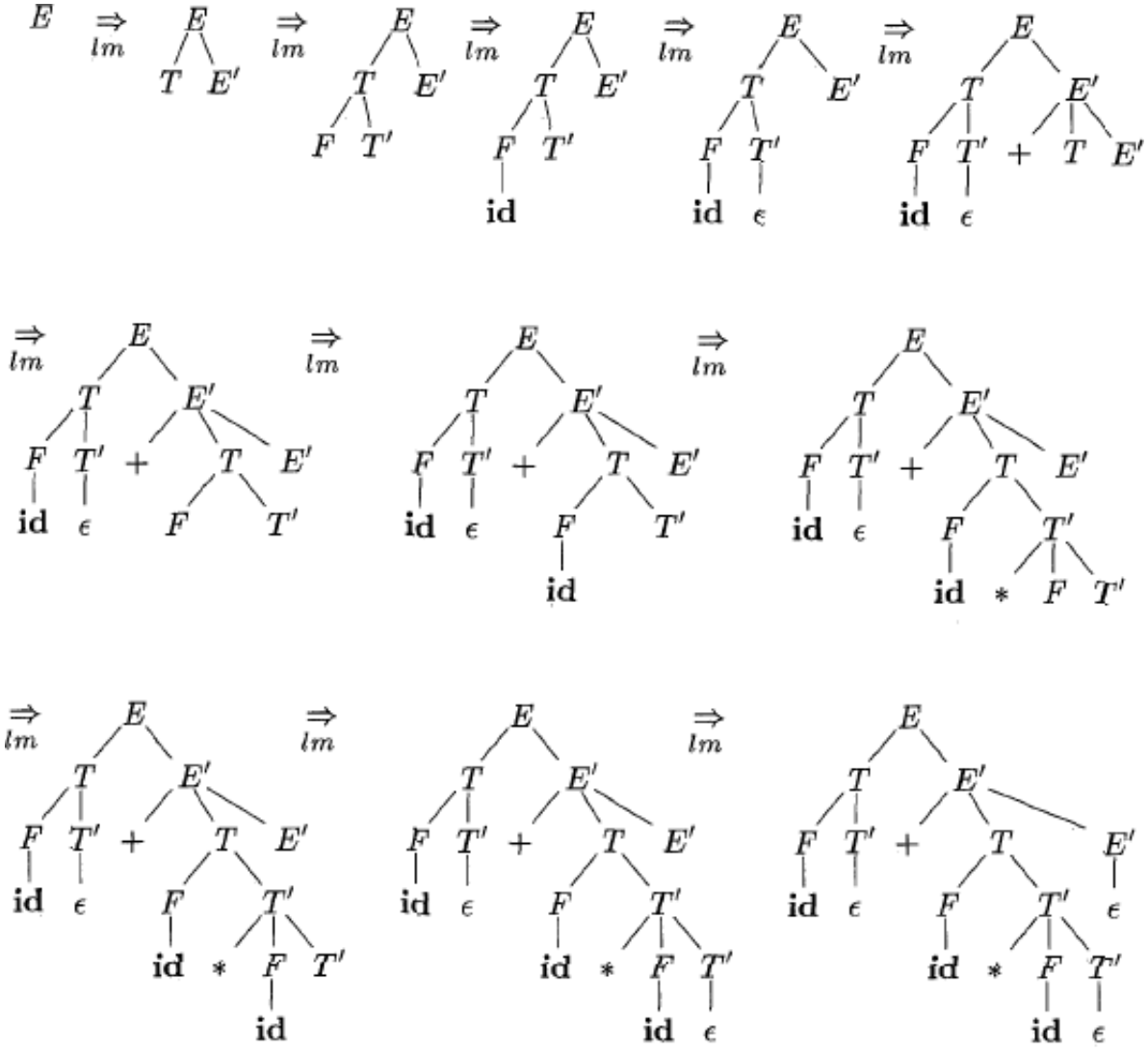
$$E \underset{lm}{\Longrightarrow} TE' \underset{lm}{\Longrightarrow} FT'E' \underset{lm}{\Longrightarrow} \mathbf{id}T'E' \underset{lm}{\Longrightarrow} \mathbf{id}E' \underset{lm}{\Longrightarrow} \mathbf{id}+TE' \underset{lm}{\Longrightarrow} \cdots$$

MATCHED	STACK	INPUT	ACTION
	$E\$$	$\mathbf{id} + \mathbf{id} * \mathbf{id}\$$	
	$TE'\$$	$\mathbf{id} + \mathbf{id} * \mathbf{id}\$$	output $E \to TE'$
	$FT'E'\$$	$\mathbf{id} + \mathbf{id} * \mathbf{id}\$$	output $T \to FT'$
	$\mathbf{id}\ T'E'\$$	$\mathbf{id} + \mathbf{id} * \mathbf{id}\$$	output $F \to \mathbf{id}$
$\mathbf{id}$	$T'E'\$$	$+\ \mathbf{id} * \mathbf{id}\$$	match $\mathbf{id}$
$\mathbf{id}$	$E'\$$	$+\ \mathbf{id} * \mathbf{id}\$$	output $T' \to \epsilon$
$\mathbf{id}$	$+\ TE'\$$	$+\ \mathbf{id} * \mathbf{id}\$$	output $E' \to +\ TE'$
$\mathbf{id} +$	$TE'\$$	$\mathbf{id} * \mathbf{id}\$$	match $+$
$\mathbf{id} +$	$FT'E'\$$	$\mathbf{id} * \mathbf{id}\$$	output $T \to FT'$
$\mathbf{id} +$	$\mathbf{id}\ T'E'\$$	$\mathbf{id} * \mathbf{id}\$$	output $F \to \mathbf{id}$
$\mathbf{id} + \mathbf{id}$	$T'E'\$$	$* \mathbf{id}\$$	match $\mathbf{id}$
$\mathbf{id} + \mathbf{id}$	$* FT'E'\$$	$* \mathbf{id}\$$	output $T' \to * FT'$
$\mathbf{id} + \mathbf{id} *$	$FT'E'\$$	$\mathbf{id}\$$	match $*$
$\mathbf{id} + \mathbf{id} *$	$\mathbf{id}\ T'E'\$$	$\mathbf{id}\$$	output $F \to \mathbf{id}$
$\mathbf{id} + \mathbf{id} * \mathbf{id}$	$T'E'\$$	$\$$	match $\mathbf{id}$
$\mathbf{id} + \mathbf{id} * \mathbf{id}$	$E'\$$	$\$$	output $T' \to \epsilon$
$\mathbf{id} + \mathbf{id} * \mathbf{id}$	$\$$	$\$$	output $E' \to \epsilon$

Moves made by a predictive parser on input $\mathbf{id} + \mathbf{id} * \mathbf{id}$

- These moves correspond to a leftmost derivation,

$$E \underset{lm}{\Longrightarrow} TE' \underset{lm}{\Longrightarrow} FT'E' \underset{lm}{\Longrightarrow} \mathbf{id}T'E' \underset{lm}{\Longrightarrow} \mathbf{id}E' \underset{lm}{\Longrightarrow} \mathbf{id}+TE' \underset{lm}{\Longrightarrow} \cdots$$



Top-down parse for $\mathbf{id} + \mathbf{id} * \mathbf{id}$

MATCHED	STACK	INPUT	ACTION
	$E\$$	$\mathbf{id} + \mathbf{id} * \mathbf{id}\$$	
	$TE'\$$	$\mathbf{id} + \mathbf{id} * \mathbf{id}\$$	output $E \to TE'$
	$FT'E'\$$	$\mathbf{id} + \mathbf{id} * \mathbf{id}\$$	output $T \to FT'$
	$\mathbf{id}\ T'E'\$$	$\mathbf{id} + \mathbf{id} * \mathbf{id}\$$	output $F \to \mathbf{id}$
$\mathbf{id}$	$T'E'\$$	$+\ \mathbf{id} * \mathbf{id}\$$	match $\mathbf{id}$
$\mathbf{id}$	$E'\$$	$+\ \mathbf{id} * \mathbf{id}\$$	output $T' \to \epsilon$
$\mathbf{id}$	$+\ TE'\$$	$+\ \mathbf{id} * \mathbf{id}\$$	output $E' \to +\ TE'$
$\mathbf{id} +$	$TE'\$$	$\mathbf{id} * \mathbf{id}\$$	match $+$
$\mathbf{id} +$	$FT'E'\$$	$\mathbf{id} * \mathbf{id}\$$	output $T \to FT'$
$\mathbf{id} +$	$\mathbf{id}\ T'E'\$$	$\mathbf{id} * \mathbf{id}\$$	output $F \to \mathbf{id}$
$\mathbf{id} + \mathbf{id}$	$T'E'\$$	$* \mathbf{id}\$$	match $\mathbf{id}$
$\mathbf{id} + \mathbf{id}$	$* FT'E'\$$	$* \mathbf{id}\$$	output $T' \to * FT'$
$\mathbf{id} + \mathbf{id} *$	$FT'E'\$$	$\mathbf{id}\$$	match $*$
$\mathbf{id} + \mathbf{id} *$	$\mathbf{id}\ T'E'\$$	$\mathbf{id}\$$	output $F \to \mathbf{id}$
$\mathbf{id} + \mathbf{id} * \mathbf{id}$	$T'E'\$$	$\$$	match $\mathbf{id}$
$\mathbf{id} + \mathbf{id} * \mathbf{id}$	$E'\$$	$\$$	output $T' \to \epsilon$
$\mathbf{id} + \mathbf{id} * \mathbf{id}$	$\$$	$\$$	output $E' \to \epsilon$

Moves made by a predictive parser on input $\mathbf{id} + \mathbf{id} * \mathbf{id}$

- Note that the sentential forms in this derivation correspond to the input that has already been matched (in column MATCHED) followed by the stack contents.

MATCHED	STACK	INPUT	ACTION
	$E\$$	$\mathbf{id} + \mathbf{id} * \mathbf{id}\$$	
	$TE'\$$	$\mathbf{id} + \mathbf{id} * \mathbf{id}\$$	output $E \to TE'$
	$FT'E'\$$	$\mathbf{id} + \mathbf{id} * \mathbf{id}\$$	output $T \to FT'$
	$\mathbf{id}\ T'E'\$$	$\mathbf{id} + \mathbf{id} * \mathbf{id}\$$	output $F \to \mathbf{id}$
$\mathbf{id}$	$T'E'\$$	$+\ \mathbf{id} * \mathbf{id}\$$	match $\mathbf{id}$
$\mathbf{id}$	$E'\$$	$+\ \mathbf{id} * \mathbf{id}\$$	output $T' \to \epsilon$
$\mathbf{id}$	$+\ TE'\$$	$+\ \mathbf{id} * \mathbf{id}\$$	output $E' \to +\ TE'$
$\mathbf{id} +$	$TE'\$$	$\mathbf{id} * \mathbf{id}\$$	match $+$
$\mathbf{id} +$	$FT'E'\$$	$\mathbf{id} * \mathbf{id}\$$	output $T \to FT'$
$\mathbf{id} +$	$\mathbf{id}\ T'E'\$$	$\mathbf{id} * \mathbf{id}\$$	output $F \to \mathbf{id}$
$\mathbf{id} + \mathbf{id}$	$T'E'\$$	$* \mathbf{id}\$$	match $\mathbf{id}$
$\mathbf{id} + \mathbf{id}$	$* FT'E'\$$	$* \mathbf{id}\$$	output $T' \to * FT'$
$\mathbf{id} + \mathbf{id} *$	$FT'E'\$$	$\mathbf{id}\$$	match $*$
$\mathbf{id} + \mathbf{id} *$	$\mathbf{id}\ T'E'\$$	$\mathbf{id}\$$	output $F \to \mathbf{id}$
$\mathbf{id} + \mathbf{id} * \mathbf{id}$	$T'E'\$$	$\$$	match $\mathbf{id}$
$\mathbf{id} + \mathbf{id} * \mathbf{id}$	$E'\$$	$\$$	output $T' \to \epsilon$
$\mathbf{id} + \mathbf{id} * \mathbf{id}$	$\$$	$\$$	output $E' \to \epsilon$

Moves made by a predictive parser on input $\mathbf{id} + \mathbf{id} * \mathbf{id}$

- The matched input is shown only to highlight the correspondence.

MATCHED	STACK	INPUT	ACTION
	$E\$$	$\mathbf{id} + \mathbf{id} * \mathbf{id}\$$	
	$TE'\$$	$\mathbf{id} + \mathbf{id} * \mathbf{id}\$$	output $E \to TE'$
	$FT'E'\$$	$\mathbf{id} + \mathbf{id} * \mathbf{id}\$$	output $T \to FT'$
	$\mathbf{id}\ T'E'\$$	$\mathbf{id} + \mathbf{id} * \mathbf{id}\$$	output $F \to \mathbf{id}$
$\mathbf{id}$	$T'E'\$$	$+\ \mathbf{id} * \mathbf{id}\$$	match $\mathbf{id}$
$\mathbf{id}$	$E'\$$	$+\ \mathbf{id} * \mathbf{id}\$$	output $T' \to \epsilon$
$\mathbf{id}$	$+\ TE'\$$	$+\ \mathbf{id} * \mathbf{id}\$$	output $E' \to +\ TE'$
$\mathbf{id} +$	$TE'\$$	$\mathbf{id} * \mathbf{id}\$$	match $+$
$\mathbf{id} +$	$FT'E'\$$	$\mathbf{id} * \mathbf{id}\$$	output $T \to FT'$
$\mathbf{id} +$	$\mathbf{id}\ T'E'\$$	$\mathbf{id} * \mathbf{id}\$$	output $F \to \mathbf{id}$
$\mathbf{id} + \mathbf{id}$	$T'E'\$$	$* \mathbf{id}\$$	match $\mathbf{id}$
$\mathbf{id} + \mathbf{id}$	$* FT'E'\$$	$* \mathbf{id}\$$	output $T' \to * FT'$
$\mathbf{id} + \mathbf{id} *$	$FT'E'\$$	$\mathbf{id}\$$	match $*$
$\mathbf{id} + \mathbf{id} *$	$\mathbf{id}\ T'E'\$$	$\mathbf{id}\$$	output $F \to \mathbf{id}$
$\mathbf{id} + \mathbf{id} * \mathbf{id}$	$T'E'\$$	$\$$	match $\mathbf{id}$
$\mathbf{id} + \mathbf{id} * \mathbf{id}$	$E'\$$	$\$$	output $T' \to \epsilon$
$\mathbf{id} + \mathbf{id} * \mathbf{id}$	$\$$	$\$$	output $E' \to \epsilon$

Moves made by a predictive parser on input $\mathbf{id} + \mathbf{id} * \mathbf{id}$

- For the same reason, the top of the stack is to the left.
- When we consider bottom-up parsing, it will be more natural to show the top of the stack to the right.

MATCHED	STACK	INPUT	ACTION
	$E\$$	$\mathbf{id} + \mathbf{id} * \mathbf{id}\$$	
	$TE'\$$	$\mathbf{id} + \mathbf{id} * \mathbf{id}\$$	output $E \to TE'$
	$FT'E'\$$	$\mathbf{id} + \mathbf{id} * \mathbf{id}\$$	output $T \to FT'$
	$\mathbf{id}\ T'E'\$$	$\mathbf{id} + \mathbf{id} * \mathbf{id}\$$	output $F \to \mathbf{id}$
$\mathbf{id}$	$T'E'\$$	$+\ \mathbf{id} * \mathbf{id}\$$	match $\mathbf{id}$
$\mathbf{id}$	$E'\$$	$+\ \mathbf{id} * \mathbf{id}\$$	output $T' \to \epsilon$
$\mathbf{id}$	$+\ TE'\$$	$+\ \mathbf{id} * \mathbf{id}\$$	output $E' \to +\ TE'$
$\mathbf{id} +$	$TE'\$$	$\mathbf{id} * \mathbf{id}\$$	match $+$
$\mathbf{id} +$	$FT'E'\$$	$\mathbf{id} * \mathbf{id}\$$	output $T \to FT'$
$\mathbf{id} +$	$\mathbf{id}\ T'E'\$$	$\mathbf{id} * \mathbf{id}\$$	output $F \to \mathbf{id}$
$\mathbf{id} + \mathbf{id}$	$T'E'\$$	$* \mathbf{id}\$$	match $\mathbf{id}$
$\mathbf{id} + \mathbf{id}$	$* FT'E'\$$	$* \mathbf{id}\$$	output $T' \to * FT'$
$\mathbf{id} + \mathbf{id} *$	$FT'E'\$$	$\mathbf{id}\$$	match $*$
$\mathbf{id} + \mathbf{id} *$	$\mathbf{id}\ T'E'\$$	$\mathbf{id}\$$	output $F \to \mathbf{id}$
$\mathbf{id} + \mathbf{id} * \mathbf{id}$	$T'E'\$$	$\$$	match $\mathbf{id}$
$\mathbf{id} + \mathbf{id} * \mathbf{id}$	$E'\$$	$\$$	output $T' \to \epsilon$
$\mathbf{id} + \mathbf{id} * \mathbf{id}$	$\$$	$\$$	output $E' \to \epsilon$

Moves made by a predictive parser on input $\mathbf{id} + \mathbf{id} * \mathbf{id}$

- The input pointer points to the leftmost symbol of the string in the INPUT column.

Example — *continued*

NON - TERMINAL	INPUT SYMBOL					
	id	+	*	(	)	$
E	$E \to TE'$			$E \to TE'$		
E'		$E' \to +TE'$			$E' \to \epsilon$	$E' \to \epsilon$
T	$T \to FT'$			$T \to FT'$		
T'		$T' \to \epsilon$	$T' \to *FT'$		$T' \to \epsilon$	$T' \to \epsilon$
F	$F \to \mathbf{id}$			$F \to (E)$		

STACK	INPUT	ACTION
$E\$$	**id** $+$ **id** $*$ **id**$\$$	
$TE'\$$	**id** $+$ **id** $*$ **id**$\$$	output $E \to TE'$

Example — *continued*

NON - TERMINAL	INPUT SYMBOL					
	id	+	*	(	)	$
E	$E \to TE'$			$E \to TE'$		
E'		$E' \to +TE'$			$E' \to \epsilon$	$E' \to \epsilon$
T	$T \to FT'$			$T \to FT'$		
T'		$T' \to \epsilon$	$T' \to *FT'$		$T' \to \epsilon$	$T' \to \epsilon$
F	$F \to$ **id**			$F \to (E)$		

STACK	INPUT	ACTION
$TE'\$$	**id** $+$ **id** $*$ **id**$\$$	
$FT'E'\$$	**id** $+$ **id** $*$ **id**$\$$	output $T \to FT'$

Example — *continued*

NON - TERMINAL	INPUT SYMBOL					
	id	+	*	(	)	$
E	$E \to TE'$			$E \to TE'$		
E'		$E' \to +TE'$			$E' \to \epsilon$	$E' \to \epsilon$
T	$T \to FT'$			$T \to FT'$		
T'		$T' \to \epsilon$	$T' \to *FT'$		$T' \to \epsilon$	$T' \to \epsilon$
F	$F \to$ **id**			$F \to (E)$		

STACK	INPUT	ACTION
$FT'E'\$$	**id** $+$ **id** $*$ **id**$\$$	
id$T'E'\$$	**id** $+$ **id** $*$ **id**$\$$	output $F \to$ **id**

Example — *continued*

STACK	INPUT	ACTION
id$T'E'\$$	**id** $+$ **id** $*$ **id**$	match **id**

Both are terminals and match. So, popped from the stack and input pointer advanced

$T'E'\$$	$+$**id** $*$ **id**$	

Example — *continued*

NON - TERMINAL	INPUT SYMBOL					
	id	$+$	$*$	$($	$)$	$\$$
E	$E \to TE'$			$E \to TE'$		
E'		$E' \to +TE'$			$E' \to \epsilon$	$E' \to \epsilon$
T	$T \to FT'$			$T \to FT'$		
T'		$T' \to \epsilon$	$T' \to *FT'$		$T' \to \epsilon$	$T' \to \epsilon$
F	$F \to$ **id**			$F \to (E)$		

STACK	INPUT	ACTION
$T'E'\$$	$+$**id** $*$ **id**$\$$	
$E'\$$	$+$**id** $*$ **id**$\$$	output $T' \to \epsilon$

Example — *continued*



Example — *continued*

NON - TERMINAL	INPUT SYMBOL					
	id	+	*	(	)	$
E	$E \to TE'$			$E \to TE'$		
E'		$E' \to +TE'$			$E' \to \epsilon$	$E' \to \epsilon$
T	$T \to FT'$			$T \to FT'$		
T'		$T' \to \epsilon$	$T' \to *FT'$		$T' \to \epsilon$	$T' \to \epsilon$
F	$F \to \mathbf{id}$			$F \to (E)$		

STACK	INPUT	ACTION
$E'\$$	$\$$	
↑	↑	
$\$$	$\$$	output $E' \to \epsilon$

Example — *continued*

STACK	INPUT	ACTION
$ ↑	$ ↑	

Both are $, the parser halts and announces successful completion of parsing.

MATCHED	STACK	INPUT	ACTION
	$E\$$	$\mathbf{id} + \mathbf{id} * \mathbf{id}\$$	
	$TE'\$$	$\mathbf{id} + \mathbf{id} * \mathbf{id}\$$	output $E \to TE'$
	$FT'E'\$$	$\mathbf{id} + \mathbf{id} * \mathbf{id}\$$	output $T \to FT'$
	$\mathbf{id}\ T'E'\$$	$\mathbf{id} + \mathbf{id} * \mathbf{id}\$$	output $F \to \mathbf{id}$
$\mathbf{id}$	$T'E'\$$	$+\ \mathbf{id} * \mathbf{id}\$$	match $\mathbf{id}$
$\mathbf{id}$	$E'\$$	$+\ \mathbf{id} * \mathbf{id}\$$	output $T' \to \epsilon$
$\mathbf{id}$	$+\ TE'\$$	$+\ \mathbf{id} * \mathbf{id}\$$	output $E' \to +\ TE'$
$\mathbf{id} +$	$TE'\$$	$\mathbf{id} * \mathbf{id}\$$	match $+$
$\mathbf{id} +$	$FT'E'\$$	$\mathbf{id} * \mathbf{id}\$$	output $T \to FT'$
$\mathbf{id} +$	$\mathbf{id}\ T'E'\$$	$\mathbf{id} * \mathbf{id}\$$	output $F \to \mathbf{id}$
$\mathbf{id} + \mathbf{id}$	$T'E'\$$	$* \mathbf{id}\$$	match $\mathbf{id}$
$\mathbf{id} + \mathbf{id}$	$* FT'E'\$$	$* \mathbf{id}\$$	output $T' \to * FT'$
$\mathbf{id} + \mathbf{id} *$	$FT'E'\$$	$\mathbf{id}\$$	match $*$
$\mathbf{id} + \mathbf{id} *$	$\mathbf{id}\ T'E'\$$	$\mathbf{id}\$$	output $F \to \mathbf{id}$
$\mathbf{id} + \mathbf{id} * \mathbf{id}$	$T'E'\$$	$\$$	match $\mathbf{id}$
$\mathbf{id} + \mathbf{id} * \mathbf{id}$	$E'\$$	$\$$	output $T' \to \epsilon$
$\mathbf{id} + \mathbf{id} * \mathbf{id}$	$\$$	$\$$	output $E' \to \epsilon$

Moves made by a predictive parser on input $\mathbf{id} + \mathbf{id} * \mathbf{id}$

For a leftmost derivation the production rules in the ACTION column (outputs only) are to be used from top to bottom.

Example — *continued*

$$
\begin{aligned}
E &\xRightarrow[E \to TE']{} TE' \\
&\xRightarrow[T \to FT']{} FT'E' \\
&\xRightarrow[F \to \mathbf{id}]{} \mathbf{id}T'E' \\
&\xRightarrow[T' \to \epsilon]{} \mathbf{id}E' \\
&\xRightarrow[E' \to +TE']{} \mathbf{id} + TE' \\
&\xRightarrow[T \to FT']{} \mathbf{id} + FT'E' \\
&\xRightarrow[F \to \mathbf{id}]{} \mathbf{id} + \mathbf{id}T'E'
\end{aligned}
$$

JT	ACTION
$*$ **id**\$	
$*$ **id**\$	output $E \to TE'$
$*$ **id**\$	output $T \to FT'$
$*$ **id**\$	output $F \to$ **id**
$*$ **id**\$	match **id**
$*$ **id**\$	output $T' \to \epsilon$
$*$ **id**\$	output $E' \to + TE'$
$*$ **id**\$	match $+$
$*$ **id**\$	output $T \to FT'$
$*$ **id**\$	output $F \to$ **id**
$*$ **id**\$	match **id**
$*$ **id**\$	output $T' \to * FT'$
id\$	match $*$
id\$	output $F \to$ **id**
\$	match **id**
\$	output $T' \to \epsilon$
\$	output $E' \to \epsilon$

Example — *continued*

$$E \xRightarrow[E \to TE']{} \ldots\ldots$$

$$\xRightarrow[F \to \mathbf{id}]{} \mathbf{id} + \mathbf{id} T' E'$$

$$\xRightarrow[T' \to *FT']{} \mathbf{id} + \mathbf{id} * F T' E'$$

$$\xRightarrow[F \to \mathbf{id}]{} \mathbf{id} + \mathbf{id} * \mathbf{id} T' E'$$

$$\xRightarrow[T' \to \epsilon]{} \mathbf{id} + \mathbf{id} * \mathbf{id} E'$$

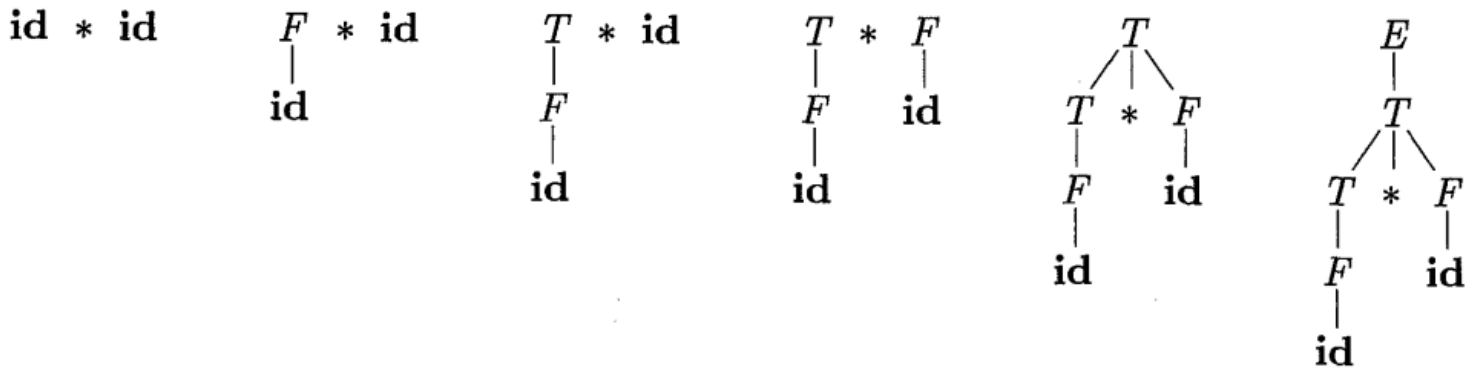
$$\xRightarrow[E' \to \epsilon]{} \mathbf{id} + \mathbf{id} * \mathbf{id}$$

JT	ACTION
$*\,\mathbf{id}\$$	
$*\,\mathbf{id}\$$	output $E \to TE'$
$*\,\mathbf{id}\$$	output $T \to FT'$
$*\,\mathbf{id}\$$	output $F \to \mathbf{id}$
$*\,\mathbf{id}\$$	match **id**
$*\,\mathbf{id}\$$	output $T' \to \epsilon$
$*\,\mathbf{id}\$$	output $E' \to +\,TE'$
$*\,\mathbf{id}\$$	match $+$
$*\,\mathbf{id}\$$	output $T \to FT'$
$*\,\mathbf{id}\$$	output $F \to \mathbf{id}$
$*\,\mathbf{id}\$$	match **id**
$*\,\mathbf{id}\$$	output $T' \to *\,FT'$
$\mathbf{id}\$$	match $*$
$\mathbf{id}\$$	output $F \to \mathbf{id}$
$\$$	match **id**
$\$$	output $T' \to \epsilon$
$\$$	output $E' \to \epsilon$

Bottom-Up Parsing

- A bottom-up parse corresponds to the construction of a parse tree for an input string,
  - beginning at the leaves (the bottom) and
  - working up towards the root (the top).
- It is convenient to describe parsing as the process of building parse trees.
- Although a front end may in fact carry out a translation directly without building an explicit tree.

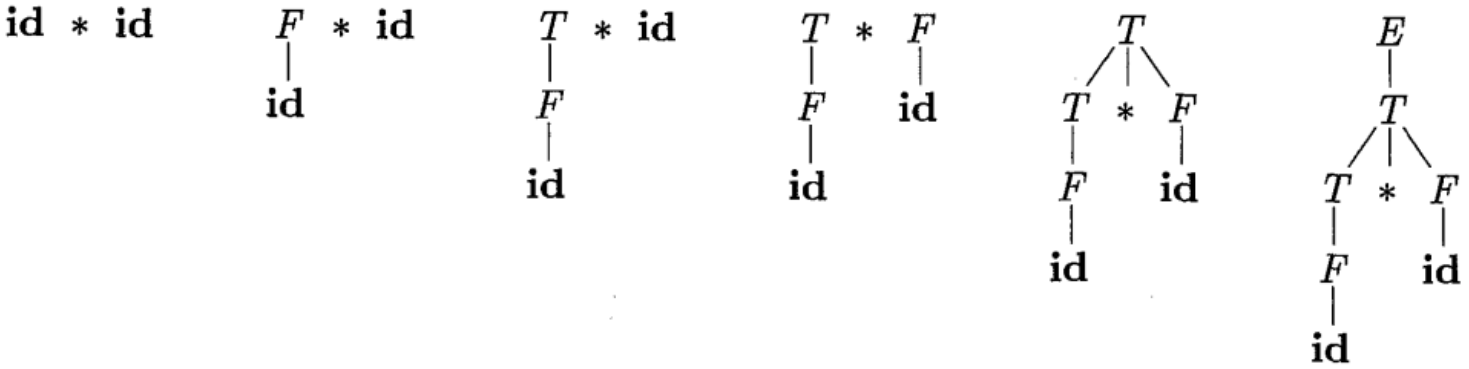
Bottom-Up Parsing — *continued*



A bottom-up parse for **id** * **id**

- The sequence of tree snapshots illustrates a bottom-up parse of the token stream **id** $*$ **id**, with respect to the expression grammar.

Bottom-Up Parsing — *continued*



A bottom-up parse for **id * id**

id $*$ **id**

$$
\begin{aligned}
E &\rightarrow E + T \mid T \\
T &\rightarrow T * F \mid F \\
F &\rightarrow (E) \mid \textbf{id}
\end{aligned}
$$

Reductions

- We can think of bottom-up parsing as the process of "reducing" a string w to the start symbol of the grammar.
- At each reduction step, a specific substring matching the body of a production is replaced by the nonterminal at the head of that production.
- The key decisions during bottom-up parsing are about,
  - when to reduce and
  - about what production to apply,

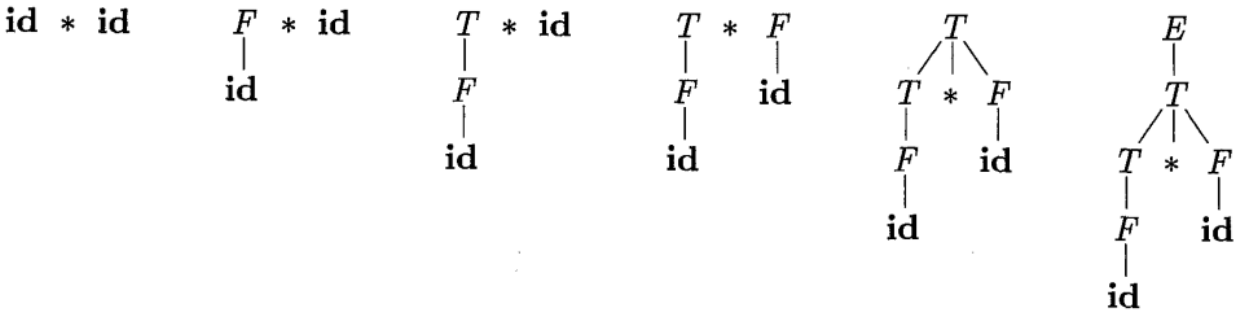
  as the parse proceeds.

Reductions

- We can think of bottom-up parsing as the process of "reducing" a string w to the start symbol of the grammar.
- At each reduction step, a specific substring matching the body of a production is replaced by the nonterminal at the head of that production.
- The key decisions during bottom-up parsing are about,
  - when to reduce and
  - about what production to apply,

  as the parse proceeds.

Example



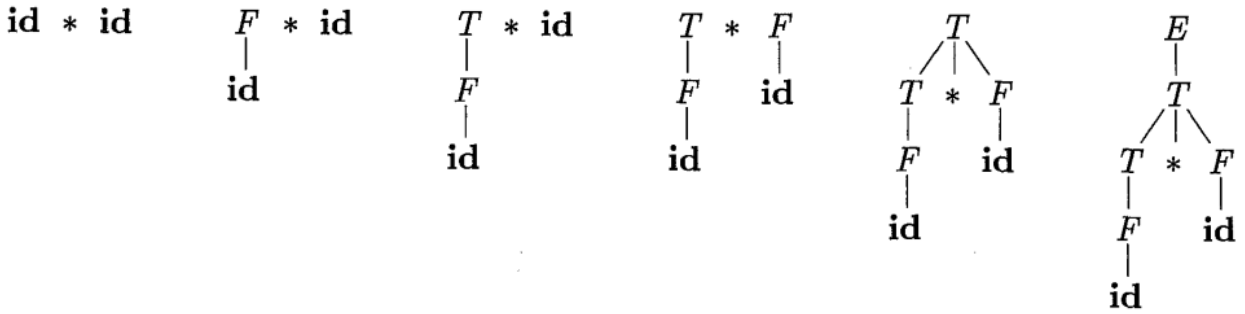
A bottom-up parse for **id * id**

$$E \to E + T \mid T$$
$$T \to T * F \mid F$$
$$F \to (E) \mid \mathbf{id}$$

$$E \Rightarrow T \Rightarrow T * F \Rightarrow T * \mathbf{id} \Rightarrow F * \mathbf{id} \Rightarrow \mathbf{id} * \mathbf{id}$$

- Illustrates a sequence of reductions.
- The grammar is the expression grammar.

Example



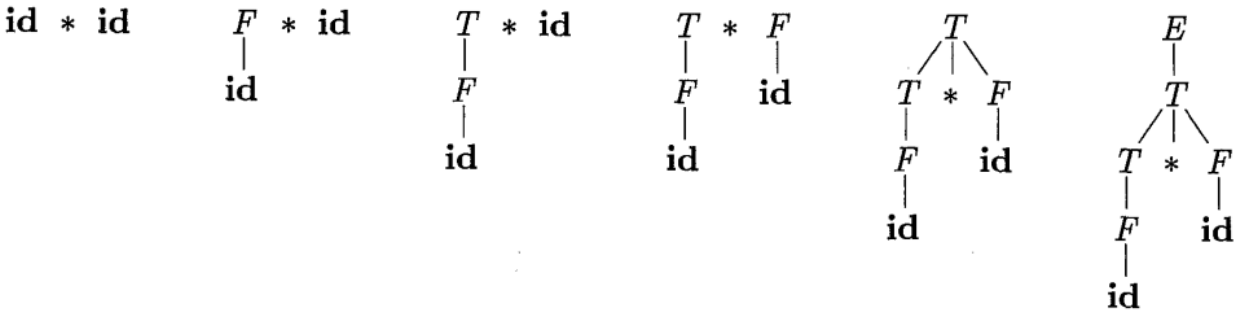
A bottom-up parse for **id** * **id**

$$E \to E + T \mid T$$
$$T \to T * F \mid F$$
$$F \to (E) \mid \mathbf{id}$$

$$E \Rightarrow T \Rightarrow T * F \Rightarrow T * \mathbf{id} \Rightarrow F * \mathbf{id} \Rightarrow \mathbf{id} * \mathbf{id}$$

- The reductions will be discussed in terms of the sequence of strings,
  - **id** $*$ **id**, $F *$ **id**, $T *$ **id**, $T * F$, T, E.

Example



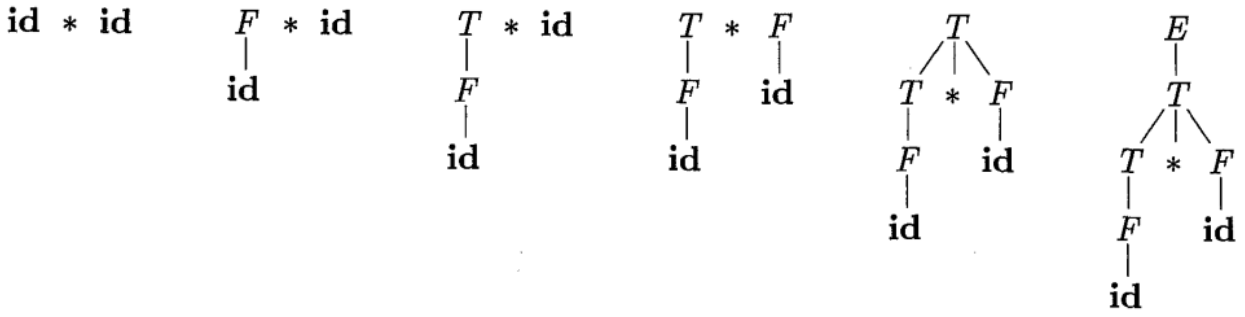
A bottom-up parse for **id** * **id**

$$E \to E + T \mid T$$
$$T \to T * F \mid F$$
$$F \to (E) \mid \mathbf{id}$$

$$E \Rightarrow T \Rightarrow T * F \Rightarrow T * \mathbf{id} \Rightarrow F * \mathbf{id} \Rightarrow \mathbf{id} * \mathbf{id}$$

- The strings in this sequence are formed from the roots of all the subtrees in the snapshots.
- The sequence starts with the input string **id** $*$ **id**.

Example



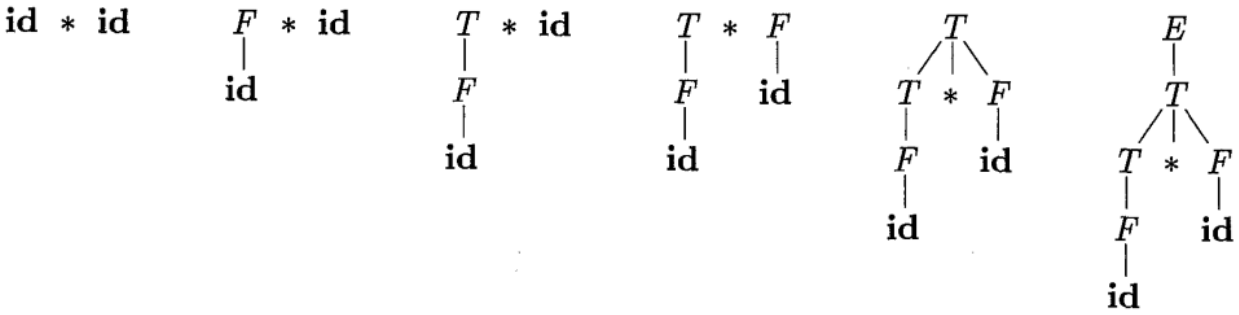
A bottom-up parse for **id** * **id**

$$E \to E + T \mid T$$
$$T \to T * F \mid F$$
$$F \to (E) \mid \mathbf{id}$$

$$E \Rightarrow T \Rightarrow T * F \Rightarrow T * \mathbf{id} \Rightarrow F * \mathbf{id} \Rightarrow \mathbf{id} * \mathbf{id}$$

- The first reduction produces $F * \mathbf{id}$ by reducing the leftmost **id** to F, using the production $F \to \mathbf{id}$.

Example



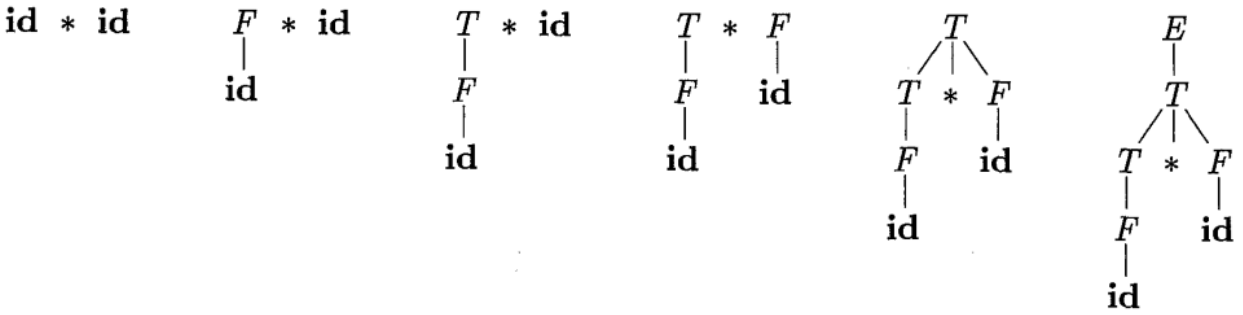
A bottom-up parse for **id * id**

$$E \to E + T \mid T$$
$$T \to T * F \mid F$$
$$F \to (E) \mid \mathbf{id}$$

$$E \Rightarrow T \Rightarrow T * F \Rightarrow T * \mathbf{id} \Rightarrow F * \mathbf{id} \Rightarrow \mathbf{id} * \mathbf{id}$$

- The second reduction produces $T * \mathbf{id}$ by reducing F to T.

Example



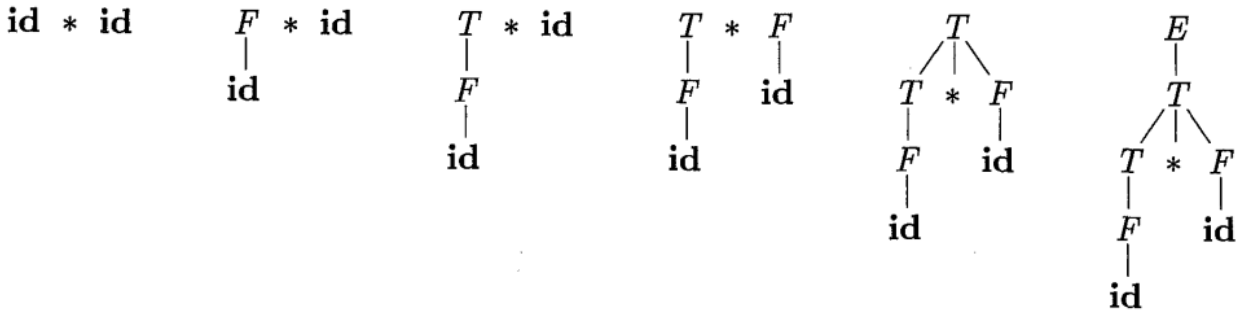
A bottom-up parse for **id** * **id**

$$E \to E + T \mid T$$
$$T \to T * F \mid F$$
$$F \to (E) \mid \mathbf{id}$$

$$E \Rightarrow T \Rightarrow T * F \Rightarrow T * \mathbf{id} \Rightarrow F * \mathbf{id} \Rightarrow \mathbf{id} * \mathbf{id}$$

- Now, we have a choice between,
  - reducing the string T, which is the body of $E \to T$, and
  - the string consisting of the second **id**, which is the body of $F \to \mathbf{id}$.

Example



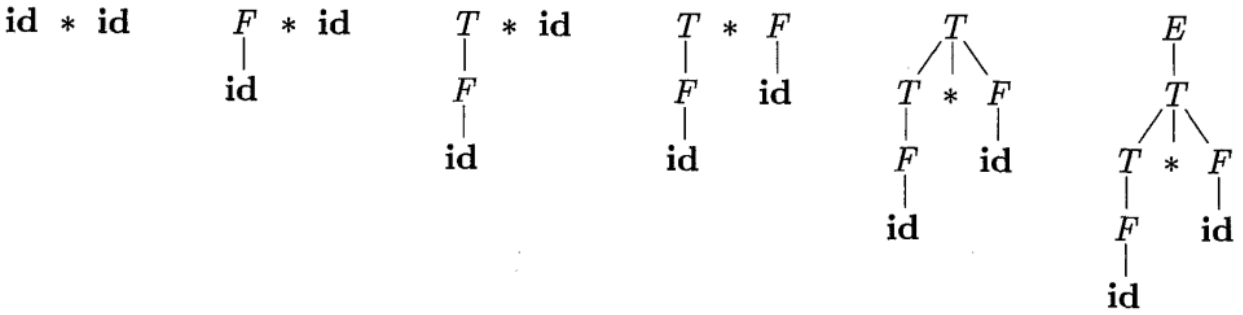
A bottom-up parse for **id** * **id**

$$E \to E + T \mid T$$
$$T \to T * F \mid F$$
$$F \to (E) \mid \mathbf{id}$$

$$E \Rightarrow T \Rightarrow T * F \Rightarrow T * \mathbf{id} \Rightarrow F * \mathbf{id} \Rightarrow \mathbf{id} * \mathbf{id}$$

- Rather than reduce T to E, the second **id** is reduced to F, resulting in the string $T * F$.

Example



A bottom-up parse for **id * id**

$$E \to E + T \mid T$$
$$T \to T * F \mid F$$
$$F \to (E) \mid \mathbf{id}$$

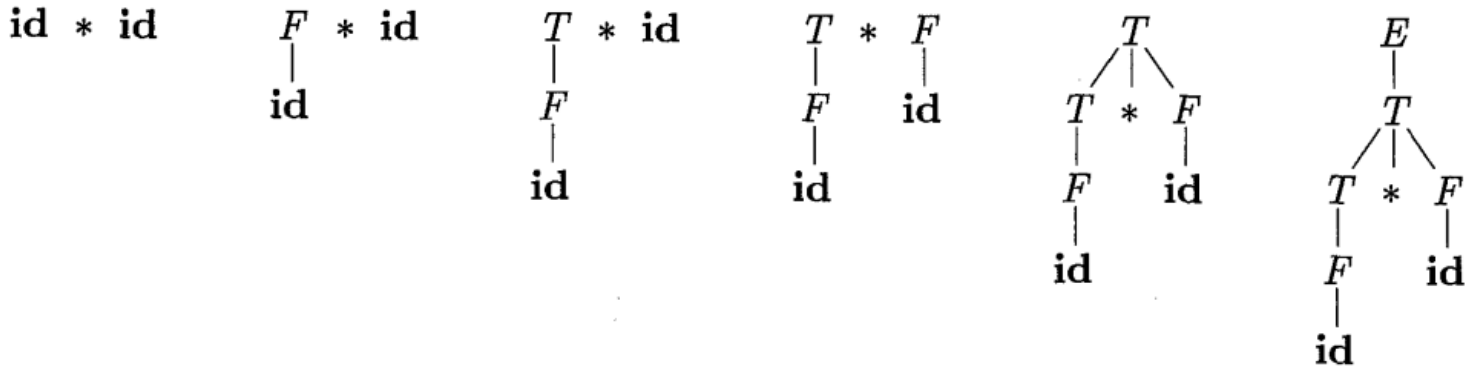
$$E \Rightarrow T \Rightarrow T * F \Rightarrow T * \mathbf{id} \Rightarrow F * \mathbf{id} \Rightarrow \mathbf{id} * \mathbf{id}$$

- This string then reduces to T.
- The parse completes with the reduction of T to the start symbol E.

Reductions — *continued*

- By definition, a reduction is the reverse of a step in a derivation.
- Recall that in a derivation, a nonterminal in a sentential form is replaced by the body of one of its productions.
- The goal of bottom-up parsing is therefore to construct a derivation in reverse.

Reductions — *continued*



A bottom-up parse for **id** * **id**

- The following derivation corresponds to the parse,

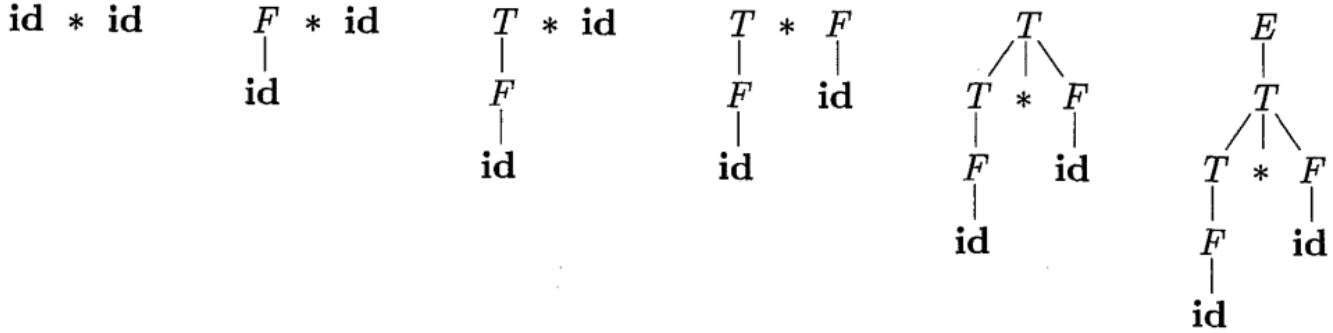
$$E \Rightarrow T \Rightarrow T * F \Rightarrow T * \mathbf{id} \Rightarrow F * \mathbf{id} \Rightarrow \mathbf{id} * \mathbf{id}.$$

- This derivation is in fact a *rightmost* derivation.

Handle Pruning

- Bottom-up parsing during a left-to-right scan of the input constructs a rightmost derivation in reverse.
- Informally, a “handle” is a substring that matches the body of a production.
- And whose reduction represents one step along the reverse of a rightmost derivation.

Handle Pruning — *continued*



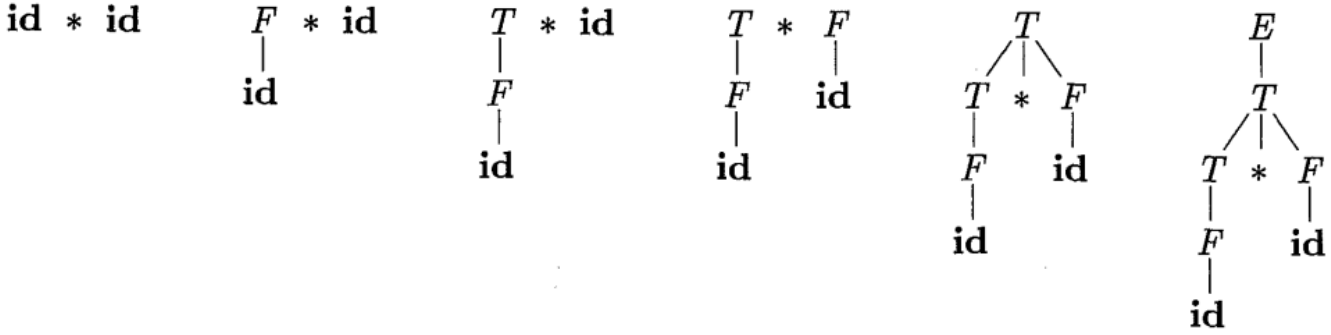
Right Sentential Form	Handle	Reducing Production
$\mathbf{id}_1 * \mathbf{id}_2$	$\mathbf{id}_1$	$F \to \mathbf{id}$
$F * \mathbf{id}_2$	F	$T \to F$
$T * \mathbf{id}_2$	$\mathbf{id}_2$	$F \to \mathbf{id}$
$T * F$	$T * F$	$E \to T * F$

Handles during a parse of $\mathbf{id}_1 * \mathbf{id}_2$

- Adding subscripts to the tokens **id** for clarity.
- The handles during the parse of $\mathbf{id}_1 * \mathbf{id}_2$ according to the expression grammar.

$$
\begin{aligned}
E &\to E + T \mid T \\
T &\to T * F \mid F \\
F &\to (E) \mid \mathbf{id}
\end{aligned}
$$

Handle Pruning — *continued*



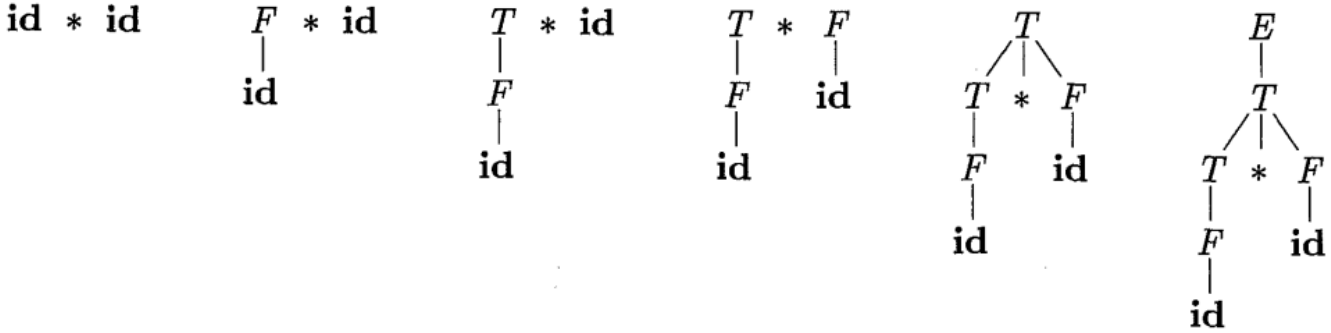
Right Sentential Form	Handle	Reducing Production
$\mathbf{id}_1 * \mathbf{id}_2$	$\mathbf{id}_1$	$F \to \mathbf{id}$
$F * \mathbf{id}_2$	F	$T \to F$
$T * \mathbf{id}_2$	$\mathbf{id}_2$	$F \to \mathbf{id}$
$T * F$	$T * F$	$E \to T * F$

Handles during a parse of $\mathbf{id}_1 * \mathbf{id}_2$

- Although T is the body of the production $E \to T$, the symbol T is not a handle in the sentential form $T * \mathbf{id}_2$.

$$
\begin{aligned}
E &\to E + T \mid T \\
T &\to T * F \mid F \\
F &\to (E) \mid \mathbf{id}
\end{aligned}
$$

Handle Pruning — *continued*



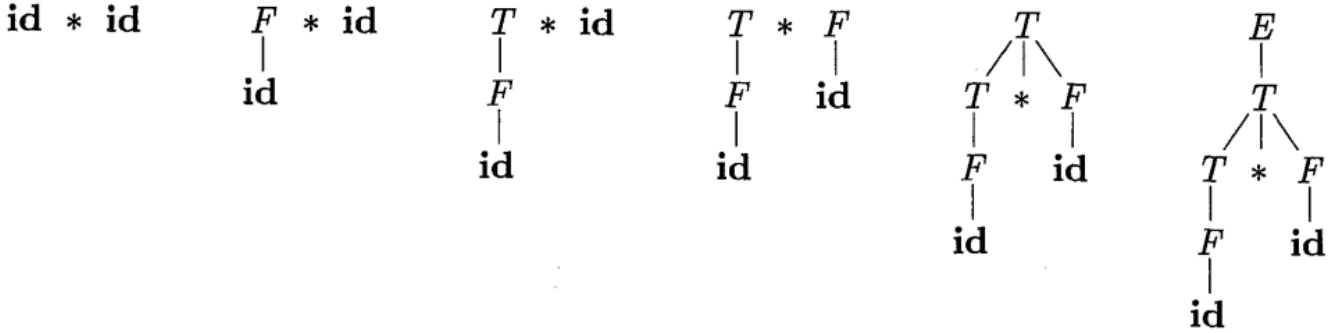
Right Sentential Form	Handle	Reducing Production
$\mathbf{id}_1 * \mathbf{id}_2$	$\mathbf{id}_1$	$F \to \mathbf{id}$
$F * \mathbf{id}_2$	F	$T \to F$
$T * \mathbf{id}_2$	$\mathbf{id}_2$	$F \to \mathbf{id}$
$T * F$	$T * F$	$E \to T * F$

Handles during a parse of $\mathbf{id}_1 * \mathbf{id}_2$

- If T were indeed replaced by E, we would get the string $E * \mathbf{id}_2$, which cannot be derived from the start symbol E.

$$
\begin{aligned}
E &\to E + T \mid T \\
T &\to T * F \mid F \\
F &\to (E) \mid \mathbf{id}
\end{aligned}
$$

Handle Pruning — *continued*



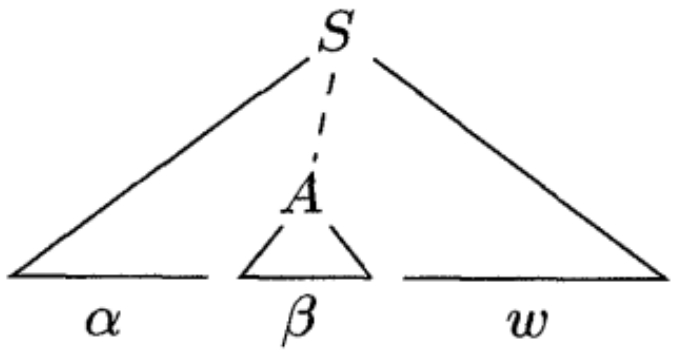
Right Sentential Form	Handle	Reducing Production
$\mathbf{id}_1 * \mathbf{id}_2$	$\mathbf{id}_1$	$F \to \mathbf{id}$
$F * \mathbf{id}_2$	F	$T \to F$
$T * \mathbf{id}_2$	$\mathbf{id}_2$	$F \to \mathbf{id}$
$T * F$	$T * F$	$E \to T * F$

Handles during a parse of $\mathbf{id}_1 * \mathbf{id}_2$

- Thus, the leftmost substring that matches the body of some production *need not* be a handle.

$$
\begin{aligned}
E &\to E + T \mid T \\
T &\to T * F \mid F \\
F &\to (E) \mid \mathbf{id}
\end{aligned}
$$

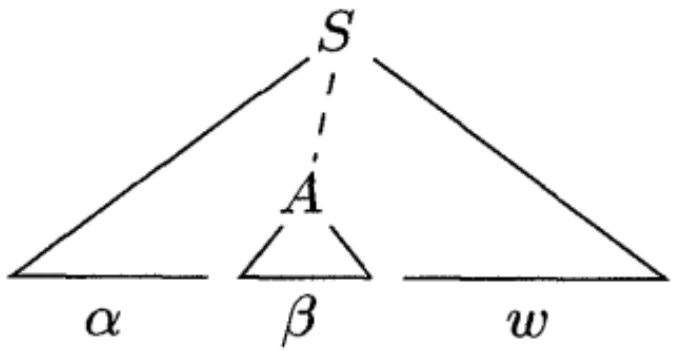
Handle Pruning — *continued*



A handle $A \to \beta$ in the parse tree for $\alpha\beta w$

- Formally, if $S \underset{rm}{\overset{*}{\Longrightarrow}} \alpha A w \underset{rm}{\Longrightarrow} \alpha\beta w$, then production $A \to \beta$, in the position following α is a handle of $\alpha\beta w$.

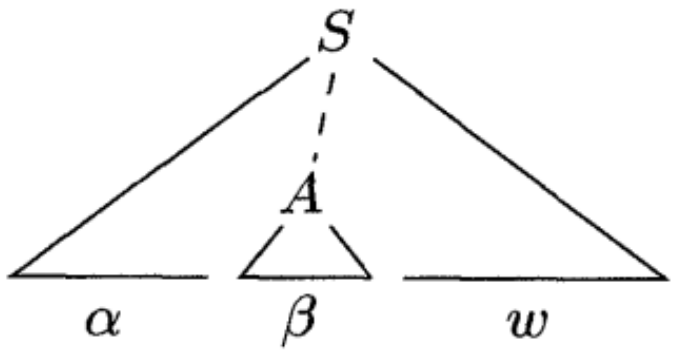
Handle Pruning — *continued*



A handle $A \to \beta$ in the parse tree for $\alpha\beta w$

- Alternatively, a handle of a right-sentential form γ is a production $A \to \beta$ and a position of γ where the string β may be found.
- Replacing β at that position by A produces the previous right-sentential form in a rightmost derivation of γ.

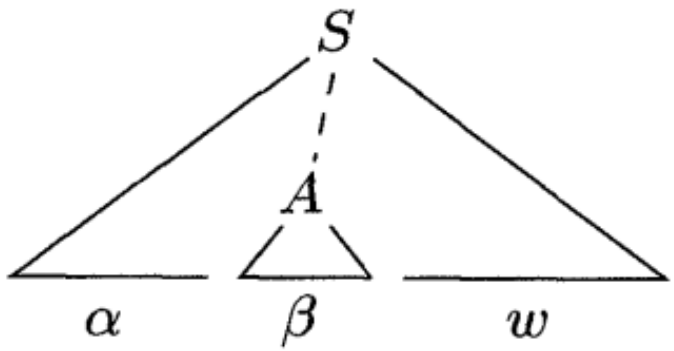
Handle Pruning — *continued*



A handle $A \to \beta$ in the parse tree for $\alpha\beta w$

- Notice that the string w to the right of the handle must contain only terminal symbols.
- For convenience, we refer to the body β rather than $A \to \beta$ as a handle.

Handle Pruning — *continued*



A handle $A \to \beta$ in the parse tree for $\alpha\beta w$

- Note we say “a handle” rather than “the handle,” because the grammar could be ambiguous, with more than one rightmost derivation of $\alpha\beta w$.
- If a grammar is unambiguous, then every right-sentential form of the grammar has exactly one handle.

Handle Pruning — *continued*

- A rightmost derivation in reverse can be obtained by "handle pruning."
- That is, we start with a string of terminals w to be parsed.
- If w is a sentence of the grammar at hand, then let $w = \gamma_n$, where γ is the nth right-sentential form of some as yet unknown rightmost derivation,

$$S = \gamma_0 \underset{rm}{\Longrightarrow} \gamma_1 \underset{rm}{\Longrightarrow} \gamma_2 \underset{rm}{\Longrightarrow} \cdots \gamma_{n-1} \underset{rm}{\Longrightarrow} \gamma_n = w.$$

Handle Pruning — *continued*

$$S = \gamma_0 \underset{rm}{\Longrightarrow} \gamma_1 \underset{rm}{\Longrightarrow} \gamma_2 \underset{rm}{\Longrightarrow} \cdots \gamma_{n-1} \underset{rm}{\Longrightarrow} \gamma_n = w$$

- To reconstruct this derivation in reverse order, we locate the handle β_n in γ_n.
- And replace β_n by the head of the relevant production $A_n \rightarrow \beta_n$ to obtain the previous right-sentential form γ_{n-1}.

Handle Pruning — *continued*

$$S = \gamma_0 \underset{rm}{\Longrightarrow} \gamma_1 \underset{rm}{\Longrightarrow} \gamma_2 \underset{rm}{\Longrightarrow} \cdots \gamma_{n-1} \underset{rm}{\Longrightarrow} \gamma_n = w$$

- We then repeat this process.
- That is, we locate the handle β_{n-1} in γ_{n-1} and reduce this handle to obtain the right-sentential form γ_{n-2}.
- By continuing this process we produce a right-sentential form consisting only of the start symbol S.
- Then we halt and announce successful completion of parsing.
- The reverse of the sequence of productions used in the reductions is a rightmost derivation for the input string.

Shift-Reduce Parsing

- Shift-reduce parsing is a form of bottom-up parsing in which a stack holds grammar symbols.
- And an input buffer holds the rest of the string to be parsed.
- The handle always appears at the top of the stack just before it is identified as the handle.

Shift-Reduce Parsing — *continued*

- We use $ to mark the bottom of the stack and also the right end of the input.
- Conventionally, when discussing bottom-up parsing, we show the top of the stack on the right, rather than on the left as we did for top-down parsing.
- Initially, the stack is empty, and the string w is on the input, as follows:

STACK	INPUT
$	$w$$

Shift-Reduce Parsing — *continued*

STACK	INPUT
$\$$	$w\$$

- During a left-to-right scan of the input string, the parser shifts zero or more input symbols onto the stack, until it is ready to reduce a string β of grammar symbols on top of the stack.
- It then reduces β to the head of the appropriate production.
- The parser repeats this cycle until it has detected an error or until the stack contains the start symbol and the input is empty.

STACK	INPUT
$\$S$	$\$$

Shift-Reduce Parsing — *continued*

STACK	INPUT
$\$S$	$\$$

- Upon entering this configuration, the parser halts and announces successful completion of parsing.

Shift-Reduce Parsing — *continued*

$$
\begin{aligned}
E &\rightarrow E + T \mid T \\
T &\rightarrow T * F \mid F \\
F &\rightarrow (E) \mid \mathbf{id}
\end{aligned}
$$

$$E \Rightarrow T \Rightarrow T * F \Rightarrow T * \mathbf{id} \Rightarrow F * \mathbf{id} \Rightarrow \mathbf{id} * \mathbf{id}$$

STACK	INPUT	ACTION
\$	$\mathbf{id}_1 * \mathbf{id}_2$ \$	shift
\$ $\mathbf{id}_1$	$* \mathbf{id}_2$ \$	reduce by $F \rightarrow \mathbf{id}$
\$ F	$* \mathbf{id}_2$ \$	reduce by $T \rightarrow F$
\$ T	$* \mathbf{id}_2$ \$	shift
\$ $T *$	$\mathbf{id}_2$ \$	shift
\$ $T * \mathbf{id}_2$	\$	reduce by $F \rightarrow \mathbf{id}$
\$ $T * F$	\$	reduce by $T \rightarrow T * F$
\$ T	\$	reduce by $E \rightarrow T$
\$ E	\$	accept

Configurations of a shift-reduce parser on input $\mathbf{id}_1 * \mathbf{id}_2$

Shift-Reduce Parsing — *continued*

- While the primary operations are shift and reduce, there are actually four possible actions a shift-reduce parser can make:

  Shift. Shift the next input symbol onto the top of the stack.

  Reduce. The right end of the string to be reduced must be at the top of the stack.
  - Locate the left end of the string within the stack.
  - And decide with what nonterminal to replace the string.

  Accept. Announce successful completion of parsing.

  Error. Discover a syntax error and call an error recovery routine.

Shift-Reduce Parsing — *continued*

- While the primary operations are shift and reduce, there are actually four possible actions a shift-reduce parser can make:

  Shift. Shift the next input symbol onto the top of the stack.

  Reduce. The right end of the string to be reduced must be at the top of the stack.
  - Locate the left end of the string within the stack.
  - And decide with what nonterminal to replace the string.

  Accept. Announce successful completion of parsing.

  Error. Discover a syntax error and call an error recovery routine.

Shift-Reduce Parsing — *continued*

- While the primary operations are shift and reduce, there are actually four possible actions a shift-reduce parser can make:

  Shift. Shift the next input symbol onto the top of the stack.

  Reduce. The right end of the string to be reduced must be at the top of the stack.
  - Locate the left end of the string within the stack.
  - And decide with what nonterminal to replace the string.

  Accept. Announce successful completion of parsing.

  Error. Discover a syntax error and call an error recovery routine.

Shift-Reduce Parsing — *continued*

- While the primary operations are shift and reduce, there are actually four possible actions a shift-reduce parser can make:

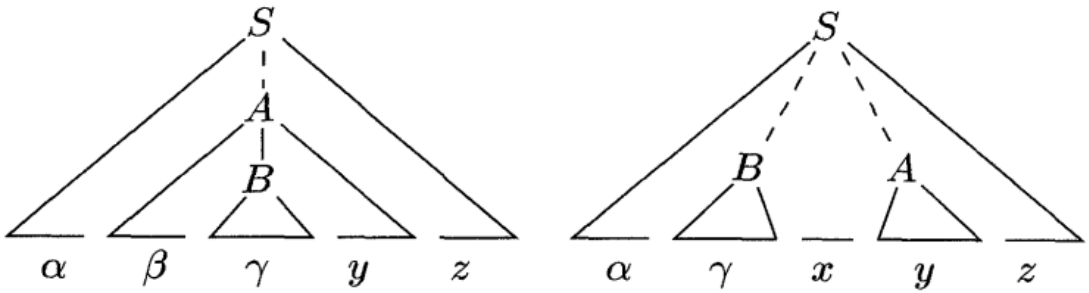
  Shift. Shift the next input symbol onto the top of the stack.

  Reduce. The right end of the string to be reduced must be at the top of the stack.
  - Locate the left end of the string within the stack.
  - And decide with what nonterminal to replace the string.

  Accept. Announce successful completion of parsing.

  Error. Discover a syntax error and call an error recovery routine.

Shift-Reduce Parsing — *continued*



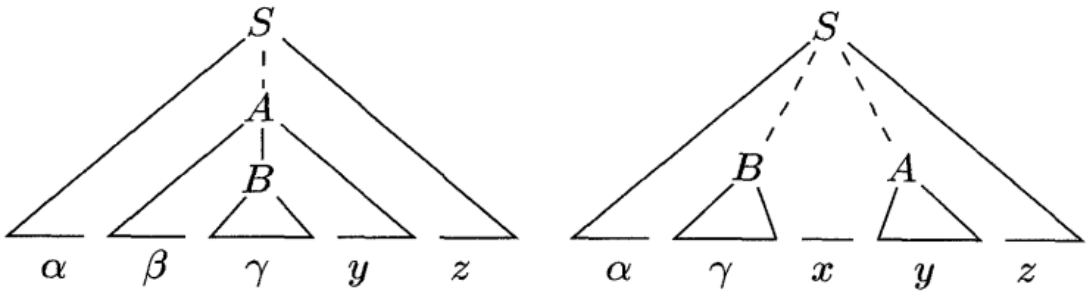
Cases for two successive steps of a rightmost derivation

$$
\begin{aligned}
S &\rightarrow \alpha Az \mid \alpha Bx Az \\
A &\rightarrow \beta By \mid y \\
B &\rightarrow \gamma
\end{aligned}
$$

(the productions are indicated in a bit loose manner)

- The use of a stack in shift-reduce parsing is justified by an important fact.
- The handle will always eventually appear on top of the stack, never inside.

Shift-Reduce Parsing — *continued*



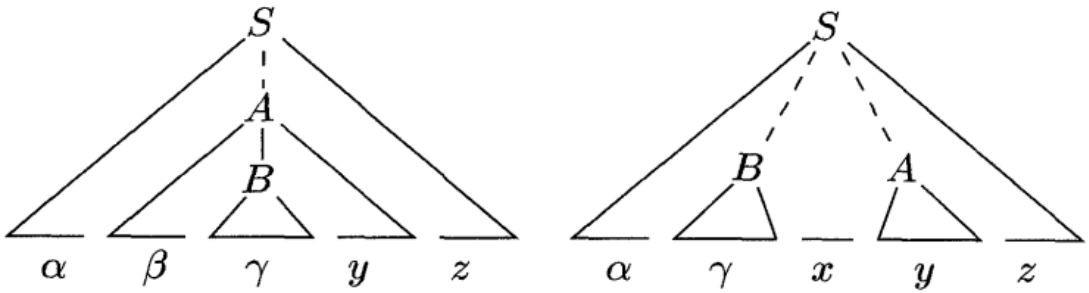
Cases for two successive steps of a rightmost derivation

$$
\begin{aligned}
S &\rightarrow \alpha Az \mid \alpha Bxz \\
A &\rightarrow \beta By \mid y \\
B &\rightarrow \gamma
\end{aligned}
$$

(the productions are indicated in a bit loose manner)

- This fact can be shown by considering the possible forms of two successive steps in any rightmost derivation.

Shift-Reduce Parsing — *continued*



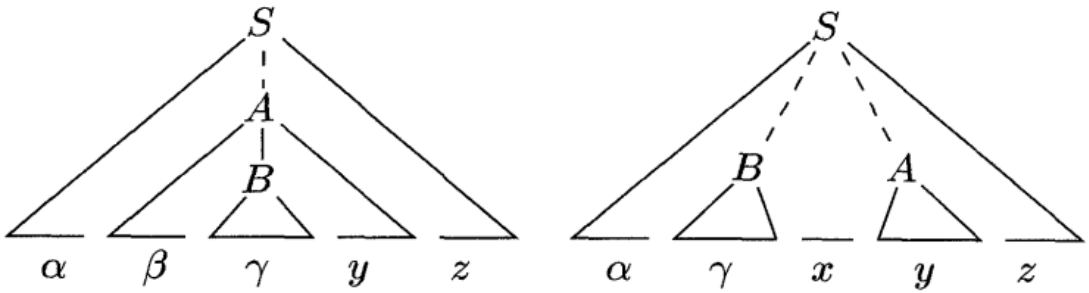
Cases for two successive steps of a rightmost derivation

$$S \rightarrow \alpha Az \mid \alpha Bxz$$
$$A \rightarrow \beta By \mid y$$
$$B \rightarrow \gamma$$

(the productions are indicated in a bit loose manner)

- In case (1), A is replaced by βBy.
- Then the rightmost nonterminal B in the body βBy is replaced by γ.

Shift-Reduce Parsing — *continued*



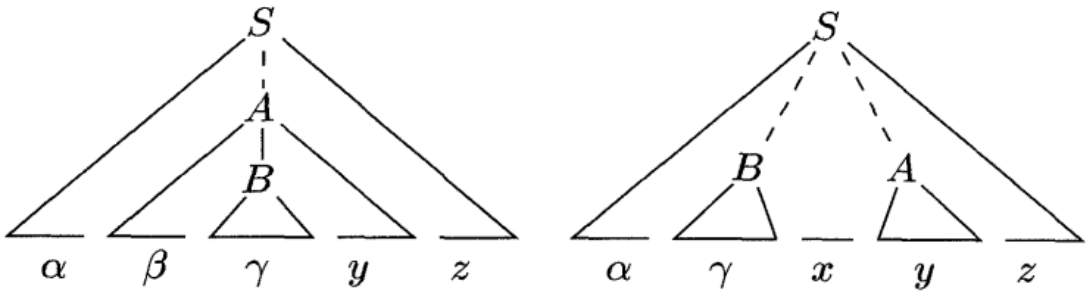
Cases for two successive steps of a rightmost derivation

$$S \rightarrow \alpha Az \mid \alpha Bxz$$
$$A \rightarrow \beta By \mid y$$
$$B \rightarrow \gamma$$

(the productions are indicated in a bit loose manner)

- In case (2), A is again expanded first, but this time the body is a string y of terminals only.
- The next rightmost nonterminal B will be somewhere to the left of y.

Shift-Reduce Parsing — *continued*



Cases for two successive steps of a rightmost derivation

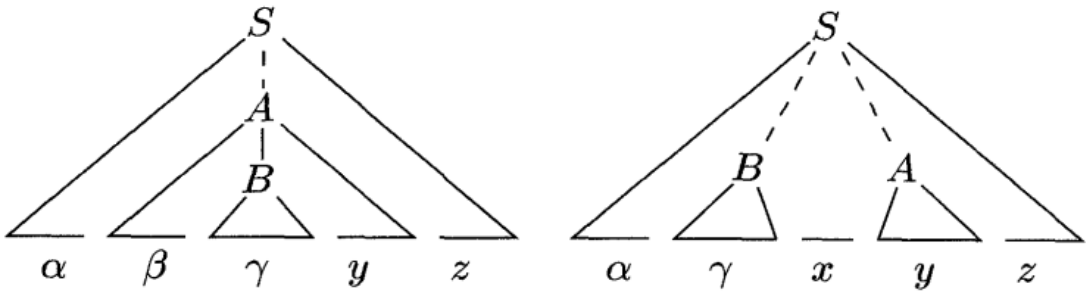
$$S \rightarrow \alpha Az \mid \alpha BxAz$$
$$A \rightarrow \beta By \mid y$$
$$B \rightarrow \gamma$$

(the productions are indicated in a bit loose manner)

- In other words:

(1) $S \underset{rm}{\overset{*}{\Longrightarrow}} \alpha Az \underset{rm}{\Longrightarrow} \alpha\beta Byz \underset{rm}{\Longrightarrow} \alpha\beta\gamma yz$

(2) $S \underset{rm}{\overset{*}{\Longrightarrow}} \alpha BxAz \underset{rm}{\Longrightarrow} \alpha Bxyz \underset{rm}{\Longrightarrow} \alpha\gamma xyz$



Cases for two successive steps of a rightmost derivation

$$S \to \alpha Az \mid \alpha BxAz$$
$$A \to \beta By \mid y$$
$$B \to \gamma$$

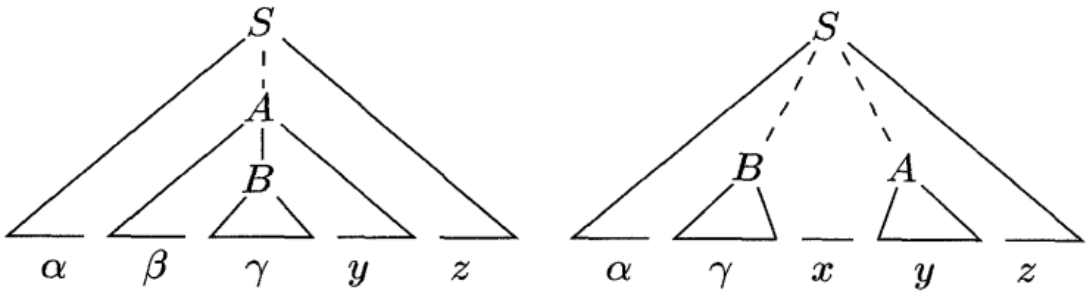
(the productions are indicated in a bit loose manner)

(1) $S \underset{rm}{\overset{*}{\Longrightarrow}} \alpha Az \underset{rm}{\Longrightarrow} \alpha\beta Byz \underset{rm}{\Longrightarrow} \alpha\beta\gamma yz$

(2) $S \underset{rm}{\overset{*}{\Longrightarrow}} \alpha BxAz \underset{rm}{\Longrightarrow} \alpha Bxyz \underset{rm}{\Longrightarrow} \alpha\gamma xyz$

- Consider case (1) in reverse, where a shift-reduce parser has just reached the configuration,

STACK	INPUT
$\$\alpha\beta\gamma$	$yz\$$



Cases for two successive steps of a rightmost derivation

$$S \to \alpha Az \mid \alpha BxAz$$
$$A \to \beta By \mid y$$
$$B \to \gamma$$

(the productions are indicated in a bit loose manner)

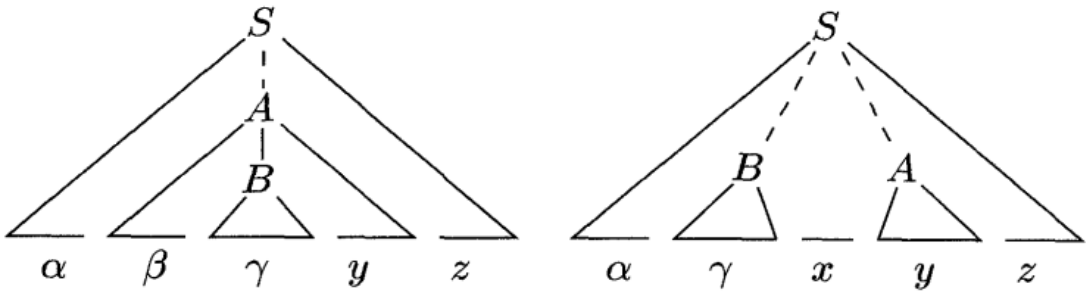
(1) $S \underset{rm}{\overset{*}{\Longrightarrow}} \alpha Az \underset{rm}{\Longrightarrow} \alpha\beta Byz \underset{rm}{\Longrightarrow} \alpha\beta\gamma yz$

(2) $S \underset{rm}{\overset{*}{\Longrightarrow}} \alpha BxAz \underset{rm}{\Longrightarrow} \alpha Bxyz \underset{rm}{\Longrightarrow} \alpha\gamma xyz$

STACK	INPUT
$\$\alpha\beta\gamma$	$yz\$$

- The parser reduces the handle γ to B to reach the configuration,

STACK	INPUT
$\$\alpha\beta B$	$yz\$$



Cases for two successive steps of a rightmost derivation

$$
\begin{aligned}
S &\rightarrow \alpha Az \mid \alpha BxAz \\
A &\rightarrow \beta By \mid y \\
B &\rightarrow \gamma
\end{aligned}
$$

(the productions are indicated in a bit loose manner)

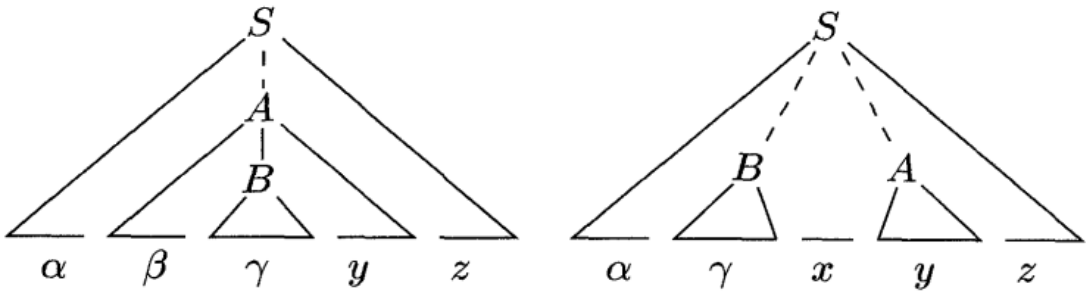
(1) $S \underset{rm}{\overset{*}{\Longrightarrow}} \alpha Az \underset{rm}{\Longrightarrow} \alpha\beta Byz \underset{rm}{\Longrightarrow} \alpha\beta\gamma yz$

(2) $S \underset{rm}{\overset{*}{\Longrightarrow}} \alpha BxAz \underset{rm}{\Longrightarrow} \alpha Bxyz \underset{rm}{\Longrightarrow} \alpha\gamma xyz$

STACK	INPUT
$\$\alpha\beta B$	$yz\$$

- The parser can now shift the string y onto the stack by a sequence of zero or more shift moves to reach the configuration,

STACK	INPUT
$\$\alpha\beta By$	$z\$$



Cases for two successive steps of a rightmost derivation

$$
\begin{aligned}
S &\rightarrow \alpha Az \mid \alpha BxAz \\
A &\rightarrow \beta By \mid y \\
B &\rightarrow \gamma
\end{aligned}
$$

(the productions are indicated in a bit loose manner)

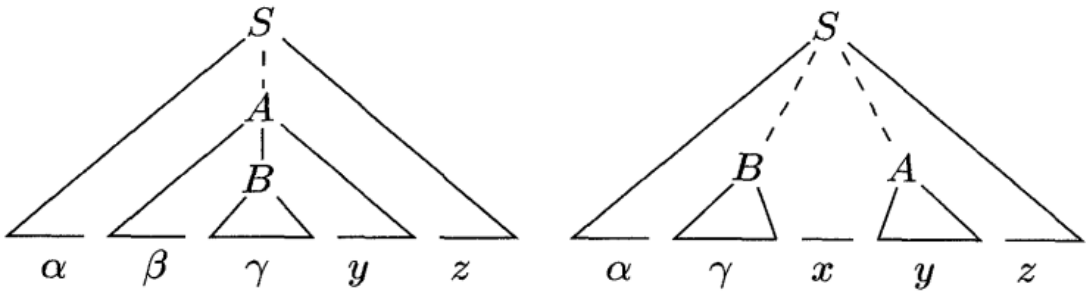
(1) $S \underset{rm}{\overset{*}{\Longrightarrow}} \alpha Az \underset{rm}{\Longrightarrow} \alpha\beta Byz \underset{rm}{\Longrightarrow} \alpha\beta\gamma yz$

(2) $S \underset{rm}{\overset{*}{\Longrightarrow}} \alpha BxAz \underset{rm}{\Longrightarrow} \alpha Bxyz \underset{rm}{\Longrightarrow} \alpha\gamma xyz$

STACK	INPUT
$\$\alpha\beta By$	$z\$$

- With the handle βBy on top of the stack, and it gets reduced to A.

STACK	INPUT
$\$\alpha A$	$z\$$



Cases for two successive steps of a rightmost derivation

$$S \to \alpha Az \mid \alpha BxAz$$
$$A \to \beta By \mid y$$
$$B \to \gamma$$

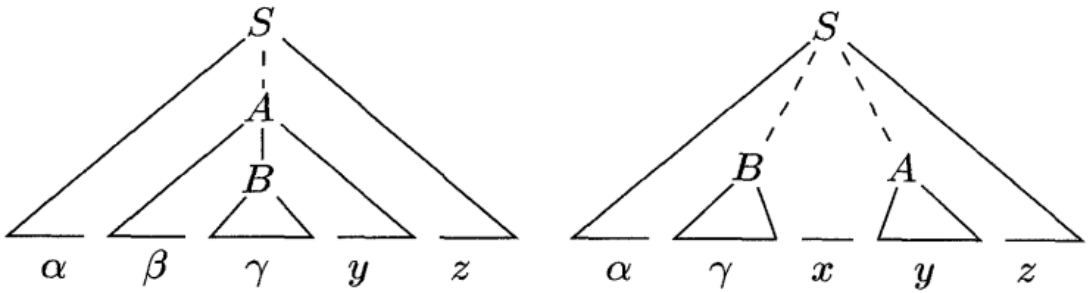
(the productions are indicated in a bit loose manner)

(1) $S \underset{rm}{\overset{*}{\Longrightarrow}} \alpha Az \underset{rm}{\Longrightarrow} \alpha\beta Byz \underset{rm}{\Longrightarrow} \alpha\beta\gamma yz$

(2) $S \underset{rm}{\overset{*}{\Longrightarrow}} \alpha BxAz \underset{rm}{\Longrightarrow} \alpha Bxyz \underset{rm}{\Longrightarrow} \alpha\gamma xyz$

- Now consider case (2).
- In configuration the handle γ is on top of the stack.

STACK	INPUT
$\$\alpha\gamma$	$xyz\$$



Cases for two successive steps of a rightmost derivation

$$S \to \alpha Az \mid \alpha BxAz$$
$$A \to \beta By \mid y$$
$$B \to \gamma$$

(the productions are indicated in a bit loose manner)

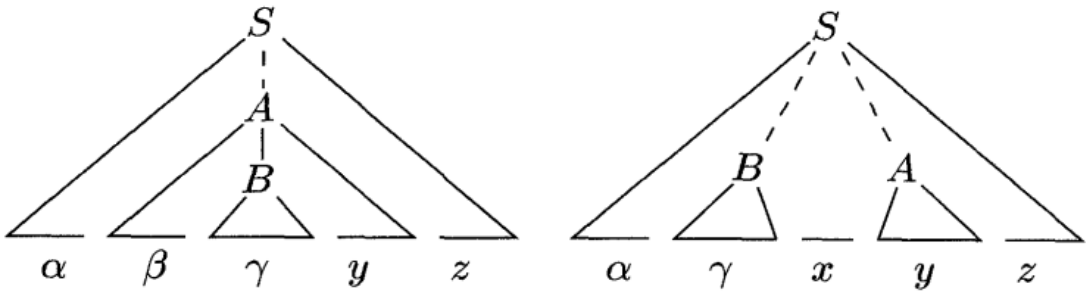
(1) $S \underset{rm}{\overset{*}{\Longrightarrow}} \alpha Az \underset{rm}{\Longrightarrow} \alpha\beta Byz \underset{rm}{\Longrightarrow} \alpha\beta\gamma yz$

(2) $S \underset{rm}{\overset{*}{\Longrightarrow}} \alpha BxAz \underset{rm}{\Longrightarrow} \alpha Bxyz \underset{rm}{\Longrightarrow} \alpha\gamma xyz$

STACK	INPUT
$\$\alpha\gamma$	$xyz\$$

- After reducing the handle γ to B, the configuration is:

STACK	INPUT
$\$\alpha B$	$xyz\$$



Cases for two successive steps of a rightmost derivation

$$
\begin{aligned}
S &\rightarrow \alpha Az \mid \alpha BxAz \\
A &\rightarrow \beta By \mid y \\
B &\rightarrow \gamma
\end{aligned}
$$

(the productions are indicated in a bit loose manner)

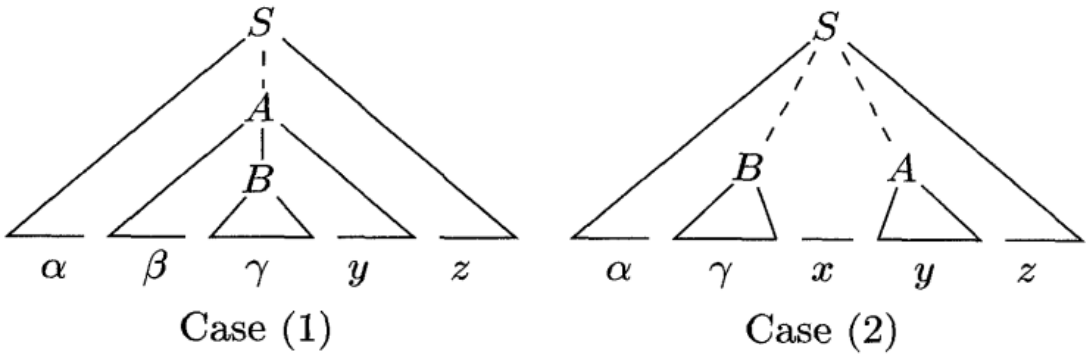
(1) $S \underset{rm}{\overset{*}{\Longrightarrow}} \alpha Az \underset{rm}{\Longrightarrow} \alpha\beta Byz \underset{rm}{\Longrightarrow} \alpha\beta\gamma yz$

(2) $S \underset{rm}{\overset{*}{\Longrightarrow}} \alpha BxAz \underset{rm}{\Longrightarrow} \alpha Bxyz \underset{rm}{\Longrightarrow} \alpha\gamma xyz$

STACK	INPUT
$\$\alpha B$	$xyz\$$

- The parser can shift the string xy to get the next handle y on top of the stack, ready to be reduced to A:

STACK	INPUT
$\$\alpha Bxy$	$z\$$



Cases for two successive steps of a rightmost derivation

$$S \rightarrow \alpha Az \mid \alpha BxAz$$
$$A \rightarrow \beta By \mid y$$
$$B \rightarrow \gamma$$

(the productions are indicated in a bit loose manner)

(1) $S \underset{rm}{\overset{*}{\Longrightarrow}} \alpha Az \underset{rm}{\Longrightarrow} \alpha\beta Byz \underset{rm}{\Longrightarrow} \alpha\beta\gamma yz$

(2) $S \underset{rm}{\overset{*}{\Longrightarrow}} \alpha BxAz \underset{rm}{\Longrightarrow} \alpha Bxyz \underset{rm}{\Longrightarrow} \alpha\gamma xyz$

- In both cases, after making a reduction the parser had to shift zero or more symbols to get the next handle onto the stack.
- It *never* had to go into the stack to find the handle.

Conflicts During Shift-Reduce Parsing

- There are context-free grammars for which shift-reduce parsing *cannot* be used.
- Every shift-reduce parser for such a grammar can reach a configuration.

Conflicts During Shift-Reduce Parsing — *continued*

- Here, the parser, knowing the entire stack contents and the next input symbol,
  - cannot decide whether to shift or to reduce (a *shift/reduce* conflict),
  - or cannot decide which of several reductions to make (a *reduce/reduce* conflict).

Conflicts During Shift-Reduce Parsing — *continued*

- The k in LR(k) refers to the number of symbols of lookahead on the input.
- Grammars used in compiling usually fall in the LR(1) class, with one symbol of lookahead at most.

Example

- An ambiguous grammar can never be LR.
- Consider the dangling-else grammar.

$$
\begin{array}{lcl}
\textit{stmt} & \rightarrow & \textbf{if } \textit{expr} \textbf{ then } \textit{stmt} \\
& | & \textbf{if } \textit{expr} \textbf{ then } \textit{stmt} \textbf{ else } \textit{stmt} \\
& | & \textbf{other}
\end{array}
$$

Example — *continued*

$$
\begin{array}{lcl}
\textit{stmt} & \rightarrow & \textbf{if}\ \textit{expr}\ \textbf{then}\ \textit{stmt} \\
& | & \textbf{if}\ \textit{expr}\ \textbf{then}\ \textit{stmt}\ \textbf{else}\ \textit{stmt} \\
& | & \textbf{other}
\end{array}
$$

- We have a shift-reduce parser in configuration,

STACK	INPUT
$\cdots$ **if** *expr* **then** *stmt*	**else** $\cdots$ \$

- We cannot tell whether "**if** *expr* then *stmt*" is the handle, no matter what appears below it on the stack.

Example — *continued*

$$
\begin{array}{lcl}
\textit{stmt} & \rightarrow & \textbf{if}\ \textit{expr}\ \textbf{then}\ \textit{stmt} \\
& | & \textbf{if}\ \textit{expr}\ \textbf{then}\ \textit{stmt}\ \textbf{else}\ \textit{stmt} \\
& | & \textbf{other}
\end{array}
$$

STACK	INPUT
$\cdots$ **if** *expr* **then** *stmt*	**else** $\cdots$ \$

- Here there is a shift/reduce conflict.

Example — *continued*

$$
\begin{array}{lcl}
\textit{stmt} & \rightarrow & \textbf{if } \textit{expr} \textbf{ then } \textit{stmt} \\
& | & \textbf{if } \textit{expr} \textbf{ then } \textit{stmt} \textbf{ else } \textit{stmt} \\
& | & \textbf{other}
\end{array}
$$

STACK	INPUT
$\cdots$ **if** *expr* **then** *stmt*	**else** $\cdots$ \$

- Depending on what follows the **else** on the input,
  - It might be correct to reduce **if** *expr* **then** *stmt* to *stmt*.
  - Or it might be correct to shift **else** and then to look for another *stmt* to complete the alternative, **if** *expr* **then** *stmt* **else** *stmt*.

Example — *continued*

$$
\begin{array}{lcl}
\textit{stmt} & \rightarrow & \textbf{if}\ \textit{expr}\ \textbf{then}\ \textit{stmt} \\
& | & \textbf{if}\ \textit{expr}\ \textbf{then}\ \textit{stmt}\ \textbf{else}\ \textit{stmt} \\
& | & \textbf{other}
\end{array}
$$

STACK	INPUT
$\cdots$ **if** *expr* **then** *stmt*	**else** $\cdots$ \$

- Note that shift-reduce parsing can be adapted to parse certain ambiguous grammars, such as the if-then-else grammar above.
- If we resolve the shift/reduce conflict on **else** in favor of shifting, the parser will behave as we expect, associating each **else** with the previous unmatched **then**.

Conflicts During Shift-Reduce Parsing — *continued*

- Another common setting for conflicts occurs when we know we have a handle.
- But the stack contents and the next input symbol are insufficient to determine which production should be used in a reduction.

Example

- Suppose we have a lexical analyzer that returns the token name **id** for all names, regardless of their type.
- Suppose also that our language,
  - invokes procedures by giving their names, with parameters surrounded by parentheses,
  - and that arrays are referenced by the same syntax.

Example — *continued*

- The translation of indices in array references and parameters in procedure calls are different.
- We want to use different productions to generate lists of actual parameters and indices.

Example — *continued*

- Our grammar might therefore have (among others) productions such as those shown.

(1)	*stmt*	→	**id** (*parameter_list*)
(2)	*stmt*	→	*expr* := *expr*
(3)	*parameter_list*	→	*parameter_list* , *parameter*
(4)	*parameter_list*	→	*parameter*
(5)	*parameter*	→	**id**
(6)	*expr*	→	**id** (*expr_list*)
(7)	*expr*	→	**id**
(8)	*expr_list*	→	*expr_list* , *expr*
(9)	*expr_list*	→	*expr*

Productions involving procedure calls and array references

Example — *continued*

```
p(i, j)
```

(1)	*stmt*	→	**id** (*parameter_list*)
(2)	*stmt*	→	*expr* := *expr*
(3)	*parameter_list*	→	*parameter_list* , *parameter*
(4)	*parameter_list*	→	*parameter*
(5)	*parameter*	→	**id**
(6)	*expr*	→	**id** (*expr_list*)
(7)	*expr*	→	**id**
(8)	*expr_list*	→	*expr_list* , *expr*
(9)	*expr_list*	→	*expr*

Productions involving procedure calls and array references

- A statement beginning with `p(i, j)` would appear as the token stream **id**(**id**, **id**) to the parser.

Example — *continued*

```
p(i, j)
```

(1)	*stmt*	→	**id** (*parameter_list*)
(2)	*stmt*	→	*expr* := *expr*
(3)	*parameter_list*	→	*parameter_list* , *parameter*
(4)	*parameter_list*	→	*parameter*
(5)	*parameter*	→	**id**
(6)	*expr*	→	**id** (*expr_list*)
(7)	*expr*	→	**id**
(8)	*expr_list*	→	*expr_list* , *expr*
(9)	*expr_list*	→	*expr*

Productions involving procedure calls and array references

- After shifting the first three tokens onto the stack, a shift-reduce parser would be in configuration,

STACK	INPUT
. . . **id** (**id**	, **id**) . . .

Example — *continued*

STACK	INPUT
. . . **id** (**id**	, **id**) . . .

(1)	*stmt*	→	**id** (*parameter_list*)
(2)	*stmt*	→	*expr* := *expr*
(3)	*parameter_list*	→	*parameter_list* , *parameter*
(4)	*parameter_list*	→	*parameter*
(5)	*parameter*	→	**id**
(6)	*expr*	→	**id** (*expr_list*)
(7)	*expr*	→	**id**
(8)	*expr_list*	→	*expr_list* , *expr*
(9)	*expr_list*	→	*expr*

Productions involving procedure calls and array references

- It is evident that the **id** on top of the stack must be reduced, but by which production?

Example — *continued*

STACK	INPUT
. . . **id** (**id**	, **id**) . . .

(1)	*stmt*	→	**id** (*parameter_list*)
(2)	*stmt*	→	*expr* := *expr*
(3)	*parameter_list*	→	*parameter_list* , *parameter*
(4)	*parameter_list*	→	*parameter*
(5)	*parameter*	→	**id**
(6)	*expr*	→	**id** (*expr_list*)
(7)	*expr*	→	**id**
(8)	*expr_list*	→	*expr_list* , *expr*
(9)	*expr_list*	→	*expr*

Productions involving procedure calls and array references

- The correct choice is production (5) if `p` is a procedure, but production (7) if `p` is an array.

Example — *continued*

STACK	INPUT
. . . **id** (**id**	, **id**) . . .

(1)	*stmt*	→	**id** (*parameter_list*)
(2)	*stmt*	→	*expr* := *expr*
(3)	*parameter_list*	→	*parameter_list* , *parameter*
(4)	*parameter_list*	→	*parameter*
(5)	*parameter*	→	**id**
(6)	*expr*	→	**id** (*expr_list*)
(7)	*expr*	→	**id**
(8)	*expr_list*	→	*expr_list* , *expr*
(9)	*expr_list*	→	*expr*

Productions involving procedure calls and array references

- The stack does not tell which information in the symbol table obtained from the declaration of `p` must be used.

Example — *continued*

STACK	INPUT
. . . **id** (**id**	, **id**) . . .

(1)	*stmt*	→	**id** (*parameter_list*)
(2)	*stmt*	→	*expr* := *expr*
(3)	*parameter_list*	→	*parameter_list* , *parameter*
(4)	*parameter_list*	→	*parameter*
(5)	*parameter*	→	**id**
(6)	*expr*	→	**id** (*expr_list*)
(7)	*expr*	→	**id**
(8)	*expr_list*	→	*expr_list* , *expr*
(9)	*expr_list*	→	*expr*

Productions involving procedure calls and array references

- One solution is to change the token **id** in production (1) to **procid**.

Example — *continued*

STACK	INPUT
. . . **id** (**id**	, **id**) . . .

(1)	*stmt*	→	**id** (*parameter_list*)
(2)	*stmt*	→	*expr* := *expr*
(3)	*parameter_list*	→	*parameter_list* , *parameter*
(4)	*parameter_list*	→	*parameter*
(5)	*parameter*	→	**id**
(6)	*expr*	→	**id** (*expr_list*)
(7)	*expr*	→	**id**
(8)	*expr_list*	→	*expr_list* , *expr*
(9)	*expr_list*	→	*expr*

Productions involving procedure calls and array references

- To use a more sophisticated lexical analyzer that returns the token name **procid** when it recognizes a lexeme that is the name of a procedure.

Example — *continued*

STACK	INPUT
. . . **id** (**id**	, **id**) . . .

(1)	*stmt*	→	**id** (*parameter_list*)
(2)	*stmt*	→	*expr* := *expr*
(3)	*parameter_list*	→	*parameter_list* , *parameter*
(4)	*parameter_list*	→	*parameter*
(5)	*parameter*	→	**id**
(6)	*expr*	→	**id** (*expr_list*)
(7)	*expr*	→	**id**
(8)	*expr_list*	→	*expr_list* , *expr*
(9)	*expr_list*	→	*expr*

Productions involving procedure calls and array references

- Doing so would require the lexical analyzer to consult the symbol table before returning a token.

Example — *continued*

(1)	*stmt*	→	**id** (*parameter_list*)
(2)	*stmt*	→	*expr* := *expr*
(3)	*parameter_list*	→	*parameter_list* , *parameter*
(4)	*parameter_list*	→	*parameter*
(5)	*parameter*	→	**id**
(6)	*expr*	→	**id** (*expr_list*)
(7)	*expr*	→	**id**
(8)	*expr_list*	→	*expr_list* , *expr*
(9)	*expr_list*	→	*expr*

Productions involving procedure calls and array references

- If we made this modification, then on processing `p(i, j)` the parser would be either in the configuration,

STACK	INPUT
. . . **procid** (**id**	, **id**) . . .

Example — *continued*

(1)	*stmt*	→	**id** (*parameter_list*)
(2)	*stmt*	→	*expr* := *expr*
(3)	*parameter_list*	→	*parameter_list* , *parameter*
(4)	*parameter_list*	→	*parameter*
(5)	*parameter*	→	**id**
(6)	*expr*	→	**id** (*expr_list*)
(7)	*expr*	→	**id**
(8)	*expr_list*	→	*expr_list* , *expr*
(9)	*expr_list*	→	*expr*

Productions involving procedure calls and array references

- Or in the configuration,

STACK	INPUT
. . . **id** (**id**	, **id**) . . .

Example — *continued*

STACK	INPUT
. . . **procid** (**id**	, **id**) . . .

STACK	INPUT
. . . **id** (**id**	, **id**) . . .

(1)	*stmt*	→	**id** (*parameter_list*)
(2)	*stmt*	→	*expr* := *expr*
(3)	*parameter_list*	→	*parameter_list* , *parameter*
(4)	*parameter_list*	→	*parameter*
(5)	*parameter*	→	**id**
(6)	*expr*	→	**id** (*expr_list*)
(7)	*expr*	→	**id**
(8)	*expr_list*	→	*expr_list* , *expr*
(9)	*expr_list*	→	*expr*

- In the former case, we choose reduction by production (5).

Example — *continued*

STACK	INPUT
. . . **procid (id**	**, id) . . .**

STACK	INPUT
. . . **id (id**	**, id) . . .**

(1)	*stmt*	→	**id** (*parameter_list*)
(2)	*stmt*	→	*expr* := *expr*
(3)	*parameter_list*	→	*parameter_list* , *parameter*
(4)	*parameter_list*	→	*parameter*
(5)	*parameter*	→	**id**
(6)	*expr*	→	**id** (*expr_list*)
(7)	*expr*	→	**id**
(8)	*expr_list*	→	*expr_list* , *expr*
(9)	*expr_list*	→	*expr*

- In the latter case we choose reduction by production (7).

Example — *continued*

STACK	INPUT
. . . **procid (id**	**, id) . . .**

STACK	INPUT
. . . **id (id**	**, id) . . .**

(1)	*stmt*	→	**id** (*parameter_list*)
(2)	*stmt*	→	*expr* := *expr*
(3)	*parameter_list*	→	*parameter_list* , *parameter*
(4)	*parameter_list*	→	*parameter*
(5)	*parameter*	→	**id**
(6)	*expr*	→	**id** (*expr_list*)
(7)	*expr*	→	**id**
(8)	*expr_list*	→	*expr_list* , *expr*
(9)	*expr_list*	→	*expr*

- Notice how the symbol third from the top of the stack determines the reduction to be made, even though it is not involved in the reduction.

Example — *continued*

STACK	INPUT
. . . **procid (id**	**, id) . . .**

STACK	INPUT
. . . **id (id**	**, id) . . .**

(1)	*stmt*	→	**id** (*parameter_list*)
(2)	*stmt*	→	*expr* := *expr*
(3)	*parameter_list*	→	*parameter_list* , *parameter*
(4)	*parameter_list*	→	*parameter*
(5)	*parameter*	→	**id**
(6)	*expr*	→	**id** (*expr_list*)
(7)	*expr*	→	**id**
(8)	*expr_list*	→	*expr_list* , *expr*
(9)	*expr_list*	→	*expr*

- Shift-reduce parsing can utilize information far down in the stack to guide the parse.

Introduction to LR Parsing: Simple LR

- The most prevalent type of bottom-up parser today is based on a concept called LR(k) parsing.
- The "L" is for left-to-right scanning of the input.
- The "R" is for constructing a rightmost derivation in reverse.
- The k is for the number of input symbols of lookahead that are used in making parsing decisions.

Introduction to LR Parsing: Simple LR — *continued*

- The cases $k = 0$ or $k = 1$ are of practical interest.
- We shall only consider LR parsers with $k \leq 1$ here.
- When (k) is omitted, k is assumed to be 1.

Introduction to LR Parsing: Simple LR — *continued*

- Introduces the basic concepts of LR parsing.
- The easiest method for constructing shift-reduce parsers, called “simple LR” (or SLR, for short).
- Some familiarity with the basic concepts is helpful even if the LR parser itself is constructed using an automatic parser generator.

Introduction to LR Parsing: Simple LR — *continued*

- “Items” and “parser states” are important concepts.
- The diagnostic output from an LR parser generator typically includes parser states.
- Can be used to isolate the sources of parsing conflicts.

Why LR Parsers?

- LR parsers are table-driven, much like the nonrecursive LL parsers.
- A grammar for which we can construct a parsing table is said to be an LR grammar.
- Intuitively, for a grammar to be LR it is sufficient that a left-to-right shift-reduce parser be able to recognize handles of right-sentential forms when they appear on top of the stack.

Why LR Parsers? — *continued*

- LR parsing is attractive for a variety of reasons:
- LR parsers can be constructed to recognize virtually all programming language constructs for which context-free grammars can be written.
- NonLR context-free grammars exist, but these can generally be avoided for typical programming-language constructs.

Why LR Parsers? — *continued*

- LR parsing is attractive for a variety of reasons:
- The LR-parsing method is the most general nonbacktracking shift-reduce parsing method known.
- Yet it can be implemented as efficiently as other, more primitive shift-reduce methods.

Why LR Parsers? — *continued*

- LR parsing is attractive for a variety of reasons:
- An LR parser can detect a syntactic error as soon as it is possible to do so on a left-to-right scan of the input.

Why LR Parsers? — *continued*

- LR parsing is attractive for a variety of reasons:
- The class of grammars that can be parsed using LR methods is a proper superset of the class of grammars that can be parsed with predictive or LL methods.
- For a grammar to be LR(k), we must be able to recognize the occurrence of the right side of a production in a right-sentential form, with k input symbols of lookahead.
- ...

Why LR Parsers? — *continued*

- LR parsing is attractive for a variety of reasons:
- ...
- This requirement is far less stringent than that for LL(k) grammars where we must be able to recognize the use of a production seeing only the first k symbols of what its right side derives.
- Thus, it should not be surprising that LR grammars can describe more languages than LL grammars.

Why LR Parsers? — *continued*

- The principal drawback of the LR method is that it is too much work to construct an LR parser by hand for a typical programming-language grammar.
- A specialized tool, an LR parser generator, is needed.

Why LR Parsers? — *continued*

- Fortunately, many such generators are available.
- One of the most commonly used ones, Yacc.
- Such a generator takes a context-free grammar and automatically produces a parser for that grammar.
- If the grammar contains ambiguities or other constructs that are difficult to parse in a left-to-right scan of the input, then the parser generator locates these constructs and provides detailed diagnostic messages.

Items and the LR(0) Automaton

- How does a shift-reduce parser know when to shift and when to reduce with stack contents $\$T$ and next input symbol $*$?
- How does the parser know that T on the top of the stack is not a handle, so the appropriate action is to shift and not to reduce T to E?

STACK	INPUT	ACTION
$\$$	$\mathbf{id}_1 * \mathbf{id}_2 \$$	shift
$\$ \mathbf{id}_1$	$* \mathbf{id}_2 \$$	reduce by $F \rightarrow \mathbf{id}$
$\$ F$	$* \mathbf{id}_2 \$$	reduce by $T \rightarrow F$
$\$ T$	$* \mathbf{id}_2 \$$	shift
$\$ T *$	$\mathbf{id}_2 \$$	shift
$\$ T * \mathbf{id}_2$	$\$$	reduce by $F \rightarrow \mathbf{id}$
$\$ T * F$	$\$$	reduce by $T \rightarrow T * F$
$\$ T$	$\$$	reduce by $E \rightarrow T$
$\$ E$	$\$$	accept

Why not reduce by using the E production?

Configurations of a shift-reduce parser on input $\mathbf{id}_1 * \mathbf{id}_2$

Items and the LR(0) Automaton — *continued*

STACK	INPUT	ACTION
$	$\mathbf{id}_1 * \mathbf{id}_2$ $	shift
$ $\mathbf{id}_1$	$* \mathbf{id}_2$ $	reduce by $F \to \mathbf{id}$
$ F	$* \mathbf{id}_2$ $	reduce by $T \to F$
$ T	$* \mathbf{id}_2$ $	shift
$ $T *$	$\mathbf{id}_2$ $	shift
$ $T * \mathbf{id}_2$	$	reduce by $F \to \mathbf{id}$
$ $T * F$	$	reduce by $T \to T * F$
$ T	$	reduce by $E \to T$
$ E	$	accept

$$
\begin{array}{lcl}
E & \to & E + T \mid T \\
T & \to & T * F \mid F \\
F & \to & (E) \mid \mathbf{id}
\end{array}
$$

- Why not reduce by using the E production?

Items and the LR(0) Automaton — *continued*

- An LR parser makes shift-reduce decisions by maintaining states to keep track of where we are in a parse.
- States represent sets of "items."
- An *LR(0) item* ("item" for short) of a grammar G is a production of G with a dot at some position of the body.

Items and the LR(0) Automaton — *continued*

- An *LR(0) item* ("item" for short) of a grammar G is a production of G with a dot at some position of the body.
- Thus, production $A \to XYZ$ yields the four items,

$$\begin{aligned} A &\to \cdot XYZ \\ A &\to X \cdot YZ \\ A &\to XY \cdot Z \\ A &\to XYZ \cdot \end{aligned}$$

- The production $A \to \epsilon$ generates only one item, $A \to \epsilon \cdot$.

Items and the LR(0) Automaton — *continued*

$$
\begin{aligned}
A &\rightarrow \cdot XYZ \\
A &\rightarrow X\cdot YZ \\
A &\rightarrow XY\cdot Z \\
A &\rightarrow XYZ\cdot
\end{aligned}
$$

- Intuitively, an item indicates how much of a production we have seen at a given point in the parsing process.
- For example, the item $A \rightarrow \cdot XYZ$ indicates that we hope to see a string derivable from XYZ next on the input.
- Item $A \rightarrow X\cdot YZ$ indicates that we have just seen on the input a string derivable from X and that we hope next to see a string derivable from YZ.
- Item $A \rightarrow XYZ\cdot$ indicates that we have seen the body XYZ and that it may be time to reduce XYZ to A.

Items and the LR(0) Automaton — *continued*

$$
\begin{aligned}
A &\rightarrow \cdot XYZ \\
A &\rightarrow X\cdot YZ \\
A &\rightarrow XY\cdot Z \\
A &\rightarrow XYZ\cdot
\end{aligned}
$$

- Intuitively, an item indicates how much of a production we have seen at a given point in the parsing process.
- For example, the item $A \rightarrow \cdot XYZ$ indicates that we hope to see a string derivable from XYZ next on the input.
- Item $A \rightarrow X\cdot YZ$ indicates that we have just seen on the input a string derivable from X and that we hope next to see a string derivable from YZ.
- Item $A \rightarrow XYZ\cdot$ indicates that we have seen the body XYZ and that it may be time to reduce XYZ to A.

Items and the LR(0) Automaton — *continued*

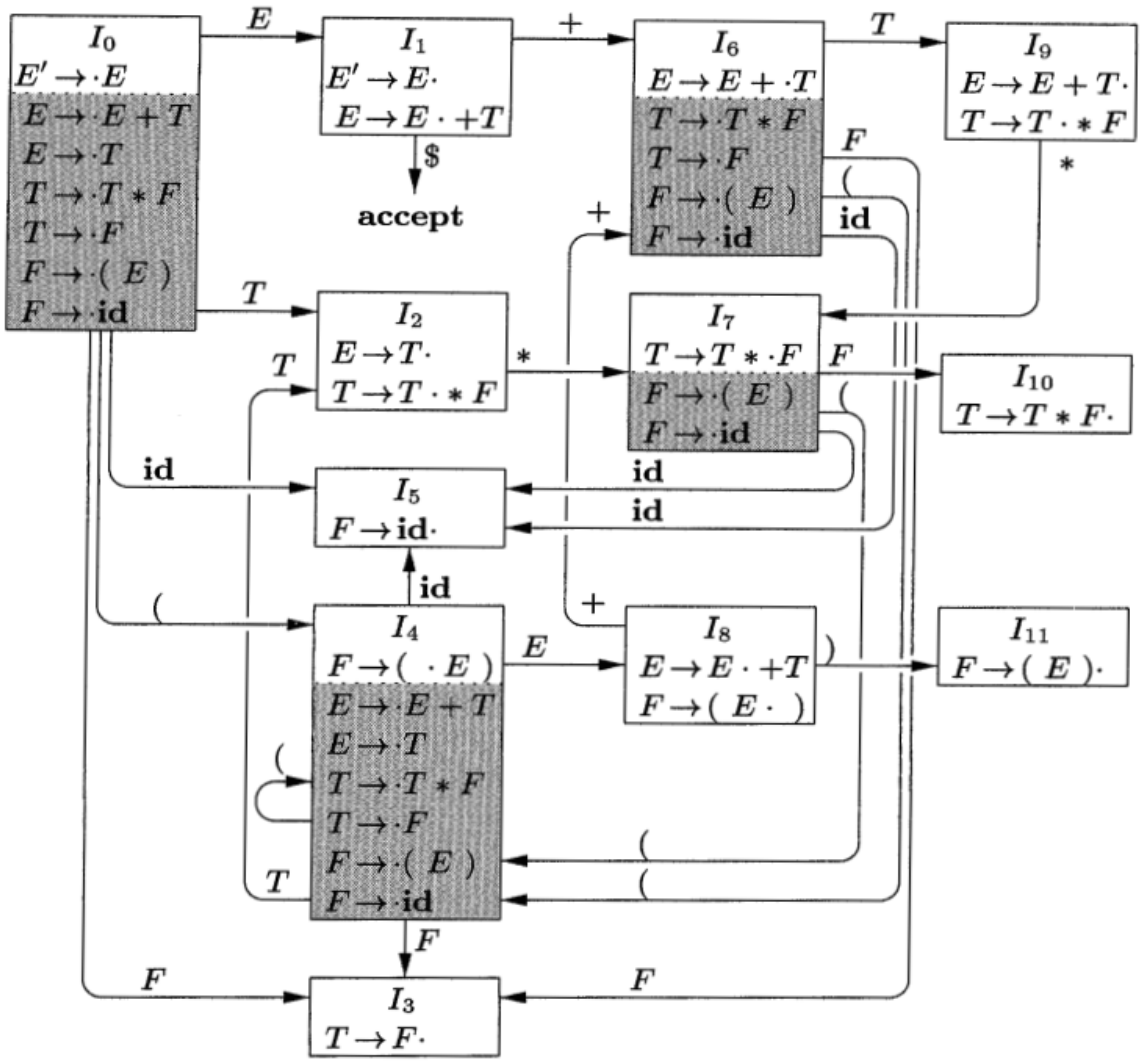
$$
\begin{aligned}
A &\rightarrow \cdot XYZ \\
A &\rightarrow X\cdot YZ \\
A &\rightarrow XY\cdot Z \\
A &\rightarrow XYZ\cdot
\end{aligned}
$$

- Intuitively, an item indicates how much of a production we have seen at a given point in the parsing process.
- For example, the item $A \rightarrow \cdot XYZ$ indicates that we hope to see a string derivable from XYZ next on the input.
- Item $A \rightarrow X\cdot YZ$ indicates that we have just seen on the input a string derivable from X and that we hope next to see a string derivable from YZ.
- Item $A \rightarrow XYZ\cdot$ indicates that we have seen the body XYZ and that it may be time to reduce XYZ to A.

Items and the LR(0) Automaton — *continued*

- One collection of sets of LR(0) items, called the canonical LR(0) collection, provides the basis for constructing a deterministic finite automaton that is used to make parsing decisions.
- Such an automaton is called an LR(0) automaton.
- In particular, each state of the LR(0) automaton represents a set of items in the canonical LR(0) collection.

Items and the LR(0) Automaton — *continued*



LR(0) automaton for the expression grammar

$$
\begin{aligned}
E &\to E + T \mid T \\
T &\to T * F \mid F \\
F &\to (E) \mid \mathbf{id}
\end{aligned}
$$

Items and the LR(0) Automaton — *continued*

- To construct the canonical LR(0) collection for a grammar, we define an augmented grammar and two functions, CLOSURE and GOTO.

Items and the LR(0) Automaton — *continued*

- If G is a grammar with start symbol S, then G', the augmented grammar for G is,
  - G with a new start symbol S' and
  - production $S' \to S$.
- The purpose of this new starting production is to indicate to the parser when it should stop parsing and announce acceptance of the input.
- That is, acceptance occurs when and only when the parser is about to reduce by $S' \to S$.

Items and the LR(0) Automaton — *continued*

- If G is a grammar with start symbol S, then G', the augmented grammar for G is,
  - G with a new start symbol S' and
  - production $S' \to S$.
- The purpose of this new starting production is to indicate to the parser when it should stop parsing and announce acceptance of the input.
- That is, acceptance occurs when and only when the parser is about to reduce by $S' \to S$.

Items and the LR(0) Automaton — Closure of Item Sets

- If I is a set of items for a grammar G, then CLOSURE(I) is the set of items constructed from I by the two rules:
  1. Initially, add every item in I to CLOSURE(I).
  2. If $A \to \alpha \cdot B\beta$ is in CLOSURE(I) and $B \to \gamma$ is a production, then add the item $B \to \cdot\gamma$ to CLOSURE(I), if it is not already there.
     - Apply this rule until no more new items can be added to CLOSURE(I).

Items and the LR(0) Automaton — Closure of Item Sets

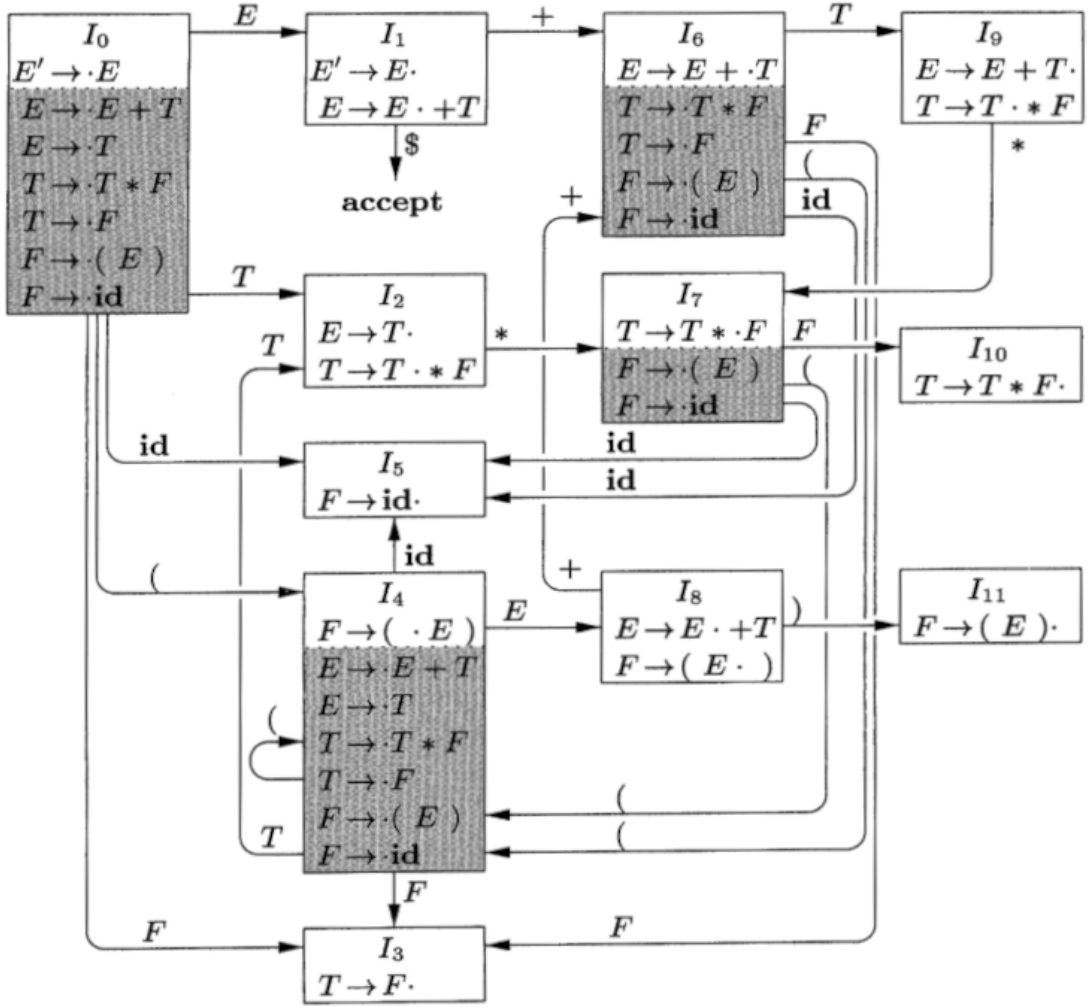
- If I is a set of items for a grammar G, then CLOSURE(I) is the set of items constructed from I by the two rules:
  1. Initially, add every item in I to CLOSURE(I).
  2. If $A \to \alpha \cdot B\beta$ is in CLOSURE(I) and $B \to \gamma$ is a production, then add the item $B \to \cdot\gamma$ to CLOSURE(I), if it is not already there.
     - Apply this rule until no more new items can be added to CLOSURE(I).

Example

- Consider the augmented expression grammar:

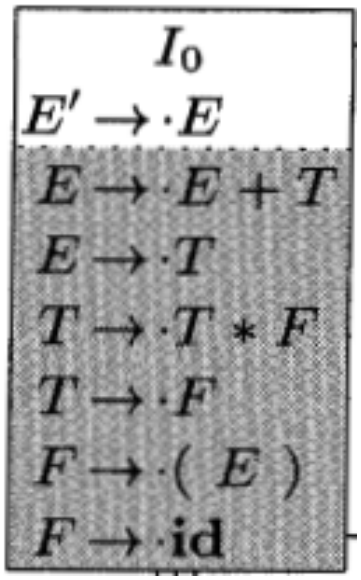
$$
\begin{aligned}
E' &\rightarrow E \\
E &\rightarrow E + T \mid T \\
T &\rightarrow T * F \mid F \\
F &\rightarrow (E) \mid \textbf{id}
\end{aligned}
$$

Example — *continued*



- If I is the set of one item $\{[E' \to \cdot E]\}$, then CLOSURE(I) contains the set of items I_0.

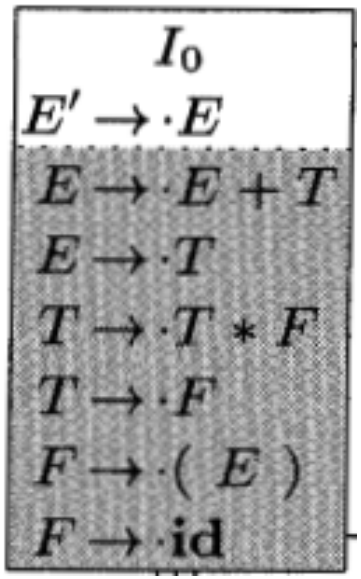
$$
\begin{aligned}
E' &\rightarrow E \\
E &\rightarrow E + T \mid T \\
T &\rightarrow T * F \mid F \\
F &\rightarrow (E) \mid \mathbf{id}
\end{aligned}
$$




```
SetOfItems CLOSURE(I) {
        J = I;
        repeat
                for ( each item A → α·Bβ in J )
                        for ( each production B → γ of G )
                                if ( B → ·γ is not in J )
                                        add B → ·γ to J;
        until no more items are added to J on one round;
        return J;
}
```

- If I is the set of one item $\{[E' \rightarrow \cdot E]\}$, then CLOSURE$(I)$ contains the set of items I_0.

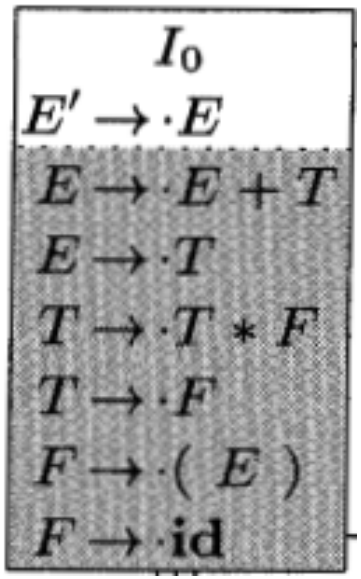
$$
\begin{aligned}
E' &\rightarrow E \\
E &\rightarrow E + T \mid T \\
T &\rightarrow T * F \mid F \\
F &\rightarrow (E) \mid \mathbf{id}
\end{aligned}
$$




```
SetOfItems CLOSURE(I) {
        J = I;
        repeat
                for ( each item A → α·Bβ in J )
                        for ( each production B → γ of G )
                                if ( B → ·γ is not in J )
                                        add B → ·γ to J;
        until no more items are added to J on one round;
        return J;
}
```

- To see how the closure is computed, $E' \rightarrow \cdot E$ is put in CLOSURE(I) by rule (1).

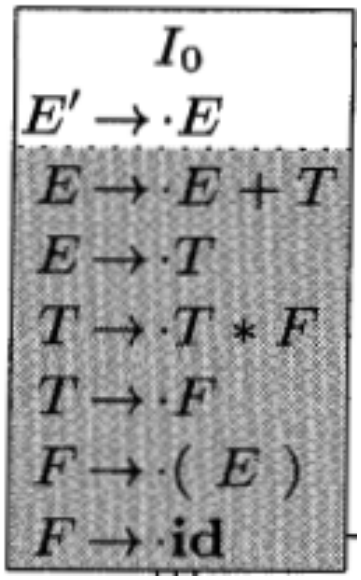
$$
\begin{aligned}
E' &\rightarrow E \\
E &\rightarrow E + T \mid T \\
T &\rightarrow T * F \mid F \\
F &\rightarrow (E) \mid \textbf{id}
\end{aligned}
$$




```
SetOfItems CLOSURE(I) {
        J = I;
        repeat
                for ( each item A → α·Bβ in J )
                        for ( each production B → γ of G )
                                if ( B → ·γ is not in J )
                                        add B → ·γ to J;
        until no more items are added to J on one round;
        return J;
}
```

- Since there is an E immediately to the right of a dot, we add the E-productions with dots at the left ends: $E \rightarrow \cdot E + T$ and $E \rightarrow \cdot T$.

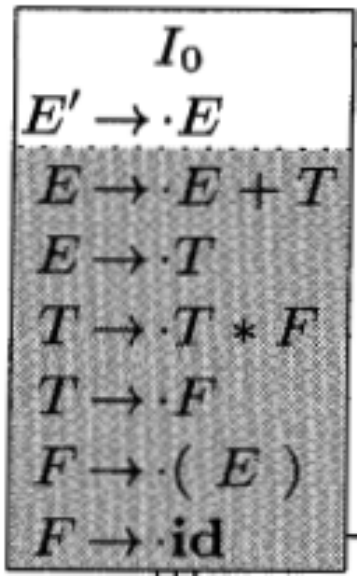
$$
\begin{aligned}
E' &\rightarrow E \\
E &\rightarrow E + T \mid T \\
T &\rightarrow T * F \mid F \\
F &\rightarrow (E) \mid \textbf{id}
\end{aligned}
$$




```
SetOfItems CLOSURE(I) {
        J = I;
        repeat
                for ( each item A → α·Bβ in J )
                        for ( each production B → γ of G )
                                if ( B → ·γ is not in J )
                                        add B → ·γ to J;
        until no more items are added to J on one round;
        return J;
}
```

- Now there is a T immediately to the right of a dot in the latter item, so we add $T \rightarrow \cdot T * F$ and $T \rightarrow \cdot F$.

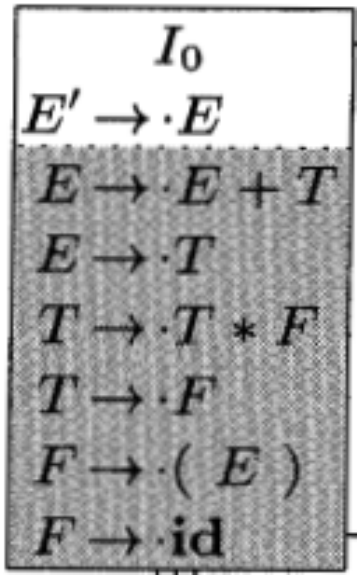
$$
\begin{aligned}
E' &\rightarrow E \\
E &\rightarrow E + T \mid T \\
T &\rightarrow T * F \mid F \\
F &\rightarrow (E) \mid \textbf{id}
\end{aligned}
$$




```
SetOfItems CLOSURE(I) {
        J = I;
        repeat
                for ( each item A → α·Bβ in J )
                        for ( each production B → γ of G )
                                if ( B → ·γ is not in J )
                                        add B → ·γ to J;
        until no more items are added to J on one round;
        return J;
}
```

- Next, the F to the right of a dot forces us to add $F \rightarrow \cdot(E)$ and $F \rightarrow$ **id**, but no other items need to be added.

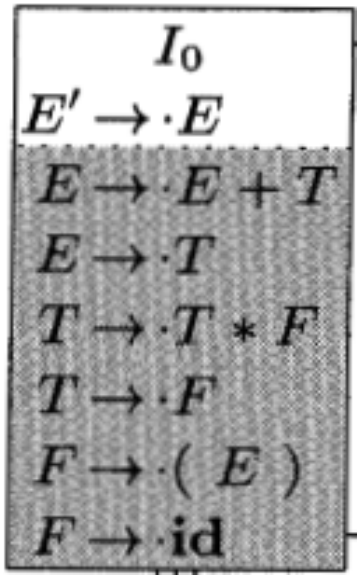
$$
\begin{aligned}
E' &\rightarrow E \\
E &\rightarrow E + T \mid T \\
T &\rightarrow T * F \mid F \\
F &\rightarrow (E) \mid \textbf{id}
\end{aligned}
$$




```
SetOfItems CLOSURE(I) {
        J = I;
        repeat
                for ( each item A → α·Bβ in J )
                        for ( each production B → γ of G )
                                if ( B → ·γ is not in J )
                                        add B → ·γ to J;
        until no more items are added to J on one round;
        return J;
}
```

- A convenient way to implement the function *closure* is to keep a Boolean array *added*, indexed by the nonterminals of G, such that *added*$[B]$ is set to **true** if and when we add the item $B \rightarrow \cdot\gamma$ for each B-production $B \rightarrow \gamma$.

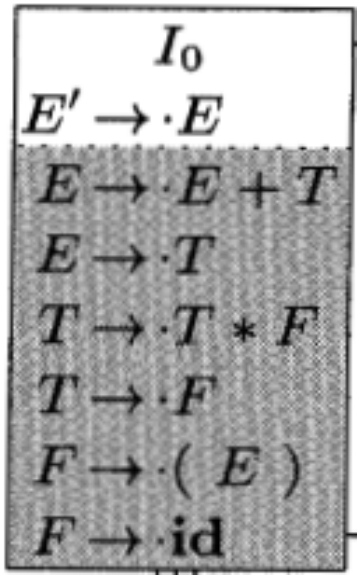
$$
\begin{aligned}
E' &\rightarrow E \\
E &\rightarrow E + T \mid T \\
T &\rightarrow T * F \mid F \\
F &\rightarrow (E) \mid \textbf{id}
\end{aligned}
$$




```
SetOfItems CLOSURE(I) {
        J = I;
        repeat
                for ( each item A → α·Bβ in J )
                        for ( each production B → γ of G )
                                if ( B → ·γ is not in J )
                                        add B → ·γ to J;
        until no more items are added to J on one round;
        return J;
}
```

- Note that if one B-production is added to the closure of I with the dot at the left end, then all B-productions will be similarly added to the closure.

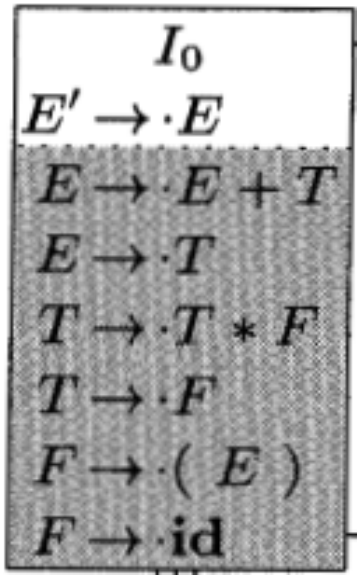
$$
\begin{aligned}
E' &\rightarrow E \\
E &\rightarrow E + T \mid T \\
T &\rightarrow T * F \mid F \\
F &\rightarrow (E) \mid \textbf{id}
\end{aligned}
$$




```
SetOfItems CLOSURE(I) {
        J = I;
        repeat
                for ( each item A → α·Bβ in J )
                        for ( each production B → γ of G )
                                if ( B → ·γ is not in J )
                                        add B → ·γ to J;
        until no more items are added to J on one round;
        return J;
}
```

- Hence, it is not necessary in some circumstances actually to list the items $B \rightarrow \gamma$ added to I by CLOSURE.

$$
\begin{aligned}
E' &\rightarrow E \\
E &\rightarrow E + T \mid T \\
T &\rightarrow T * F \mid F \\
F &\rightarrow (E) \mid \textbf{id}
\end{aligned}
$$

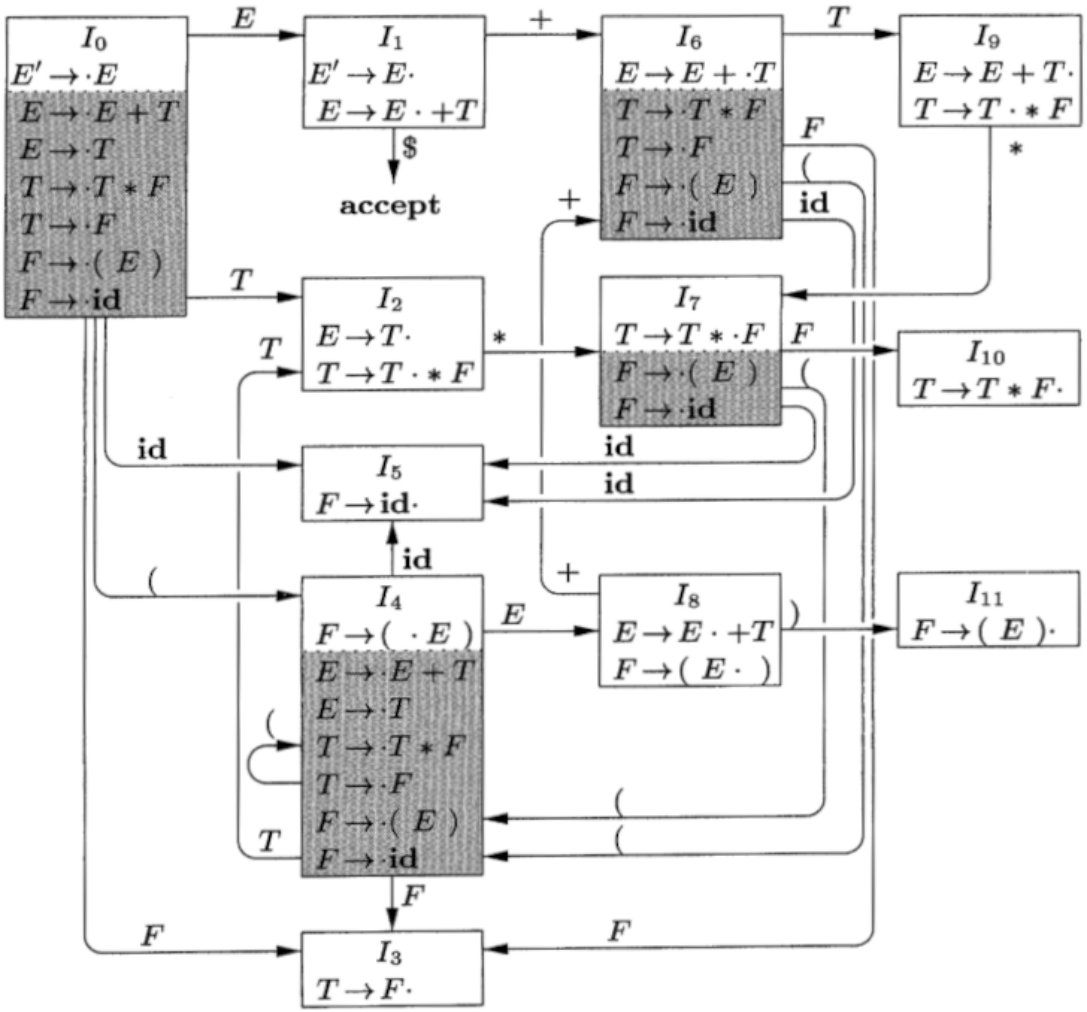



```
SetOfItems CLOSURE(I) {
        J = I;
        repeat
                for ( each item A → α·Bβ in J )
                        for ( each production B → γ of G )
                                if ( B → ·γ is not in J )
                                        add B → ·γ to J;
        until no more items are added to J on one round;
        return J;
}
```

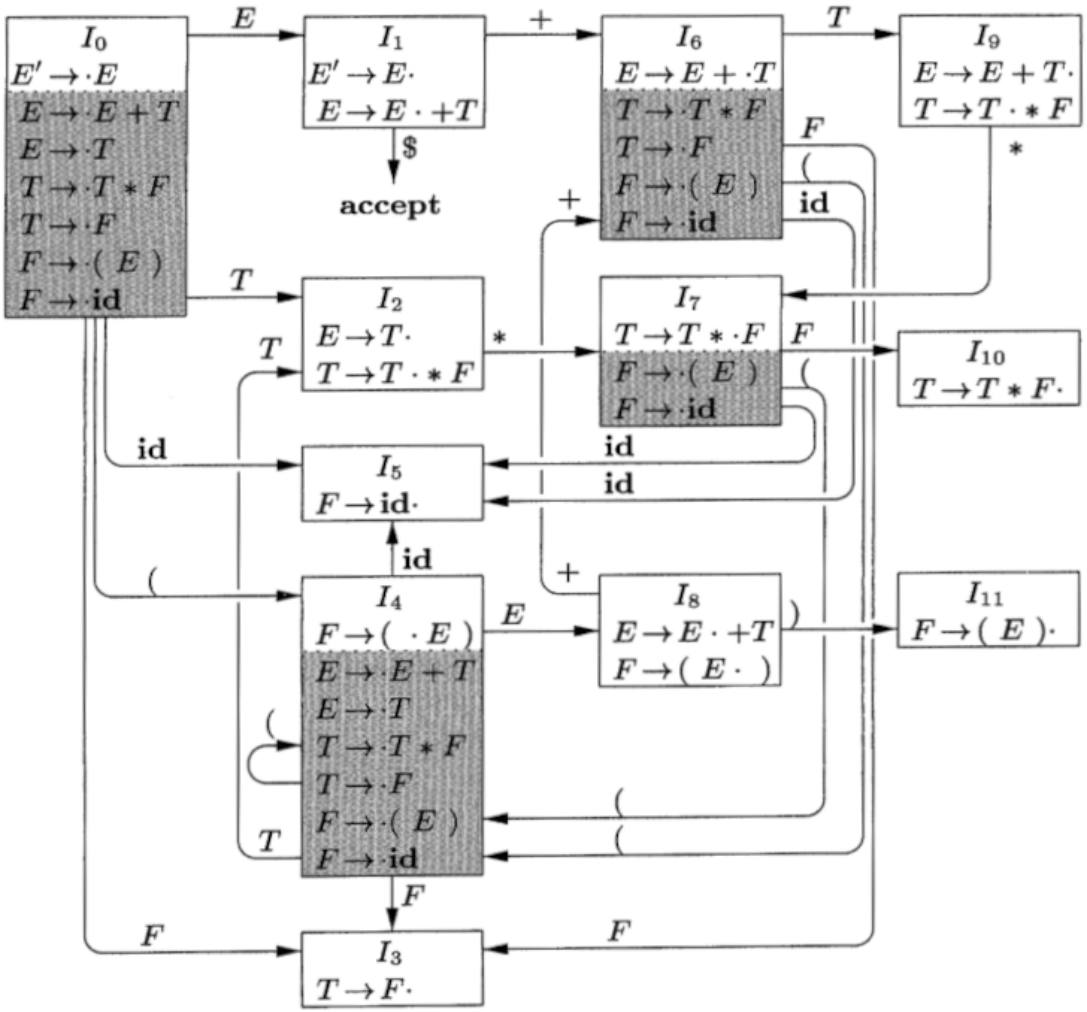
- A list of the nonterminals B whose productions were so added will suffice.

Items and the LR(0) Automaton — Closure of Item Sets — *continued*

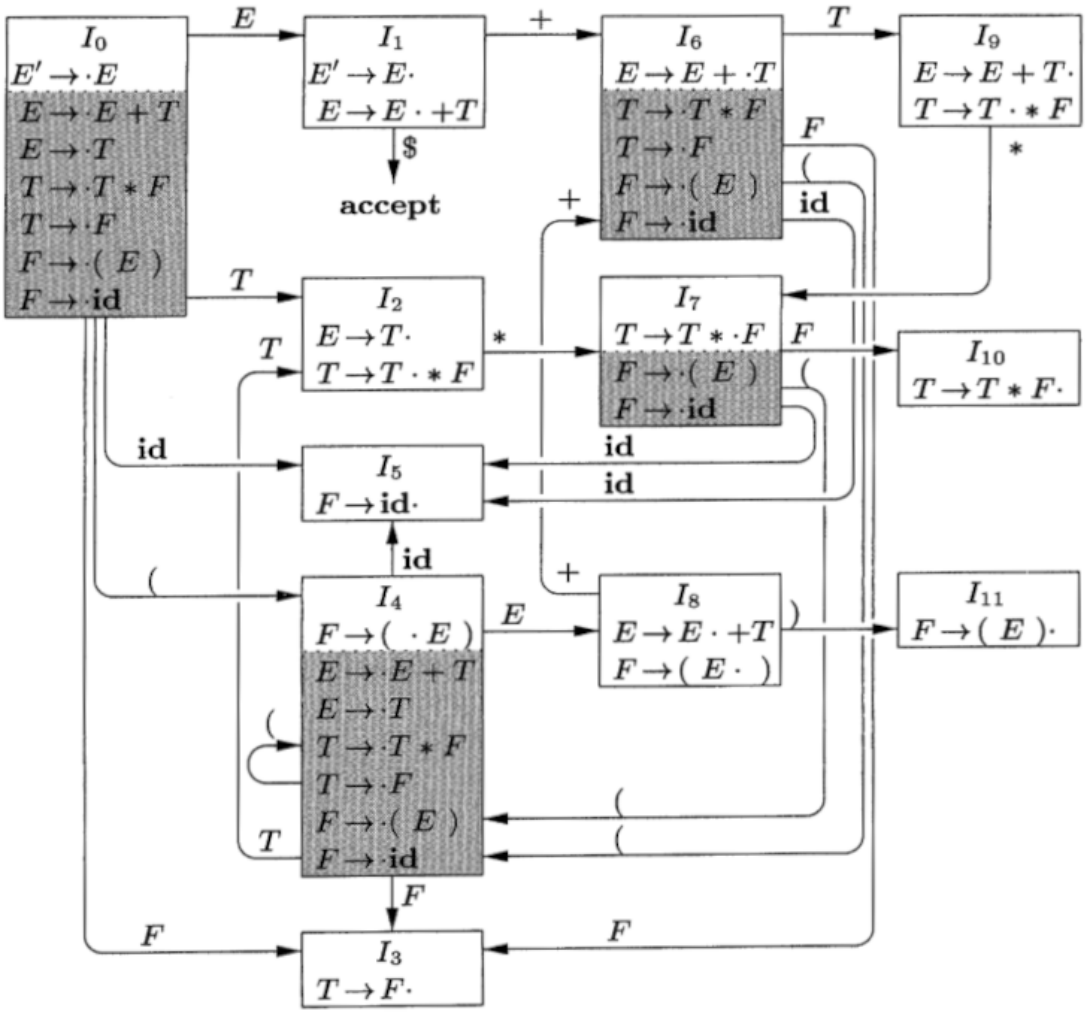
- We divide all the sets of items of interest into two classes:
  1. Kernel items: the initial item, $S' \rightarrow \cdot S$, and all items whose dots are not at the left end.
  2. Nonkernel items: all items with their dots at the left end, except for $S' \rightarrow \cdot S$.



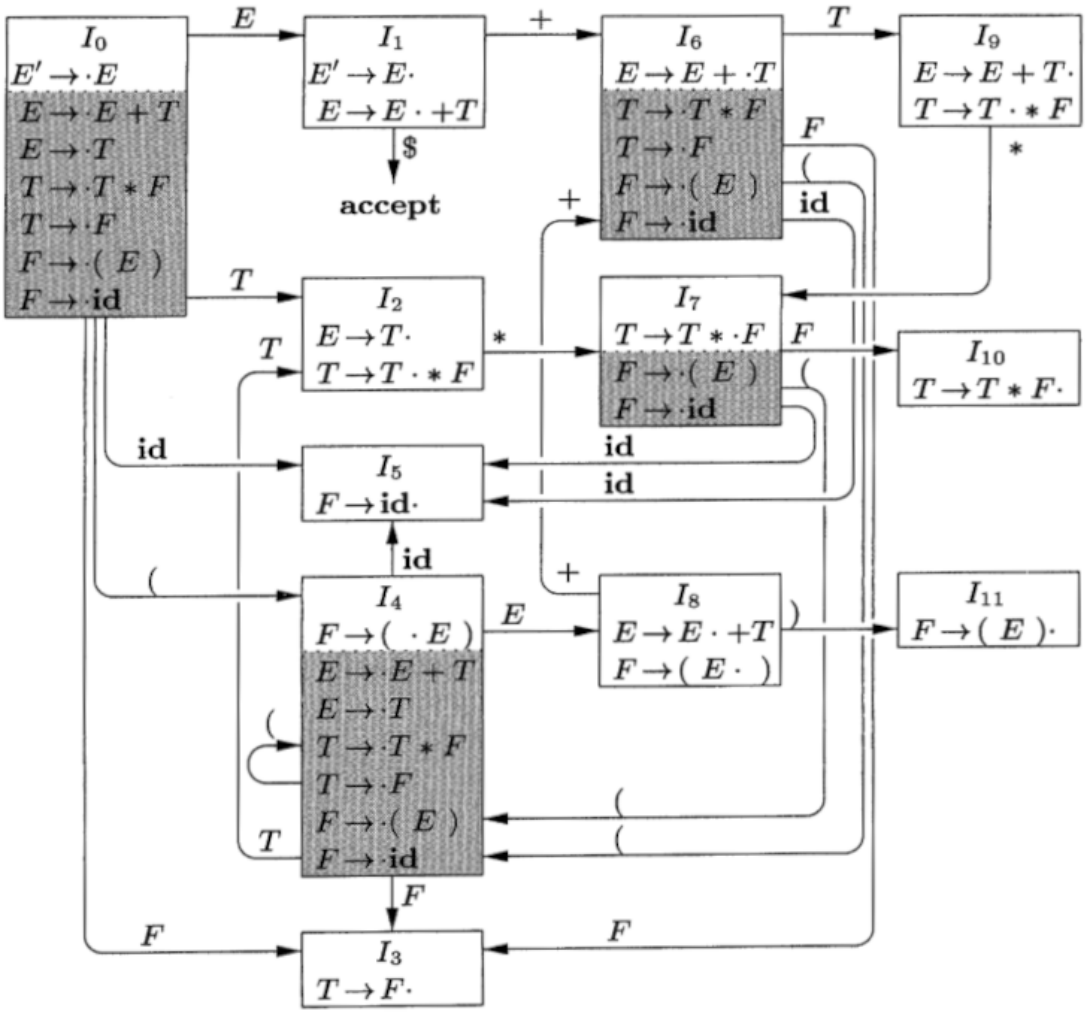
- Moreover, each set of items of interest is formed by taking the closure of a set of kernel items.



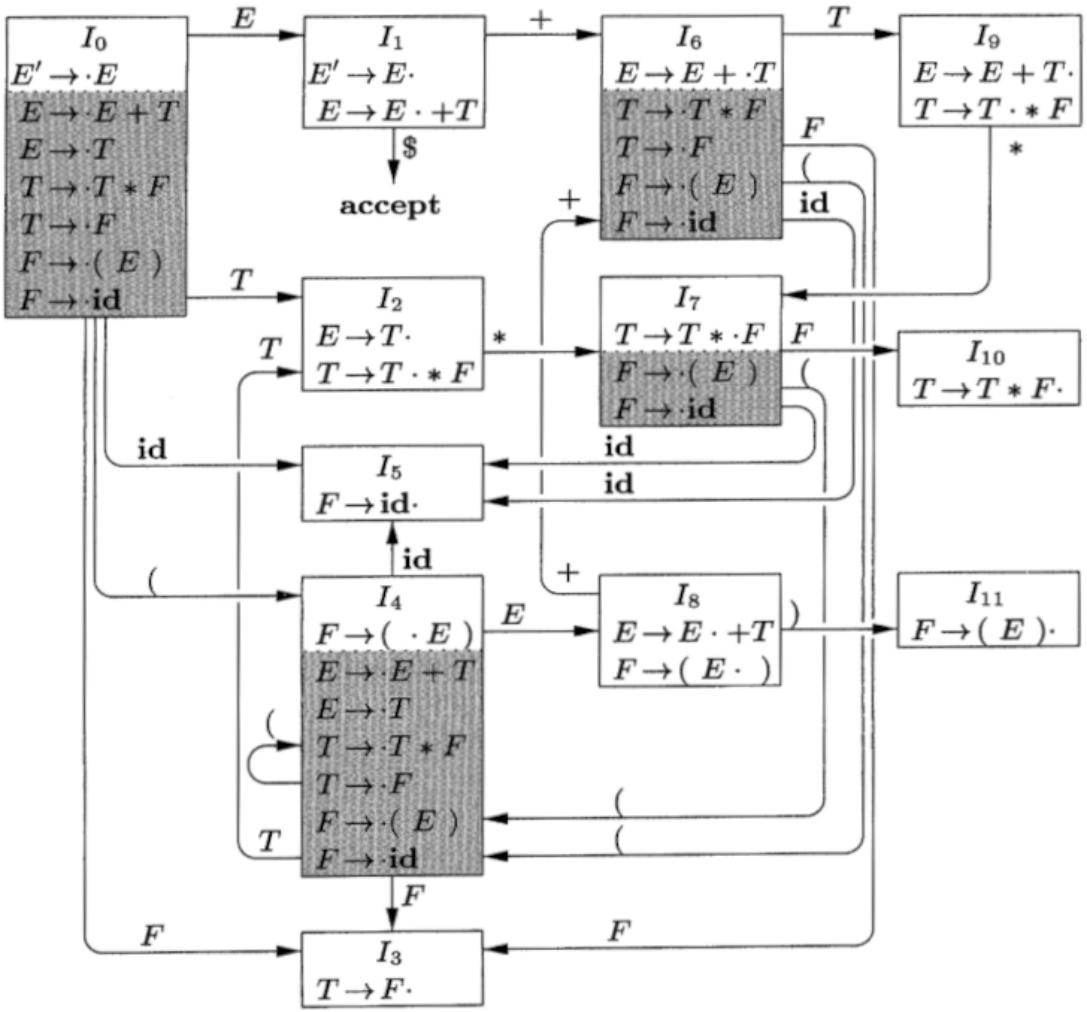
- The items added in the closure can never be kernel items, of course.



- Thus, we can represent the sets of items we are really interested in with very little storage if we throw away all nonkernel items.



- Knowing that they could be regenerated by the closure process.



- Nonkernel items are in the shaded part of the box for a state.

Items and the LR(0) Automaton — the Function GOTO

- The second useful function is GOTO(I, X) where I is a set of items and X is a grammar symbol.
- GOTO(I, X) is defined to be the closure of the set of all items $[A \to \alpha X \cdot \beta]$ such that $[A \to \alpha \cdot X \beta]$ is in I.
- Intuitively, the GOTO function is used to define the transitions in the LR(0) automaton for a grammar.
- The states of the automaton correspond to sets of items.
- GOTO(I, X) specifies the transition from the state for I under input X.

Example

- If I is the set of two items $\{[E' \to E \cdot\,], [E \to E \cdot\, + T]\}$, then GOTO$(I, +)$ contains the items,

$$
\begin{array}{lcl}
E & \to & E + \cdot T \\
T & \to & \cdot T * F \\
T & \to & \cdot F \\
F & \to & \cdot (E) \\
F & \to & \cdot \mathbf{id}
\end{array}
$$

- We computed GOTO$(I, +)$ by examining I for items with $+$ immediately to the right of the dot.
- $E' \to E \cdot$ is not such an item, but $E \to E \cdot + T$ is.
- We moved the dot over the $+$ to get $E \to E + \cdot T$ and then took the closure of this singleton set.

Example

- If I is the set of two items $\{[E' \to E \cdot\,], [E \to E \cdot\, + T]\}$, then GOTO$(I, +)$ contains the items,

$$
\begin{array}{lcl}
E & \to & E + \cdot T \\
T & \to & \cdot T * F \\
T & \to & \cdot F \\
F & \to & \cdot (E) \\
F & \to & \cdot \textbf{id}
\end{array}
$$

- We computed GOTO$(I, +)$ by examining I for items with $+$ immediately to the right of the dot.
- $E' \to E \cdot$ is not such an item, but $E \to E \cdot + T$ is.
- We moved the dot over the $+$ to get $E \to E + \cdot T$ and then took the closure of this singleton set.

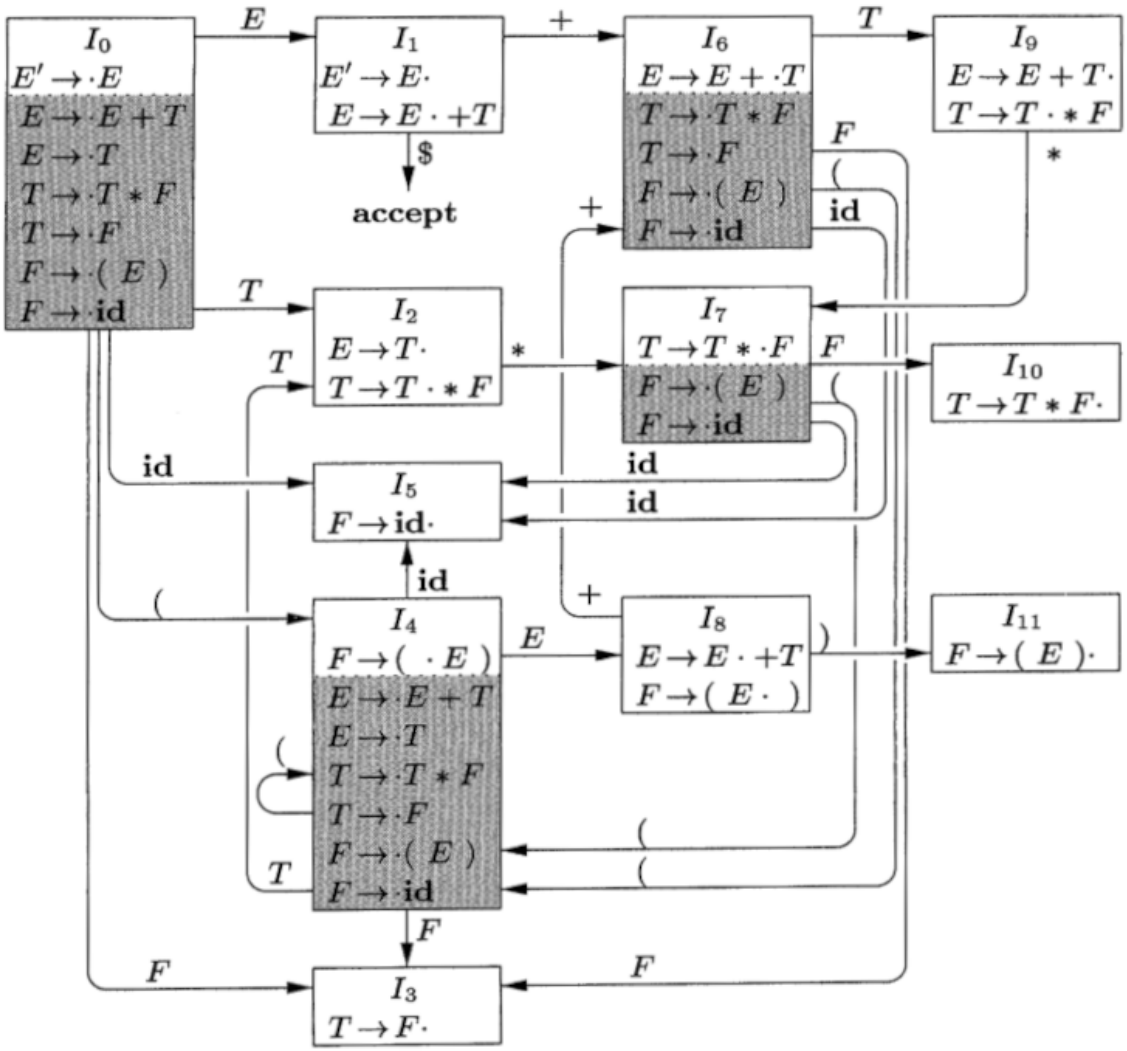
Items and the LR(0) Automaton — the Function GOTO — *continued*

```
void items(G') {
        C = CLOSURE({[S' → ·S]});
        repeat
                for ( each set of items I in C )
                        for ( each grammar symbol X )
                                if ( GOTO(I, X) is not empty and not in C )
                                        add GOTO(I, X) to C;
        until no new sets of items are added to C on a round;
}
```

Computation of the canonical collection of sets of LR(0) items

- The algorithm to construct C, the canonical collection of sets of LR(0) items for an augmented grammar G'.

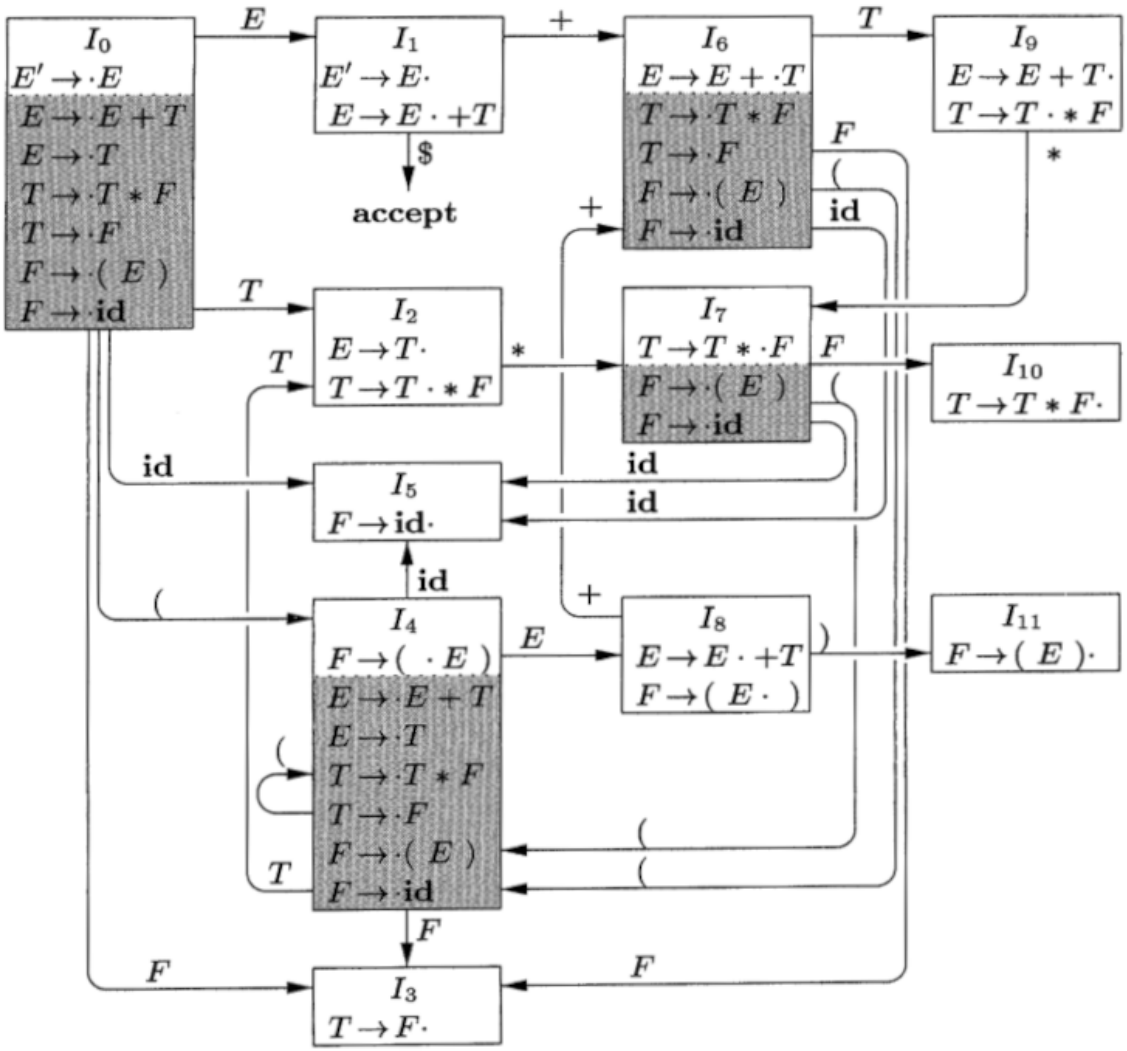
Example



$$
\begin{aligned}
E' &\to E \\
E &\to E + T \mid T \\
T &\to T * F \mid F \\
F &\to (E) \mid \mathbf{id}
\end{aligned}
$$

- The canonical collection of sets of LR(0) items for the grammar and the GOTO function are shown.

Example

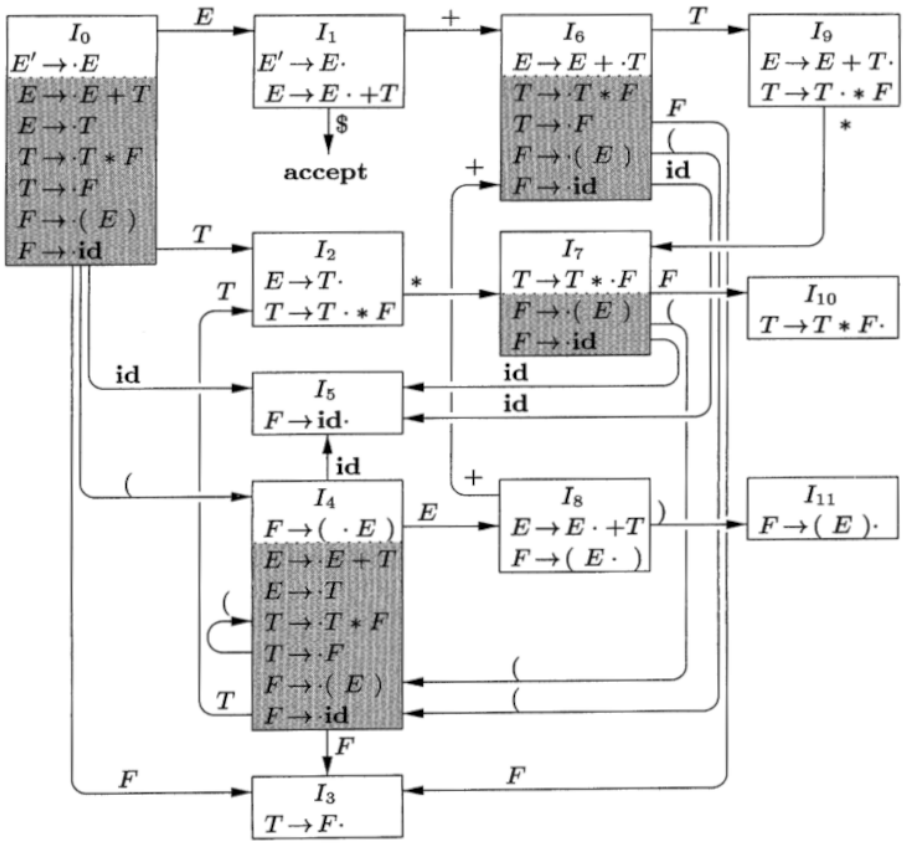


$$
\begin{aligned}
E' &\to E \\
E &\to E + T \mid T \\
T &\to T * F \mid F \\
F &\to (E) \mid \mathbf{id}
\end{aligned}
$$

- GOTO is encoded by the transitions in the figure.

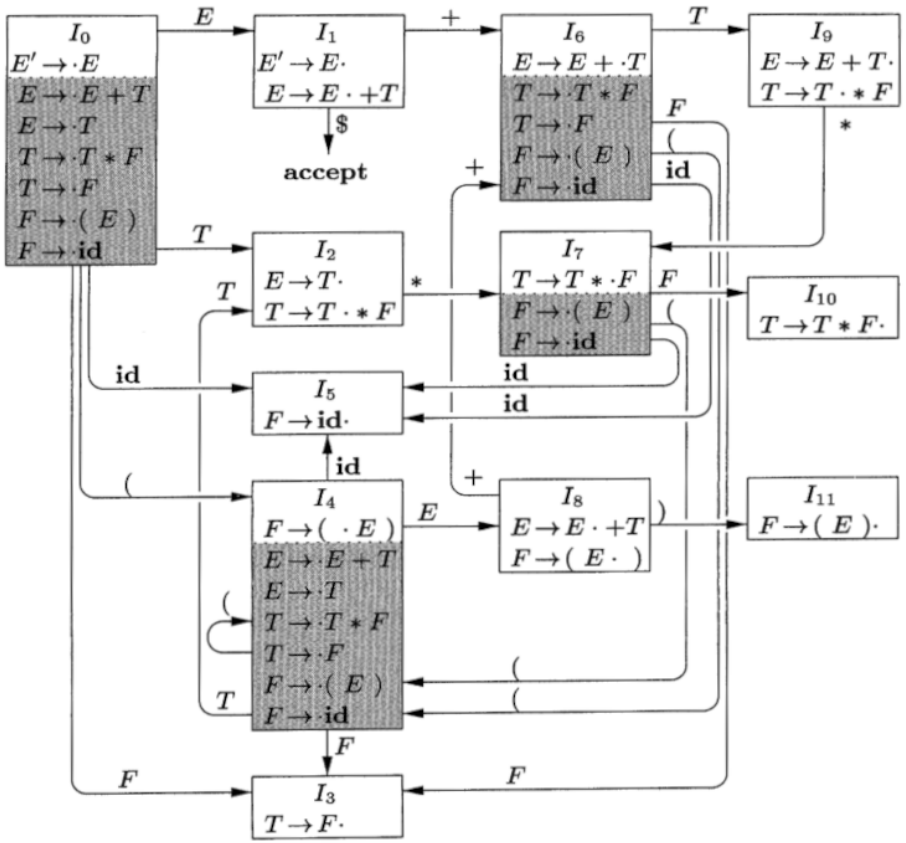
Items and the LR(0) Automaton — Use of the LR(0) Automaton

- The central idea behind “Simple LR,” or SLR, parsing is the construction from the grammar of the LR(0) automaton.
- The states of this automaton are the sets of items from the canonical LR(0) collection, and the transitions are given by the GOTO function.



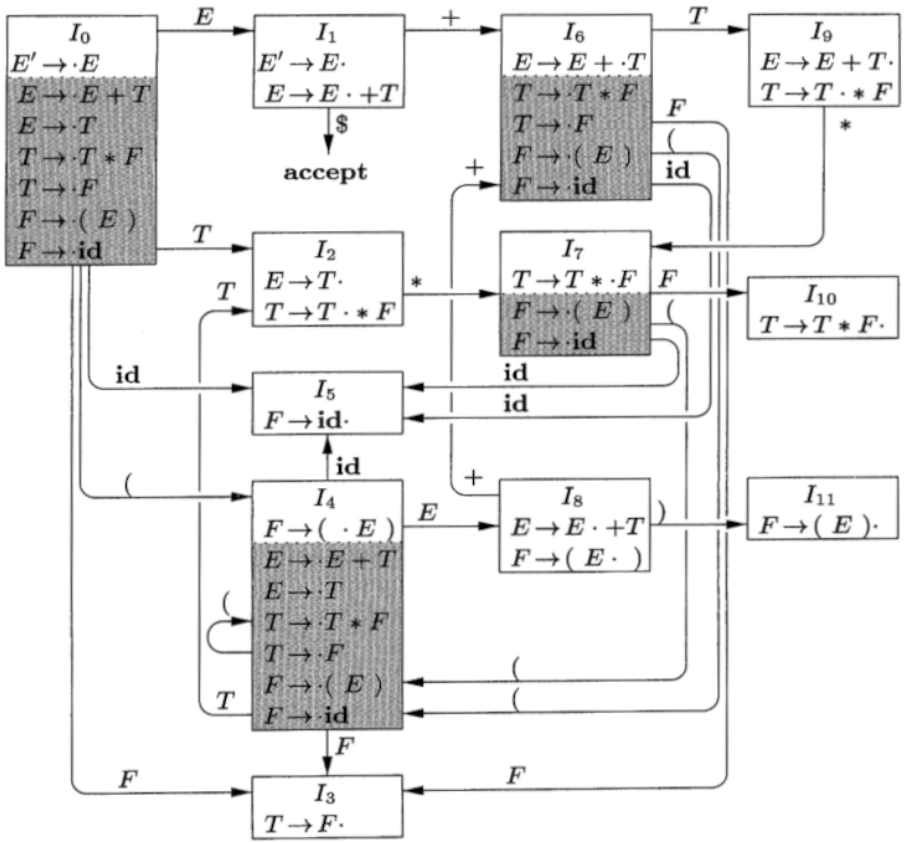
$$
\begin{aligned}
E' &\to E \\
E &\to E + T \mid T \\
T &\to T * F \mid F \\
F &\to (E) \mid \mathbf{id}
\end{aligned}
$$

- The LR(0) automaton for the expression grammar appeared earlier.
- The start state of the LR(0) automaton is CLOSURE$(\{[S' \to \cdot S]\})$, where S' is the start symbol of the augmented grammar.



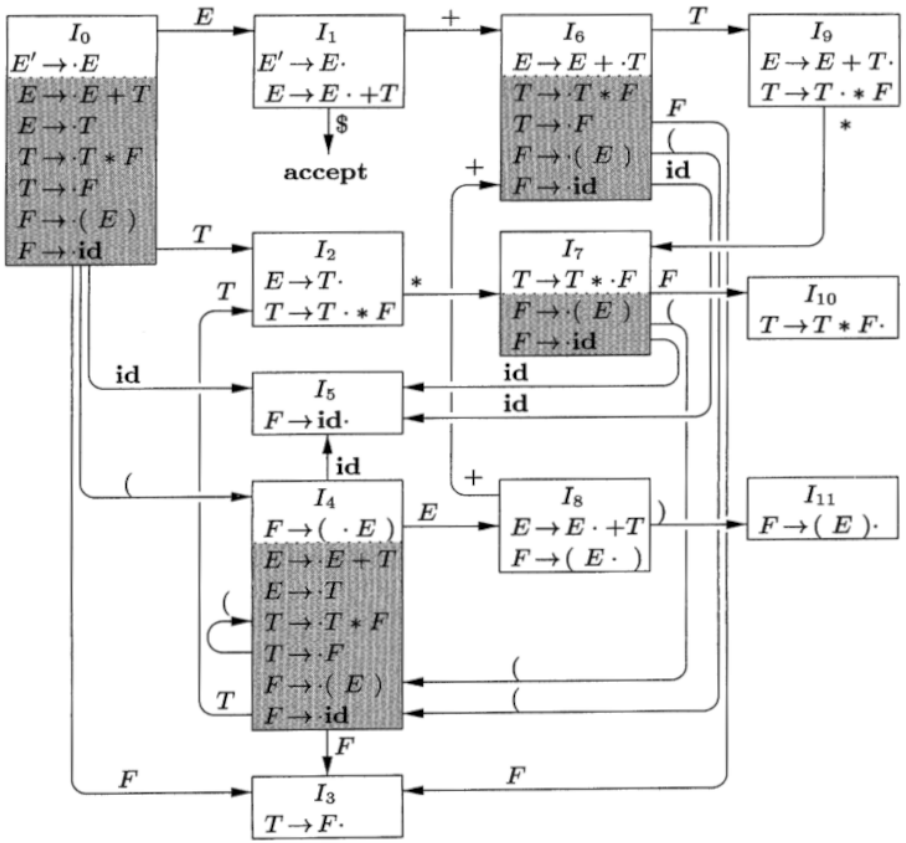
$$
\begin{aligned}
E' &\to E \\
E &\to E + T \mid T \\
T &\to T * F \mid F \\
F &\to (E) \mid \mathbf{id}
\end{aligned}
$$

- All states are accepting states.
- We say "state j" to refer to the state corresponding to the set of items I_j .



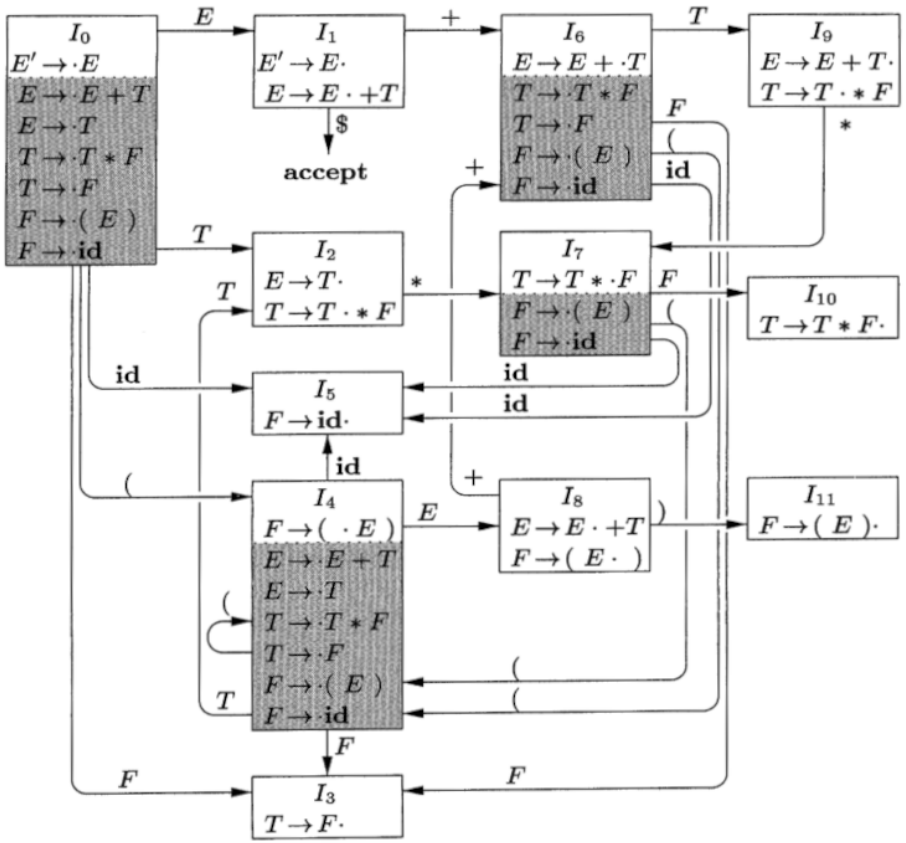
$$
\begin{aligned}
E' &\to E \\
E &\to E + T \mid T \\
T &\to T * F \mid F \\
F &\to (E) \mid \mathbf{id}
\end{aligned}
$$

- How can LR(0) automata help with shift-reduce decisions?



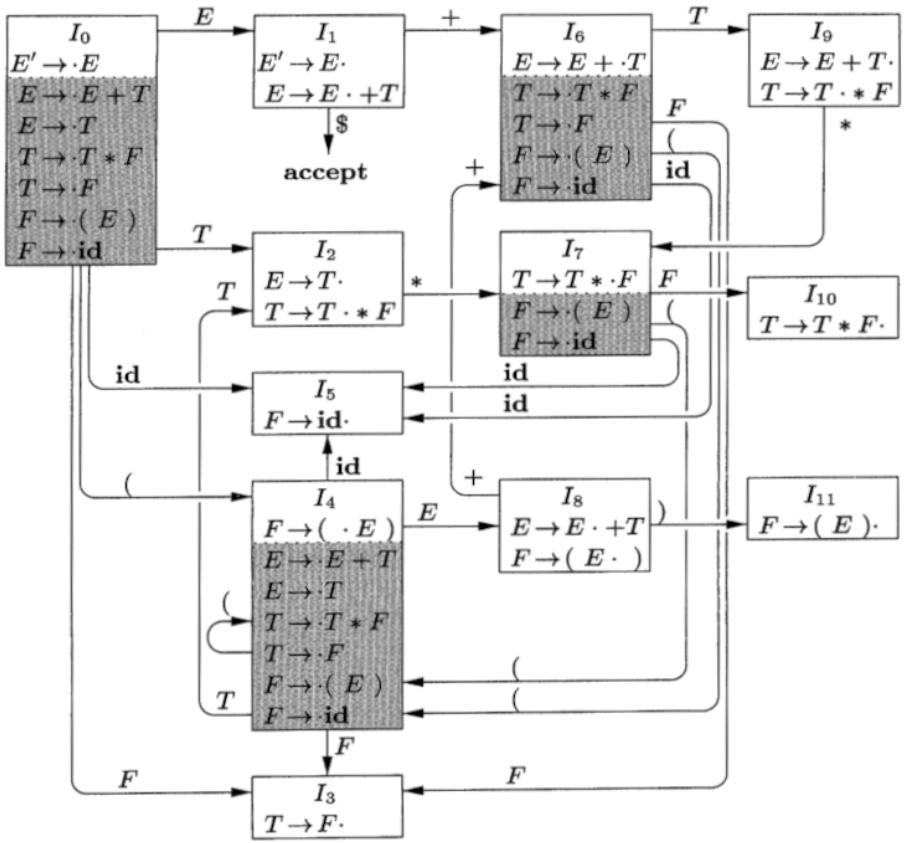
$$
\begin{aligned}
E' &\to E \\
E &\to E + T \mid T \\
T &\to T * F \mid F \\
F &\to (E) \mid \mathbf{id}
\end{aligned}
$$

- Shift-reduce decisions can be made as follows.
- Suppose that the string γ of grammar symbols takes the LR(0) automaton from the start state 0 to some state j.



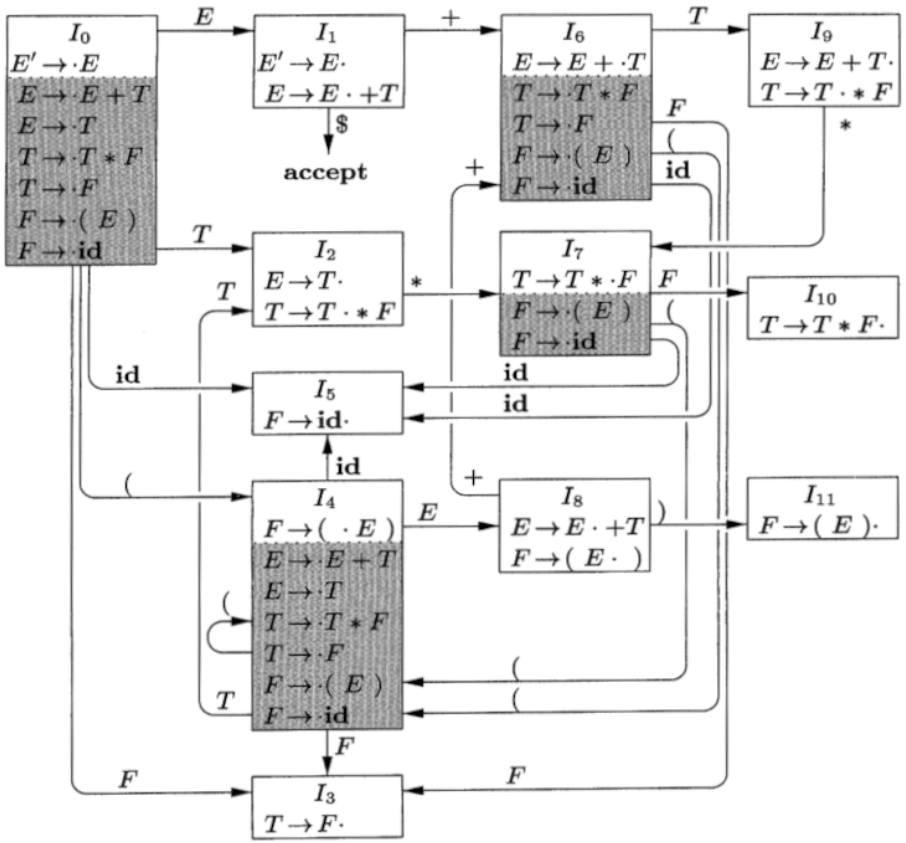
$$
\begin{aligned}
E' &\to E \\
E &\to E + T \mid T \\
T &\to T * F \mid F \\
F &\to (E) \mid \mathbf{id}
\end{aligned}
$$

- Then, shift on next input symbol a if state j has a transition on a.



$$
\begin{aligned}
E' &\to E \\
E &\to E + T \mid T \\
T &\to T * F \mid F \\
F &\to (E) \mid \mathbf{id}
\end{aligned}
$$

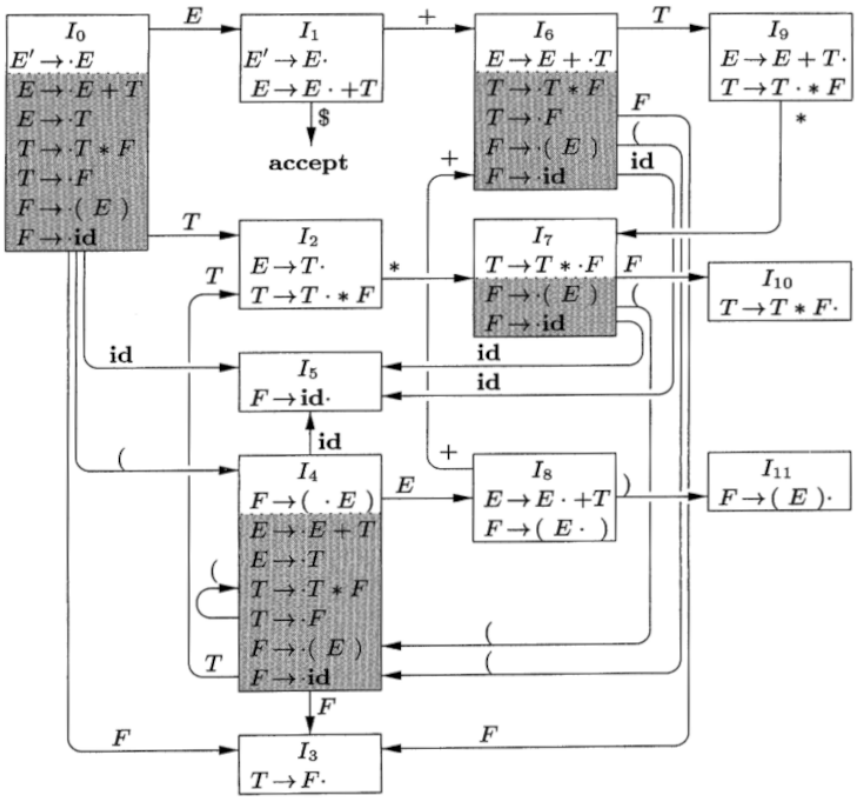
- Otherwise, we choose to reduce.
- The items in state j will tell us which production to use.



$$
\begin{aligned}
E' &\to E \\
E &\to E + T \mid T \\
T &\to T * F \mid F \\
F &\to (E) \mid \mathbf{id}
\end{aligned}
$$

- The LR-parsing algorithm uses its stack to keep track of states as well as grammar symbols.
- In fact, the grammar symbol can be recovered from the state, so the stack holds states.

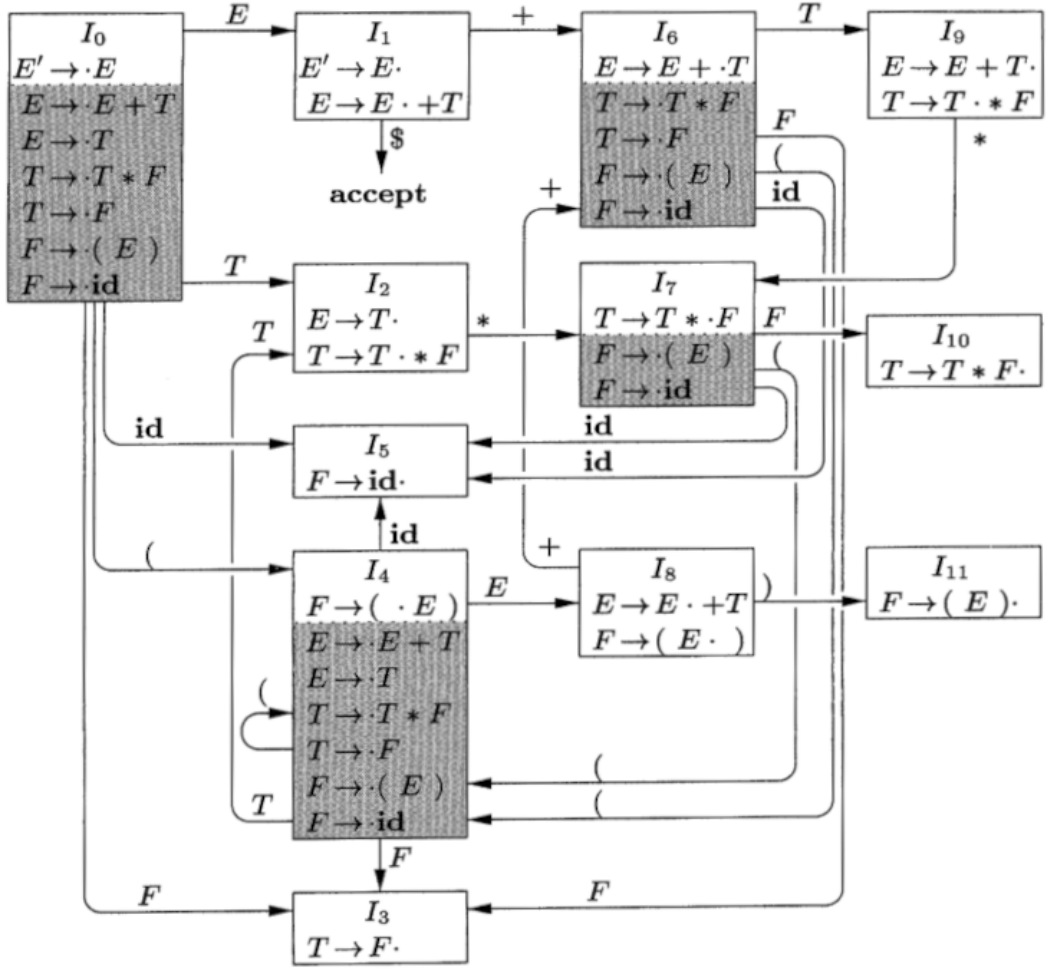
Example



LINE	STACK	SYMBOLS	INPUT	ACTION
(1)	0	\$	**id** $*$ **id** \$	shift to 5
(2)	0 5	\$ **id**	$*$ **id** \$	reduce by $F \to$ **id**
(3)	0 3	\$ F	$*$ **id** \$	reduce by $T \to F$
(4)	0 2	\$ T	$*$ **id** \$	shift to 7
(5)	0 2 7	\$ $T *$	**id** \$	shift to 5
(6)	0 2 7 5	\$ $T *$ **id**	\$	reduce by $F \to$ **id**
(7)	0 2 7 10	\$ $T * F$	\$	reduce by $T \to T * F$
(8)	0 2	\$ T	\$	reduce by $E \to T$
(9)	0 1	\$ E	\$	accept

- Illustrates the actions of a shift-reduce parser on input **id** $*$ **id**, using the LR(0) automaton.

id ∗ id



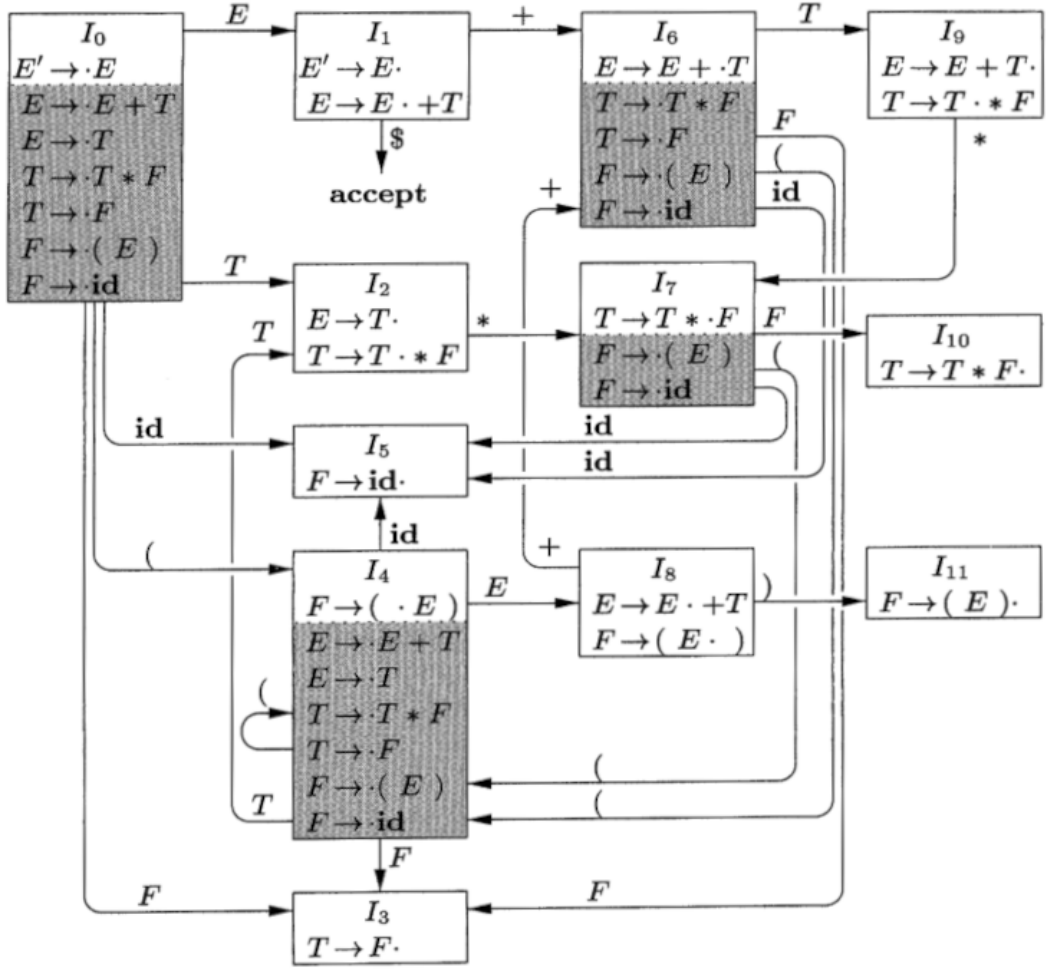
- We use a stack to hold states.
- For clarity, the grammar symbols corresponding to the states on the stack appear in column SYMBOLS.

id $*$ id

LINE	STACK	SYMBOLS	INPUT	ACTION
(1)	0	\$	**id** $*$ **id** \$	shift to 5
(2)	0 5	\$ **id**	$*$ **id** \$	reduce by $F \to$ **id**
(3)	0 3	\$ F	$*$ **id** \$	reduce by $T \to F$
(4)	0 2	\$ T	$*$ **id** \$	shift to 7
(5)	0 2 7	\$ T $*$	**id** \$	shift to 5
(6)	0 2 7 5	\$ T $*$ **id**	\$	reduce by $F \to$ **id**
(7)	0 2 7 10	\$ T $*$ F	\$	reduce by $T \to T * F$
(8)	0 2	\$ T	\$	reduce by $E \to T$
(9)	0 1	\$ E	\$	accept

- We use a stack to hold states.
- For clarity, the grammar symbols corresponding to the states on the stack appear in column SYMBOLS.

id ∗ id



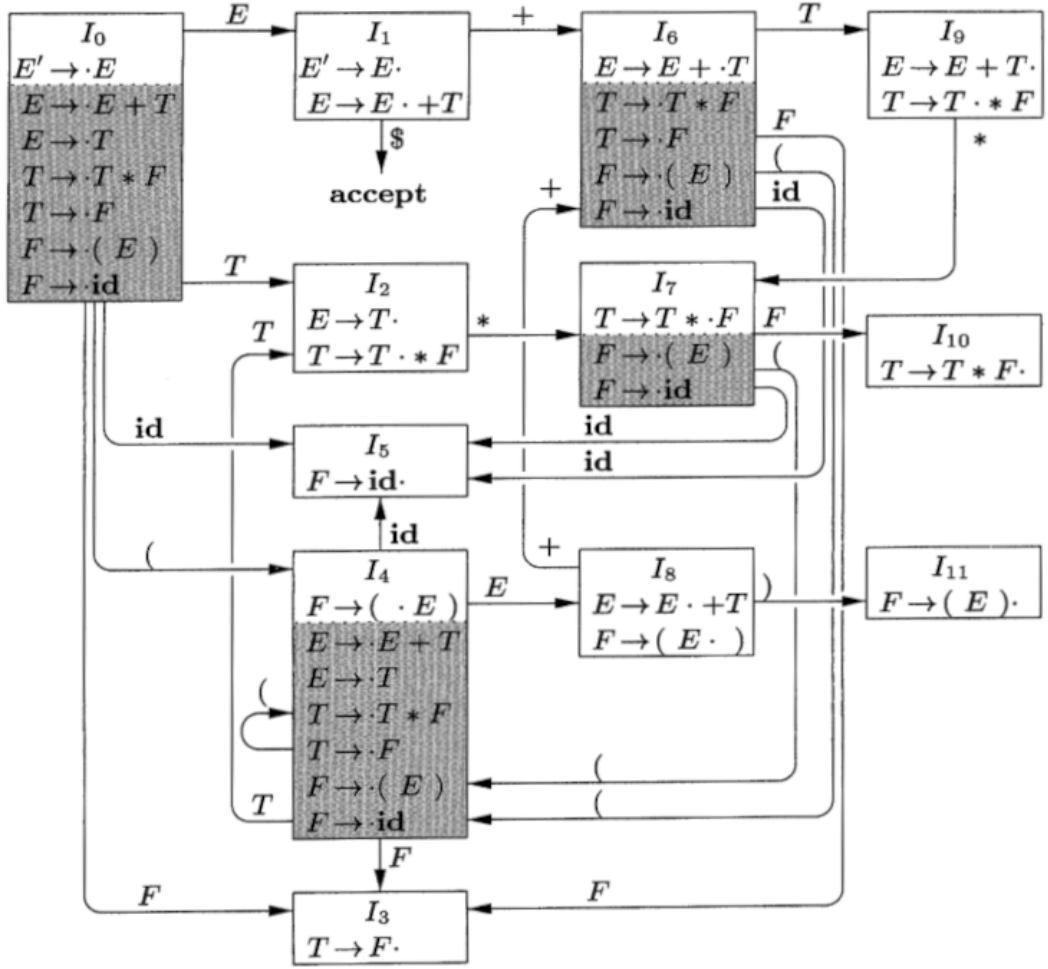
- At line (1), the stack holds the start state 0 of the automaton.
- The corresponding symbol is the bottom-of-stack marker $.

id $*$ id

LINE	STACK	SYMBOLS	INPUT	ACTION
(1)	0	\$	**id** $*$ **id** \$	shift to 5
(2)	0 5	\$ **id**	$*$ **id** \$	reduce by $F \to$ **id**
(3)	0 3	\$ F	$*$ **id** \$	reduce by $T \to F$
(4)	0 2	\$ T	$*$ **id** \$	shift to 7
(5)	0 2 7	\$ T $*$	**id** \$	shift to 5
(6)	0 2 7 5	\$ T $*$ **id**	\$	reduce by $F \to$ **id**
(7)	0 2 7 10	\$ T $*$ F	\$	reduce by $T \to T * F$
(8)	0 2	\$ T	\$	reduce by $E \to T$
(9)	0 1	\$ E	\$	accept

- At line (1), the stack holds the start state 0 of the automaton.
- The corresponding symbol is the bottom-of-stack marker \$.

id ∗ id



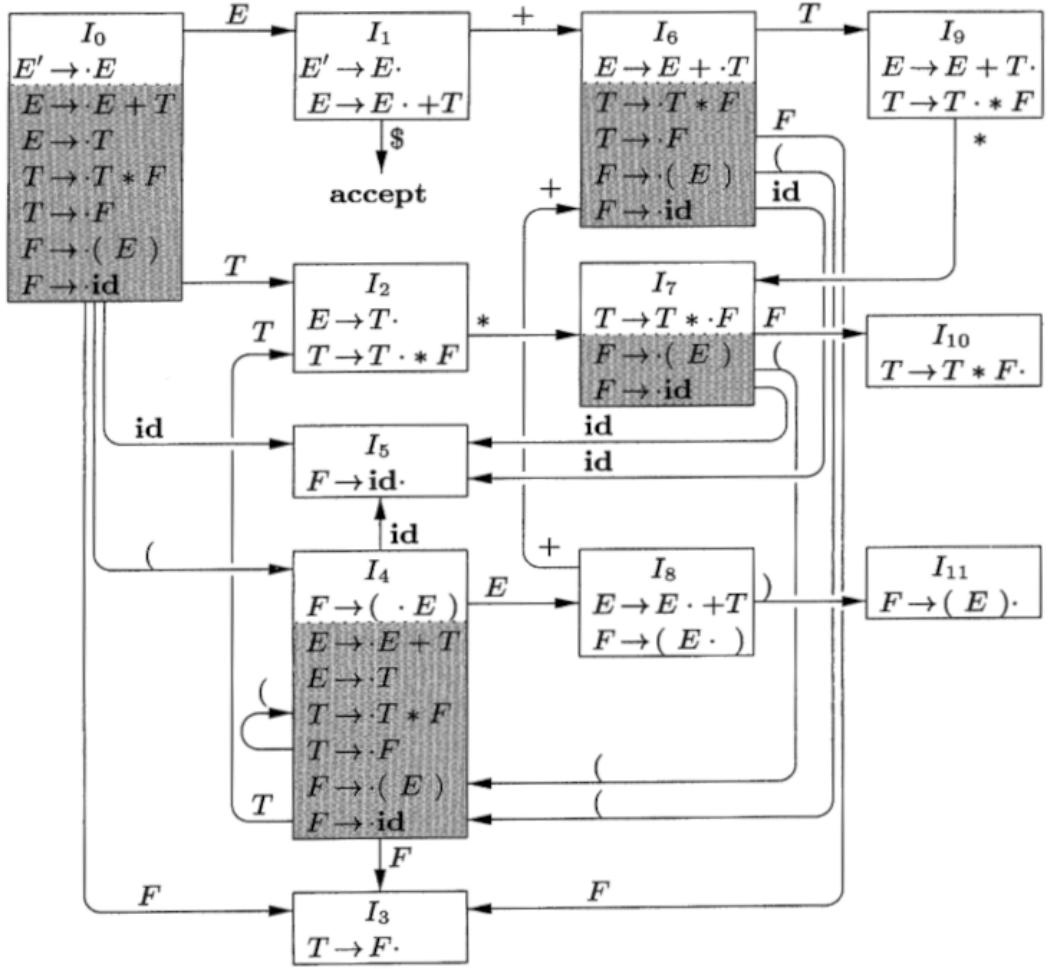
- The next input symbol is **id** and state 0 has a transition on **id** to state 5.
- We therefore shift.

id $*$ id

LINE	STACK	SYMBOLS	INPUT	ACTION
(1)	0	\$	**id** $*$ **id** \$	shift to 5
(2)	0 5	\$ **id**	$*$ **id** \$	reduce by $F \to$ **id**
(3)	0 3	\$ F	$*$ **id** \$	reduce by $T \to F$
(4)	0 2	\$ T	$*$ **id** \$	shift to 7
(5)	0 2 7	\$ T $*$	**id** \$	shift to 5
(6)	0 2 7 5	\$ T $*$ **id**	\$	reduce by $F \to$ **id**
(7)	0 2 7 10	\$ T $*$ F	\$	reduce by $T \to T * F$
(8)	0 2	\$ T	\$	reduce by $E \to T$
(9)	0 1	\$ E	\$	accept

- The next input symbol is **id** and state 0 has a transition on **id** to state 5.
- We therefore shift.

id ∗ id



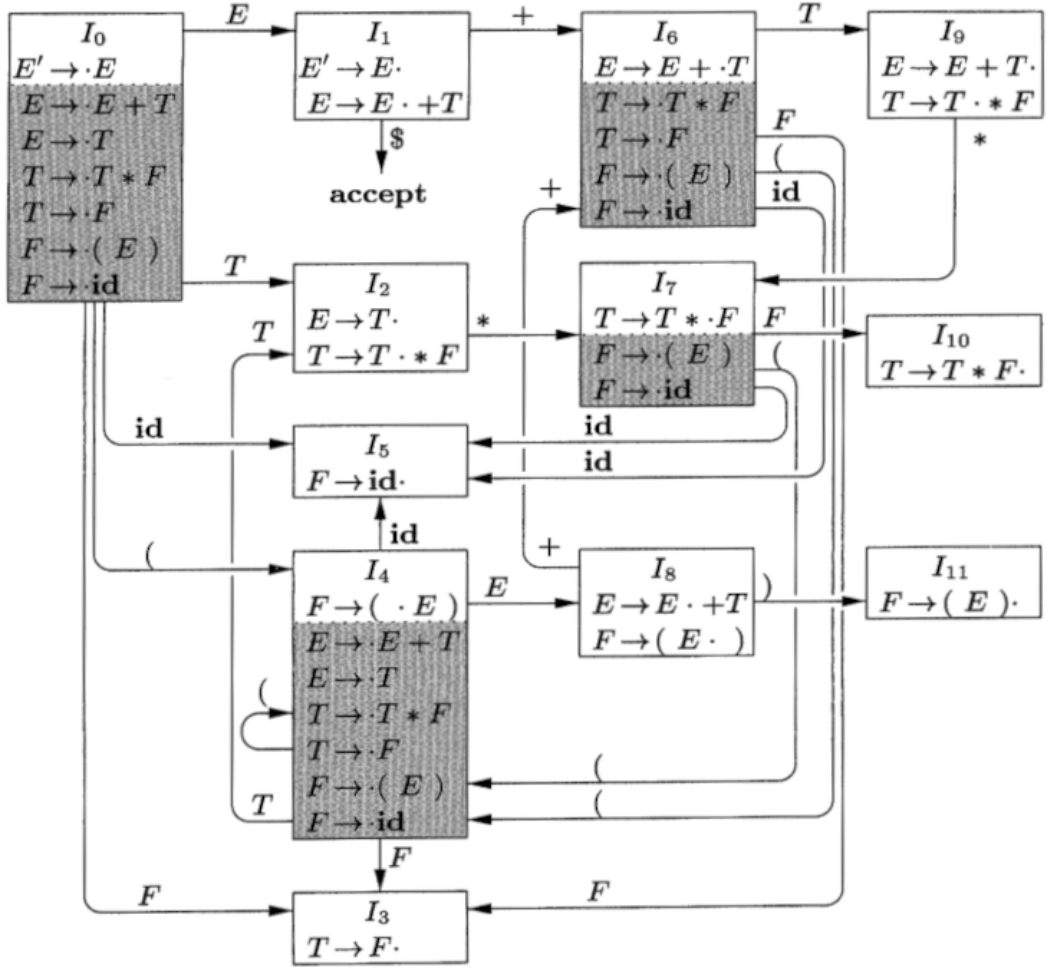
- At line (2), state 5 (symbol **id**) has been pushed onto the stack.

id $*$ id

LINE	STACK	SYMBOLS	INPUT	ACTION
(1)	0	\$	**id** $*$ **id** \$	shift to 5
(2)	0 5	\$ **id**	$*$ **id** \$	reduce by $F \to$ **id**
(3)	0 3	\$ F	$*$ **id** \$	reduce by $T \to F$
(4)	0 2	\$ T	$*$ **id** \$	shift to 7
(5)	0 2 7	\$ T $*$	**id** \$	shift to 5
(6)	0 2 7 5	\$ T $*$ **id**	\$	reduce by $F \to$ **id**
(7)	0 2 7 10	\$ T $*$ F	\$	reduce by $T \to T * F$
(8)	0 2	\$ T	\$	reduce by $E \to T$
(9)	0 1	\$ E	\$	accept

- At line (2), state 5 (symbol **id**) has been pushed onto the stack.

id $*$ id



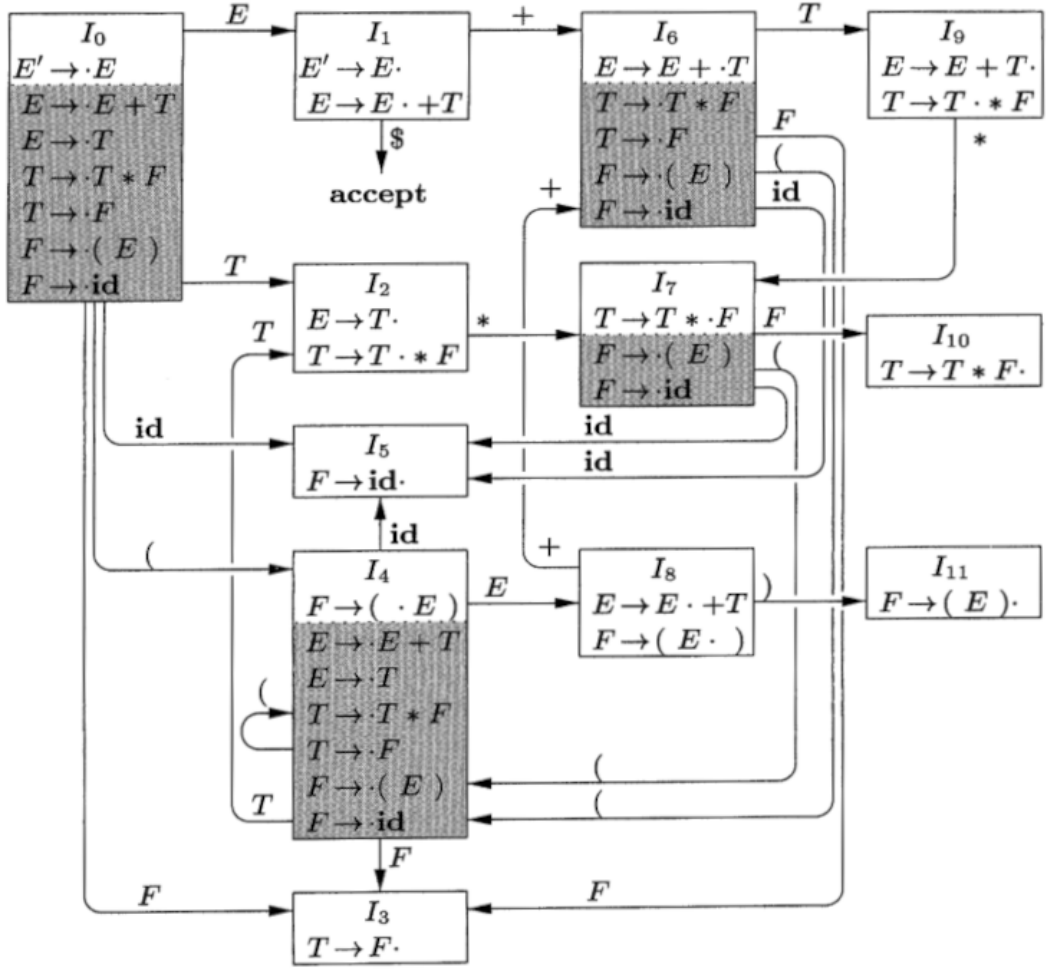
- There is no transition from state 5 on input $*$, so we reduce.
- From item $[F \to \mathbf{id} \cdot\,]$ in state 5, the reduction is by production $F \to \mathbf{id}$.

id $*$ id

LINE	STACK	SYMBOLS	INPUT	ACTION
(1)	0	\$	**id** $*$ **id** \$	shift to 5
(2)	0 5	\$ **id**	$*$ **id** \$	reduce by $F \to$ **id**
(3)	0 3	\$ F	$*$ **id** \$	reduce by $T \to F$
(4)	0 2	\$ T	$*$ **id** \$	shift to 7
(5)	0 2 7	\$ T $*$	**id** \$	shift to 5
(6)	0 2 7 5	\$ T $*$ **id**	\$	reduce by $F \to$ **id**
(7)	0 2 7 10	\$ T $*$ F	\$	reduce by $T \to T * F$
(8)	0 2	\$ T	\$	reduce by $E \to T$
(9)	0 1	\$ E	\$	accept

- There is no transition from state 5 on input $*$, so we reduce.
- From item $[F \to \textbf{id} \cdot]$ in state 5, the reduction is by production $F \to$ **id**.

id ∗ id



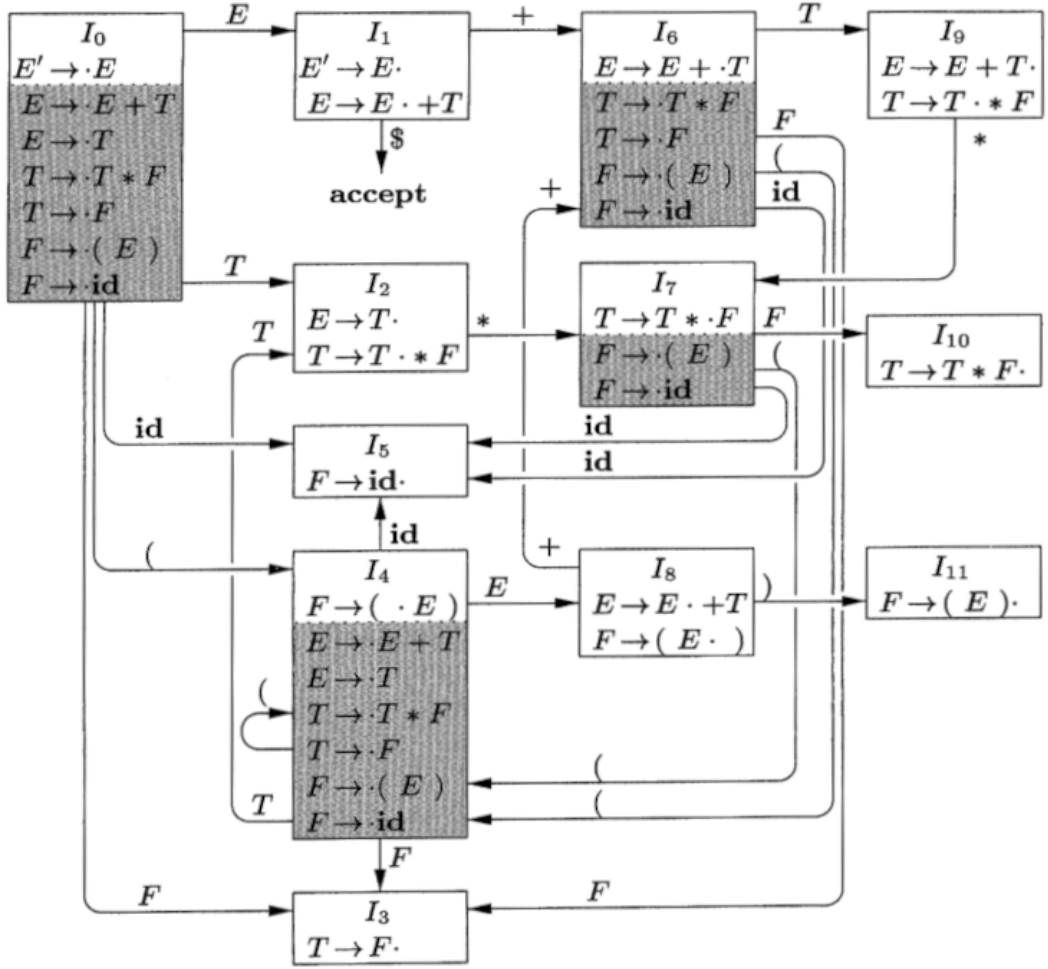
- With symbols, a reduction is implemented by,
  - popping the body of the production from the stack (on line (2), the body is **id**) and
  - pushing the head of the production (in this case, F).

id $*$ id

LINE	STACK	SYMBOLS	INPUT	ACTION
(1)	0	\$	**id** $*$ **id** \$	shift to 5
(2)	0 5	\$ **id**	$*$ **id** \$	reduce by $F \to$ **id**
(3)	0 3	\$ F	$*$ **id** \$	reduce by $T \to F$
(4)	0 2	\$ T	$*$ **id** \$	shift to 7
(5)	0 2 7	\$ $T*$	**id** \$	shift to 5
(6)	0 2 7 5	\$ $T*$ **id**	\$	reduce by $F \to$ **id**
(7)	0 2 7 10	\$ $T*F$	\$	reduce by $T \to T*F$
(8)	0 2	\$ T	\$	reduce by $E \to T$
(9)	0 1	\$ E	\$	accept

- With symbols, a reduction is implemented by,
  - popping the body of the production from the stack (on line (2), the body is **id**) and
  - pushing the head of the production (in this case, F).

id ∗ id



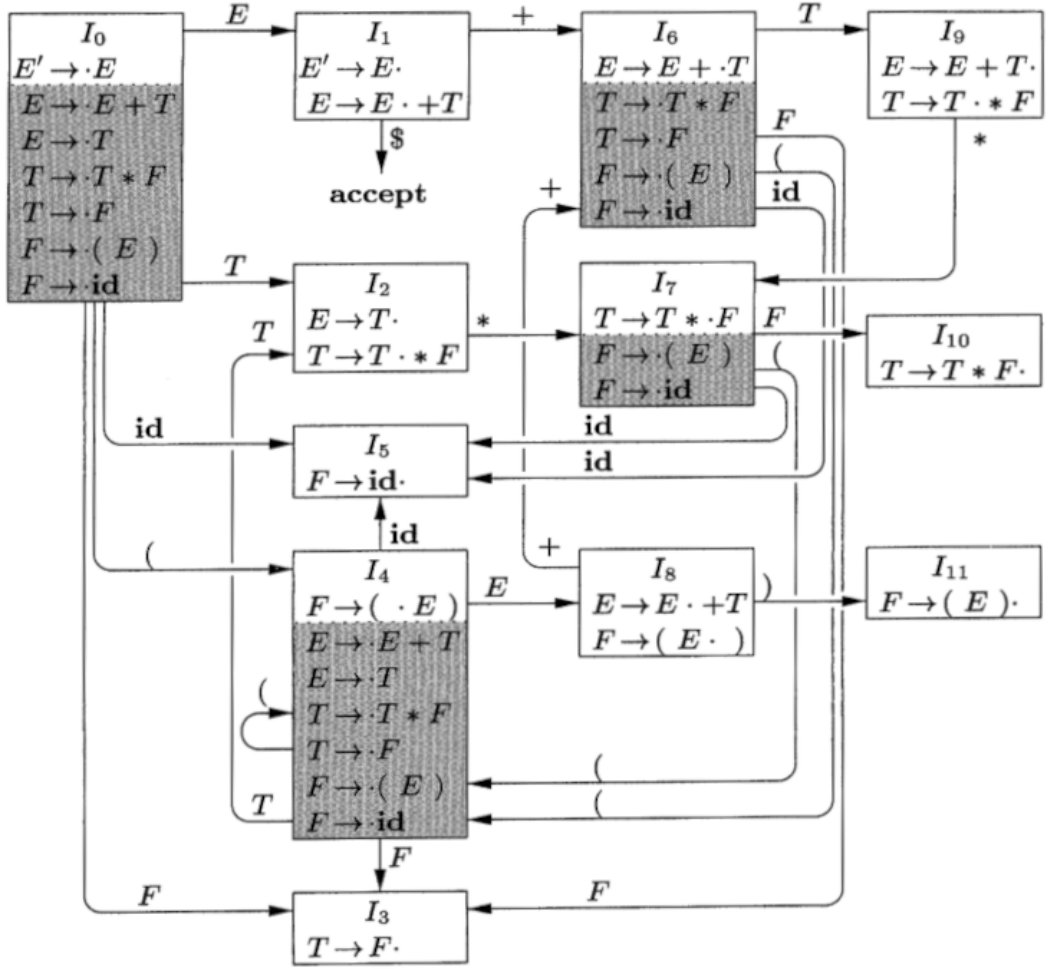
- With states, we pop state 5 for symbol **id**.
- Which brings state 0 to the top and look for a transition on F, the head of the production.

id $*$ id

LINE	STACK	SYMBOLS	INPUT	ACTION
(1)	0	\$	**id** $*$ **id** \$	shift to 5
(2)	0 5	\$ **id**	$*$ **id** \$	reduce by $F \to$ **id**
(3)	0 3	\$ F	$*$ **id** \$	reduce by $T \to F$
(4)	0 2	\$ T	$*$ **id** \$	shift to 7
(5)	0 2 7	\$ $T*$	**id** \$	shift to 5
(6)	0 2 7 5	\$ $T*$ **id**	\$	reduce by $F \to$ **id**
(7)	0 2 7 10	\$ $T*F$	\$	reduce by $T \to T*F$
(8)	0 2	\$ T	\$	reduce by $E \to T$
(9)	0 1	\$ E	\$	accept

- With states, we pop state 5 for symbol **id**.
- Which brings state 0 to the top and look for a transition on F, the head of the production.

id ∗ id



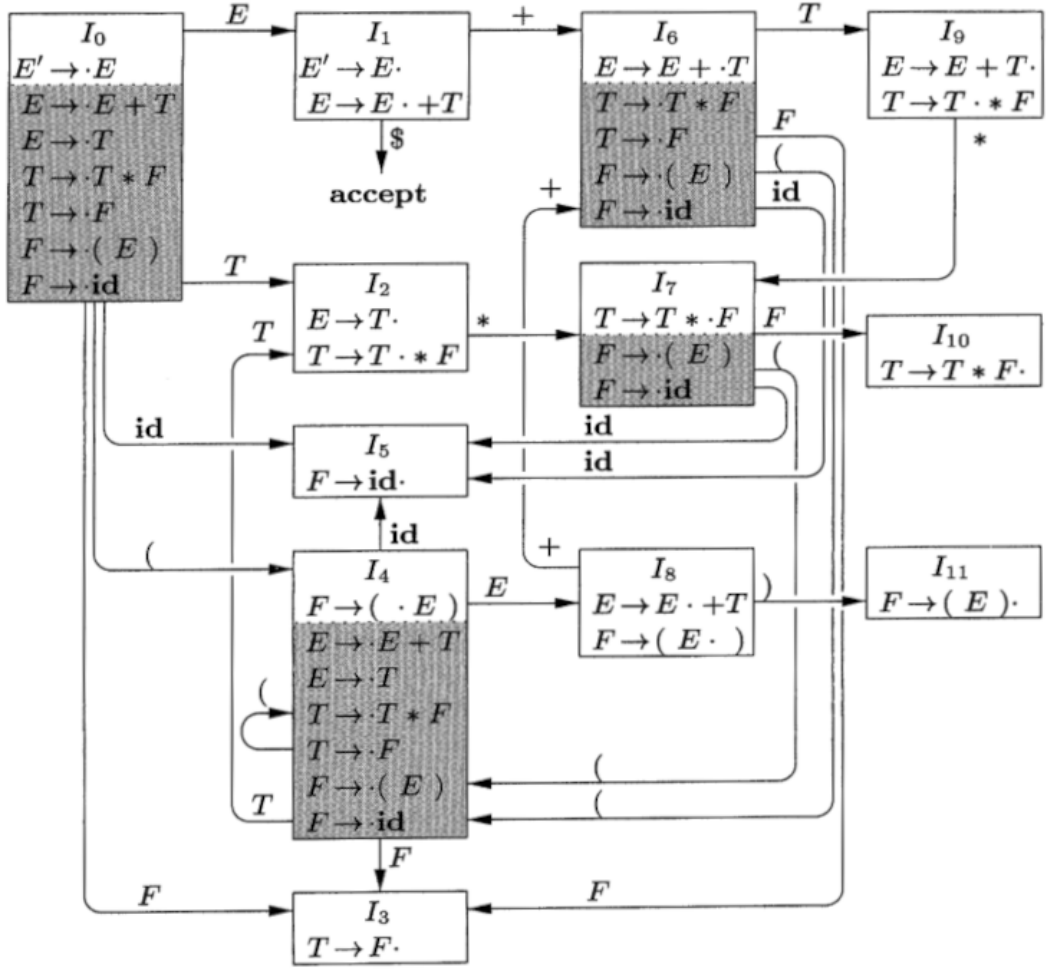
- State 0 has a transition on F to state 3, so we push state 3, with corresponding symbol F; see line (3).

id $*$ id

LINE	STACK	SYMBOLS	INPUT	ACTION
(1)	0	\$	**id** $*$ **id** \$	shift to 5
(2)	0 5	\$ **id**	$*$ **id** \$	reduce by $F \to$ **id**
(3)	0 3	\$ F	$*$ **id** \$	reduce by $T \to F$
(4)	0 2	\$ T	$*$ **id** \$	shift to 7
(5)	0 2 7	\$ T $*$	**id** \$	shift to 5
(6)	0 2 7 5	\$ T $*$ **id**	\$	reduce by $F \to$ **id**
(7)	0 2 7 10	\$ T $*$ F	\$	reduce by $T \to T * F$
(8)	0 2	\$ T	\$	reduce by $E \to T$
(9)	0 1	\$ E	\$	accept

- State 0 has a transition on F to state 3, so we push state 3, with corresponding symbol F; see line (3).

id ∗ id



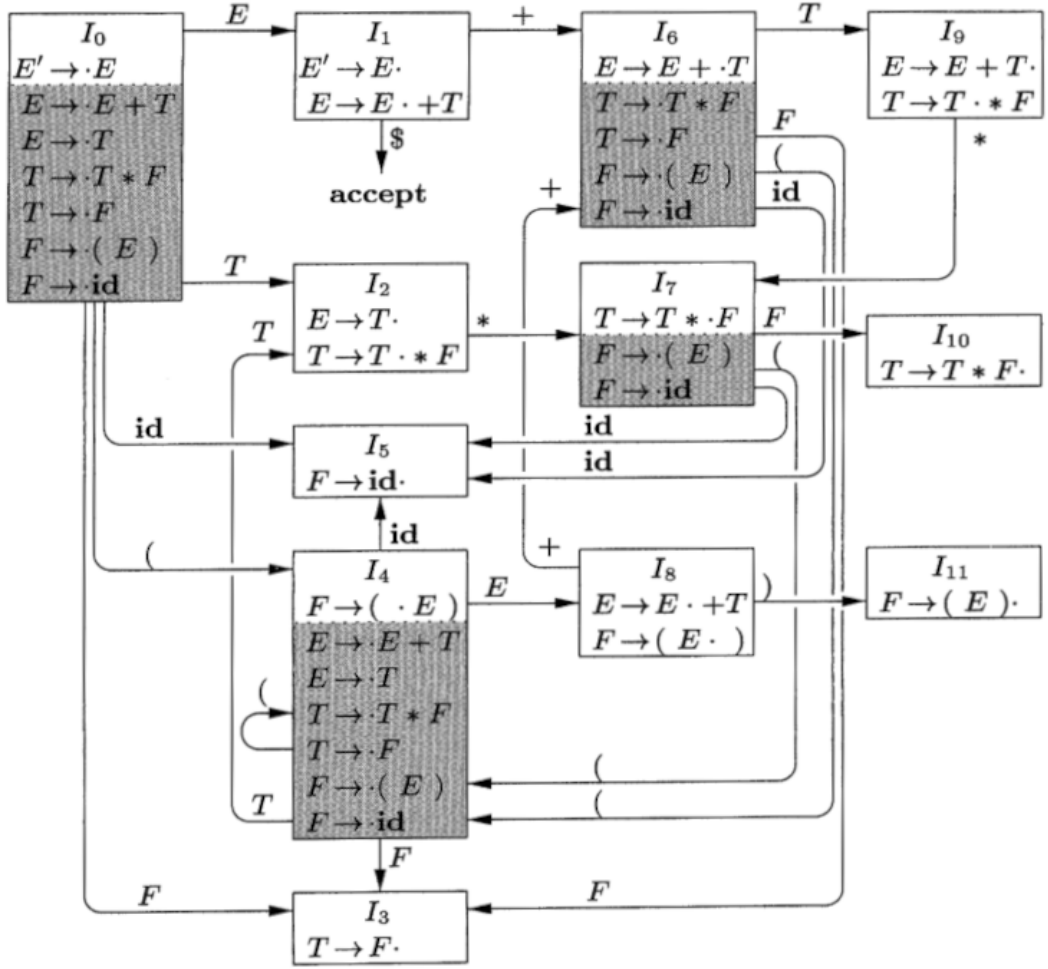
- As another example, consider line (5), with state 7 (symbol $*$) on top of the stack.
- This state has a transition to state 5 on input **id**, so we push state 5 (symbol **id**).

id $*$ id

LINE	STACK	SYMBOLS	INPUT	ACTION
(1)	0	\$	**id** $*$ **id** \$	shift to 5
(2)	0 5	\$ **id**	$*$ **id** \$	reduce by $F \to$ **id**
(3)	0 3	\$ F	$*$ **id** \$	reduce by $T \to F$
(4)	0 2	\$ T	$*$ **id** \$	shift to 7
(5)	0 2 7	\$ $T*$	**id** \$	shift to 5
(6)	0 2 7 5	\$ $T*$ **id**	\$	reduce by $F \to$ **id**
(7)	0 2 7 10	\$ $T*F$	\$	reduce by $T \to T*F$
(8)	0 2	\$ T	\$	reduce by $E \to T$
(9)	0 1	\$ E	\$	accept

- As another example, consider line (5), with state 7 (symbol $*$) on top of the stack.
- This state has a transition to state 5 on input **id**, so we push state 5 (symbol **id**).

id ∗ id



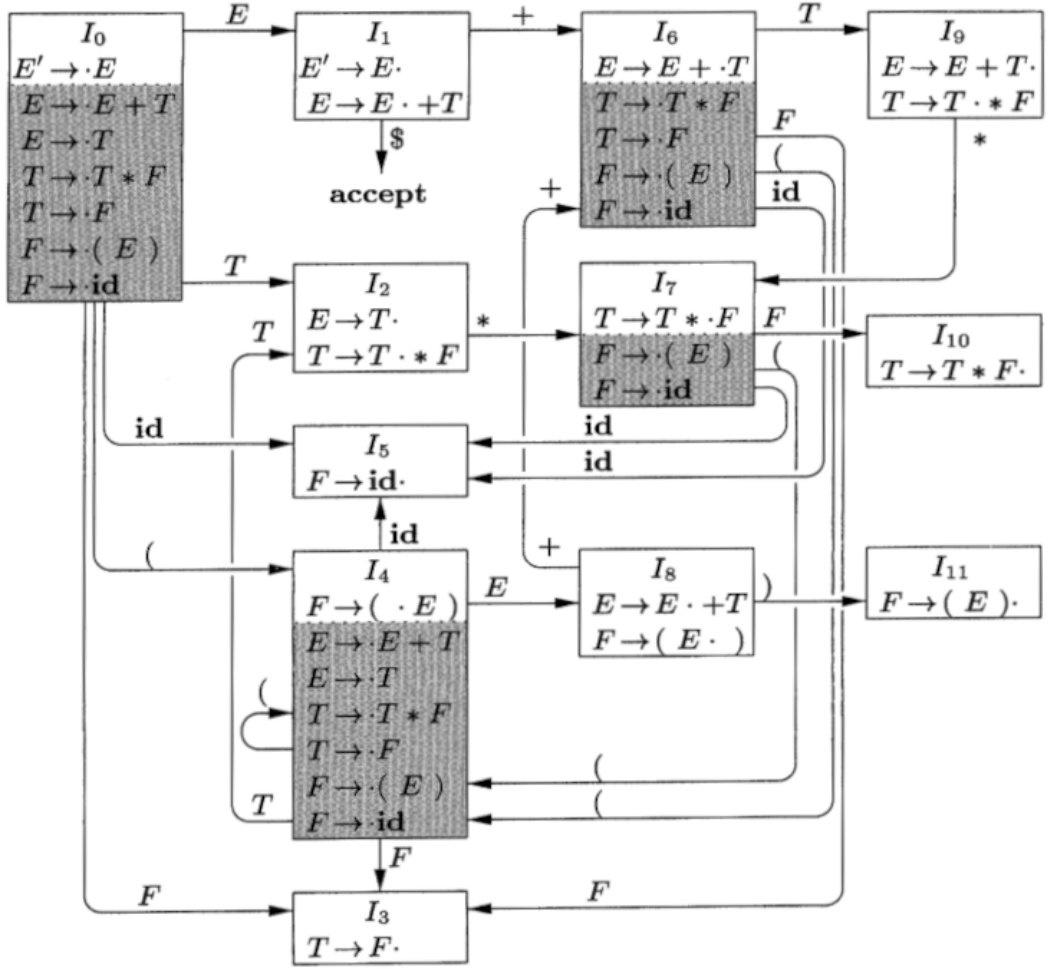
- State 5 has no transitions, so we reduce by $F \to$ **id**.
- When we pop state 5 for the body **id**, state 7 comes to the top of the stack.

id $*$ id

LINE	STACK	SYMBOLS	INPUT	ACTION
(1)	0	\$	**id** $*$ **id** \$	shift to 5
(2)	0 5	\$ **id**	$*$ **id** \$	reduce by $F \to$ **id**
(3)	0 3	\$ F	$*$ **id** \$	reduce by $T \to F$
(4)	0 2	\$ T	$*$ **id** \$	shift to 7
(5)	0 2 7	\$ $T*$	**id** \$	shift to 5
(6)	0 2 7 5	\$ $T*$ **id**	\$	reduce by $F \to$ **id**
(7)	0 2 7 10	\$ $T*F$	\$	reduce by $T \to T*F$
(8)	0 2	\$ T	\$	reduce by $E \to T$
(9)	0 1	\$ E	\$	accept

- State 5 has no transitions, so we reduce by $F \to$ **id**.
- When we pop state 5 for the body **id**, state 7 comes to the top of the stack.

id ∗ id



- Since state 7 has a transition on F to state 10, we push state 10 (symbol F).

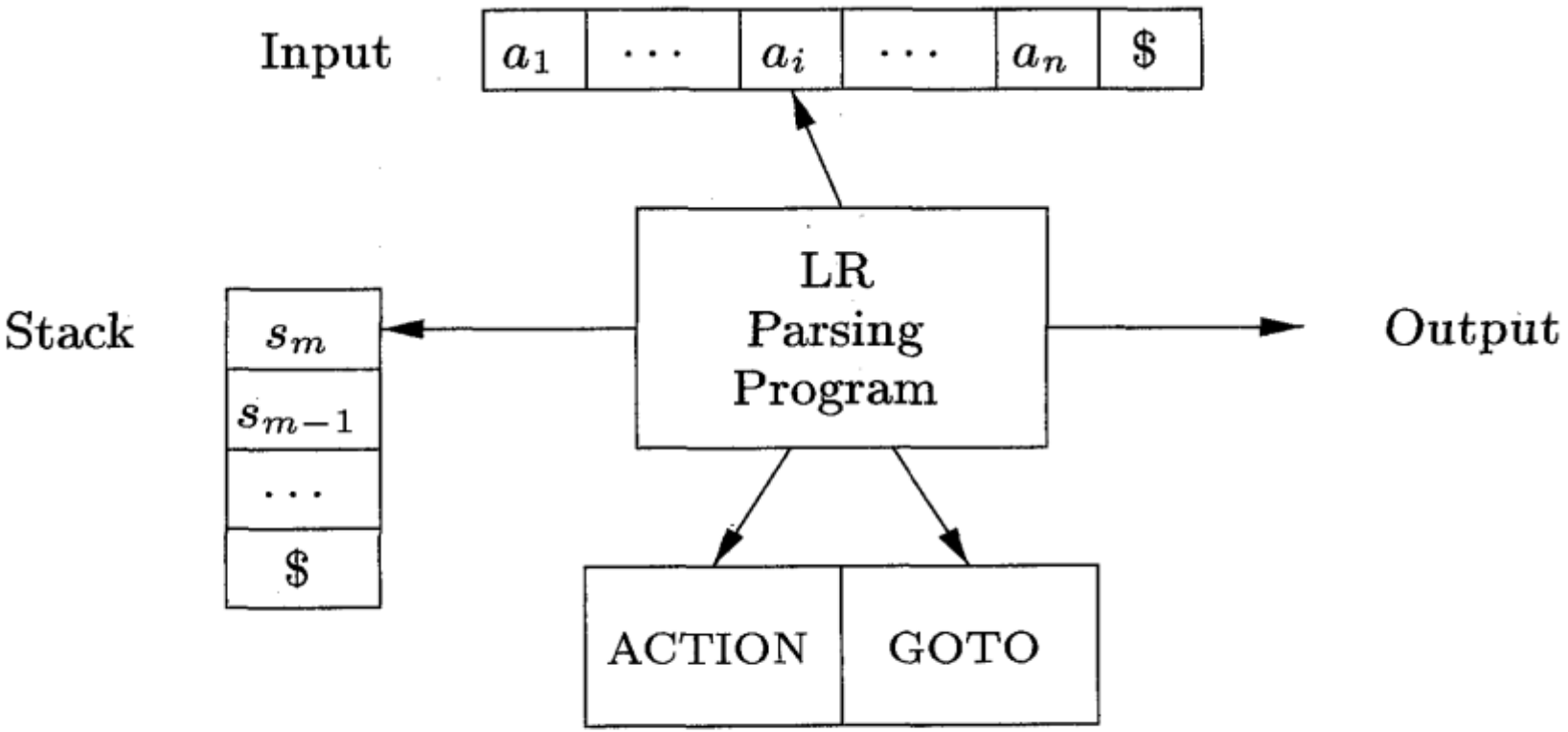
id $*$ id

LINE	STACK	SYMBOLS	INPUT	ACTION
(1)	0	\$	**id** $*$ **id** \$	shift to 5
(2)	0 5	\$ **id**	$*$ **id** \$	reduce by $F \to$ **id**
(3)	0 3	\$ F	$*$ **id** \$	reduce by $T \to F$
(4)	0 2	\$ T	$*$ **id** \$	shift to 7
(5)	0 2 7	\$ $T*$	**id** \$	shift to 5
(6)	0 2 7 5	\$ $T*$ **id**	\$	reduce by $F \to$ **id**
(7)	0 2 7 10	\$ $T*F$	\$	reduce by $T \to T*F$
(8)	0 2	\$ T	\$	reduce by $E \to T$
(9)	0 1	\$ E	\$	accept

- Since state 7 has a transition on F to state 10, we push state 10 (symbol F).

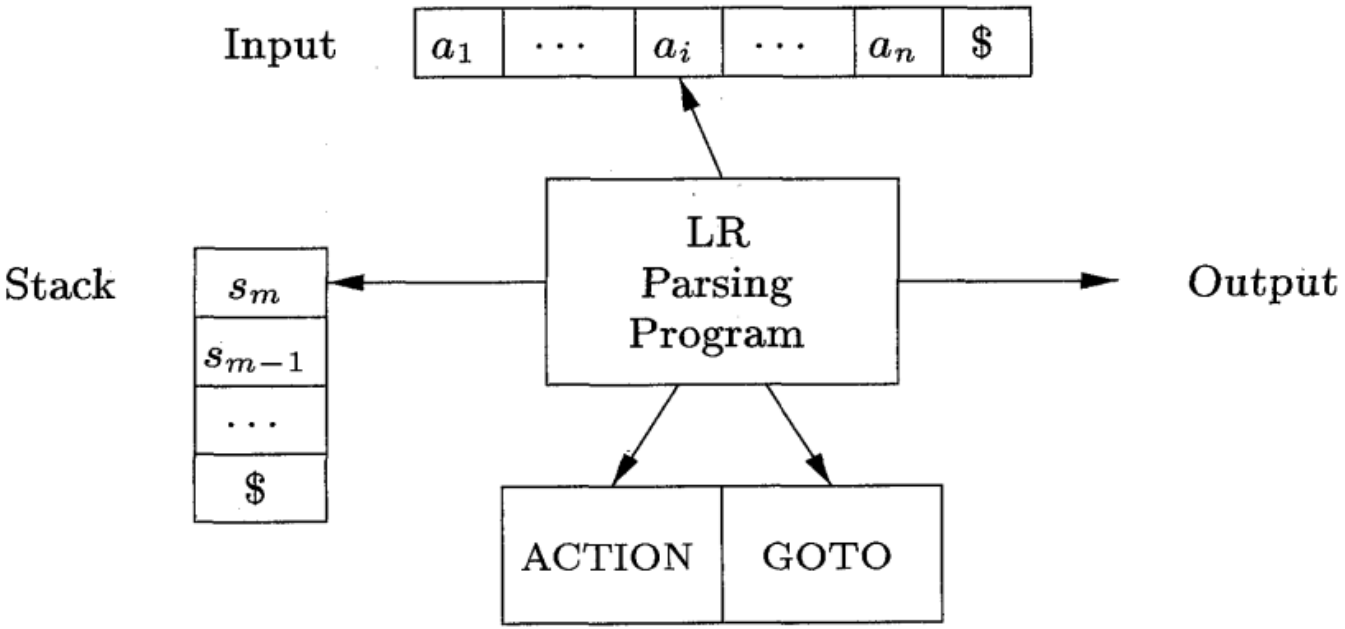
The LR-Parsing Algorithm

- A schematic of an LR parser is shown.



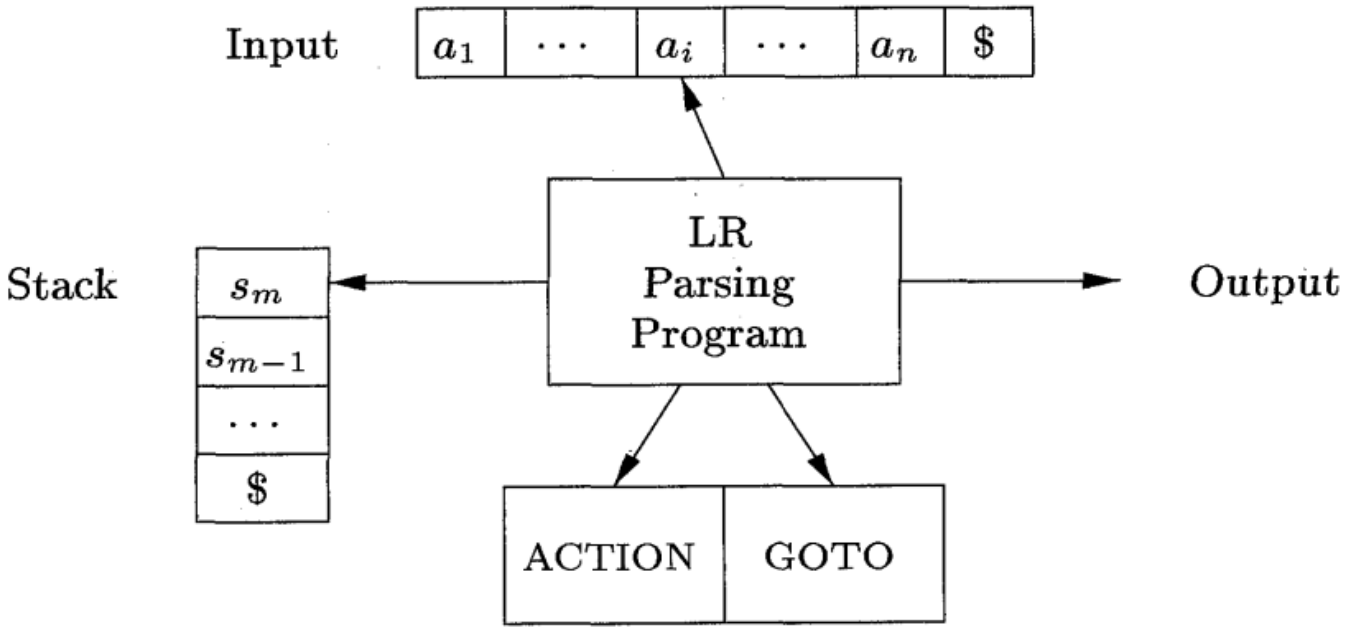
Model of an LR parser

The LR-Parsing Algorithm — *continued*



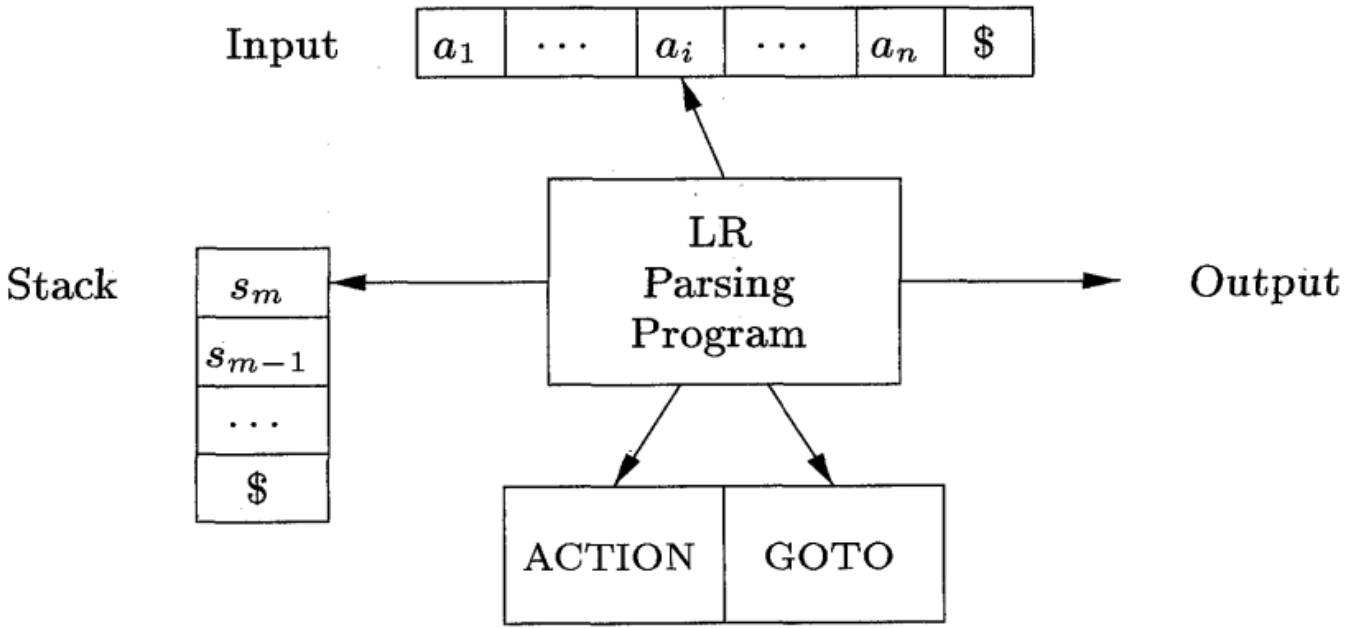
- It consists of an input, an output, a stack, a driver program, and a parsing table that has two parts (ACTION and GOTO).
- The driver program is the same for all LR parsers.
- Only the parsing table changes from one parser to another.

The LR-Parsing Algorithm — *continued*



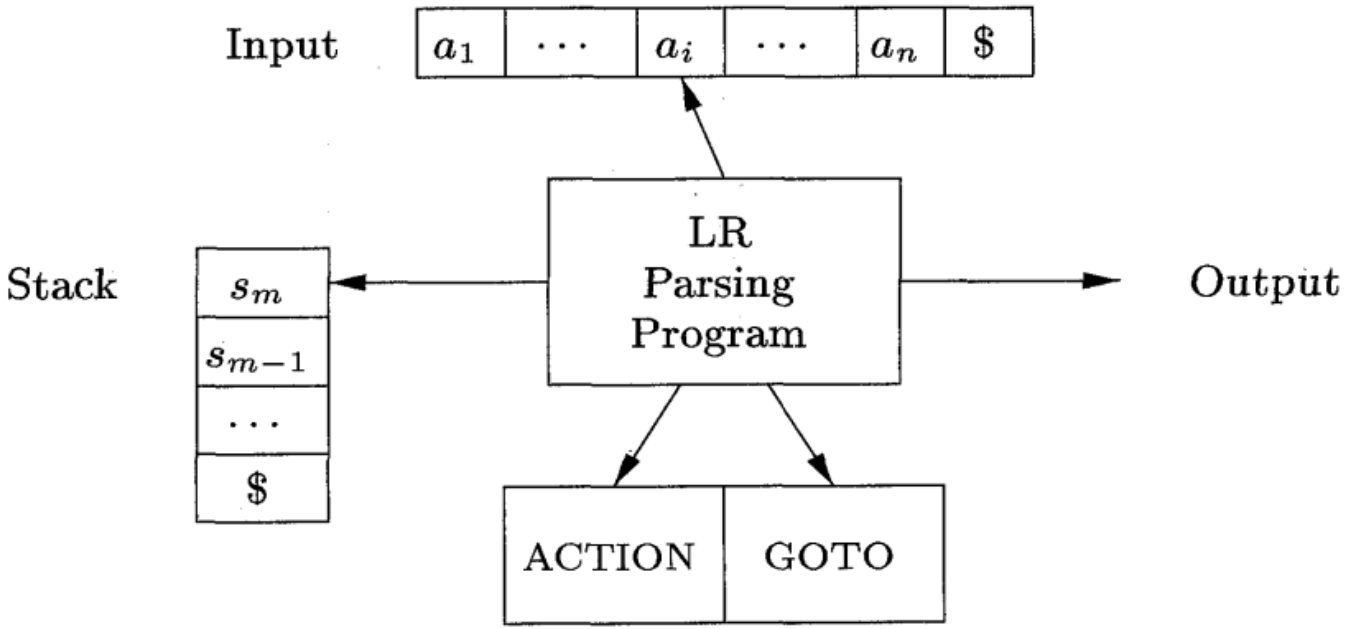
- The parsing program reads characters from an input buffer one at a time.

The LR-Parsing Algorithm — *continued*



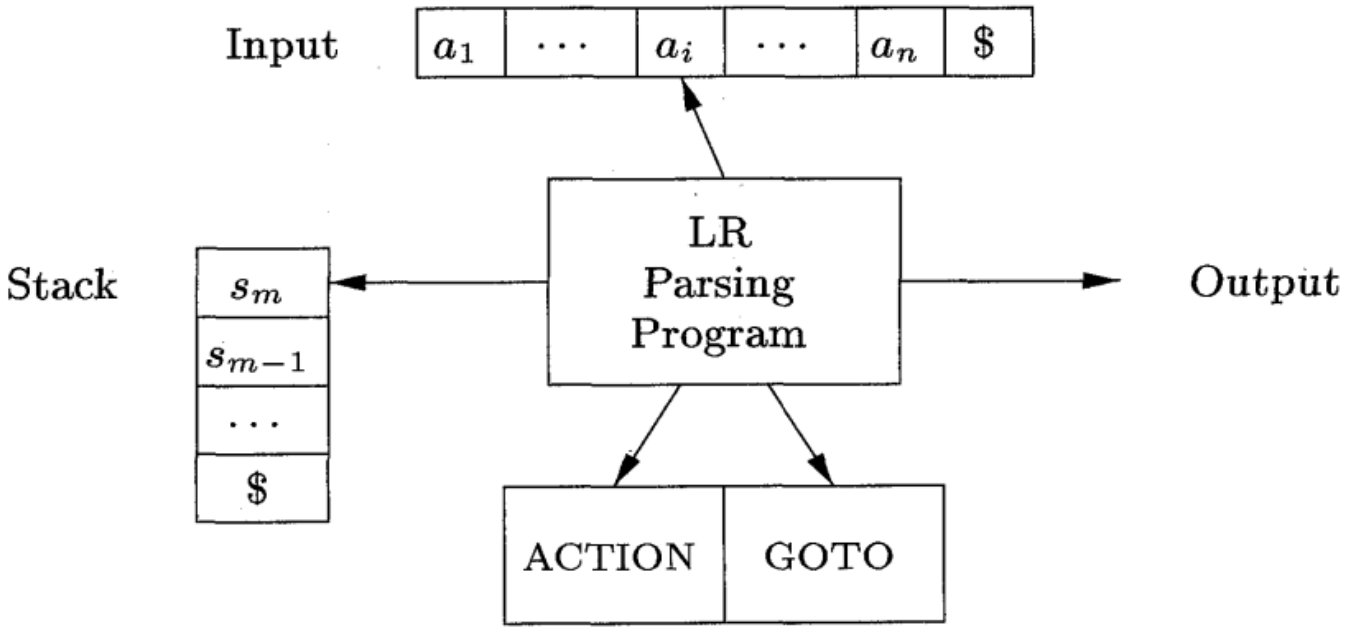
- Where a shift-reduce parser would shift a symbol, an LR parser shifts a state.
- Each state summarizes the information contained in the stack below it.

The LR-Parsing Algorithm — *continued*



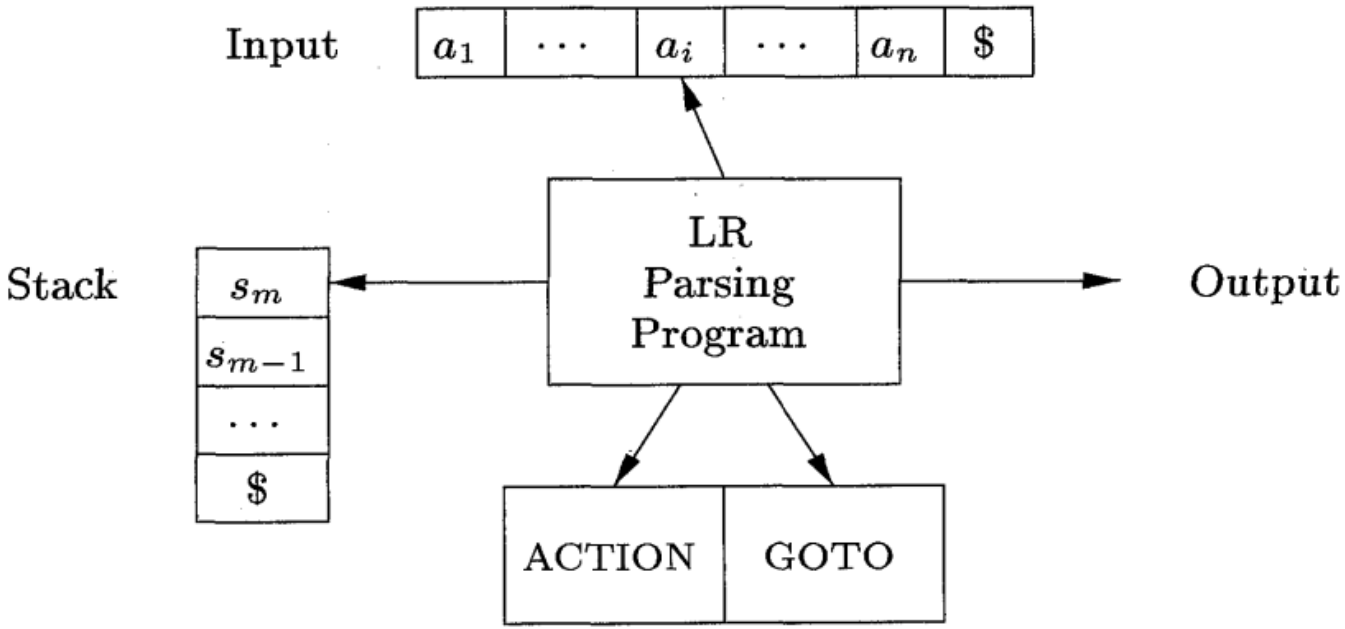
- The stack holds a sequence of states, $s_0 s_1 s_2 \ldots s_m$, where s_m is on top.

The LR-Parsing Algorithm — *continued*



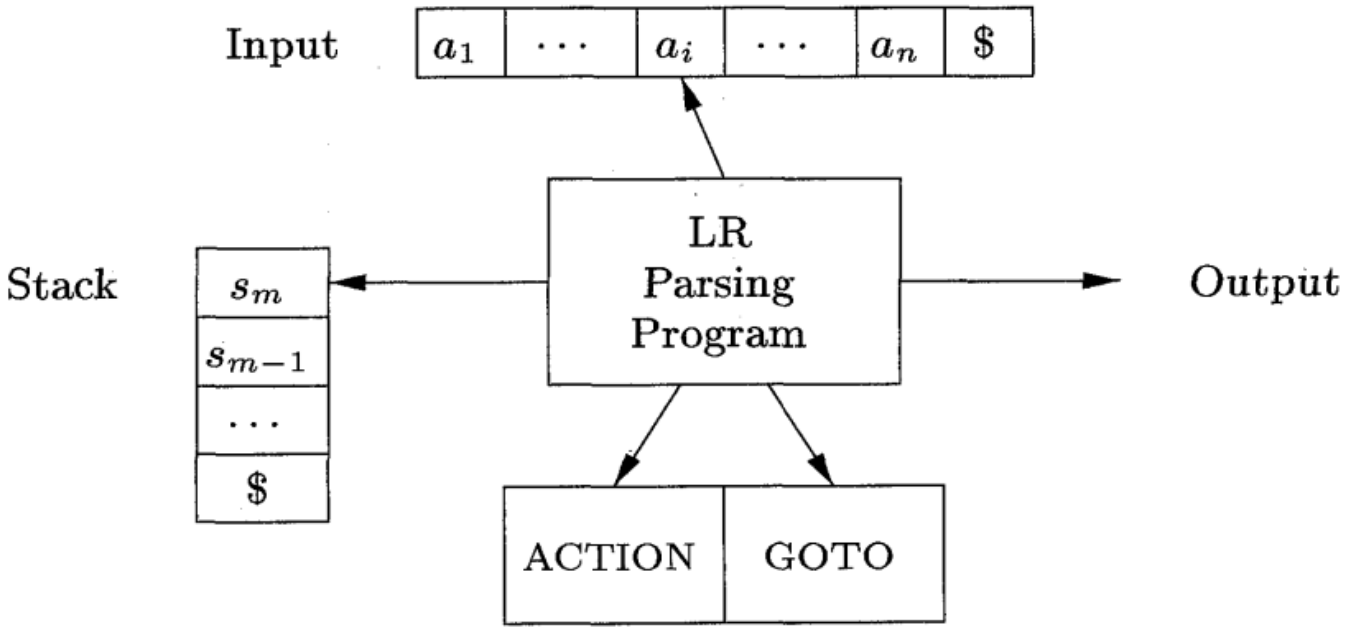
- In the SLR method, the stack holds states from the LR(0) automaton.
- The canonical LR and LALR methods are similar.

The LR-Parsing Algorithm — *continued*



- By construction, each state has a corresponding grammar symbol.
- Recall that states correspond to sets of items.

The LR-Parsing Algorithm — *continued*



- That there is a transition from state i to state j if $\text{GOTO}(I_i, X) = I_j$.
- All transitions to state j must be for the same grammar symbol X.
- Thus, each state, except the start state 0, has a unique grammar symbol associated with it.

The LR-Parsing Algorithm — Structure of the LR Parsing Table

- The parsing table consists of two parts.
- A parsing-action function ACTION.
- A goto function GOTO.

The LR-Parsing Algorithm — Structure of the LR Parsing Table — *continued*

1. The ACTION function takes as arguments a state i and a terminal a (or \$, the input endmarker).

- The value of ACTION$[i, a]$ can have one of four forms:
  (a) Shift j, where j is a state.
  - The action taken by the parser effectively shifts input a to the stack, but uses state j to represent a.

  (b) Reduce $A \to \beta$.
  - The action of the parser effectively reduces β on the top of the stack to head A.

  (c) Accept.
  - The parser accepts the input and finishes parsing.

  (d) Error.
  - The parser discovers an error in its input and takes some corrective action.

The LR-Parsing Algorithm — Structure of the LR Parsing Table — *continued*

1. The ACTION function takes as arguments a state i and a terminal a (or \$, the input endmarker).

- The value of ACTION$[i, a]$ can have one of four forms:
  (a) Shift j, where j is a state.
  - The action taken by the parser effectively shifts input a to the stack, but uses state j to represent a.

  (b) Reduce $A \to \beta$.
  - The action of the parser effectively reduces β on the top of the stack to head A.

  (c) Accept.
  - The parser accepts the input and finishes parsing.

  (d) Error.
  - The parser discovers an error in its input and takes some corrective action.

The LR-Parsing Algorithm — Structure of the LR Parsing Table — *continued*

1. The ACTION function takes as arguments a state i and a terminal a (or \$, the input endmarker).
   - The value of ACTION$[i, a]$ can have one of four forms:
     (a) Shift j, where j is a state.
     - The action taken by the parser effectively shifts input a to the stack, but uses state j to represent a.

     (b) Reduce $A \rightarrow \beta$.
     - The action of the parser effectively reduces β on the top of the stack to head A.

     (c) Accept.
     - The parser accepts the input and finishes parsing.

     (d) Error.
     - The parser discovers an error in its input and takes some corrective action.

The LR-Parsing Algorithm — Structure of the LR Parsing Table — *continued*

1. The ACTION function takes as arguments a state i and a terminal a (or \$, the input endmarker).
   - The value of ACTION$[i, a]$ can have one of four forms:
     (a) Shift j, where j is a state.
     - The action taken by the parser effectively shifts input a to the stack, but uses state j to represent a.

     (b) Reduce $A \rightarrow \beta$.
     - The action of the parser effectively reduces β on the top of the stack to head A.

     (c) Accept.
     - The parser accepts the input and finishes parsing.

     (d) Error.
     - The parser discovers an error in its input and takes some corrective action.

The LR-Parsing Algorithm — Structure of the LR Parsing Table — *continued*

2. We extend the GOTO function, defined on sets of items, to states.
   - If GOTO$[I_i, A] = I_j$, then GOTO also maps a state i and a nonterminal A to state j.

The LR-Parsing Algorithm — LR-Parser Configurations

- To describe the behavior of an LR parser, it helps to have a notation representing the complete state of the parser:
  - its stack and
  - the remaining input.
- A configuration of an LR parser is a pair:

$$\Big(\underbrace{s_0 s_1 s_2 \ldots s_m}_{\text{stack contents}} , \quad \underbrace{a_i a_{i+1} a_{i+2} \ldots a_n \$}_{\text{remaining input}} \Big)$$

- The first component is the stack contents (top on the right).
- The second component is the remaining input.

The LR-Parsing Algorithm — LR-Parser Configurations

- To describe the behavior of an LR parser, it helps to have a notation representing the complete state of the parser:
  - its stack and
  - the remaining input.
- A configuration of an LR parser is a pair:

$$\Big(\underbrace{s_0 s_1 s_2 \ldots s_m}_{\text{stack contents}} , \quad \underbrace{a_i a_{i+1} a_{i+2} \ldots a_n \$}_{\text{remaining input}} \Big)$$

- The first component is the stack contents (top on the right).
- The second component is the remaining input.

The LR-Parsing Algorithm — LR-Parser Configurations

$$(s_0 s_1 s_2 \ldots s_m, \quad a_i a_{i+1} a_{i+2} \ldots a_n \$)$$

- This configuration represents the right-sentential form,

$$X_1 X_2 \ldots X_m a_i a_{i+1} a_{i+2} \ldots a_n \$$$

- This is essentially the same way as a shift-reduce parser would.

The LR-Parsing Algorithm — LR-Parser Configurations

$$(s_0 s_1 s_2 \ldots s_m, \quad a_i a_{i+1} a_{i+2} \ldots a_n \$)$$

- The only difference is that instead of grammar symbols, the stack holds states from which grammar symbols can be recovered.
- That is, X_i is the grammar symbol represented by state s_i.
- s_o, the start state of the parser, does not represent a grammar symbol, and serves as a bottom-of-stack marker, as well as playing an important role in the parse.

The LR-Parsing Algorithm — Behavior of the LR Parser

- The next move of the parser from the configuration above is determined,
  - by reading a_i, the current input symbol,
  - and s_m, the state on top of the stack,
  - and then consulting the entry ACTION$[s_m, a_i]$ in the parsing action table.
- The configurations resulting after each of the four types of move.

The LR-Parsing Algorithm — Behavior of the LR Parser — *continued*

$$(s_0 s_1 s_2 \ldots s_m, a_i a_{i+1} a_{i+2} \ldots a_n \$)$$

1. If ACTION$[s_m, a_i] =$ shift s
   - The parser executes a shift move.
   - It shifts the next state s onto the stack.
   - Enters the configuration,

$$(s_0 s_1 s_2 \ldots s_m s, a_{i+1} a_{i+2} \cdots a_n \$)$$

   - The symbol a_i need not be held on the stack.
   - It can be recovered from s, if needed (which in practice it never is).
   - The current input symbol is now a_{i+1}.

The LR-Parsing Algorithm — Behavior of the LR Parser — *continued*

$$(s_0 s_1 s_2 \ldots s_m, a_i a_{i+1} a_{i+2} \ldots a_n \$)$$

2. If ACTION$[s_m, a_i] =$ reduce $A \to \beta$

- The parser executes a reduce move.
- Enters the configuration,

$$(s_0 s_1 s_2 \ldots s_{m-r} s, a_i a_{i+1} a_{i+2} \ldots a_n \$)$$

- r is the length of β.
- $s =$ GOTO$[s_{m-r}, A]$.

The LR-Parsing Algorithm — Behavior of the LR Parser — *continued*

$$(s_0 s_1 s_2 \ldots s_m, a_i a_{i+1} a_{i+2} \ldots a_n \$)$$

2. If ACTION$[s_m, a_i] =$ reduce $A \to \beta$

- Here the parser first popped r state symbols off the stack, exposing state s_{m-r}.
- The parser then pushed s, the entry for GOTO$[s_{m-r}, A]$, onto the stack.
- The current input symbol is not changed in a reduce move.

The LR-Parsing Algorithm — Behavior of the LR Parser — *continued*

$$(s_0 s_1 s_2 \ldots s_m, a_i a_{i+1} a_{i+2} \ldots a_n \$)$$

2. If ACTION$[s_m, a_i] =$ reduce $A \to \beta$

   - For the LR parsers we shall construct, $X_{m-r+1} \cdots X_m$, the sequence of grammar symbols corresponding to the states popped off the stack, will always match β, the right side of the reducing production.

The LR-Parsing Algorithm — Behavior of the LR Parser — *continued*

$$(s_0 s_1 s_2 \ldots s_m, a_i a_{i+1} a_{i+2} \ldots a_n \$)$$

2. If ACTION$[s_m, a_i] =$ reduce $A \to \beta$

- The output of an LR parser is generated after a reduce move by executing the semantic action associated with the reducing production.
- For the time being, we shall assume the output consists of just printing the reducing production.

The LR-Parsing Algorithm — Behavior of the LR Parser — *continued*

$$(s_0 s_1 s_2 \ldots s_m, a_i a_{i+1} a_{i+2} \ldots a_n \$)$$

3. If ACTION$[s_m, a_i] =$ accept
   - Parsing is completed.

The LR-Parsing Algorithm — Behavior of the LR Parser — *continued*

$$(s_0 s_1 s_2 \ldots s_m, a_i a_{i+1} a_{i+2} \ldots a_n \$)$$

4. If ACTION$[s_m, a_i] =$ error
   - The parser has discovered an error and calls an error recovery routine.

The LR-Parsing Algorithm — Behavior of the LR Parser — *continued*

```
let a be the first symbol of w$;
while(1) { /* repeat forever */
        let s be the state on top of the stack;
        if ( ACTION[s, a] = shift t ) {
                push t onto the stack;
                let a be the next input symbol;
        } else if ( ACTION[s, a] = reduce A → β ) {
                pop |β| symbols off the stack;
                let state t now be on top of the stack;
                push GOTO[t, A] onto the stack;
                output the production A → β;
        } else if ( ACTION[s, a] = accept ) break; /* parsing is done */
        else call error-recovery routine;
}
```

Figure 4.36: LR-parsing program

- The LR-parsing algorithm.

The LR-Parsing Algorithm — Behavior of the LR Parser — *continued*

```
let a be the first symbol of w$;
while(1) { /* repeat forever */
        let s be the state on top of the stack;
        if ( ACTION[s, a] = shift t ) {
                push t onto the stack;
                let a be the next input symbol;
        } else if ( ACTION[s, a] = reduce A → β ) {
                pop |β| symbols off the stack;
                let state t now be on top of the stack;
                push GOTO[t, A] onto the stack;
                output the production A → β;
        } else if ( ACTION[s, a] = accept ) break; /* parsing is done */
        else call error-recovery routine;
}
```

Figure 4.36: LR-parsing program

- All LR parsers behave in this fashion.
- The only difference between one LR parser and another is the information in the ACTION and GOTO fields of the parsing table.

The LR-Parsing Algorithm — Behavior of the LR Parser — *continued*

```
let a be the first symbol of w$;
while(1) { /* repeat forever */
        let s be the state on top of the stack;
        if ( ACTION[s, a] = shift t ) {
                push t onto the stack;
                let a be the next input symbol;
        } else if ( ACTION[s, a] = reduce A → β ) {
                pop |β| symbols off the stack;
                let state t now be on top of the stack;
                push GOTO[t, A] onto the stack;
                output the production A → β;
        } else if ( ACTION[s, a] = accept ) break; /* parsing is done */
        else call error-recovery routine;
}
```

- **INPUT:** An input string w and an LR-parsing table with functions ACTION and GOTO for a grammar G.

The LR-Parsing Algorithm — Behavior of the LR Parser — *continued*

```
let a be the first symbol of w$;
while(1) { /* repeat forever */
        let s be the state on top of the stack;
        if ( ACTION[s, a] = shift t ) {
                push t onto the stack;
                let a be the next input symbol;
        } else if ( ACTION[s, a] = reduce A → β ) {
                pop |β| symbols off the stack;
                let state t now be on top of the stack;
                push GOTO[t, A] onto the stack;
                output the production A → β;
        } else if ( ACTION[s, a] = accept ) break; /* parsing is done */
        else call error-recovery routine;
}
```

- **OUTPUT:** If w is in $L(G)$, the reduction steps of a bottom-up parse for w; otherwise, an error indication.

The LR-Parsing Algorithm — Behavior of the LR Parser — *continued*

```
let a be the first symbol of w$;
while(1) { /* repeat forever */
        let s be the state on top of the stack;
        if ( ACTION[s, a] = shift t ) {
                push t onto the stack;
                let a be the next input symbol;
        } else if ( ACTION[s, a] = reduce A → β ) {
                pop |β| symbols off the stack;
                let state t now be on top of the stack;
                push GOTO[t, A] onto the stack;
                output the production A → β;
        } else if ( ACTION[s, a] = accept ) break; /* parsing is done */
        else call error-recovery routine;
}
```

- **METHOD:** Initially, the parser has s_0 on its stack, where s_0 is the initial state, and w in the input buffer.
- The parser then executes the above program.

Example

STATE	ACTION						GOTO		
	id	+	*	(	)	$	*E*	*T*	*F*
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

Figure 4.37: Parsing table for expression grammar

- The ACTION and GOTO functions of an LR-parsing table for the expression grammar.

Example — *continued*

STATE	ACTION						GOTO		
	id	+	*	(	)	$	*E*	*T*	*F*
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

(1) $E \to E + T$

(2) $E \to T$

(3) $T \to T * F$

(4) $T \to F$

(5) $F \to (E)$

(6) $F \to \mathbf{id}$

- The codes for the actions are:
  1. si means shift and stack state i,
  2. rj means reduce by the production numbered j,
  3. acc means accept,
  4. blank means error.

Example — *continued*

STATE	ACTION						GOTO		
	id	+	*	(	)	$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

(1) $E \to E + T$

(2) $E \to T$

(3) $T \to T * F$

(4) $T \to F$

(5) $F \to (E)$

(6) $F \to$ **id**

- Note that the value of GOTO$[s, a]$ for terminal a is found in the ACTION field connected with the shift action on input a for state s.
- The GOTO field gives GOTO$[s, A]$ for nonterminals A.

Example — *continued*

	STACK	SYMBOLS	INPUT	ACTION
(1)	0		**id** $*$ **id** $+$ **id** \$	shift
(2)	0 5	**id**	$*$ **id** $+$ **id** \$	reduce by $F \to$ **id**
(3)	0 3	F	$*$ **id** $+$ **id** \$	reduce by $T \to F$
(4)	0 2	T	$*$ **id** $+$ **id** \$	shift
(5)	0 2 7	$T*$	**id** $+$ **id** \$	shift
(6)	0 2 7 5	$T*$ **id**	$+$ **id** \$	reduce by $F \to$ **id**
(7)	0 2 7 10	$T*F$	$+$ **id** \$	reduce by $T \to T*F$
(8)	0 2	T	$+$ **id** \$	reduce by $E \to T$
(9)	0 1	E	$+$ **id** \$	shift
(10)	0 1 6	$E+$	**id** \$	shift
(11)	0 1 6 5	$E+$ **id**	\$	reduce by $F \to$ **id**
(12)	0 1 6 3	$E+F$	\$	reduce by $T \to F$
(13)	0 1 6 9	$E+T$	\$	reduce by $E \to E+T$
(14)	0 1	E	\$	accept

Moves of an LR parser on **id** $*$ **id** $+$ **id**

id $*$ id $+$ id

- On input **id** $*$ **id** $+$ **id**, the sequence of stack and input contents is shown.

Example — *continued*

	Stack	Symbols	Input	Action
(1)	0		**id** $*$ **id** $+$ **id** \$	shift
(2)	0 5	**id**	$*$ **id** $+$ **id** \$	reduce by $F \to$ **id**
(3)	0 3	F	$*$ **id** $+$ **id** \$	reduce by $T \to F$
(4)	0 2	T	$*$ **id** $+$ **id** \$	shift
(5)	0 2 7	$T*$	**id** $+$ **id** \$	shift
(6)	0 2 7 5	$T*$ **id**	$+$ **id** \$	reduce by $F \to$ **id**
(7)	0 2 7 10	$T*F$	$+$ **id** \$	reduce by $T \to T*F$
(8)	0 2	T	$+$ **id** \$	reduce by $E \to T$
(9)	0 1	E	$+$ **id** \$	shift
(10)	0 1 6	$E+$	**id** \$	shift
(11)	0 1 6 5	$E+$ **id**	\$	reduce by $F \to$ **id**
(12)	0 1 6 3	$E+F$	\$	reduce by $T \to F$
(13)	0 1 6 9	$E+T$	\$	reduce by $E \to E+T$
(14)	0 1	E	\$	accept

id $*$ id $+$ id

(1) $E \to E+T$

(2) $E \to T$

(3) $T \to T*F$

(4) $T \to F$

(5) $F \to (E)$

(6) $F \to$ **id**

- Also shown for clarity, are the sequences of grammar symbols corresponding to the states held on the stack.
- For example, at line (1) the LR parser is in state 0, the initial state with no grammar symbol, and with **id** the first input symbol.

Example — *continued*

STATE	ACTION						GOTO		
	id	+	*	(	)	$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

id $*$ **id** $+$ **id**

(1) $E \to E + T$

(2) $E \to T$

(3) $T \to T * F$

(4) $T \to F$

(5) $F \to (E)$

(6) $F \to$ **id**

- Also shown for clarity, are the sequences of grammar symbols corresponding to the states held on the stack.
- For example, at line (1) the LR parser is in state 0, the initial state with no grammar symbol, and with **id** the first input symbol.

Example — *continued*

	STACK	SYMBOLS	INPUT	ACTION
(1)	0		**id** $*$ **id** $+$ **id** \$	shift
(2)	0 5	**id**	$*$ **id** $+$ **id** \$	reduce by $F \to$ **id**
(3)	0 3	F	$*$ **id** $+$ **id** \$	reduce by $T \to F$
(4)	0 2	T	$*$ **id** $+$ **id** \$	shift
(5)	0 2 7	$T*$	**id** $+$ **id** \$	shift
(6)	0 2 7 5	$T*$ **id**	$+$ **id** \$	reduce by $F \to$ **id**
(7)	0 2 7 10	$T*F$	$+$ **id** \$	reduce by $T \to T*F$
(8)	0 2	T	$+$ **id** \$	reduce by $E \to T$
(9)	0 1	E	$+$ **id** \$	shift
(10)	0 1 6	$E+$	**id** \$	shift
(11)	0 1 6 5	$E+$ **id**	\$	reduce by $F \to$ **id**
(12)	0 1 6 3	$E+F$	\$	reduce by $T \to F$
(13)	0 1 6 9	$E+T$	\$	reduce by $E \to E+T$
(14)	0 1	E	\$	accept

id $*$ id $+$ id

(1) $E \to E+T$

(2) $E \to T$

(3) $T \to T*F$

(4) $T \to F$

(5) $F \to (E)$

(6) $F \to$ **id**

- The action in row 0 and column **id** of the action field of is $s5$, meaning shift by pushing state 5.

Example — *continued*

STATE	ACTION						GOTO		
	id	+	*	(	)	$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

id $*$ **id** $+$ **id**

(1) $E \to E + T$

(2) $E \to T$

(3) $T \to T * F$

(4) $T \to F$

(5) $F \to (E)$

(6) $F \to$ **id**

- The action in row 0 and column **id** of the action field of is $s5$, meaning shift by pushing state 5.

Example — *continued*

	Stack	Symbols	Input	Action
(1)	0		**id** $*$ **id** $+$ **id** \$	shift
(2)	0 5	**id**	$*$ **id** $+$ **id** \$	reduce by $F \to$ **id**
(3)	0 3	F	$*$ **id** $+$ **id** \$	reduce by $T \to F$
(4)	0 2	T	$*$ **id** $+$ **id** \$	shift
(5)	0 2 7	$T*$	**id** $+$ **id** \$	shift
(6)	0 2 7 5	$T*$ **id**	$+$ **id** \$	reduce by $F \to$ **id**
(7)	0 2 7 10	$T*F$	$+$ **id** \$	reduce by $T \to T*F$
(8)	0 2	T	$+$ **id** \$	reduce by $E \to T$
(9)	0 1	E	$+$ **id** \$	shift
(10)	0 1 6	$E+$	**id** \$	shift
(11)	0 1 6 5	$E+$ **id**	\$	reduce by $F \to$ **id**
(12)	0 1 6 3	$E+F$	\$	reduce by $T \to F$
(13)	0 1 6 9	$E+T$	\$	reduce by $E \to E+T$
(14)	0 1	E	\$	accept

id $*$ **id** $+$ **id**

(1) $E \to E+T$

(2) $E \to T$

(3) $T \to T*F$

(4) $T \to F$

(5) $F \to (E)$

(6) $F \to$ **id**

- That is what has happened at line (2).
- The state symbol 5 has been pushed onto the stack.
- And **id** has been removed from the input.

Example — *continued*

STATE	ACTION						GOTO		
	id	+	*	(	)	$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

id $*$ **id** $+$ **id**

(1) $E \rightarrow E + T$

(2) $E \rightarrow T$

(3) $T \rightarrow T * F$

(4) $T \rightarrow F$

(5) $F \rightarrow (E)$

(6) $F \rightarrow$ **id**

- That is what has happened at line (2).
- The state symbol 5 has been pushed onto the stack.
- And **id** has been removed from the input.

Example — *continued*

	STACK	SYMBOLS	INPUT	ACTION
(1)	0		**id** $*$ **id** $+$ **id** \$	shift
(2)	0 5	**id**	$*$ **id** $+$ **id** \$	reduce by $F \to$ **id**
(3)	0 3	F	$*$ **id** $+$ **id** \$	reduce by $T \to F$
(4)	0 2	T	$*$ **id** $+$ **id** \$	shift
(5)	0 2 7	$T*$	**id** $+$ **id** \$	shift
(6)	0 2 7 5	$T*$ **id**	$+$ **id** \$	reduce by $F \to$ **id**
(7)	0 2 7 10	$T*F$	$+$ **id** \$	reduce by $T \to T*F$
(8)	0 2	T	$+$ **id** \$	reduce by $E \to T$
(9)	0 1	E	$+$ **id** \$	shift
(10)	0 1 6	$E+$	**id** \$	shift
(11)	0 1 6 5	$E+$ **id**	\$	reduce by $F \to$ **id**
(12)	0 1 6 3	$E+F$	\$	reduce by $T \to F$
(13)	0 1 6 9	$E+T$	\$	reduce by $E \to E+T$
(14)	0 1	E	\$	accept

id $*$ **id** $+$ **id**

(1) $E \to E+T$

(2) $E \to T$

(3) $T \to T*F$

(4) $T \to F$

(5) $F \to (E)$

(6) $F \to$ **id**

- Then, $*$ becomes the current input symbol.
- The action of state 5 on input $*$ is to reduce by $F \to$ **id**.

Example — *continued*

STATE	ACTION						GOTO		
	id	+	*	(	)	$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

id $*$ **id** $+$ **id**

(1) $E \to E + T$

(2) $E \to T$

(3) $T \to T * F$

(4) $T \to F$

(5) $F \to (E)$

(6) $F \to$ **id**

- Then, $*$ becomes the current input symbol.
- The action of state 5 on input $*$ is to reduce by $F \to$ **id**.

Example — *continued*

	Stack	Symbols	Input	Action
(1)	0		**id** $*$ **id** $+$ **id** \$	shift
(2)	0 5	**id**	$*$ **id** $+$ **id** \$	reduce by $F \to$ **id**
(3)	0 3	F	$*$ **id** $+$ **id** \$	reduce by $T \to F$
(4)	0 2	T	$*$ **id** $+$ **id** \$	shift
(5)	0 2 7	$T*$	**id** $+$ **id** \$	shift
(6)	0 2 7 5	$T*$ **id**	$+$ **id** \$	reduce by $F \to$ **id**
(7)	0 2 7 10	$T*F$	$+$ **id** \$	reduce by $T \to T*F$
(8)	0 2	T	$+$ **id** \$	reduce by $E \to T$
(9)	0 1	E	$+$ **id** \$	shift
(10)	0 1 6	$E+$	**id** \$	shift
(11)	0 1 6 5	$E+$ **id**	\$	reduce by $F \to$ **id**
(12)	0 1 6 3	$E+F$	\$	reduce by $T \to F$
(13)	0 1 6 9	$E+T$	\$	reduce by $E \to E+T$
(14)	0 1	E	\$	accept

id $*$ id $+$ id

(1) $E \to E + T$

(2) $E \to T$

(3) $T \to T * F$

(4) $T \to F$

(5) $F \to (E)$

(6) $F \to$ **id**

- One state symbol is popped off the stack.
- State 0 is then exposed.
- Since the goto of state 0 on F is 3, state 3 is pushed onto the stack.

Example — *continued*

STATE	ACTION						GOTO		
	id	+	*	(	)	$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

id $*$ **id** $+$ **id**

(1) $E \to E + T$

(2) $E \to T$

(3) $T \to T * F$

(4) $T \to F$

(5) $F \to (E)$

(6) $F \to$ **id**

- One state symbol is popped off the stack.
- State 0 is then exposed.
- Since the goto of state 0 on F is 3, state 3 is pushed onto the stack.

Example — *continued*

	Stack	Symbols	Input	Action
(1)	0		**id** $*$ **id** $+$ **id** \$	shift
(2)	0 5	**id**	$*$ **id** $+$ **id** \$	reduce by $F \to$ **id**
(3)	0 3	F	$*$ **id** $+$ **id** \$	reduce by $T \to F$
(4)	0 2	T	$*$ **id** $+$ **id** \$	shift
(5)	0 2 7	$T*$	**id** $+$ **id** \$	shift
(6)	0 2 7 5	$T*$ **id**	$+$ **id** \$	reduce by $F \to$ **id**
(7)	0 2 7 10	$T*F$	$+$ **id** \$	reduce by $T \to T*F$
(8)	0 2	T	$+$ **id** \$	reduce by $E \to T$
(9)	0 1	E	$+$ **id** \$	shift
(10)	0 1 6	$E+$	**id** \$	shift
(11)	0 1 6 5	$E+$ **id**	\$	reduce by $F \to$ **id**
(12)	0 1 6 3	$E+F$	\$	reduce by $T \to F$
(13)	0 1 6 9	$E+T$	\$	reduce by $E \to E+T$
(14)	0 1	E	\$	accept

id $*$ id $+$ id

(1) $E \to E + T$

(2) $E \to T$

(3) $T \to T * F$

(4) $T \to F$

(5) $F \to (E)$

(6) $F \to$ **id**

- We now have the configuration in line (3).
- Each of the remaining moves is determined similarly.

Example — *continued*

STATE	ACTION						GOTO		
	id	+	*	(	)	$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

id $*$ **id** $+$ **id**

(1) $E \to E + T$

(2) $E \to T$

(3) $T \to T * F$

(4) $T \to F$

(5) $F \to (E)$

(6) $F \to$ **id**

- We now have the configuration in line (3).
- Each of the remaining moves is determined similarly.

Constructing SLR-Parsing Tables

- The SLR method for constructing parsing tables is a good starting point for studying LR parsing.
- We shall refer to the parsing table constructed by this method as an SLR table.
- An LR parser using an SLR-parsing table is an SLR parser.
- The other two methods augment the SLR method with lookahead information.

Constructing SLR-Parsing Tables

- The SLR method for constructing parsing tables is a good starting point for studying LR parsing.
- We shall refer to the parsing table constructed by this method as an SLR table.
- An LR parser using an SLR-parsing table is an SLR parser.
- The other two methods augment the SLR method with lookahead information.

Constructing SLR-Parsing Tables — *continued*

- The SLR method begins with LR(0) items and LR(0) automata.
- That is, given a grammar, G, we augment G to produce G', with a new start symbol S'.
- From G', we construct C, the canonical collection of sets of items for G' together with the GOTO function.

Constructing SLR-Parsing Tables — *continued*

- The ACTION and GOTO entries in the parsing table are then constructed using the following algorithm.
- It requires us to know FOLLOW(A) for each nonterminal A of a grammar.

Constructing SLR-Parsing Tables — *continued*

Algorithm: Constructing an SLR-parsing table

- **INPUT:** An augmented grammar G'.
- **OUTPUT:** The SLR-parsing table functions ACTION and GOTO for G'.

Constructing SLR-Parsing Tables — *continued*

Algorithm: Constructing an SLR-parsing table

METHOD:

1. Construct $C = \{I_0, I_1, I_2, \ldots, I_n\}$, the collection of sets of LR(0) items for G'.

Constructing SLR-Parsing Tables — *continued*

Algorithm: Constructing an SLR-parsing table

METHOD:

2. State i is constructed from I_i.

- The parsing actions for state i are determined as follows:
  - (a) If $[A \to \alpha \cdot a\beta]$ is in I_i, and $\text{GOTO}(I_i, a) = I_j$, then set $\text{ACTION}[i, a]$ to "shift j."
    - Here a must be a terminal.
  - (b) If $[A \to \alpha \cdot\,]$ is in I_i, then set $\text{ACTION}[i, a]$ to "reduce $A \to \alpha$" for all a in $\text{FOLLOW}(A)$.
    - Here A may not be S'.
  - (c) If $[S' \to S \cdot\,]$ is in I_i, then set $\text{ACTION}[i, \$]$ to "accept."
- If any conflicting actions result from the above rules, we say the grammar is not SLR(1).
  - The algorithm fails to produce a parser in this case.

Constructing SLR-Parsing Tables — *continued*

Algorithm: Constructing an SLR-parsing table

METHOD:

3. The goto transitions for state i are constructed for all nonterminals A using the rule: If $\text{GOTO}(I_i, A) = I_j$, then $\text{GOTO}[i, A] = j$.

Constructing SLR-Parsing Tables — *continued*

Algorithm: Constructing an SLR-parsing table

METHOD:

4. All entries not defined by rules (2) and (3) are made “error.”

Constructing SLR-Parsing Tables — *continued*

Algorithm: Constructing an SLR-parsing table

METHOD:

5. The initial state of the parser is the one constructed from the set of items containing $[S' \rightarrow \cdot S]$.

Constructing SLR-Parsing Tables — *continued*

- The parsing table consisting of the ACTION and GOTO functions determined by the previous algorithm is called the SLR(1) table for G.
- An LR parser using the SLR(1) table for G is called the SLR(1) parser for G.
- And a grammar having an SLR(1) parsing table is said to be SLR(1).
- We usually omit the "(1)" after the "SLR," since we shall not deal here with parsers having more than one symbol of lookahead.

Example

- Let us construct the SLR table for the augmented expression grammar.

$$
\begin{aligned}
E' &\rightarrow E \\
E &\rightarrow E + T \mid T \\
T &\rightarrow T * F \mid F \\
F &\rightarrow (E) \mid \textbf{id}
\end{aligned}
$$

(1) $E \rightarrow E + T$

(2) $E \rightarrow T$

(3) $T \rightarrow T * F$

(4) $T \rightarrow F$

(5) $F \rightarrow (E)$

(6) $F \rightarrow \textbf{id}$

Example — *continued*

$$
\begin{aligned}
E' &\rightarrow E \\
E &\rightarrow E + T \mid T \\
T &\rightarrow T * F \mid F \\
F &\rightarrow (E) \mid \textbf{id}
\end{aligned}
$$

(1) $E \rightarrow E + T$

(2) $E \rightarrow T$

(3) $T \rightarrow T * F$

(4) $T \rightarrow F$

(5) $F \rightarrow (E)$

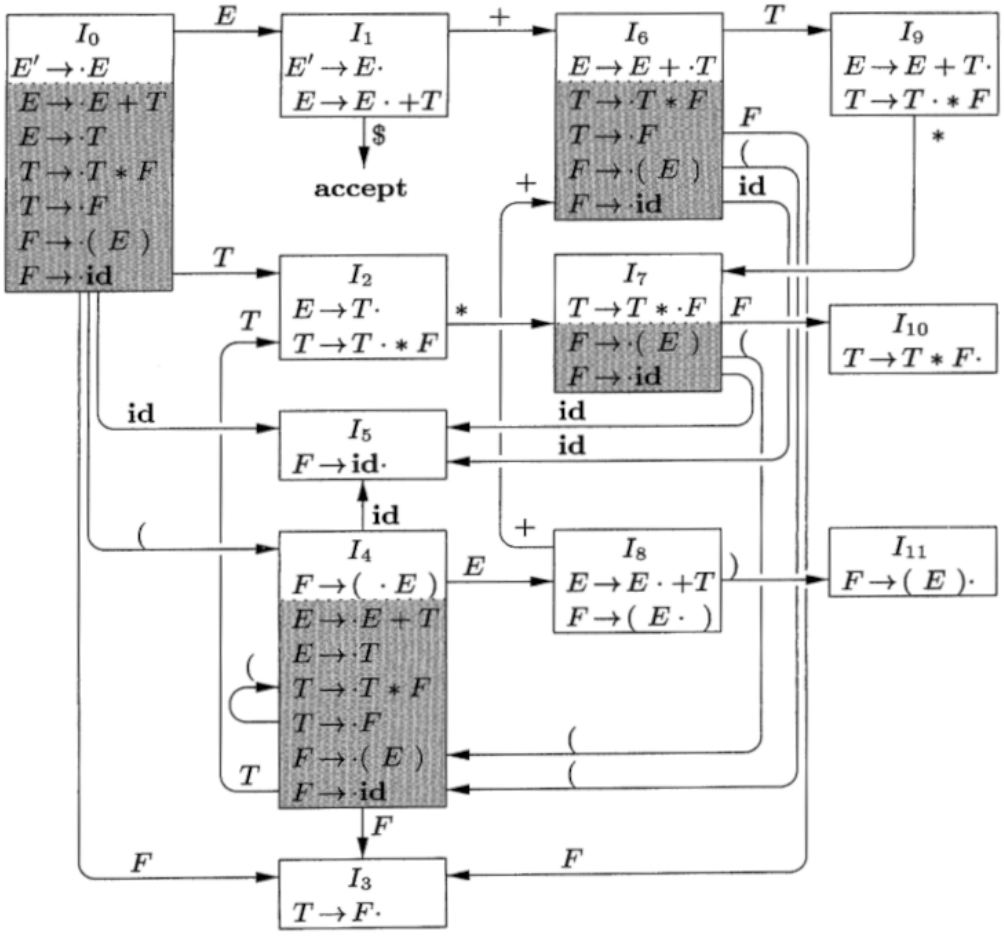
(6) $F \rightarrow \textbf{id}$

- First consider the set of items I_0.

2. State i is constructed from I_i.

- The parsing actions for state i are determined as follows:
  - (a) If $[A \to \alpha \cdot a\beta]$ is in I_i, and GOTO$(I_i, a) = I_j$, then set ACTION$[i, a]$ to "shift j."
    - Here a must be a terminal.
  - (b) If $[A \to \alpha \cdot\,]$ is in I_i, then set ACTION$[i, a]$ to "reduce $A \to \alpha$" for all a in FOLLOW(A).
    - Here A may not be S'.
  - (c) If $[S' \to S \cdot\,]$ is in I_i, then set ACTION$[i, \$]$ to "accept."
- If any conflicting actions result from the above rules, we say the grammar is not SLR(1).
  - The algorithm fails to produce a parser in this case.

- The item $F \to \cdot(E)$ gives rise to the entry ACTION$[0, (] =$ shift 4.
- And the item $F \to \cdot$**id** to the entry ACTION$[0,$ **id**$] =$ shift 5.
- Other items in I_0 yield no actions.



(1) $E \to E + T$

(2) $E \to T$

(3) $T \to T * F$

(4) $T \to F$

(5) $F \to (E)$

(6) $F \to \mathbf{id}$

- The item $F \to \cdot(E)$ gives rise to the entry ACTION$[0, ($] = shift 4.
- And the item $F \to \cdot\mathbf{id}$ to the entry ACTION$[0, \mathbf{id}] =$ shift 5.
- Other items in I_0 yield no actions.

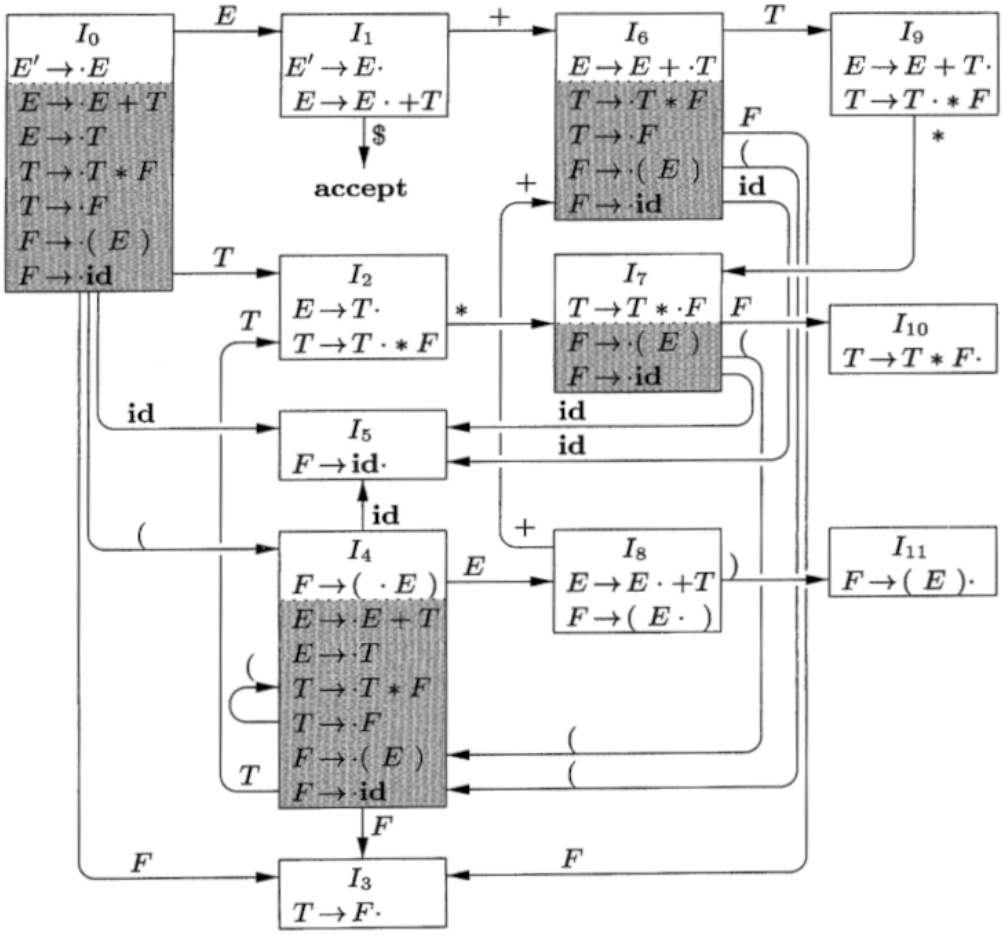
STATE	ACTION						GOTO		
	id	+	*	(	)	$	*E*	*T*	*F*
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

- The item $F \to \cdot(E)$ gives rise to the entry ACTION$[0, (] =$ shift 4.
- And the item $F \to \cdot$**id** to the entry ACTION$[0,$ **id**$] =$ shift 5.
- Other items in I_0 yield no actions.

2. State i is constructed from I_i.

- The parsing actions for state i are determined as follows:
  - (a) If $[A \to \alpha \cdot a\beta]$ is in I_i, and GOTO$(I_i, a) = I_j$, then set ACTION$[i, a]$ to "shift j."
    - Here a must be a terminal.
  - (b) If $[A \to \alpha \cdot\,]$ is in I_i, then set ACTION$[i, a]$ to "reduce $A \to \alpha$" for all a in FOLLOW(A).
    - Here A may not be S'.
  - (c) If $[S' \to S \cdot\,]$ is in I_i, then set ACTION$[i, \$]$ to "accept."
- If any conflicting actions result from the above rules, we say the grammar is not SLR(1).
  - The algorithm fails to produce a parser in this case.

- Now consider I_1, $E' \to E \cdot$, $E \to E \cdot + T$.
- The first item yields ACTION$[1, \$] =$ accept.
- And the second yields ACTION$[1, +] =$ shift 6.



(1) $E \to E + T$

(2) $E \to T$

(3) $T \to T * F$

(4) $T \to F$

(5) $F \to (E)$

(6) $F \to \mathbf{id}$

- Now consider I_1, $E' \to E \cdot$, $E \to E \cdot + T$.
- The first item yields ACTION$[1, \$] =$ accept.
- And the second yields ACTION$[1, +] =$ shift 6.

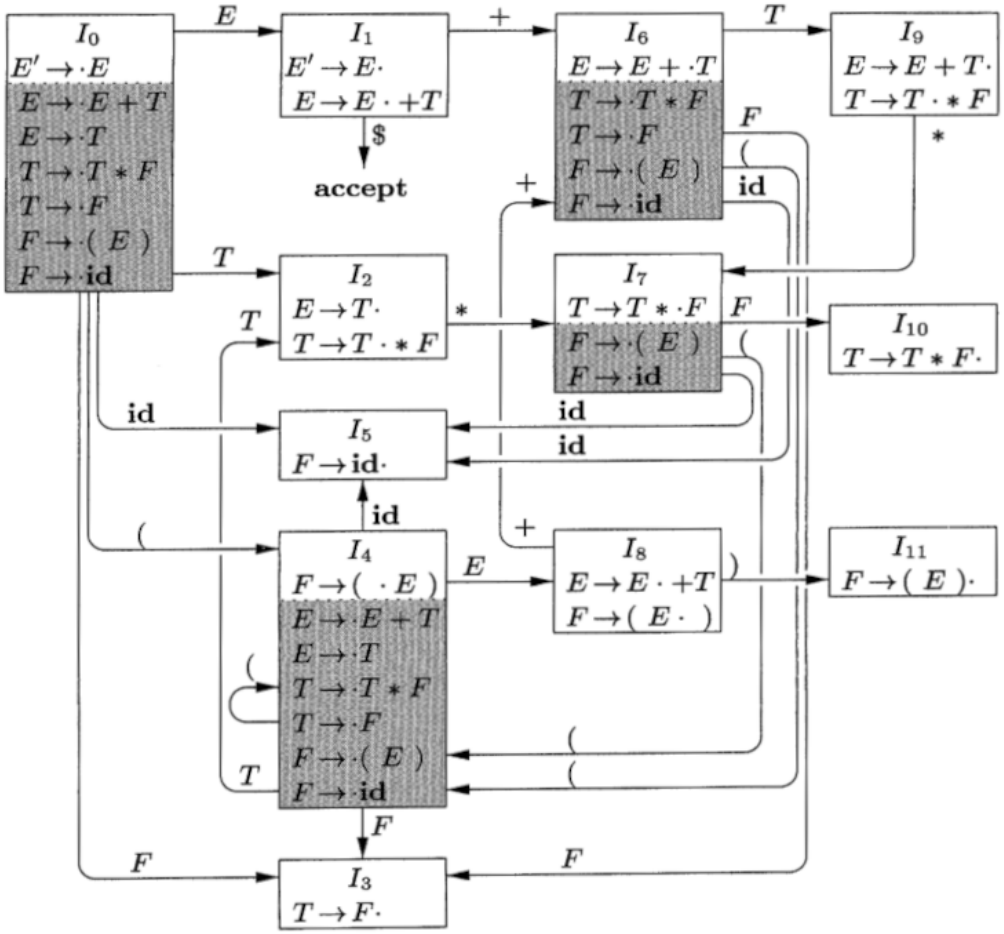
STATE	ACTION						GOTO		
	id	+	*	(	)	$	*E*	*T*	*F*
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

- Now consider I_1, $E' \to E \cdot$, $E \to E \cdot + T$.
- The first item yields ACTION$[1, \$] =$ accept.
- And the second yields ACTION$[1, +] =$ shift 6.

2. State i is constructed from I_i.

- The parsing actions for state i are determined as follows:
  - (a) If $[A \to \alpha \cdot a\beta]$ is in I_i, and GOTO$(I_i, a) = I_j$, then set ACTION$[i, a]$ to "shift j."
    - Here a must be a terminal.
  - (b) If $[A \to \alpha \cdot\ ]$ is in I_i, then set ACTION$[i, a]$ to "reduce $A \to \alpha$" for all a in FOLLOW(A).
    - Here A may not be S'.
  - (c) If $[S' \to S \cdot\ ]$ is in I_i, then set ACTION$[i, \$]$ to "accept."
- If any conflicting actions result from the above rules, we say the grammar is not SLR(1).
  - The algorithm fails to produce a parser in this case.

- Next consider I_2, $E \to T \cdot\ , T \to T \cdot\ * F$.
- Since FOLLOW$(E) = \{\$, +,)\}$, the first item makes ACTION$[2, \$]$ = ACTION$[2, +]$ = ACTION$[2,)]$ = reduce $E \to T$.
- The second item makes ACTION$[2, *]$ = shift 7.



(1) $E \to E + T$

(2) $E \to T$

(3) $T \to T * F$

(4) $T \to F$

(5) $F \to (E)$

(6) $F \to \mathbf{id}$

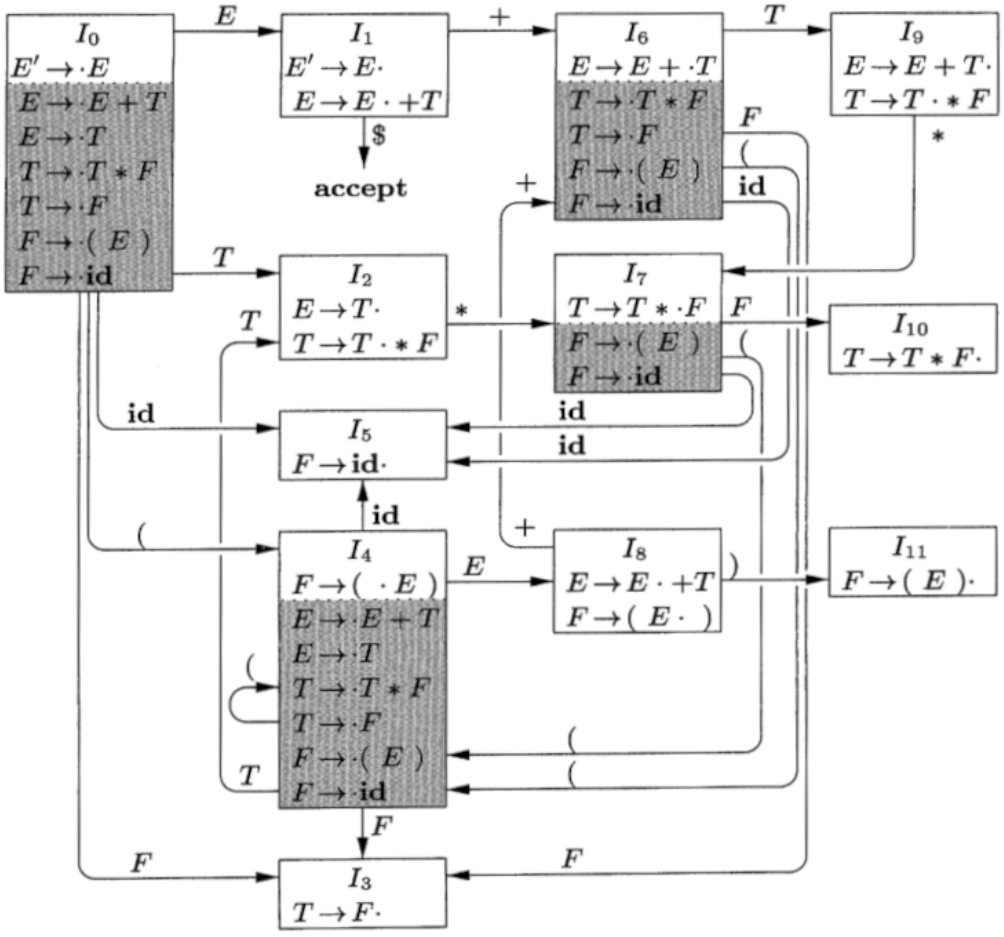
- Next consider I_2, $E \to T \cdot$, $T \to T \cdot * F$.
- Since FOLLOW$(E) = \{\$, +,)\}$, the first item makes ACTION$[2, \$]$ = ACTION$[2, +]$ = ACTION$[2,)]$ = reduce $E \to T$.
- The second item makes ACTION$[2, *]$ = shift 7.

STATE	ACTION						GOTO		
	id	+	*	(	)	$	*E*	*T*	*F*
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

- Next consider I_2, $E \to T \cdot$, $T \to T \cdot * F$.
- Since FOLLOW$(E) = \{\$, +,)\}$, the first item makes ACTION$[2, \$]$ = ACTION$[2, +]$ = ACTION$[2,)]$ = reduce $E \to T$.
- The second item makes ACTION$[2, *]$ = shift 7.

3. The goto transitions for state i are constructed for all nonterminals A using the rule: If GOTO$(I_i, A) = I_j$, then GOTO$[i, A] = j$.

- Now consider the GOTO transitions.
- Since GOTO$(I_0, E) = 1$, GOTO$(I_0, T) = 2$, and GOTO$(I_0, F) = 3$, we get GOTO$(0, E) = 1$, GOTO$(0, T) = 2$, and GOTO$(0, F) = 3$.



(1) $E \to E + T$

(2) $E \to T$

(3) $T \to T * F$

(4) $T \to F$

(5) $F \to (E)$

(6) $F \to \mathbf{id}$

- Now consider the GOTO transitions.
- Since $\text{GOTO}(I_0, E) = 1$, $\text{GOTO}(I_0, T) = 2$, and $\text{GOTO}(I_0, F) = 3$, we get $\text{GOTO}(0, E) = 1$, $\text{GOTO}(0, T) = 2$, and $\text{GOTO}(0, F) = 3$.

STATE	ACTION						GOTO		
	id	+	*	(	)	$	*E*	*T*	*F*
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

- Now consider the GOTO transitions.
- Since GOTO$(I_0, E) = 1$, GOTO$(I_0, T) = 2$, and GOTO$(I_0, F) = 3$, we get GOTO$(0, E) = 1$, GOTO$(0, T) = 2$, and GOTO$(0, F) = 3$.

STATE	ACTION						GOTO		
	id	+	*	(	)	$	*E*	*T*	*F*
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

- Continuing in this fashion we obtain the ACTION and GOTO tables

Example

- Every SLR(1) grammar is unambiguous, but there are many unambiguous grammars that are not SLR(1).
- Consider the grammar with productions,

$$\begin{aligned} S &\rightarrow L = R \mid R \\ L &\rightarrow *R \mid \textbf{id} \\ R &\rightarrow L \end{aligned}$$

- Think of L and R as standing for ℓ-value and r-value, respectively.
- And $*$ as an operator indicating “contents of.”

Example — *continued*

- The canonical collection of sets of LR(0) items for the grammar.

I_0: $S' \to \cdot S$
$S \to \cdot L = R$
$S \to \cdot R$
$L \to \cdot * R$
$L \to \cdot \mathbf{id}$
$R \to \cdot L$

I_1: $S' \to S\cdot$

I_2: $S \to L\cdot = R$
$R \to L\cdot$

I_3: $S \to R\cdot$

I_4: $L \to *\cdot R$
$R \to \cdot L$
$L \to \cdot * R$
$L \to \cdot \mathbf{id}$

I_5: $L \to \mathbf{id}\cdot$

I_6: $S \to L = \cdot R$
$R \to \cdot L$
$L \to \cdot * R$
$L \to \cdot \mathbf{id}$

I_7: $L \to *R\cdot$

I_8: $R \to L\cdot$

I_9: $S \to L = R\cdot$

Figure 4.39: Canonical LR(0) collection for grammar (4.49)

I_0: $S' \to \cdot S$
$S \to \cdot L = R$
$S \to \cdot R$
$L \to \cdot * R$
$L \to \cdot \mathbf{id}$
$R \to \cdot L$

I_1: $S' \to S\cdot$

I_2: $S \to L\cdot = R$
$R \to L\cdot$

I_3: $S \to R\cdot$

I_4: $L \to *\cdot R$
$R \to \cdot L$
$L \to \cdot * R$
$L \to \cdot \mathbf{id}$

I_5: $L \to \mathbf{id}\cdot$

I_6: $S \to L = \cdot R$
$R \to \cdot L$
$L \to \cdot * R$
$L \to \cdot \mathbf{id}$

I_7: $L \to *R\cdot$

I_8: $R \to L\cdot$

I_9: $S \to L = R\cdot$

- Consider the set of items I_2.

I_0: $S' \to \cdot S$
$S \to \cdot L = R$
$S \to \cdot R$
$L \to \cdot * R$
$L \to \cdot \mathbf{id}$
$R \to \cdot L$

I_1: $S' \to S \cdot$

I_2: $S \to L \cdot = R$
$R \to L \cdot$

I_3: $S \to R \cdot$

I_4: $L \to * \cdot R$
$R \to \cdot L$
$L \to \cdot * R$
$L \to \cdot \mathbf{id}$

I_5: $L \to \mathbf{id} \cdot$

I_6: $S \to L = \cdot R$
$R \to \cdot L$
$L \to \cdot * R$
$L \to \cdot \mathbf{id}$

I_7: $L \to * R \cdot$

I_8: $R \to L \cdot$

I_9: $S \to L = R \cdot$

- The first item in this set makes ACTION$[2, =]$ be "shift 6."
- FOLLOW(R) contains $=$ ($S \Rightarrow L = R \Rightarrow * R = R$).
- The second item sets ACTION$[2, =]$ to "reduce $R \to L$."

I_0: $S' \to \cdot S$
$S \to \cdot L = R$
$S \to \cdot R$
$L \to \cdot * R$
$L \to \cdot \mathbf{id}$
$R \to \cdot L$

I_1: $S' \to S\cdot$

I_2: $S \to L\cdot = R$
$R \to L\cdot$

I_3: $S \to R\cdot$

I_4: $L \to *\cdot R$
$R \to \cdot L$
$L \to \cdot * R$
$L \to \cdot \mathbf{id}$

I_5: $L \to \mathbf{id}\cdot$

I_6: $S \to L = \cdot R$
$R \to \cdot L$
$L \to \cdot * R$
$L \to \cdot \mathbf{id}$

I_7: $L \to *R\cdot$

I_8: $R \to L\cdot$

I_9: $S \to L = R\cdot$

- Since there is both a shift and a reduce entry in ACTION$[2, =]$, state 2 has a shift/reduce conflict on input symbol $=$.

Example — *continued*

$$
\begin{aligned}
S &\rightarrow L = R \mid R \\
L &\rightarrow *R \mid \textbf{id} \\
R &\rightarrow L
\end{aligned}
$$

- The grammar is not ambiguous.
- This shift/reduce conflict arises from the fact that the SLR parser construction method is not powerful enough to remember enough left context to decide what action the parser should take on input $=$, having seen a string reducible to L.

Example — *continued*

$$
\begin{aligned}
S &\rightarrow L = R \mid R \\
L &\rightarrow *R \mid \textbf{id} \\
R &\rightarrow L
\end{aligned}
$$

- The canonical and LALR methods, succeeds on a larger collection of grammars, including the above grammar.

Example — *continued*

$$
\begin{aligned}
S &\rightarrow L = R \mid R \\
L &\rightarrow *R \mid \textbf{id} \\
R &\rightarrow L
\end{aligned}
$$

- Note, however, that there are unambiguous grammars for which every LR parser construction method will produce a parsing action table with parsing action conflicts.
- Fortunately, such grammars can generally be avoided in programming language applications.